

Abdul Bari – Algorithms

84 Video İçin Ayrıntılı Türkçe Çalışma Notları

Teknik kavramlar English, anlatım Türkçe

2026

İçindekiler

01 – Introduction to Algorithms	3
02 – A Priori Analysis and A Posteriori Testing	8
03 – Characteristics of Algorithm	11
04 – How to Write and Analyze an Algorithm	15
05 – Frequency Count Method	19
06 – Time Complexity #1	24
07 – Time Complexity Example #2	29
08 – Time Complexity of While and If	34
09 – Classes of Functions	40
10 – Compare Classes of Functions	44
11 – Asymptotic Notations: Big O, Omega and Theta #1	48
12 – Asymptotic Notations: Big O, Omega and Theta #2	53
13 – Properties of Asymptotic Notations	57
14 – Comparison of Functions #1	62
15 – Comparison of Functions #2	66
16 – Best, Worst and Average Case Analysis	70
17 – Disjoint Sets: Weighted Union and Collapsing Find	76
18 – Divide and Conquer	81
19 – Recurrence Relation: $T(n) = T(n-1) + 1$	85
20 – Recurrence Relation: $T(n) = T(n-1) + n$	89
21 – Recurrence Relation: $T(n) = T(n-1) + \log n$	93
22 – Recurrence Relation: $T(n) = 2T(n-1) + 1$	98
23 – “Master Theorem” for Decreasing Functions	102
24 – Recurrence Relation: $T(n) = T(n/2) + 1$	106
25 – Recurrence Relation: $T(n) = T(n/2) + n$	110
26 – Recurrence Relation: $T(n) = 2T(n/2) + n$	113
27 – Master’s Theorem for Dividing Functions	118
28 – Examples for Master Theorem	124
29 – Root Function Recurrence Relation	130
30 – Binary Search: Iterative Method	135
31 – Binary Search: Recursive Method	141
32 – Heap, Heap Sort, Heapify and Priority Queues	147
33 – Two-Way Merge Sort: Iterative Method	154
34 – Merge Sort Algorithm	159
35 – Merge Sort In-Depth Analysis	164
36 – Quick Sort Algorithm	170
37 – Quick Sort Analysis	176
38 – Strassen’s Matrix Multiplication	181
39 – Greedy Method: Introduction	186
40 – Fractional Knapsack: Greedy Method	191

41 – Job Sequencing with Deadlines	197
42 – Optimal Merge Pattern	203
43 – Huffman Coding	207
44 – Prim's and Kruskal's Algorithms	212
45 – Dijkstra Algorithm: Single-Source Shortest Path	218
46 – Principle of Optimality: Dynamic Programming Introduction	224
47 – MultiStage Graph	230
48 – MultiStage Graph Program	235
49 – All-Pairs Shortest Path: Floyd-Warshall	241
50 – Matrix Chain Multiplication	246
51 – Matrix Chain Multiplication Using DP Formula	252
52 – Matrix Chain Multiplication Program	260
53 – Bellman-Ford: Single Source Shortest Path	266
54 – 0/1 Knapsack: Two Methods	271
55 – 0/1 Knapsack Program	277
56 – Optimal Binary Search Tree: Successful Search Only	283
57 – Optimal BST: Successful and Unsuccessful Probability	290
58 – Traveling Salesman Problem Using DP Formula	297
59 – Reliability Design	302
60 – Longest Common Subsequence: Recursion and DP	308
61 – Graph Traversals: BFS and DFS	314
62 – Articulation Point and Biconnected Components	319
63 – Backtracking: Brute Force Approach	324
64 – N-Queens Using Backtracking	329
65 – Sum of Subsets Using Backtracking	334
66 – Graph Coloring Using Backtracking	339
67 – Hamiltonian Cycle Using Backtracking	344
68 – Branch and Bound: Introduction	349
69 – Job Sequencing with Deadline: Branch and Bound	354
70 – 0/1 Knapsack Using Branch and Bound	359
71 – Traveling Salesman Problem: Branch and Bound	365
72 – NP-Hard and NP-Complete Problems	372
73 – Clique Decision Problem	377
74 – Knuth-Morris-Pratt (KMP) String Matching	382
75 – Rabin-Karp String Matching	388
76 – AVL Tree: Insertion and Rotations	394
77 – B-Trees and B+ Trees in Databases	403
78 – Asymptotic Notations: Simplified	408
79 – Hashing Technique: Simplified	411
80 – Shortest Path: Dijkstra and Bellman-Ford, Simplified	414
81 – BFS and DFS: Simplified	418
82 – Tower of Hanoi: Made Easy	422
83 – Row-Major and Column-Major Mapping	425
84 – Merge Sort Algorithm (Hindi)	428

01 – Introduction to Algorithms

Video:

1. Introduction to Algorithms – Abdul Bari (11:49)

Bu videonun ana fikri Bir Algorithm, bilgisayarın doğrudan çalıştıracağı kod değildir. Bir problemi çözmek için tasarladığımız, açık ve sıralı çözüm yöntemidir. Program ise bu yöntemin belirli bir Programming Language, Operating System ve çalışma ortamı dikkate alınarak uygulanmış hâlidir.

Videodaki temel ayırım şudur:

Önce çözümü tasarlarız (Algorithm), sonra bu tasarımı çalışan yazılıma dönüştürürüz (Program).

Bu ayrımı anlamadan Time Complexity, Divide and Conquer veya Dynamic Programming gibi sonraki konuların neden var olduğunu anlamak zorlaşır.

1. Algorithm nedir? Algorithm, bir computational problem çözmek için kullanılan sonlu ve adım adım prosedürdür.

Bu tanımın parçalarını açalım:

- **Problem:** Başlangıçta elimizde bazı girdiler ve ulaşmak istediğimiz bir sonuç vardır.
- **Input:** Problemin algoritmaya verilen verisidir.
- **Procedure:** Sonuca ulaşmak için izlenecek işlemler dizisidir.
- **Output:** Algoritmanın ürettiği sonuçtur.
- **Step-by-step:** İşlemler belirli bir sıraya sahiptir; sıra değişirse sonuç da değişebilir.

Örneğin “bir dizideki en büyük elemanı bul” problemi için:

- **Input:** Sayılardan oluşan bir dizi
- **Output:** Dizideki en büyük sayı
- **Algorithm:** İlk elemanı geçici en büyük kabul et; kalan elemanları sırayla karşılaştır; daha büyüğünü gördüğünde geçici sonucu güncelle.

Burada henüz C, Java veya Python seçmek zorunda değiliz. Önce çözüm fikrini kesinleştiriyoruz.

2. Algorithm ile Program arasındaki fark

Boyut	Algorithm	Program
Yaşam döngüsündeki yeri	Design phase	Implementation phase
Amaç	Çözüm yöntemini tarif etmek	Çözümü bilgisayarda çalıştırmak
Yazım biçimi	Doğal dil, pseudocode, matematiksel gösterim	C, C++, Java, Python gibi Programming Language
Sözdizimi	Katı bir dil sözdizimine bağlı olmak zorunda değildir	Seçilen dilin syntax kurallarına uymalıdır
Ortama bağımlılık	Kavramsal olarak hardware ve Operating System bağımsızdır	Derleyici, çalışma ortamı ve bazen platform ayrıntılarına bağlıdır
Değerlendirme	Analysis: doğruluk, zaman ve bellek ihtiyacı incelenir	Testing: program çalıştırılır, gerçek çıktılar ve hatalar kontrol edilir

Bu tablo “algorithm tamamen hiçbir ayrıntıya bağlı değildir” demek değildir. Örneğin seçtiğimiz data structure, işlemlerin maliyetini etkiler. Bağımsızlık, çözüm fikrinin Windows/macOS veya Python/Java seçmeden de ifade edilebilmesi anlamındadır.

3. Neden önce Design, sonra Implementation? Software Development Life Cycle (SDLC)

içinde tasarım ve uygulama farklı soruları cevaplar:

1. Design phase: **Problemi hangi yöntemle çözeceğiz?**
2. Implementation phase: **Bu yöntemi seçtiğimiz teknolojiyle nasıl çalıştıracağız?**

Doğrudan kod yazmaya başlarsak aynı anda iki farklı zorlukla uğraşırız:

- Çözüm mantığını bulmak
- Programlama dilinin ayrıntılarını doğru kullanmak

Algoritmayı önce kâğıt üzerinde kurmak bu iki yükü ayırır. Yanlış bir tasarımı değiştirmek, binlerce satırlık yanlış bir uygulamayı değiştirmekten genellikle daha ucuzdur.

Basit örnek Problem: Pozitif bir n sayısının faktöriyelini bulmak.

Önce dil bağımsız düşünce:

1. Sonucu 1 yap.
2. 2'den n'e kadar tüm sayıları sonuçla çarp.
3. Sonucu döndür.

Sonra pseudocode:

```
Algorithm Factorial(n)
  result <- 1
  for i <- 2 to n
    result <- result * i
  return result
```

Aynı Algorithm Python, C veya Java ile uygulanabilir. Dil değişir; temel yöntem değişmez.

4. Algorithm'ı kim tasarlar? Video, domain knowledge kavramını özellikle vurgular. Bir problemi çözebilmek için yalnızca kod yazmayı bilmek yeterli olmayabilir; problemin ait olduğu alanı da anlamak gerekir.

Örnekler:

- Muhasebe uygulamasında muhasebe kurallarını bilmek gerekir.
- Hastane yazılımında doktorların ve idari personelin iş akışları anlaşılmalıdır.
- Bir rota planlayıcıda yolların, kısıtların ve maliyetlerin nasıl modelleneceği bilinmelidir.

Bu nedenle Algorithm Designer ile Programmer aynı kişi olabilir, fakat olmak zorunda değildir. Gerçek projelerde alan uzmanı ve yazılım ekibi birlikte çalışabilir:

```
Domain Expert -> problem ve kurallar -> Algorithm Designer
Algorithm Designer -> çözüm yöntemi -> Programmer
Programmer -> çalışan Program
```

Önemli sonuç: Yanlış anlaşılan bir problem için teknik olarak kusursuz kod yazılsa bile doğru ürün ortaya çıkmaz.

5. Algorithm hangi dille yazılır? Bir Algorithm şu biçimlerde anlatılabilir:

- Açık ve sade doğal dil
- Pseudocode
- Matematiksel gösterimler
- Gerektiğinde diyagramlar

Amaç bilgisayarın değil, çözümü uygulayacak insanların anlamasıdır. Anlatım belirsiz olmamalı; bir adım farklı kişiler tarafından farklı yorumlanmamalıdır.

Videoda C benzeri sözdiziminin sık kullanılmasının nedeni, hedef kitlenin buna aşina olmasıdır. Bu, algoritmanın C programı olduğu anlamına gelmez. C benzeri pseudocode, yalnızca ortak ve kısa bir anlatım aracıdır.

Kötü ve iyi ifade farkı Belirsiz ifade:

Sayıları uygun şekilde kontrol et ve en büyüğünü bul.

Daha iyi pseudocode:

```
Algorithm FindMaximum(A, n)
  max ← A[0]
  for i ← 1 to n - 1
    if A[i] > max
      max ← A[i]
  return max
```

İkinci anlatımda başlangıç değeri, döngü sınırları, karşılaştırma ve çıktı nettir.

6. Analysis ile Testing aynı şey değildir Videonun sonraki derslere hazırlayan en önemli noktalarından biri budur.

Algorithm Analysis Bir algoritmayı çalıştırmadan, kullandığı yöntemle bakarak değerlendiririz. Başlıca sorular:

- Doğru sonucu üretiyor mu?
- Girdi büyüdükçe kaç işlem yapıyor?
- Ne kadar ek bellek kullanıyor?
- Başka bir algoritmadan daha verimli mi?

Bunlar ilerde Time Complexity ve Space Complexity ile sistematik hâle gelecek.

Program Testing Uygulanan programı gerçek veya üretilmiş girdilerle çalıştırırız:

- Beklenen çıktıyı veriyor mu?
- Sınır durumlarında hata oluşuyor mu?
- Seçilen makinede ne kadar sürüyor?
- Bellek taşması veya çalışma zamanı hatası var mı?

Analysis ve Testing birbirinin yerine geçmez. Analiz, bütün olası büyük girdiler hakkında matematiksel fikir verir. Test ise yalnızca denediğimiz girdiler ve ortam için gözlem sağlar.

Örneğin bir program 100 elemanda hızlı çalışabilir. Bu test, 100 milyon elemanda da hızlı çalışacağını kanıtlamaz. Algoritmanın büyüme davranışını anlamak için Analysis gerekir.

7. Örnek: FindMaximum Algorithm Şu diziyi kullanalım:

A = [7, 2, 11, 4, 9]

Adımlar:

Adım	İncelenen değer	Önceki max	Yeni max
Başlangıç	7	–	7
1	2	7	7
2	11	7	11
3	4	11	11
4	9	11	11

Sonuç 11'dir.

n elemanlı bir dizi için tam olarak $n - 1$ karşılaştırma yapılır. Şimdilik yalnızca şu sezgiyi kur: Girdi iki katına çıkarsa karşılaştırma sayısı da yaklaşık iki katına çıkar. Sonraki videolarda bunu $T(n)$ ve $O(n)$ ile ifade edeceğiz. Ek bellek olarak yalnızca max ve döngü değişkeni kullanılır. Girdi büyüse de bu değişkenlerin sayısı büyümmez; ilerde buna $O(1)$ auxiliary space diyeceğiz.

8. Sık yapılan yanlış anlamalar “Algorithm, açıklama satırları silinmiş koddur.” Hayır. Kod bir uygulamadır; algoritma çözüm yöntemidir. Aynı algoritmanın çok sayıda programı olabilir.

“Bir program testleri geçiyorsa algoritması kesinlikle iyidir.” Testler doğruluğa dair güven verebilir, fakat verimliliği her girdi boyutu için kanıtlamaz. Küçük testlerde iki farklı büyüme hızı benzer görünebilir.

“Algorithm mutlaka İngilizce yazılır.” Hayır. Ekip tarafından açıkça anlaşılabilen bir doğal dil veya gösterim kullanılabilir. Bu notlarda uluslararası teknik iletişimi kolaylaştırmak için kavram adları İngilizce tutulur.

“Hardware-independent demek bütün makinelerde aynı sürede çalışır demektir.” Hayır. Gerçek süre işlemciye, dile, derleyiciye ve sisteme göre değişir. Bağımsız olan şey çözüm yönteminin tanımıdır; ölçülen saniye değildir.

9. Kavram sözlüğü

English term	Türkçe açıklama
Algorithm	Bir problemi çözmek için açık, sıralı ve sonlu işlem yöntemi
Program	Bir algoritmanın programlama diliyle uygulanmış, çalıştırılabilir hâli
Computational Problem	Girdiden belirli bir çıktı üretmeyi gerektiren hesaplama problemi
Design Phase	Çözümün yapısının ve yönteminin tasarlandığı aşama
Implementation Phase	Tasarımın kod ve çalışan sistem hâline getirildiği aşama
Pseudocode	Programlama diline benzer, fakat belirli bir dilin katı kurallarına bağlı olmayan anlatım
Domain Knowledge	Problemin ait olduğu iş veya uzmanlık alanına ilişkin bilgi
Hardware-independent	Çözüm fikrinin belirli bir bilgisayar donanımına bağımlı olmaması
Operating System-independent	Çözüm fikrinin belirli bir işletim sistemine bağımlı olmaması
Analysis	Algoritmanın doğruluğunu ve kaynak ihtiyacını ku-ramsal olarak inceleme
Testing	Programı seçilmiş girdilerle çalıştırıp davranışını gözleme
Time Complexity	İşlem miktarının girdi büyüklüğüyle nasıl büyüdü-ğünün ifadesi
Space Complexity	Bellek ihtiyacının girdi büyüklüğüyle nasıl büyüdü-ğünün ifadesi

10. Kendini sinama

Önce cevaplara bakmadan yanıtla.

1. Aynı Algorithm neden hem Python hem Java ile uygulanabilir?

2. Design phase ile Implementation phase hangi farklı soruları cevaplar?
3. Bir programın birkaç testte hızlı çalışması neden algoritmasının verimli olduğunu kanıtlamaz?
4. Hastane yazılımı örneğinde domain knowledge neden önemlidir?
5. Pseudocode ile gerçek programlama dili arasındaki temel fark nedir?
6. FindMaximum neden $n - 1$ karşılaştırma yapar?

Kısa cevaplar

1. Çünkü Algorithm çözüm yöntemidir; programlama dili ise bu yöntemin uygulanacağı araçtır.
2. Design, "hangi yöntemle çözeceğiz?"; Implementation, "bu yöntemi seçilen teknolojiyle nasıl çalıştırac-ğız?" sorusunu cevaplar.
3. Test yalnızca seçilen girdi ve makineyi gözler; büyük girdilerdeki büyüme davranışını kanıtlamaz.
4. Sağlık alanının kuralları ve iş akışları anlaşılmadan doğru problem ve çözüm tanımlanamaz.
5. Pseudocode anlatım ve anlaşılabilirlik içindir; derlenmek zorunda değildir ve katı bir syntaxa bağlı değildir.
6. İlk eleman başlangıçta max seçilir; kalan $n - 1$ elemanın her biri bir kez karşılaştırılır.

Bir sonraki videoya geçmeden önce Şunları kendi cümlelerinle açıklayabiliyorsan hazırsın:

- Algorithm ile Program arasındaki fark
- Design ile Implementation arasındaki ilişki
- Analysis ile Testing arasındaki fark
- domain knowledge neden gerekli olabilir?

Sonraki ders: **02 – Priori Analysis and Posteriori Testing.**

02 – A Priori Analysis and A Posteriori Testing

Video: 1.1 Priori Analysis and Posteriori Testing (1:48)

Bu videonun ana fikri Bir çözümün kaynak ihtiyacını iki farklı şekilde değerlendirebiliriz:

- A Priori Analysis: Programı çalıştırmadan, algoritmanın yapısını matematiksel olarak incelemek.
- A Posteriori Testing: Programı yazıp çalıştırdıktan sonra gerçek süre ve bellek kullanımını ölçmek.

İkisi rakip değil, birbirini tamamlayan yöntemlerdir. İlki büyüme davranışını açıklar; ikincisi belirli bir uygulamanın belirli bir sistemde nasıl davrandığını gösterir.

1. A Priori Analysis A priori, Latince “önceden” anlamına gelir. Algoritmayı henüz belirli bir programlama diliyle uygulamadan önce yapılan kuramsal incelemedir.

Şunlara bakarız:

- Temel işlemler kaç kez gerçekleştiriliyor?
- İşlem sayısı input size büyüdükçe nasıl değişiyor?
- Algoritmanın ihtiyaç duyduğu bellek nasıl büyüyor?
- En kötü, en iyi veya ortalama durumda davranış değişiyor mu?

Sonuç genellikle saniye veya byte değildir. Girdi büyüklüğüne bağlı fonksiyonlardır:

Time function: $T(n)$
Space function: $S(n)$

Örneğin bir dizinin bütün elemanlarını bir kez gezen algoritma için:

$T(n) = an + b$

Buradaki a ve b , kullandığımız ayrıntı seviyesine göre değişebilir. Önemli olan, zaman ihtiyacının n ile doğrusal büyümesidir.

2. A Posteriori Testing A posteriori, “sonradan” anlamına gelir. Algoritma bir Program hâline getirildikten sonra çalıştırılır ve gerçek sistemde ölçüm yapılır.

Ölçülebilecek değerler:

- wall-clock time: Kullanıcının gözlediği toplam geçen süre
- CPU time: İşlemcinin program için harcadığı süre
- Bellek tüketimi: byte, KB, MB veya GB
- Gecikme, enerji tüketimi, veri transferi gibi uygulamaya özel ölçüler

Örnek bir test sonucu şöyle olabilir:

Input size: 1,000,000
Elapsed time: 42 ms
Peak memory: 8.4 MB

Bu sonuç yalnızca kullanılan program, girdi, dil, derleyici, Operating System ve donanım bağlamında anlamlıdır.

3. Temel karşılaştırma

Özellik	A Priori Analysis	A Posteriori Testing
İncelenen şey	Algorithm	Program
Zaman	Uygulamadan önce	Uygulamadan sonra
Yöntem	Matematiksel analiz	Çalıştırma ve ölçme
Zaman sonucu	$T(n)$ gibi fonksiyon	ms, s, dakika
Bellek sonucu	$S(n)$ gibi fonksiyon	byte, KB, MB
Dil bağımlılığı	Büyük ölçüde bağımsız	Seçilen dile bağlı
Donanım bağımlılığı	Makine hızından bağımsız model	Donanıma bağlı
Genelleme gücü	Tüm girdi büyüklükleri hakkında fikir verir	Test edilen durumları gösterir

4. Neden yalnızca saniye ölçmek yetmez? İki farklı bilgisayarda aynı program farklı sürede çalışabilir. Hatta aynı bilgisayarda arka plandaki işlemler yüzünden iki çalıştırmanın sonuçları değişebilir.

Şu iki algoritmayı düşün:

Algorithm A: $T(n) = 1000n$
Algorithm B: $T(n) = n^2$

Küçük bir n için B daha hızlı ölçülebilir. Fakat n büyüdükçe n^2 , $1000n$ 'i geçer ve aradaki fark sürekli açılır. A Priori Analysis, tek bir ölçümün gösteremediği bu büyüme davranışını ortaya çıkarır.

5. Neden yalnızca kuramsal analiz yetmez? Kuramsal olarak aynı büyüme sınıfındaki iki program pratikte farklı çalışabilir:

- Sabit katsayıları farklı olabilir.
- Biri cache kullanımına daha uygun olabilir.
- Dil ve derleyici optimizasyonları farklı olabilir.
- Bellek ayırma veya ağ erişimi gerçek maliyeti değiştirebilir.

Bu nedenle iyi mühendislik yaklaşımı şöyledir:

A Priori Analysis → doğru algoritma ailesini seç
A Posteriori Testing → gerçek uygulamayı ölç ve iyileştir

6. Örnek

```
Algorithm Sum(A, n)
  sum ← 0
  for i ← 0 to n - 1
    sum ← sum + A[i]
  return sum
```

A Priori Analysis Her eleman bir kez ziyaret edilir. Toplama işlemi n kez gerçekleşir:

$T(n) = an + b$

Dolayısıyla büyüme doğrusaldır. İleride bunu $\theta(n)$ olarak yazacağız.

A Posteriori Testing Aynı yöntem Python ve C ile uygulanıp bir milyon eleman üzerinde çalıştırılabilir. İki program farklı milisaniye sonuçları verebilir; buna rağmen ikisinin temel algoritmik büyümesi yine doğrusaldır.

7. Sık yapılan hatalar “ $T(n)$, programın kaç saniye süreceğidir.” Hayır. $T(n)$ çoğunlukla temel işlem sayısını veya onun matematiksel modelini gösterir. Saniyeye çevirmek için uygulama ve makine ayrıntıları gerekir.

“Benchmark hızlıysa algoritma her zaman iyidir.” Tek bir benchmark, farklı büyüklüklerdeki ve dağılımlardaki girdileri temsil etmeyebilir.

“Big O ile milisaniyeyi karşılaştırabilirim.” Bunlar farklı türde değerlerdir. Big O büyüme üst sınırını; milisaniye gerçek bir deneyin sonucunu ifade eder.

8. Kavram sözlüğü

English term	Türkçe açıklama
A Priori Analysis	Uygulamadan önce yapılan kuramsal algoritma analizi
A Posteriori Testing	Uygulamadan sonra programı çalıştırarak yapılan ölçüm
Time Function	İşlem miktarını input size cinsinden veren $T(n)$ fonksiyonu
Space Function	Bellek ihtiyacını input size cinsinden veren $S(n)$ fonksiyonu
Wall-clock Time	Başlangıç ile bitiş arasında gerçek dünyada geçen süre
Benchmark	Performansı ölçmek için yapılan kontrollü çalıştırma
Input Size	Girdinin büyüklüğünü temsil eden, çoğunlukla n ile gösterilen değer

9. Kendini sınamaya

1. A Priori Analysis neden saniye sonucu vermez?
2. Aynı algoritmanın Python ve C uygulamalarının süreleri neden farklı olabilir?
3. Bir benchmark neden algoritmanın bütün girdilerde hızlı olduğunu kanıtlamaz?
4. $T(n)$ ile wall-clock time arasındaki fark nedir?

Kısa cevaplar

1. Çünkü belirli bir donanımı çalıştırmak yerine algoritmadaki işlem sayısını matematiksel olarak modeller.
2. Dil, çalışma zamanı, derleyici ve sabit maliyetler farklıdır.
3. Yalnızca seçilmiş girdileri ve sistemi gözlemler.
4. $T(n)$ girdiye bağlı kuramsal büyümeyi, wall-clock time belirli bir çalıştırmada geçen gerçek süreyi gösterir.

Sonraki ders: **03 – Characteristics of Algorithm.**

03 – Characteristics of Algorithm

Video: 1.2 Characteristics of Algorithm (5:37)

Bu videonun ana fikri Her işlem listesi iyi tanımlanmış bir Algorithm değildir. Bir prosedürün algoritma sayılabilmesi için girdisi ve çıktısı tanımlı olmalı, adımları açık ve uygulanabilir olmalı, ayrıca sonlu sürede bitmelidir.

Videoda beş temel özellik ele alınır:

1. Input
2. Output
3. Definiteness
4. Finiteness
5. Effectiveness

1. Input: sıfır veya daha fazla girdi Bir Algorithm, dışarıdan **zero or more inputs** alabilir.

Örnekler:

- FindMaximum(A, n) algoritması A dizisini ve uzunluğunu alır.
- BinarySearch(A, target) bir dizi ve aranacak değer alır.
- Günün tarihini sistem saatinden okuyan bir prosedür açık parametre almayabilir.

“Sıfır girdi” hiçbir bilgi kullanılmadığı anlamına gelmek zorunda değildir. Algoritma bir sabit, sistem durumu veya önceden belirlenmiş başlangıç koşulu kullanabilir. Biçimsel olarak dış parametre sayısı sıfır olabilir.

2. Output: en az bir sonuç Bir algoritma **one or more outputs** üretmelidir. Sonuç yalnızca return edilen bir sayı olmayabilir:

- Bir değer döndürmek
- Bir diziyi sıralanmış hâle getirmek
- Bir dosyaya veri yazmak
- Bir cihazın durumunu değiştirmek
- “Çözüm yok” bilgisini üretmek

Örneğin bir arama algoritmasının -1 döndürmesi de anlamlı bir çıktıdır; hedefin bulunamadığını bildirir.

3. Definiteness: her adım açık ve tek anlamlı olmalı Definiteness, algoritmadaki her ifadenin kesin, anlaşılır ve unambiguous olmasıdır. Aynı girdiler ve aynı durum verildiğinde, adımın ne istediği farklı kişilerce farklı yorumlanmamalıdır.

Belirsiz ifade:

```
Liste yeterince büyükse birkaç elemanı kontrol et.
```

Sorunlar:

- “Yeterince büyük” kaç elemandır?
- “Biraz” kaçtır?
- Hangi elemanlar kontrol edilecektir?

Kesin ifade:

```
if n >= 10
    inspect A[0] through A[4]
```

Burada koşul, sınırlar ve işlem açıktır.

Definiteness ile determinism aynı şey değildir Bir randomized algorithm, rastgele seçim yapabilir ve yine de iyi tanımlı olabilir. "1 ile n arasında eşit olasılıkla rastgele bir indeks seç" kesin bir adımdır; hangi indeksin çıkacağı önceden belirli değildir ama işlemin kendisi belirsiz değildir.

4. Finiteness: sonlu sayıda adımdan sonra bitmeli Finiteness, algoritmanın geçerli her girdi için sonlu sayıda adım sonra durmasıdır.

Problemli örnek:

```
while true
  x <- x + 1
```

Bu süreç kendiliğinden bir sonuçla sona ermez.

Doğru örnek:

```
while x < n
  x <- x + 1
```

x sonlu bir başlangıç değerine sahipse ve her adımda artıyorsa döngü sonunda biter.

Sürekli çalışan sistemler ne olacak? Web server veya database server uzun süre çalışacak şekilde tasarlanır. Videoda bunlar, bütünüyle ele alındığında sonlanan algoritma örnekleri olarak görülmez. Bununla birlikte sunucunun işlediği her tekil istek sonlu bir prosedür olabilir:

```
Server sürekli çalışır
└─ Her request için çalışan handler sonlu sürede biter
```

Bu ayrım, sürekli hizmet veren Program ile belirli bir görevi çözen Algorithm arasındadır.

5. Effectiveness: adımlar uygulanabilir ve işe yarar olmalı Effectiveness, her adımın:

- Sonlu sürede gerçekleştirilebilir,
- Temel ve uygulanabilir,
- Çözümün ilerlemesine katkı sağlayan

bir işlem olmasıdır.

Videoda bu fikir "gereksiz adım yazmamak" üzerinden anlatılır. Daha teknik ifade şudur: Bir adım yalnızca güzel görünen bir talimat değil, gerçekten yürütülebilecek etkili bir işlem olmalıdır.

Uygulanamaz ifade:

```
Doğru cevabı sihirli biçimde tahmin et.
```

Uygulanabilir ifade:

```
Her candidate için constraint'leri kontrol et.
```

İkinci yöntem pahalı olabilir, fakat her adımın nasıl yapılacağı tanımlıdır.

6. Tarif ve laboratuvar benzetmesi Video, algoritmanın özelliklerini günlük prosedürlerle açıklar.

Bir yemek tarifi:

- Input: Malzemeler
- Output: Hazırlanan yemek
- Definiteness: "200°C'de 20 dakika pişir" gibi açık adımlar
- Finiteness: Tarif bir noktada sona erer
- Effectiveness: Her adım uygulanabilir ve yemeğin hazırlanmasına hizmet eder

Bir kimya deneyi de benzer yapıdadır: kimyasallar ve araçlar girdi, elde edilen ölçüm veya çözelti çıktı, deney protokolü ise prosedürdür.

Benzetmenin sınırı şudur: Bilgisayar, insanın "göz kararı" veya bağlam bilgisiyle doldurduğu belirsizlikleri kendiliğinden çözemez. Algoritmik anlatım daha kesin olmalıdır.

7. Özellikleri bir örnekte kontrol edelim

```
Algorithm LinearSearch(A, n, target)
  for i ← 0 to n - 1
    if A[i] = target
      return i
  return -1
```

Özellik	Nasıl karşılanıyor?
Input	A, n ve target alınır
Output	Bulunan indeks veya -1 döndürülür
Definiteness	Döngü sınırı ve karşılaştırma açıktır
Finiteness	En fazla n eleman incelenir
Effectiveness	İndeksleme, karşılaştırma ve döndürme uygulanabilir temel işlemlerdir

8. Sık yapılan hatalar "Output yalnızca ekrana yazılan değerdir." Hayır. Döndürülen değer, değiştirilen veri yapısı veya dış dünyada oluşturulan etki de çıktı olabilir.

"Çok uzun süren algoritma finite değildir." Bir algoritma trilyon adım sürse bile sonunda duruyorsa finite olabilir. Verimsizlik ile sonsuzluk farklı sorunlardır.

"Effectiveness yalnızca hızlı olmak demektir." Hayır. Öncelikle adımların gerçekleştirilebilir ve açıkça yürütülebilir olmasını anlatır. Hız ayrıca analiz edilir.

"Her algoritma en az bir input almak zorundadır." Hayır. Sıfır veya daha fazla girdi mümkündür; fakat en az bir anlamlı sonuç beklenir.

9. Kavram sözlüğü

English term	Türkçe açıklama
Input	Algoritmaya verilen veri veya başlangıç durumu
Output	Algoritmanın ürettiği sonuç veya etki
Definiteness	Her adımın açık ve tek anlamlı olması
Finiteness	Algoritmanın sonlu adım sonunda durması
Effectiveness	Adımların uygulanabilir ve çözüm açısından işlevsel olması
Unambiguous	Birden fazla yoruma izin vermeyen
Deterministic	Aynı durumda aynı sonraki adımı veya sonucu üretten
Randomized Algorithm	İşleyişinde tanımlı rastgele seçim kullanan algoritma

10. Kendini sinama

1. Bir algoritma hiç parametre almadan geçerli olabilir mi?
2. return -1 neden bir Output sayılır?

3. Definiteness ile determinism neden aynı değildir?
4. Bir web server'ın sürekli çalışması Finiteness özelliğiyle nasıl bağdaştırılabilir?
5. Çok yavaş fakat sonunda biten bir yöntem finite midir?

Kısa cevaplar

1. Evet; algoritmalar sıfır veya daha fazla girdi alabilir.
2. Aranacak değerin bulunamadığını bildiren anlamlı bir sonuçtur.
3. Rastgele seçim kullanan bir adım bile nasıl seçim yapılacağını kesin olarak tanımlayabilir.
4. Sunucu sürekli çalışan programdır; işlediği her tekil isteğin algoritması sonlu olabilir.
5. Evet. Verimlilik ile sonluluk farklı özelliklerdir.

Sonraki ders: **04 – How to Write and Analyze an Algorithm.**

04 – How to Write and Analyze an Algorithm

Video: 1.3 How Write and Analyze Algorithm (10:37)

Bu videonun ana fikri Bir Algorithm yazarken belirli bir programlama dilinin bütün ayrıntılarına bağlı kalmayız. Çözüm açıkça anlaşılacak kadar kesin olmalıdır; ancak data type, değişken bildirimi veya derleyici kuralları gibi Implementation ayrıntılarını erteleyebiliriz.

Yazılan algoritmayı analiz ederken videonun iki temel ölçütü şunlardır:

- Time: Algoritmanın gerçekleştirdiği işlem miktarı
- Space: Algoritmanın ihtiyaç duyduğu bellek miktarı

1. Pseudocode nasıl yazılır? Pseudocode, insan tarafından okunmak üzere hazırlanan, programlama diline benzeyen algoritma anlatımıdır. Evrensel ve zorunlu bir sözdizimi yoktur.

Şunların hepsi kullanılabilir:

```
temp = a
```

```
temp <- a
```

```
ASSIGN a TO temp
```

Önemli olan, dokümanı kullanacak kişilerin ifadeyi aynı şekilde anlamasıdır.

Neleri yazmak zorunda değiliz? Basit bir algoritmada çoğunlukla şu dil ayrıntılarını atlayabiliriz:

- int, float, double gibi kesin data type bildirimleri
- include, import, sınıf veya paket yapısı
- Bellek adresi ve platform ayrıntıları
- Bir programlama dilinin zorunlu noktalama işaretleri

Fakat atlanan ayrıntı algoritmanın anlamını belirsizleştiriyorsa mutlaka belirtilmelidir. Örneğin tam sayı bölmesi ile gerçek sayı bölmesi farklı sonuç veriyorsa kullanılan bölme türü yazılmalıdır.

2. Swap Algorithm Video iki değerini yerini değiştiren Swap algoritmasını kullanır:

```
Algorithm Swap(a, b)
  temp <- a
  a <- b
  b <- temp
```

Başlangıçta:

```
a = 10
b = 20
```

Adımlar:

Adım	a	b	temp
Başlangıç	10	20	–
temp <- a	10	20	10
a <- b	20	20	10
b <- temp	20	10	10

Sonuçta değerler yer değiştirmiştir.

Uygulama ayrıntısı Gerçek bir programlama dilinde parametrelerin `pass by value` veya `pass by reference` olması önemlidir. Yalnızca kopyalar değiştirilirse çağırının değişkenleri yer değiştirmez. Bu, temel çözüm fikrinden çok dilin çağrı modeline ait bir `Implementation` ayrıntısıdır.

3. Analiz ölçütleri Time Algoritmanın çalışması için gereken temel işlem sayısının `input size` ile nasıl büyüdüğünü inceleriz.

Videodaki basitleştirilmiş modelde her basit statement bir `unit time` kabul edilir:

```
temp <- a    // 1 unit
a <- b       // 1 unit
b <- temp    // 1 unit
```

Bu durumda:

$T(n) = 3$

İşlem miktarı girdinin büyüklüğüne göre artmadığı için:

Time Complexity = $O(1)$

Buradaki 1, "bir işlem" demek değildir. Sabit miktarda işlem demektir. 3, 20 veya 300 sabit işlem de asimptotik olarak $O(1)$ olabilir.

Space Videoda `a`, `b` ve `temp` için birer word sayılır:

$S(n) = 3 \text{ words}$
Space Complexity = $O(1)$

Neden byte değil word? Çünkü algoritma düzeyinde değerlerin ileride `int`, `float` veya başka bir tür olarak uygulanacağını henüz bilmiyoruz. Word, bir değeri saklayan soyut bellek birimidir.

4. Neden her statement'a bir unit time veriyoruz? Gerçekte şu ifade tek makine komutu olmak zorunda değildir:

```
x <- 5 * a + 6 * b
```

Çarpma, toplama, yükleme ve saklama için birden fazla `machine instruction` üretilebilir. Üstelik sayı türü ve işlemci mimarisi maliyeti değiştirebilir.

Temel algoritma analizinde bu ayrıntıları soyutlarız. Sabit sayıda temel işlem içeren ifadeleri $O(1)$ kabul ederiz. Böylece makine modelinden çok, girdi büyüdüğünde davranışın nasıl değiştiğine odaklanırsınız.

Bu model gerektiğinde ayrıntılandırılabilir:

- `comparison` ve `assignment` ayrı ayrı sayılabilir.
- Disk veya ağ erişimi farklı maliyetle değerlendirilebilir.
- `floating-point operation` sayısı ölçülebilir.
- Belirli işlemci için `machine instruction` düzeyine inilebilir.

Analiz ayrıntısı, çözmek istediğimiz soruya uygun seçilir.

5. Time ve Space dışındaki ölçütler Video, projenin türüne göre başka kaynakların da analiz edilebileceğini söyler:

- `Data transfer`: Ağ üzerinden taşınan veri miktarı
- `Network usage`: İstek sayısı, bant genişliği ve gecikme
- `Power consumption`: Özellikle mobil ve gömülü cihazlarda enerji
- `Register usage`: Düşük seviyeli ve sistem programlamada işlemci register ihtiyacı
- `I/O cost`: Disk veya kalıcı depolama erişimi

Örneğin mobil cihazdaki iki algoritma aynı Time Complexity sınıfında olabilir; fakat biri sürekli ağ bağlantısı kullandığı için daha fazla enerji tüketebilir.

6. Cost model seçimi

Cost model, hangi işlemleri ve kaynakları ne şekilde sayacağımızı belirler.

Swap örneğinde:

Her assignment = 1 unit time
Her scalar variable = 1 word

Matris algoritmasında ise matrix multiplication tek bir basit işlem kabul edilmemelidir; onun içindeki eleman işlemleri incelenmelidir.

Doğru ilke:

Girdi büyüdükçe kendi içinde büyüyen bir işi, sabit maliyetli tek statement gibi gizleme.

Örneğin $\text{sort}(A)$ çağırısı tek satırdır ama n elemanı sıraladığı için $O(1)$ değildir. Çağrılan algoritmanın maliyeti hesaba katılmalıdır.

7. Total Space ve Auxiliary Space ayrımı

Videonun temel hesabında bütün değişkenler sayılır. İleride iki farklı tanımla karşılaşabilirsin:

- Total Space: Girdi, çıktı ve çalışma alanının tamamı
- Auxiliary Space: Girdi ve çıktı dışında algoritmanın kullandığı ek çalışma alanı

$\text{Swap}(a, b)$ için her iki bakış da sabit büyür, dolayısıyla sonuç $O(1)$ değişmez. Fakat büyük dizilerde bu ayrım önem kazanır.

8. Sık yapılan hatalar “Pseudocode’da syntax hiç önemli değildir.”

Derleyici sözdizimi zorunlu değildir, fakat insanın anlayacağı tutarlı bir yazım gerekir. Belirsizlik kabul edilemez.

“Tek satır her zaman $O(1)$ ’dir.” Hayır. Satır başka bir algoritmayı çağırabilir veya uzunluğu n olan veriyi işleyebilir.

“ $O(1)$, tam olarak bir işlem demektir.” Hayır. Girdi boyutundan bağımsız herhangi bir sabit işlem miktarını ifade eder.

“Space Complexity yalnızca değişken sayısıdır.” Diziler, recursive call stack, dinamik veri yapıları ve çağrılan prosedürlerin belleği de hesaba katılabilir.

9. Kavram sözlüğü

English term	Türkçe açıklama
Pseudocode	Algoritmayı dil bağımsız ve okunabilir biçimde anlatan sözde kod
Assignment	Bir değeri değişkene atama işlemi
Unit Time	Temel analiz modelinde bir işleme verilen soyut sabit maliyet
Word	Bir değeri sakladığı varsayılan soyut bellek birimi
Cost Model	Hangi işlemlerin ve kaynakların nasıl sayılacağını belirleyen model
Total Space	Girdi ve çıktı dâhil toplam bellek ihtiyacı
Auxiliary Space	Algoritmanın girdi/çıktı dışında kullandığı ek bellek
Machine Instruction	İşlemcinin doğrudan gerçekleştirdiği düşük seviyeli komut

10. Kendini sinama

1. Pseudocode neden kesin bir programlama dili olmak zorunda değildir?
2. Swap algoritmasının Time Complexity değeri neden $O(1)$ 'dir?
3. Tek satırlık sort(A) çağrısını neden bir unit time kabul edemeyiz?
4. word kullanılmasının nedeni nedir?
5. Total Space ile Auxiliary Space arasındaki fark nedir?

Kısa cevaplar

1. Amaç bilgisayar tarafından derlenmek değil, çözüm yöntemini insanlar arasında açık biçimde aktarmaktır.
2. Üç atamanın sayısı girdi büyüklüğüyle değişmez.
3. Çağrının içinde girdiyle büyüyen bir sıralama algoritması çalışır.
4. Algoritma aşamasında gerçek veri türü ve byte miktarı henüz seçilmemiş olabilir.
5. Total Space girdi/çıktıyı da sayar; Auxiliary Space yalnızca ek çalışma alanını sayar.

Sonraki ders: **05 – Frequency Count Method.**

05 – Frequency Count Method

Video: 1.4 Frequency Count Method (12:22)

Bu videonun ana fikri Frequency Count Method, bir algoritmadaki statement'ların kaç kez çalıştığını sayarak Time Function elde etme yöntemidir.

Temel süreç:

1. Her statement'ın frequency değerini bul.
2. Maliyet ile frekansı çarp.
3. Bütün maliyetleri toplayarak $T(n)$ oluştur.
4. En hızlı büyüyen terime bakarak order of growth belirle.

Video bunu üç örnekle gösterir:

- Array elemanlarının toplamı: doğrusal büyüme
- İki matrix'in toplamı: karesel büyüme
- İki matrix'in çarpımı: kübik büyüme

1. Frequency ne demektir? Frequency, bir statement'ın belirli input size için kaç defa yürütüldüğüdür.

```
for i <- 0 to n - 1
  process(A[i])
```

Burada $process(A[i])$ tam n kez çalışır. Döngü koşulu ise son başarısız kontrol dâhil $n + 1$ kez değerlendirilir.

Bir for döngüsünü açık biçimde düşünersek:

```
i <- 0           // 1 kez
while i < n      // n + 1 kez
  process(A[i])  // n kez
  i <- i + 1     // n kez
```

Hangi ayrıntıların sayıldığına göre tam polinom değişebilir. Fakat en yüksek dereceli terim değişmediği sürece büyüme sınıfı aynı kalır.

2. Örnek 1: Array Sum

```
Algorithm Sum(A, n)
  sum <- 0
  for i <- 0 to n - 1
    sum <- sum + A[i]
  return sum
```

Frequency tablosu

İşlem	Frequency
$sum \leftarrow 0$	1
Döngü başlangıcı	1
$i < n$ kontrolü	$n + 1$
$sum \leftarrow sum + A[i]$	n
$i \leftarrow i + 1$	n
return sum	1

Her birini bir unit time kabul edersek ayrıntılı model şu biçimde olur:

$$T(n) = 1 + 1 + (n + 1) + n + n + 1 \\ = 3n + 4$$

Videodaki daha kaba sayım, for yapısının bazı parçalarını tek kalemde ele alıp $2n + 3$ benzeri bir fonksiyon elde eder. Bu bir çelişki değildir; kullanılan cost model farklıdır.

Her iki durumda da baskın terim n 'dir:

$$\text{Time Complexity} = O(n)$$

İleride sıkı sınırı öğrendiğimizde bu algoritma için $\theta(n)$ demek daha bilgilendirici olacaktır; çünkü her durumda bütün elemanlar gezilir.

Space hesabı Videoda girdi dizisi de sayılır:

```
A      -> n words
n      -> 1 word
sum    -> 1 word
i      -> 1 word

S(n) = n + 3
Total Space = O(n)
```

Ancak girdi dışındaki ek alanı sorarsak:

$$\text{Auxiliary Space} = O(1)$$

Bu ayrım sınavlarda çok önemlidir: Sorunun total space mi yoksa auxiliary space mi istediğini kontrol et.

3. Örnek 2: Matrix Addition A ve B, $n \times n$ boyutlu iki matrix olsun. Sonuç C matrix'ine yazılsın.

```
Algorithm AddMatrices(A, B, C, n)
  for i <- 0 to n - 1
    for j <- 0 to n - 1
      C[i][j] <- A[i][j] + B[i][j]
```

Statement kaç kez çalışır? Dış döngü n farklı satır için çalışır. Her satırda iç döngü n farklı sütunu gezer:

$$n * n = n^2$$

Dolayısıyla toplama ve atama yapılan iç statement n^2 kez yürütülür.

Tam sayımın polinomu kullanılan modele bağlı olarak değişebilir:

$$T(n) = an^2 + bn + c$$

En yüksek dereceli terim n^2 olduğu için:

$$\text{Time Complexity} = O(n^2)$$

Matrix'in n^2 hücresinin her birini üretmek zorunda olduğumuz için sıkı büyüme $\theta(n^2)$ 'dir.

Space hesabı Videonun Total Space yaklaşımı:

```
A -> n^2 words
B -> n^2 words
C -> n^2 words
n, i, j -> 3 words
```

$$S(n) = 3n^2 + 3$$
$$\text{Total Space} = O(n^2)$$

Girdi ve çıktı matrix'lerini hariç tutarsak döngü değişkenleri sabittir:

$$\text{Auxiliary Space} = O(1)$$

4. Örnek 3: Matrix Multiplication Klasik $n \times n$ matrix multiplication üç iç içe döngü kullanır:

```
Algorithm MultiplyMatrices(A, B, C, n)
  for i <- 0 to n - 1
    for j <- 0 to n - 1
      C[i][j] <- 0
      for k <- 0 to n - 1
        C[i][j] <- C[i][j] + A[i][k] * B[k][j]
```

İndekslerin görevleri:

- i: Sonuç matrix'inin satırını seçer.
- j: Sonuç matrix'inin sütununu seçer.
- k: A satırı ile B sütununun dot product hesabını yürütür.

En iç statement:

$$n * n * n = n^3$$

kez çalışır.

Videodaki sadeleştirilmiş frequency hesabı şu tip bir polinom verir:

$$T(n) = 2n^3 + 3n^2 + 2n + 1$$

Tam katsayıdan bağımsız olarak en yüksek derece 3'tür:

$$\text{Time Complexity} = O(n^3)$$

Klasik üç döngülü yöntem için sıkı büyüme $\Theta(n^3)$ 'tür.

Space hesabı Videodaki Total Space hesabı:

A, B, C $\rightarrow 3n^2$ words
n, i, j, k $\rightarrow 4$ words

$$S(n) = 3n^2 + 4$$
$$\text{Total Space} = O(n^2)$$

Girdi ve sonuç matrix'leri dışarıdan sağlanıyorsa ek çalışma alanı yine sabit olabilir:

$$\text{Auxiliary Space} = O(1)$$

Bu sonuç n^3 zaman kullanırken n^2 veri saklanması mümkün olduğunu gösterir. Time Complexity ile Space Complexity aynı olmak zorunda değildir.

5. Dominant term neden yeterlidir? Şu fonksiyonu düşün:

$$T(n) = 2n^3 + 3n^2 + 2n + 1$$

n büyüdükçe terimlerin etkisi:

$2n^3$ → en hızlı büyür
 $3n^2$ → n^3 'e göre giderek önemsizleşir
 $2n$ → daha da yavaş büyür
 1 → sabittir

Oranı n^3 'e bölersek:

$$T(n) / n^3 = 2 + 3/n + 2/n^2 + 1/n^3$$

n büyüdükçe kesirli terimler sıfıra yaklaşır. Bu nedenle asimptotik büyümeyi n^3 belirler.

6. Nested loop görünce doğrudan çarpılır mı? Yalnızca döngü sınırları bağımsızsa.

```
for i <- 0 to n - 1
  for j <- 0 to n - 1
    work()
```

Burada $n * n = n^2$ doğrudur. Fakat iç sınır $j < i$ veya $j < n / i$ olursa her dış iterasyondaki iç çalışma aynı değildir. O zaman toplam yazmak gerekir:

Total work = sum of inner-loop frequencies

Bu ayrım bir sonraki videoda ayrıntılı işlenir.

7. Üç örneğin karşılaştırması

Algorithm	Baskın işlem sayısı	Time	Video yaklaşımıyla Total Space
Array Sum	n	$O(n)$	$O(n)$
Matrix Addition	n^2	$O(n^2)$	$O(n^2)$
Matrix Multiplication	n^3	$O(n^3)$	$O(n^2)$

Girdi boyutu 10 katına çıkarsa baskın işlem sayısı yaklaşık olarak:

- $O(n)$: 10 kat
- $O(n^2)$: 100 kat
- $O(n^3)$: 1000 kat

Bu, algoritmik büyümenin neden sabit saniye ölçümünden daha açıklayıcı olduğunu gösterir.

8. Sık yapılan hatalar Döngü koşulunu n kez saymak Başarısız son kontrol nedeniyle koşul çoğunlukla $n + 1$ kez değerlendirilir.

Her nested loop'u otomatik çarpmak İç döngünün sınırı dış değişkene bağlıysa frekansları toplamak gerekir.

Total Space ile Auxiliary Space'i karıştırmak Video girdiyi de sayar. Başka kaynaklar yalnızca ek alanı sorabilir.

En büyük katsayıyı complexity sanmak $2n^3$ içindeki 2 değil, n^3 büyüme derecesini belirler.

9. Kavram sözlüğü

English term	Türkçe açıklama
Frequency Count Method	Statement yürütülme sayılarını kullanarak maliyet çıkarma yöntemi
Frequency	Bir statement'ın yürütülme sayısı
Dominant Term	Büyük girdilerde büyümeyi belirleyen en hızlı büyüyen terim
Degree of Polynomial	Polinomdaki en büyük üs
Nested Loop	Başka bir döngünün gövdesinde bulunan döngü
Matrix Addition	Aynı konumdaki matrix elemanlarını toplama işlemi
Matrix Multiplication	Satır ve sütunların dot product değerleriyle sonuç matrix'i üretme

10. Kendini sına

1. n iterasyonlu bir döngünün koşulu neden $n + 1$ kez kontrol edilir?
2. $3n + 4$ ile $2n + 3$ neden aynı büyüme sınıfındadır?
3. Matrix addition'daki temel statement kaç kez çalışır?
4. Klasik matrix multiplication neden n^3 zaman kullanır?
5. Array Sum için Total Space ve Auxiliary Space neden farklı yazılabilir?

Kısa cevaplar

1. n başarılı kontrolden sonra döngüyü bitiren bir başarısız kontrol daha yapılır.
2. Her ikisinin de baskın terimi birinci dereceden n 'dir.
3. n^2 kez.
4. Satır, sütun ve dot product için üç bağımsız n uzunluklu dolaşım vardır.
5. Total Space girdi dizisini sayar; Auxiliary Space yalnızca sabit sayıdaki ek değişkeni sayar.

Sonraki ders: **06 – Time Complexity #1.**

06 – Time Complexity #1

Video: 1.5.1 Time Complexity #1 (9:44)

Bu videonun ana fikri Bir döngünün Time Complexity değerini görünüşüne bakarak değil, gövdesindeki temel işlemin kaç kez çalıştığını bularak çıkarırız.

Videoda beş önemli kalıp incelenir:

1. Birer birer artan veya azalan loop: $O(n)$
2. Sabit miktarda atlayan loop: $O(n)$
3. Bağımsız iki nested loop: $O(n^2)$
4. Üçgensel nested loop: $O(n^2)$
5. Birikimli toplamla duran loop: $O(\sqrt{n})$

1. Standard Linear Loop

```
for i <- 0 to n - 1  
  work()
```

i değerleri:

```
0, 1, 2, ..., n - 1
```

work() tam n kez çalışır:

```
T(n) = cn + d  
Time Complexity =  $O(n)$ 
```

Burada döngü kontrolünün n + 1 kez çalışması sonucu değiştirmez. Hem n hem n + 1 doğrusal büyür.

2. Decrementing Loop

```
for i <- n down to 1  
  work()
```

Değerler ters yönde ilerler:

```
n, n - 1, n - 2, ..., 1
```

Yön değişse de adım sayısı değişmez; work() yine n kez çalışır:

```
Time Complexity =  $O(n)$ 
```

Temel fikir: 1'den n'e gitmek ile n'den 1'e gelmek aynı sayıda sabit adım gerektirir.

3. Constant Step Loop

```
for i <- 0; i < n; i <- i + 2  
  work()
```

Değerler:

```
0, 2, 4, 6, ...
```

Yaklaşık n / 2 iterasyon vardır:

$$T(n) = c(n/2) + d$$

Sabit katsayılar asimptotik sınıfı değiştirmez:

$$\text{Time Complexity} = O(n)$$

Aynı sonuç $i += 20$, $i += 1000$ gibi n 'den bağımsız pozitif sabit adımlar için de geçerlidir:

$$\begin{aligned} n / 20 &\rightarrow O(n) \\ n / 1000 &\rightarrow O(n) \end{aligned}$$

Kritik ayırım Adım sabit değilse sonuç değişebilir:

$$i \leftarrow i * 2$$

Bu durumda değerler 1, 2, 4, 8, ... biçiminde büyür ve iterasyon sayısı $O(\log n)$ olur. Yani "kaç artırıldığı" değil, güncellenmenin n ile nasıl ilişkilendiği önemlidir.

4. Independent Nested Loops

```
for i <- 0 to n - 1
  for j <- 0 to n - 1
    work()
```

Her i değeri için iç loop n kez çalışır. Dış loop da n kez çalıştığı için:

$$\begin{aligned} n * n &= n^2 \\ \text{Time Complexity} &= O(n^2) \end{aligned}$$

Örnek olarak $n = 4$:

i	İç loop iterasyonu
0	4
1	4
2	4
3	4
Toplam	$16 = 4^2$

Burada çarpabilmemizin nedeni, iç loop'un her dış iterasyonda aynı n uzunluğunda ve i 'den bağımsız olmasıdır.

5. Triangular Nested Loop

```
for i <- 0 to n - 1
  for j <- 0 to i - 1
    work()
```

Bu kez iç loop'un sınırı i 'ye bağlıdır. Bu nedenle doğrudan "iki loop var, n^2 " demek doğru bir analiz yöntemi değildir. Frekansları toplamalıyız.

i	j değerleri	work() sayısı
0	yok	0
1	0	1
2	0, 1	2
3	0, 1, 2	3
...	...	...
n - 1	0 ... n - 2	n - 1

Toplam:

$$\begin{aligned}
 &0 + 1 + 2 + \dots + (n - 1) \\
 &= n(n - 1) / 2 \\
 &= (n^2 - n) / 2
 \end{aligned}$$

Baskın terim n^2 olduğu için:

$$\text{Time Complexity} = O(n^2)$$

Aslında her durumda bu kadar çalıştığından sıkı ifade $\theta(n^2)$ 'dir.

Neden önceki loop'tan daha az çalıştığı hâlde sınıf aynı? Bağımsız nested loop yaklaşık n^2 kez, üçgensel loop yaklaşık $n^2 / 2$ kez çalışır. İkincisi pratikte yarı miktarda iş yapar; fakat ikisinin büyüme derecesi aynıdır. Asimptotik gösterim sabit çarpanı atar.

6. Cumulative Sum Loop Videonun en öğretici örneği:

```

p <- 0
for i <- 1; p <= n; i <- i + 1
  p <- p + i

```

Burada koşul $i \leq n$ değildir. Bu yüzden döngünün n kez çalıştığını varsayamayız.

Trace

İterasyon	Eklenen i	Yeni p
1	1	1
2	2	3
3	3	6
4	4	10
5	5	15

k iterasyon sonunda:

$$\begin{aligned}
 p &= 1 + 2 + 3 + \dots + k \\
 &= k(k + 1) / 2
 \end{aligned}$$

Döngü, p değeri n sınırını geçtiğinde durur. Yaklaşık eşitlik kurarsak:

$$k(k + 1) / 2 \approx n$$

Büyük k için $k + 1 \approx k$:

$$k^2 / 2 \approx n$$

$$k^2 \approx 2n$$

$$k \approx \sqrt{2n}$$

$\sqrt{2}$ sabit olduğu için:

$$\text{Time Complexity} = O(\sqrt{n})$$

Daha kesin çözüm:

$$k^2 + k - 2n \approx 0$$

$$k \approx (-1 + \sqrt{1 + 8n}) / 2$$

Bu ifade de asimptotik olarak $O(\sqrt{n})$ büyür.

7. Analiz için genel yöntem Bir loop gördüğünde şu sırayı izle:

1. Loop variable hangisi?
2. Başlangıç değeri nedir?
3. Devam koşulu hangi değişkene bağlıdır?
4. Her iterasyonda bu değişken nasıl değişir?
5. k iterasyon sonraki değerini formülle yaz.
6. Durma koşuluyla denkleştir ve k'yi n cinsinden çöz.

Özellikle koşul ve güncelleme farklı değişkenlerdeyse trace tablosu hazırla. Cumulative sum örneğinde yanlış yapmayı önleyen şey budur.

8. Kalıpların özeti

Güncelleme / yapı	Yaklaşık iterasyon	Complexity
i++	n	$O(n)$
i--	n	$O(n)$
i += c, sabit c	n/c	$O(n)$
İki bağımsız n loop	n^2	$O(n^2)$
İç sınır j < i	$n(n-1)/2$	$O(n^2)$
Her adımda p += i, koşul p <= n	yaklaşık $\sqrt{2n}$	$O(\sqrt{n})$

9. Sık yapılan hatalar Loop sayısını complexity sanmak İki nested loop her zaman $O(n^2)$ değildir.

Sınırlar ve güncellemeler incelenmelidir.

Koşulda yanlış değişkeni takip etmek p <= n koşullu loop'ta yalnızca i'nin artışına bakmak yanlış sonuç verir. p'nin nasıl büyüdüğü bulunmalıdır.

n/2 görünce $O(n/2)$ yazmak Standart asimptotik gösterimde sabit çarpan atılır; sonuç $O(n)$ 'dir.

Triangular sum'ı $O(n)$ sanmak Toplamda n terim vardır ama terimlerin kendileri büyür. $1 + 2 + \dots + n = n(n+1)/2$, yani karesel büyür.

Döngü yönünün sınıfı değiştirdiğini düşünmek Sabit adımla aynı aralığı ters yönde geçmek iterasyon sayısının derecesini değiştirmez.

10. Kavram sözlüğü

English term	Türkçe açıklama
Loop Variable	Döngünün ilerleyişini kontrol eden değişken
Incrementing Loop	Kontrol değişkeni artan döngü
Decrementing Loop	Kontrol değişkeni azalan döngü
Constant Step	Her iterasyonda aynı sabit miktarda artış veya azalış
Independent Nested Loops	Sınırları birbirinden bağımsız iç içe döngüler
Triangular Loop	İç iterasyon sayısı dış indekse bağlı olarak artan/azalan yapı
Cumulative Sum	Değerlerin adım adım birikimli olarak toplanması
Trace	Değişken değerlerini iterasyon iterasyon izleme yöntemi

11. Kendini sına

1. $i += 5$ kullanan ve $i < n$ koşuluyla çalışan loop neden $O(n)$ 'dir?
2. $j < i$ koşullu nested loop'ta neden frekansları çarpmak yerine toplarız?
3. $1 + 2 + \dots + (n - 1)$ toplamının complexity değeri nedir?
4. $p += i$ örneği neden n değil yaklaşık $\sqrt{n}$ kez çalışır?
5. n^2 ile $n^2/2$ neden aynı asimptotik sınıftadır?

Kısa cevaplar

1. Yaklaşık $n/5$ iterasyon vardır ve sabit $1/5$ büyüme derecesini değiştirmez.
2. İç loop uzunluğu her i için farklıdır; toplam iş $0 + 1 + \dots + (n-1)$ olur.
3. $O(n^2)$; sıkı olarak $\theta(n^2)$.
4. k iterasyonda $p \approx k^2/2$ olur; $p \approx n$ için $k \approx \sqrt{2n}$ gerekir.
5. Aralarındaki $1/2$ sabit çarpanı büyük girdilerde büyüme derecesini değiştirmez.

Sonraki ders: **07 – Time Complexity Example #2.**

07 – Time Complexity Example #2

Video: 1.5.2 Time Complexity Example #2 (14:13)

Bu videonun ana fikri Loop variable sabit miktarda artmak yerine her adımda çarpılıyor veya bölünüyorsa iterasyon sayısı çoğunlukla $\log$ arithmic olur. Analizin güvenilir yöntemi, k iterasyon sonraki değeri yazıp durma koşuluyla eşitlemektir.

Bu derste şu kalıplar incelenir:

- $i *= c$ ve $i /= c$: $\theta(\log_c n)$
- $i * i \leq n$: $\theta(\sqrt{n})$
- Ardışık loop'lar: maliyetler toplanır
- Nested loop'lar: uygun durumlarda maliyetler çarpılır
- $\log \log n$ üreten iki aşamalı yapı

1. Multiplicative Increment

```
for i <- 1; i < n; i <- i * 2  
  work()
```

i değerleri:

1, 2, 4, 8, 16, ...

k iterasyon sonra:

$i = 2^k$

Loop, $i \geq n$ olduğunda biter:

$2^k \geq n$
 $k \geq \log_2 n$

Dolayısıyla iterasyon sayısı yaklaşık $\log_2 n$ 'dir:

Time Complexity = $\theta(\log n)$

Ceiling ayrıntısı n tam olarak 2'nin kuvveti değilse sonuç tam sayı çıkmaz. Örneğin:

- $n = 8$: $\log_2 8 = 3$
- $n = 10$: $\log_2 10 \approx 3.322$, fakat dört çarpma gerekir

Bu loop için iterasyon sayısı sınır koşuluna bağlı olarak $\lceil \log_2 n \rceil$ biçimindedir. Complexity yazarken ceiling sonucu değiştirmez:

$\theta(\lceil \log_2 n \rceil) = \theta(\log n)$

2. Logarithm Base

```
for i <- 1; i < n; i <- i * 3  
  work()
```

k adım sonra $i = 3^k$ olur:

$k = \log_3 n$

Asimptotik gösterimde logaritmanın tabanı sabit çarpan farkı yaratır:

$$\log_3 n = \log_2 n / \log_2 3$$

$1 / \log_2 3$ sabit olduğu için:

$$\theta(\log_2 n) = \theta(\log_3 n) = \theta(\log n)$$

Taban, kesin iterasyon sayısında önemlidir; Big O veya Theta sınıfında çoğunlukla yazılmaz.

3. Repeated Division

```
i <- n
while i >= 1
  work()
  i <- i / 2
```

Değerler:

$n, n/2, n/4, n/8, \dots$

k iterasyon sonra:

$$i = n / 2^k$$

Durma sınırında:

$$\begin{aligned} n / 2^k &\approx 1 \\ n &\approx 2^k \\ k &\approx \log_2 n \end{aligned}$$

Sonuç:

$$\text{Time Complexity} = \theta(\log n)$$

Her adımda sabit tabanla çarpma ve aynı tabanla bölme, ters yönlü olsalar da aynı logarithmic büyümeyi üretir.

4. Square-root Loop

```
i <- 1
while i * i <= n
  work()
  i <- i + 1
```

Loop şu koşula kadar devam eder:

$$i^2 \leq n$$

Her iki tarafın karekökü alınır:

$$i \leq \sqrt{n}$$

i birer birer arttığı için:

$$\text{Time Complexity} = \theta(\sqrt{n})$$

Burada loop variable linearly artar; fakat hedef sınır n değil $\sqrt{n}$ 'dir. Sadece $i++$ görmek $\theta(n)$ demek için yeterli değildir.

5. Sequential Loops

```
for i <- 0 to n - 1
  workA()

for j <- 0 to n - 1
  workB()
```

Loop'lar iç içe değil, ardışıktır:

```
T(n) = n + n = 2n
Time Complexity =  $\theta(n)$ 
```

Toplama kuralı Ardışık bölümlerde maliyetler toplanır; toplamın en hızlı büyüyen terimi baskın olur:

```
 $\theta(n) + \theta(n^2) = \theta(n^2)$ 
 $\theta(\log n) + \theta(n) = \theta(n)$ 
```

Nested loop'ta ise bir bölüm diğerinin her iterasyonunda tekrarlandığından çoğu zaman çarpım ortaya çıkar.

6. Log of Log n örneği

Videodaki iki aşamalı fikri açık pseudocode ile yazalım:

```
p <- 0
for i <- 1; i < n; i <- i * 2
  p <- p + 1

for j <- 1; j < p; j <- j * 2
  work()
```

İlk loop:

```
p =  $\theta(\log n)$ 
```

İkinci loop, j değerini p'ye ulaşana kadar ikiyle çarpıp:

```
work() frequency =  $\theta(\log p)$ 
                  =  $\theta(\log(\log n))$ 
```

Önemli düzeltme: statement ile bütün kod aynı değildir İkinci loop içindeki `work()` gerçekten $\theta(\log \log n)$ kez çalışır. Ancak iki loop'un birlikte toplam süresi:

```
T(n) =  $\theta(\log n) + \theta(\log \log n)$ 
      =  $\theta(\log n)$ 
```

Yani soru "work() kaç kez çalışır?" ise $\theta(\log \log n)$; "bütün kodun complexity değeri nedir?" ise $\theta(\log n)$ cevabı verilir.

7. Linear x Logarithmic Nested Loop

```
for i <- 0 to n - 1
  for j <- 1; j < n; j <- j * 2
    work()
```

- Dış loop: n kez
- İç loop: Her dış iterasyonda $\log_2 n$ kez

Toplam:

```
T(n) = n * log2 n  
Time Complexity =  $\theta(n \log n)$ 
```

$n \log n$, Merge Sort, Heap Sort ve ortalama durumdaki birçok verimli sıralama algoritmasında karşımıza çıkar.

8. Genel çözüm şablonu

Şu loop için:

```
x <- x0  
while x < n  
    x <- x * c
```

1. k iterasyon sonraki değeri yaz: $x = x_0 c^k$
2. Durma sınırına eşitle: $x_0 c^k \approx n$
3. Çöz:

```
c^k ≈ n/x0  
k ≈ logc(n/x0)
```

x_0 ve $c > 1$ sabitse sonuç $\theta(\log n)$ olur.

9. Kalıpların özeti

Code pattern	İterasyon	Complexity
i++, sınır n	n	$\theta(n)$
i *= 2, sınır n	$\log_2 n$	$\theta(\log n)$
i *= 3, sınır n	$\log_3 n$	$\theta(\log n)$
i /= 2, başlangıç n	$\log_2 n$	$\theta(\log n)$
i++, koşul $i^2 \leq n$	$\sqrt{n}$	$\theta(\sqrt{n})$
Ardışık iki linear loop	n + n	$\theta(n)$
Linear dış, logarithmic iç	n log n	$\theta(n \log n)$

10. Sık yapılan hatalar i *= 2 loop'unu linear sanmak Değer her adımda sabit eklenmiyor, katlanıyor; hedefe logarithmic sayıda adımda ulaşır.

Logarithm base'i complexity sınıfı sanmak Sabit tabanlar yalnızca sabit katsayı farkı oluşturur.

Ardışık loop'ları çarpmak Ardışık bölümlerin maliyetleri toplanır; nested bölümlerde tekrarlanma nedeniyle çarpım oluşabilir.

Tek bir statement'ın frequency değeriyle bütün kodu karıştırmak work() $\theta(\log \log n)$ kez çalışırken onu hazırlayan ilk loop $\theta(\log n)$ sürebilir.

11. Kendini sına

1. $i *= 4$ ve $i < n$ loop'unun complexity değeri nedir?
2. $i /= 3$ ile n'den 1'e inen loop kaç kez çalışır?
3. İki ardışık $\theta(n)$ loop neden $\theta(n^2)$ değildir?
4. $i*i \leq n$ koşulu neden $\theta(\sqrt{n})$ üretir?
5. Yukarıdaki iki aşamalı örnekte work() ile bütün kodun complexity değerleri nelerdir?

Kısa cevaplar

1. $\theta(\log_4 n)$, yani $\theta(\log n)$.
2. Yaklaşık $\log_3 n$, yani $\theta(\log n)$ kez.
3. Maliyetler $n+n=2n$ olarak toplanır; iç içe tekrarlanmaz.

4. Koşulun en büyük geçerli i değeri yaklaşık $\sqrt{n}$ 'dir ve i birer artar.
5. `work()` frequency $\theta(\log \log n)$; bütün kod $\theta(\log n)$.

Sonraki ders: **08 – Time Complexity of While and If.**

08 – Time Complexity of While and If

Video: 1.5.3 Time Complexity of While and if #3 (21:54)

Bu videonun ana fikri Bir loop'un Time Complexity değeri for, while veya do-while yazılmasına göre değil; başlangıç, koşul, güncelleme ve gövdenin kaç kez çalıştığına göre belirlenir. if statement ise seçilen branch'e bağlı olarak farklı çalışma süreleri oluşturabilir, fakat her if otomatik olarak best case ve worst case üretmez.

1. Loop türleri ve semantics While Loop

```
while condition
  body
```

condition önce kontrol edilir. Başlangıçta false ise body hiç çalışmaz. Bu nedenle while, pre-test loop olarak adlandırılır.

Do-While Loop

```
do
  body
while condition
```

Koşul gövdeden sonra kontrol edilir. Başlangıçta false olsa bile body en az bir kez çalışır; bu bir post-test looptur.

Repeat-Until Loop

```
repeat
  body
until condition
```

Bu da en az bir kez çalışır fakat mantığı do-while'in tersidir:

- do-while: Koşul true olduğu sürece devam et.
- repeat-until: Koşul true olduğunda dur.

Modern For Loop C benzeri dillerde:

```
for initialization; condition; update
  body
```

Bu yapı genel bir while loop'a dönüştürülebilir:

```
initialization
while condition
  body
  update
```

Bu nedenle C'de "for loop her zaman linear çalışır" denemez. Update bölümü $i++$, $i*=2$ veya başka bir ifade olabilir.

2. Linear While Loop

```
i <- 0
while i < n
  work()
  i <- i + 1
```

Frekanslar:

Statement	Frequency
<code>i <- 0</code>	1
<code>i < n</code>	$n + 1$
<code>work()</code>	n
<code>i <- i + 1</code>	n

Birer unit cost verilirse:

$$T(n) = 1 + (n+1) + n + n \\ = 3n + 2$$

Sonuç:

$$\text{Time Complexity} = \theta(n)$$

Aynı işlem for ile yazıldığında da aynı frequency ve complexity elde edilir. Syntax değişir, yapılan iş değişmez.

3. Multiplicative While Loop

```
a <- 1
while a < b
  work()
  a <- a * 2
```

k iterasyon sonra:

$$a = 2^k$$

Durma sınırı:

$$2^k \geq b \\ k \geq \log_2 b$$

b input size olarak adlandırılırsa:

$$\text{Time Complexity} = \theta(\log n)$$

Eşdeğer for biçimi:

```
for a <- 1; a < b; a <- a * 2
  work()
```

Yapının adının değil, variable'ın büyüme biçiminin belirleyici olduğunu görürüz.

4. Dividing While Loop

```
i <- n
while i > 1
  work()
  i <- i / 2
```

i her adımda yarıya iner:

```
n, n/2, n/4, ..., 1
```

Bu nedenle:

```
Time Complexity =  $\theta(\log n)$ 
```

5. Cumulative Growth ile Square Root

```
i <- 1
k <- 1
while k < n
  k <- k + i
  i <- i + 1
```

m iterasyon sonra k, yaklaşık doğal sayıların toplamıdır:

```
k ≈ 1 + 2 + 3 + ... + m
    = m(m+1)/2
```

Loop $k \geq n$ olduğunda durur:

```
m(m+1)/2 ≈ n
m2 ≈ 2n
m ≈  $\sqrt{2n}$ 
```

Sonuç:

```
Time Complexity =  $\theta(\sqrt{n})$ 
```

Bu örnekte $i++$ bulunmasına rağmen sınırı kontrol eden asıl variable k 'dir. Analizde condition'daki variable takip edilmelidir.

6. Repeated-subtraction GCD

```
Algorithm GCD(m, n)
  while m != n
    if m > n
      m <- m - n
    else
      n <- n - m
  return m
```

Bu yöntem büyük sayıdan küçük olanı tekrar tekrar çıkarır. Sayılar eşitlendiğinde ortak değer $\text{gcd}(m,n)$ olur.

Örnekler

```
m = 6, n = 3
(6,3) -> (3,3)
```

Bir iterasyon.

```
m = 16, n = 2
(16,2) -> (14,2) -> ... -> (2,2)
```

Yedi iterasyon.

Best ve worst davranışı

- Başlangıçta $m = n$ ise loop gövdesi 0 kez çalışır: $\theta(1)$ toplam kontrol maliyeti.

- Sayılardan biri 1, diğeri N ise yaklaşık $N-1$ çıkarma gerekir: $\Theta(N)$.

Dolayısıyla en büyük giriş değeri N kabul edilirse bu repeated-subtraction sürümünün worst-case süresi $\Theta(N)$ olabilir. Modulo kullanan Euclidean Algorithm çok daha hızlıdır ve ileride ayrı analiz edilir.

7. If Statement ve branch maliyeti

```
if n < 5
    print(n)
else
    for i <- 0 to n - 1
        work()
```

İki branch'in maliyeti farklıdır:

- $n < 5$: $\Theta(1)$
- $n \geq 5$: $\Theta(n)$

Bu kod için girdi koşuluna bağlı farklı sürelerden söz edebiliriz. Ancak dikkat: $n < 5$ zaten sabit bir aralık olduğundan buradaki case ayrımı, her algoritmada kullanılan klasik input dağılımı analizinin basit bir örneğidir.

Her if, best/worst case oluşturmaz

```
if n > 5
    for i <- 0 to n - 1
        work()
```

Sorunun domain'i yalnızca $n > 5$ girdileri içeriyorsa algoritma her geçerli girdide linear çalışır. Sırf if yazılmış olması otomatik olarak farklı case'ler olduğu anlamına gelmez.

Başka bir örnek:

```
if condition
    constantWorkA()
else
    constantWorkB()
```

Her iki branch de $\Theta(1)$ ise seçim değişse bile complexity sınıfı değişmez.

8. Conditional analiz yöntemi

Bir if/else yapısı için:

1. Condition maliyetini belirle.
2. Her branch'i ayrı analiz et.
3. Hangi girdilerin hangi branch'i seçtiğini tanımla.
4. İstenen case'e göre sonucu raporla.

```
Worst-case = condition + max(branch costs)
Best-case  = condition + min(branch costs)
```

Ancak average-case için branch olasılıkları gerekir:

```
Expected cost = P(A) * cost(A) + P(B) * cost(B)
```

Olasılık modeli verilmeden gerçek bir average-case sonucu çıkarılamaz.

9. Termination kontrolü

C benzeri for veya while içinde update serbest olduğundan algoritmanın gerçekten durduğu doğrulanmalıdır.

```
i <- 1
while i < n
  i <- i - 1
```

Pozitif n için i hedeften uzaklaşır ve loop bitmez. Böyle bir yapı için sonlu Time Complexity sınıfı vermeden önce termination sorunu çözülmelidir.

Kontrol listesi:

- Update condition'ı false yapmaya yaklaşıyor mu?
- Variable overflow veya integer division ilerlemeyi durdurabilir mi?
- Bütün branch'lerde ilerleme var mı?

10. Sık yapılan hatalar For görünce $O(n)$ demek Modern for loop'un update ve condition bölümleri keyfi olabilir. $i*=2$, $\theta(\log n)$ üretir.

Do-while ile while'ı aynı sanmak Asimptotik sınıfları çoğu zaman aynı olsa da do-while gövdeyi en az bir kez çalıştırır.

If görünce mutlaka best/worst case yazmak Branch'lerin maliyetleri ve geçerli girdiler incelenmeden case analizi yapılamaz.

GCD örneğini modern Euclidean Algorithm ile karıştırmak Videodaki yöntem modulo değil repeated subtraction kullanır; complexity değerleri farklıdır.

11. Kavram sözlüğü

English term	Türkçe açıklama
Pre-test Loop	Koşulu gövdeden önce kontrol eden loop
Post-test Loop	Koşulu gövdeden sonra kontrol eden loop
Repeat-Until	Koşul true olduğunda duran post-test loop
Branch	Conditional statement içindeki alternatif yürütme yolu
Best Case	Belirlenen input size için en az maliyetli geçerli durum
Worst Case	Belirlenen input size için en yüksek maliyetli geçerli durum
Average Case	Bir olasılık dağılımına göre beklenen maliyet
Repeated Subtraction	Büyük değerden küçüğü eşitlik oluşana kadar çıkarma yöntemi

12. Kendini sinama

1. while ile do-while arasındaki temel execution farkı nedir?
2. C benzeri bir for loop neden otomatik olarak $\theta(n)$ değildir?
3. $k += i$; $i++$ yapısı neden yaklaşık $\sqrt{n}$ iterasyon üretir?
4. Repeated-subtraction GCD'nin worst case'i hangi tür girdide linear olur?
5. Her if neden farklı complexity case'leri üretmez?

Kısa cevaplar

1. while başlangıçta hiç çalışmayabilir; do-while gövdeyi en az bir kez çalıştırır.
2. Condition ve update bölümleri linear ilerlemek zorunda değildir.
3. m iterasyonda biriken değer yaklaşık $m^2/2$ olur; n 'e ulaşmak için $m \approx \sqrt{2n}$ gerekir.
4. Değerlerden biri 1, diğeri N gibi büyük olduğunda yaklaşık N çıkarma gerekir.
5. Branch'ler aynı sınıfta olabilir veya bazı branch'ler geçerli input domain'inde hiç seçilmeyebilir.

Sonraki ders: **09 – Classes of Functions.**

09 – Classes of Functions

Video: 1.6 Classes of Functions (3:10)

Bu videonun ana fikri Algoritmaların exact Time Function değerleri farklı olsa da aynı büyüme davranışını gösteren fonksiyonları ortak complexity class altında toplarız.

Örneğin:

$$\begin{aligned}T_1(n) &= 2n + 3 \\T_2(n) &= 500n + 700 \\T_3(n) &= n/5000 + 6\end{aligned}$$

Üçünün katsayıları çok farklıdır; fakat her biri büyük n için n ile orantılı büyür. Bu nedenle hepsi Linear class içindedir.

1. Neden class kullanıyoruz? Exact function şu ayrıntıları içerebilir:

- Statement'lara verilen unit cost
- Dil ve compiler farkları
- Sabit başlangıç maliyetleri
- Donanım ayrıntıları

Algoritma seçerken çoğunlukla asıl soru şudur:

Input size büyüdüğünde kaynak ihtiyacı hangi hızla büyüyor?

Complexity class, bu büyüme biçimini kısa ve karşılaştırılabilir hâle getirir.

2. Constant – $O(1)$

$$\begin{aligned}T(n) &= 2 \\T(n) &= 5000\end{aligned}$$

Maliyet input size n 'e bağlı değildir:

$$\begin{aligned}\text{Complexity} &= O(1) \\ \text{Class} &= \text{Constant}\end{aligned}$$

Örnekler:

- Array'de bilinen bir indeksi okumak
- İki scalar değerini yerini değiştirmek
- Sabit sayıda arithmetic operation yapmak

$O(1)$, işlemin mutlaka hızlı olduğu anlamına gelmez. 5000 unit, 2 unit'ten yavaştır; ikisi de input büyüdüğünde artmaz.

3. Logarithmic – $O(\log n)$

$$\begin{aligned}T(n) &= \log_2 n \\T(n) &= 7 \log_3 n + 20\end{aligned}$$

Input her adımda sabit oranda küçülüyor veya aranan aralık yarıya iniyorsa sık görülür:

$$\begin{aligned}\text{Complexity} &= O(\log n) \\ \text{Class} &= \text{Logarithmic}\end{aligned}$$

Örnekler:

- Binary Search
- Balanced search tree yüksekliği
- $i \neq 2$ veya $i \neq 2$ kalıbı

Input iki katına çıktığında eklenen iş miktarı sabittir. Örneğin $\log_2(2n) = \log_2 n + 1$.

4. Linear – $O(n)$

$$T(n) = an + b$$

Burada $a > 0$ ve b sabittir:

Complexity = $O(n)$
Class = Linear

Örnekler:

- Array elemanlarının toplamı
- Unsorted array içinde Linear Search
- Bir linked list'i baştan sona gezmek

Input yaklaşık iki katına çıktığında baskın iş miktarı da iki katına çıkar.

5. Linearithmic – $O(n \log n)$ Video ana listede kısa geçse de karşılaştırmada kullanılacak önemli class'tır:

Complexity = $O(n \log n)$
Class = Linearithmic

Örnekler:

- Merge Sort
- Heap Sort
- Ortalama durumda Quick Sort

Genellikle n eleman üzerinde $\log n$ seviyeli çalışma veya $\log n$ maliyetli işlemin n kez yapılmasıyla oluşur.

6. Quadratic – $O(n^2)$

$$T(n) = an^2 + bn + c$$

En yüksek dereceli terim n^2 :

Complexity = $O(n^2)$
Class = Quadratic

Örnekler:

- $n \times n$ matrix addition
- Bütün eleman çiftlerini karşılaştırma
- Basit nested loop kalıpları

Input iki katına çıktığında baskın iş yaklaşık dört katına çıkar.

7. Cubic – $O(n^3)$

$$T(n) = an^3 + bn^2 + cn + d$$

Complexity = $O(n^3)$
Class = Cubic

Klasik $n \times n$ matrix multiplication temel örnektir. Input dimension iki katına çıktığında işlem miktarı yaklaşık sekiz katına çıkar.

8. Polynomial – $O(n^k)$ k sabit pozitif sayı olmak üzere:

$O(n)$, $O(n^2)$, $O(n^3)$, ..., $O(n^k)$

fonksiyonları genel olarak Polynomial class ailesindedir. k büyük olsa bile sabitse sınıf polynomial'dır.

Örneğin $O(n^{100})$ pratikte çok pahalıdır ama teknik olarak hâlâ polynomial büyümedir.

9. Exponential – $O(c^n)$ $c > 1$ sabit olduğunda:

2^n , 3^n , 1.5^n

Exponential büyümedir:

Complexity = $O(c^n)$
Class = Exponential

Örnek bağlamlar:

- Bütün subset'leri denemek: yaklaşık 2^n
- Bazı naive recursive algoritmalar
- Birden fazla seçeneğin her seviyede dallandığı exhaustive search

Input'a yalnızca 1 eklemek maliyeti sabit bir oranla çarpabilir.

n^n hakkında teknik not Video n^n ifadesini de exponential örneklerle birlikte anar. Standart terminolojide c^n için taban c sabittir; n^n ise tabanı da büyüdüğü için 2^n gibi fixed-base exponential fonksiyonlardan daha hızlı, super-exponential bir büyümedir.

10. Özet tablo

Function	Class	Input 2 katına çıkınca yaklaşık etki
1	Constant	Değişmez
$\log n$	Logarithmic	Sabit miktar eklenir
n	Linear	2 kat
$n \log n$	Linearithmic	2 kattan biraz fazla
n^2	Quadratic	4 kat
n^3	Cubic	8 kat
2^n	Exponential	Önceki değerin karesi

Son satırın nedeni:

$$2^{(2n)} = (2^n)^2$$

11. Exact değer ile growth class farkı $1000n$ ile n^2 karşılaştırmasında küçük girdilerde n^2 daha düşük olabilir:

$$n = 10: \quad 1000n = 10000, \quad n^2 = 100$$

Fakat $n > 1000$ olduğunda $n^2 > 1000n$ olur ve fark büyür. Complexity class, küçük input'lardaki exact yarıştan çok uzun vadeli büyümeyi anlatır.

Bu nedenle performans kararında iki bilgi birlikte değerlidir:

- Asymptotic Complexity: Ölçeklenme davranışı
- Gerçek ölçüm ve sabitler: İlgilendiğimiz input aralığındaki pratik performans

12. Sık yapılan hatalar Büyük sabiti daha yüksek class sanmak $5000n$, katsayısı büyük olsa da linear'dır; n^2 değildir.

$O(1)$ 'i "çok hızlı" diye çevirmek Yalnızca input size'dan bağımsız olduğunu söyler.

Her üs içeren fonksiyona exponential demek n^2 , polynomial; 2^n , exponential'dır. Değişkenin tabanda mı üste mi olduğuna dikkat et.

n^n ile 2^n 'i aynı hızda sanmak İkisi de çok hızlı büyür, fakat n^n fixed-base exponential'dan daha hızlıdır.

13. Kavram sözlüğü

English term	Türkçe açıklama
Complexity Class	Aynı asimptotik büyüme davranışına sahip fonksiyonlar grubu
Constant	Input size'dan bağımsız büyüme
Logarithmic	Logaritma hızında büyüme
Linear	Input ile doğru orantılı büyüme
Linearithmic	$n \log n$ biçimindeki büyüme
Quadratic	n^2 baskın terimli büyüme
Cubic	n^3 baskın terimli büyüme
Polynomial	Sabit k için n^k türü büyüme
Exponential	Sabit $c > 1$ için c^n türü büyüme
Super-exponential	Fixed-base exponential'dan daha hızlı büyüme; örneğin n^n

14. Kendini sınamaya

1. $700n + 500$ hangi class'a aittir?
2. n^2 ile 2^n arasındaki yapısal fark nedir?
3. $O(1)$ neden "bir işlem" anlamına gelmez?
4. Input iki katına çıktığında n^3 maliyeti yaklaşık kaç katına çıkar?
5. n^{100} polynomial mıdır?

Kısa cevaplar

1. Linear, yani $O(n)$.
2. İlkinde variable tabanda, sabit üs vardır; ikincisinde sabit tabanda, variable üs vardır.
3. Herhangi bir sabit miktarda işlem olabilir; önemli olan n ile büyümemesidir.
4. Yaklaşık 8 katına.
5. Evet; 100 sabit bir üstür.

Sonraki ders: **10 – Compare Classes of Functions.**

10 – Compare Classes of Functions

Video: 1.7 Compare Class of Functions (5:11)

Bu videonun ana fikri Complexity class'ları büyük input değerlerindeki büyüme hızlarına göre sıralayabiliriz. Standart sıralamanın önemli bir bölümü şöyledir:

$$1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < \dots < 2^n < 3^n < n^n$$

Bu sembol dizisi her küçük n için sayısal eşitsizlik iddiası değildir. Anlamı, sağdaki fonksiyonun yeterince büyük n değerlerinde soldakinden daha hızlı büyümesidir.

1. Increasing order of growth Sabit $k > 1$ ve $c > 1$ için genel düzen:

$$\begin{aligned} &O(1) \\ &< O(\log n) \\ &< O(n^a) \quad \text{where } 0 < a < 1 \\ &< O(n) \\ &< O(n \log n) \\ &< O(n^2) \\ &< O(n^3) \\ &< O(n^k) \\ &< O(c^n) \\ &< O(n^n) \end{aligned}$$

$\sqrt{n}$, n^a ailesinde $a = 1/2$ örneğidir.

2. Sayısal karşılaştırma Tabanı 2 olan logaritmayı kullanalım:

n	$\log_2 n$	n	n^2	2^n
1	0	1	1	2
2	1	2	4	4
4	2	4	16	16
8	3	8	64	256
16	4	16	256	65,536
32	5	32	1,024	4,294,967,296

Küçük değerlerde bazı fonksiyonlar eşit olabilir:

$$\begin{aligned} n = 2: n^2 &= 2^n = 4 \\ n = 4: n^2 &= 2^n = 16 \end{aligned}$$

Ancak n büyüdüğünde 2^n , n^2 'yi hızla geride bırakır.

3. Crossover point İki fonksiyon küçük input'larda beklenen sıranın tersinde olabilir:

$$\begin{aligned} f(n) &= 1000n \\ g(n) &= n^2 \end{aligned}$$

$n < 1000$ için $n^2 < 1000n$; $n > 1000$ için $n^2 > 1000n$.

Eşit oldukları $n = 1000$ değeri bir crossover pointtir. Asimptotik karşılaştırma, belirli bir başlangıçtan sonraki kalıcı davranışla ilgilenir.

Bu yüzden:

Daha düşük complexity class, her küçük input'ta mutlaka daha hızlı gerçek program demek değildir; daha iyi ölçeklenme demektir.

4. Logarithmic neden Linear'dan küçüktür? Oranı inceleyelim:

$$\log n / n$$

$n \rightarrow \text{infinity}$ iken bu oran 0'a yaklaşır. Yani $\log n$, n 'in giderek daha küçük bir kesri hâline gelir:

$$\log n = o(n)$$

Benzer biçimde:

$$\begin{aligned} n / n^2 &= 1/n \rightarrow 0 \\ n^2 / n^3 &= 1/n \rightarrow 0 \end{aligned}$$

Soldaki fonksiyon sağdakine göre asimptotik olarak daha yavaş büyür.

5. $n \log n$ neden n ile n^2 arasındadır? $\log n$ büyüyen bir değer olduğu için:

$$n \log n > n$$

yeterince büyük n için geçerlidir. Diğer taraftan $\log n < n$ olduğu için:

$$n \log n < n * n = n^2$$

Dolayısıyla:

$$n < n \log n < n^2$$

6. Polynomial ve Exponential Her sabit k ve $c > 1$ için:

$$n^k < c^n$$

yeterince büyük n değerlerinde geçerli olur. $k = 100$ gibi çok büyük bir sabit olsa bile exponential fonksiyon sonunda polynomial fonksiyonu geçer.

Bu, 2^n 'in her küçük n için n^{100} den büyük olduğu anlamına gelmez. Crossover değeri çok büyük olabilir; asimptotik ifade yalnızca sonunda ve kalıcı olarak geçeceğini söyler.

7. Exponential fonksiyonların kendi sırası

$$2^n < 3^n < 10^n$$

çünkü:

$$3^n / 2^n = (3/2)^n \rightarrow \text{infinity}$$

Bu nedenle logaritmanın tabanı complexity class'ında sabit katsayı farkıyla, exponential fonksiyonun tabanı büyüme hızını gerçekten değiştirir:

$$\begin{aligned} \log_2 n &= \theta(\log_3 n) \\ 2^n &\neq \theta(3^n) \end{aligned}$$

8. Practical scale örneği Bir bilgisayar saniyede yaklaşık 10^8 temel işlem gerçekleştirebilir. Katsayıları yok sayan kaba sezgiyle:

- $n = 10^6$ için n : yaklaşık bir milyon işlem

- $n \log_2 n$: yaklaşık yirmi milyon işlem
- n^2 : 10^{12} işlem
- 2^n : hesaplanamayacak kadar büyük

Bu, class seçiminin büyük input'larda neden mikro-optimizasyondan çok daha etkili olabildiğini gösterir.

9. Hangi class "iyi"dir? Tek başına class etiketi yeterli değildir:

- n en fazla 20 ise 2^n bazen uygulanabilir olabilir.
- n milyonlar düzeyindeyse n^2 fazla pahalı olabilir.
- Gerçek zamanlı sistemde $n \log n$ bile gecikme hedefini aşabilir.
- Küçük, sabit input'larda yüksek katsayılı $O(n)$ yerine optimize edilmiş $O(n^2)$ kod daha hızlı ölçülebilir.

Doğru soru şudur:

Beklenen input aralığı ve kaynak sınırları için bu büyüme kabul edilebilir mi?

10. Sık yapılan hatalar Sıralamanın $n=1$ 'den itibaren kesin olduğunu düşünmek

Asimptotik sıra, yeterince büyük input'lardaki büyümeyi anlatır.

log tabanı ile exponential tabanını aynı şekilde ele almak Log tabanı yalnızca sabit çarpan; exponential tabanı farklı growth rate oluşturur.

n^k ifadesinde k 'nin n 'e bağlı olabileceğini unutmak "Polynomial" denmesi için k sabit olmalıdır. n^n polynomial değildir.

Daha düşük class'ın her ölçümde hızlı olacağını sanmak Constants, cache, compiler ve input aralığı küçük değerlerde sonucu değiştirebilir.

11. Kavram sözlüğü

English term	Türkçe açıklama
Order of Growth	Input büyürken fonksiyonun artış hızı
Increasing Order	Fonksiyonların yavaştan hızlı büyümeye sıralanması
Crossover Point	İki fonksiyonun eşitlenip görece sırasının değiştiği input
Asymptotic Dominance	Bir fonksiyonun yeterince büyük input'larda diğerinden kalıcı hızlı büyümesi
Little-o	Bir fonksiyonun diğerine göre kesin olarak daha yavaş büyüdüğünü belirten ilişki
Scalability	Input ve yük büyürken sistemin maliyet artışını yönetebilme özelliği

12. Kendini sına

1. $n \log n$, neden n ile n^2 arasındadır?
2. n^{100} , 2^n nden her zaman küçük müdür?
3. $\log_2 n$ ile $\log_3 n$ neden aynı Theta class'ındadır?
4. 2^n ile 3^n neden aynı Theta class'ında değildir?
5. Daha düşük class neden her küçük input'ta daha hızlı program garantisi vermez?

Kısa cevaplar

1. $\log n$ büyür ama n den yavaş büyür; dolayısıyla $n < n \log n < n^2$ olur.
2. Hayır; fakat yeterince büyük n den sonra 2^n daha büyük olur.
3. Change-of-base formülü aralarında yalnızca sabit çarpan olduğunu gösterir.
4. Oranları $(3/2)^n$ sınırsız büyür; sabit katsayıyla birbirlerini sınırlayamazlar.

5. Constants, sistem ayrıntıları ve crossover point küçük input sonuçlarını etkileyebilir.

Sonraki ders: **11 – Asymptotic Notations: Big O, Omega and Theta #1.**

11 – Asymptotic Notations: Big O, Omega and Theta #1

Video: 1.8.1 Asymptotic Notations Big Oh - Omega - Theta #1 (15:46)

Bu videonun ana fikri Asymptotic Notations, bir fonksiyonun büyük input değerlerindeki büyümesini sabit katsayı ve düşük dereceli ayrıntılardan arındırarak ifade eder.

Üç temel notation:

- Big O: Asymptotic upper bound
- Big Omega (Ω): Asymptotic lower bound
- Theta (Θ): Asymptotic tight bound

Önemli terminoloji düzeltmesi: Videoda Theta için “average bound” ifadesi kullanılıyor. Standart matematiksel karşılığı **tight bound**dur. Theta, average-case anlamına gelmez.

1. Neden asymptotic notation kullanıyoruz? Bir algoritmanın exact time function’ı şöyle olabilir:

$$T(n) = 7n^2 + 12n + 43$$

Bu formül ayrıntılıdır fakat karşılaştırma yaparken asıl büyümeyi n^2 belirler. Kısa ifade:

$$T(n) = \Theta(n^2)$$

Bu, “ $T(n)$ tam olarak n^2 ’ye eşittir” demek değildir. Yeterince büyük n için $T(n)$ fonksiyonunun pozitif sabit katlarla n^2 ’nin hem altında hem üstünde tutulabildiğini söyler.

2. Matematiksel notation hakkında küçük not Teknik olarak $O(g(n))$, $\Omega(g(n))$ ve $\Theta(g(n))$ birer fonksiyon kümesidir. En doğru yazım:

$$f(n) \in O(g(n))$$

Algoritma kaynaklarında şu yazım da gelenekselleşmiştir:

$$f(n) = O(g(n))$$

Bu notlarda yaygın ikinci biçim kullanılabilir; sembolün sıradan eşitlik gibi simetrik olmadığı unutulmamalıdır.

3. Big O – Upper Bound Formal definition $f(n) = O(g(n))$ denebilmesi için pozitif c ve n_0 sabitleri bulunmalıdır; bütün $n \geq n_0$ değerleri için:

$$0 \leq f(n) \leq c * g(n)$$

Burada:

- $g(n)$: Karşılaştırma yapılan growth function
- c : Dikey ölçekleme yapan pozitif sabit
- n_0 : Eşitsizliğin kalıcı olarak geçerli olmaya başladığı eşik

Big O, fonksiyonun bundan daha hızlı büyümediğini belirten asymptotic upper bound verir.

4. Big O örneği: $f(n) = 2n + 3$ $g(n) = n$ seçelim. $c = 5$, $n_0 = 1$ için:

$$2n + 3 \leq 5n$$

çünkü $n \geq 1$ olduğunda $3 \leq 3n$ dir. Dolayısıyla:

$$2n + 3 = O(n)$$

$c = 5$ tek seçenek değildir. Örneğin uygun daha büyük bir n_0 ile $c = 3$ de kullanılabilir. Definition yalnızca **en az bir** geçerli sabit çifti bulunmasını ister.

$O(n^2)$ de doğru mu? Evet. $n \geq 1$ için:

$$2n + 3 \leq 5n^2$$

Bu nedenle:

$$2n + 3 = O(n^2)$$

Hatta $O(2^n)$ gibi daha gevşek upper bound'lar da bulunabilir. Fakat bunlar fonksiyonun gerçek büyümesini iyi anlatmaz.

Tercih edilen bilgi:

$O(n)$ → doğru ve tight upper bound
 $O(n^2)$ → doğru fakat loose upper bound
 $O(2^n)$ → doğru fakat çok loose upper bound
 $O(\log n)$ → yanlış

Algoritma konuşmalarında "Big O'su nedir?" sorusu çoğu zaman en tight bilinen upper bound'ı ister.

5. Big Omega – Lower Bound Formal definition $f(n) = \Omega(g(n))$ denebilmesi için pozitif c ve n_0 sabitleri bulunmalıdır; bütün $n \geq n_0$ değerleri için:

$$0 \leq c * g(n) \leq f(n)$$

Bu kez $c * g(n)$, fonksiyonun altında kalır. Big Omega, fonksiyonun en az bu hızda büyüdüğünü söyler.

6. Big Omega örneği: $f(n) = 2n + 3$ $g(n) = n$, $c = 2$, $n_0 = 1$ seçersek:

$$2n \leq 2n + 3$$

Dolayısıyla:

$$2n + 3 = \Omega(n)$$

Daha yavaş büyüyen fonksiyonlar da geçerli lower bound olabilir:

$$\begin{aligned} 2n + 3 &= \Omega(\log n) \\ 2n + 3 &= \Omega(1) \end{aligned}$$

Fakat $\Omega(n)$ daha tight ve bilgilendiricidir.

Şu ifade yanlıştır:

$$2n + 3 = \Omega(n^2)$$

Çünkü hiçbir pozitif sabit c , yeterince büyük bütün n değerleri için $cn^2 \leq 2n+3$ eşitsizliğini koruyamaz. Karesel fonksiyon sonunda linear fonksiyonu geçer.

7. Theta – Tight Bound Formal definition $f(n) = \Theta(g(n))$ denebilmesi için pozitif c_1 , c_2 ve n_0 sabitleri bulunmalıdır; bütün $n \geq n_0$ değerleri için:

$$0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$$

Yani $f(n)$, aynı $g(n)$ fonksiyonunun iki pozitif sabit katı arasında sıkışır.

Başka bir ifadeyle:

$$\begin{aligned} f(n) &= \theta(g(n)) \\ \text{iff} \\ f(n) &= O(g(n)) \text{ and } f(n) = \Omega(g(n)) \end{aligned}$$

8. Theta örneği: $f(n) = 2n + 3$ $g(n) = n$ seçelim. $n \geq 1$ için:

$$2n \leq 2n + 3 \leq 5n$$

Burada:

$$\begin{aligned} c_1 &= 2 \\ c_2 &= 5 \\ n_0 &= 1 \end{aligned}$$

Dolayısıyla:

$$2n + 3 = \theta(n)$$

Fakat:

$$\begin{aligned} 2n + 3 &\neq \theta(n^2) \\ 2n + 3 &\neq \theta(\log n) \end{aligned}$$

n^2 upper bound olabilir ama matching lower bound değildir. $\log n$ lower bound olabilir ama matching upper bound değildir. Aynı $g(n)$ iki tarafta da çalışmalıdır.

9. Üç notation'ın birlikte görünümü $f(n) = 2n + 3$ için:

Lower bounds: $1, \log n, \sqrt{n}, n$
Tight bound: n
Upper bounds: $n, n \log n, n^2, n^3, 2^n, \dots$

Burada n , hem lower hem upper bound olduğu için tight bound'dır.

Notation	Söylediği şey	Bu örnekte tight seçim
$O(g(n))$	f, g den asymptotically daha hızlı büyümeyiz	$O(n)$
$\Omega(g(n))$	f, g den asymptotically daha yavaş büyümeyiz	$\Omega(n)$
$\theta(g(n))$	f ile g aynı asymptotic hızda büyür	$\theta(n)$

10. n_0 neden vardır? Asymptotic analysis küçük input değerlerindeki geçici davranışları önemsemez. Eşitsizliğin $n = 0$ veya $n = 1$ de bozulması sorun değildir; yeterli bir n_0 sonrasında kalıcı olması yeterlidir.

Örnek:

$$\begin{aligned} f(n) &= n^2 \\ g(n) &= 1000n \end{aligned}$$

Küçük n için $n^2 < 1000n$ olabilir. Fakat $n \geq 1000$ sonrasında $n^2 \geq 1000n$ olur. n_0 , bu crossover sonrasındaki bölgeyi tanımlar.

11. Constants neden atılabilir?

$$f(n) = 100n$$

$$g(n) = n$$

$c_1 = 100$ ve $c_2 = 100$ seçilirse doğrudan aynı sabit kat görülür:

$$100n = \theta(n)$$

Benzer şekilde:

$$3n^2 + 50n + 700 = \theta(n^2)$$

Yeterince büyük n için düşük dereceli terimler n^2 'nin sabit katı içinde tutulabilir.

12. Big O worst case demek değildir Çok yaygın fakat yanlış eşleştirme:

Big O = Worst Case
 Big Omega = Best Case
 Theta = Average Case

Doğrusu:

- Case analysis, hangi input durumunun incelendiğini söyler.
- Asymptotic notation, o case için elde edilen fonksiyonun bound türünü söyler.

Örneğin Insertion Sort için ayrı ayrı şunlar yazılabilir:

Best-case time = $\theta(n)$
 Worst-case time = $\theta(n^2)$

Burada hem best hem worst case Theta ile ifade edilmiştir. Aynı şekilde worst-case fonksiyon için loose bir $\theta(n^3)$ upper bound yazmak teknik olarak mümkün olsa da tight olan $\theta(n^2)$ daha yararlıdır.

13. "Theta average bound" neden yanlış? Average-case, girdilerin olasılık dağılımına göre expected running time hesabıdır. Theta ise tek bir fonksiyonun iki taraftan aynı growth function ile sınırlanmasıdır.

Average-case -> probability / expectation kavramı
 Theta -> asymptotic tightness kavramı

Bir average-case fonksiyonu $\theta(n \log n)$ olabilir; bir worst-case fonksiyonu da $\theta(n \log n)$ olabilir. Notation, case'in türünü belirlemez.

14. Hızlı polinom kuralı Pozitif leading coefficient'e sahip bir polynomial için en yüksek dereceli terim growth class'ı belirler:

$$f(n) = a_k n^k + a_{(k-1)} n^{(k-1)} + \dots + a_0$$

$a_k > 0$ ise:

$$f(n) = \theta(n^k)$$

Örnekler:

$$7n^3 + 2n^2 + 100 = \theta(n^3)$$

$$0.5n^2 - 20n + 8 = \theta(n^2) \quad // \text{ yeterince büyük } n \text{ için pozitif}$$

$$42 = \theta(1)$$

15. Sık yapılan hatalar Sadece bir n değeri deneyerek bound kanıtlamak Definition, bütün $n \geq n_0$ değerlerinde eşitsizlik ister.

Big O'yu exact class sanmak Linear fonksiyon $O(n^2)$ de olabilir. Tight sınıfı belirtmek için $\Theta(n)$ daha kesindir.

Theta'yı average-case sanmak Theta tight bound; average-case ise olasılıksal case analizidir.

c veya n_0 'ın n'e bağlı olmasına izin vermek c, c_1 , c_2 ve n_0 sabit olmalıdır. $c=n$ seçmek definition'ı karşılamaz.

Eşitsizliğin yönünü karıştırmak

Big O: $f(n) \leq c g(n)$
Big Omega: $f(n) \geq c g(n)$
Theta: $c_1 g(n) \leq f(n) \leq c_2 g(n)$

16. Kavram sözlüğü

English term	Türkçe açıklama
Asymptotic Notation	Büyük input davranışını bounds ile ifade eden gösterim
Upper Bound	Fonksiyonun asymptotically aşmadığı büyüme sınırı
Lower Bound	Fonksiyonun asymptotically altına düşmediği büyüme sınırı
Tight Bound	Aynı growth function ile hem upper hem lower bound
Loose Bound	Doğru fakat gerçek büyümeyi yakın ifade etmeyen sınır
Positive Constant	n'den bağımsız, sıfırdan büyük sabit
Threshold (n_0)	Bound'un bütün sonraki input'larda geçerli olduğu başlangıç değeri
Case Analysis	Best, worst veya average gibi input durumlarına göre analiz

17. Kendini sına

- $3n + 7 = O(n^2)$ ifadesi doğru mudur? Faydalı mıdır?
- $3n + 7 = \Omega(1)$ neden doğrudur?
- $3n + 7 = \Theta(n)$ göstermek için ne gerekir?
- Theta neden average-case anlamına gelmez?
- Big O definition'ındaki n_0 ne işe yarar?
- $5n^2 + 2n + 9$ için tight bound nedir?

Kısa cevaplar

- Doğrudur fakat loose olduğu için $O(n)$ kadar bilgilendirici değildir.
- Fonksiyon yeterince büyük tüm n'lerde pozitif sabit bir değerin üzerinde kalır.
- Pozitif c_1, c_2, n_0 bulup $c_1 n \leq 3n+7 \leq c_2 n$ eşitsizliğini göstermek gerekir.
- Theta bir fonksiyonun tight bound'unu, average-case ise olasılık dağılımına göre beklenen maliyeti anlatır.
- Küçük input'lardaki geçici davranışı dışarıda bırakıp bound'un kalıcı olduğu bölgeyi seçer.
- $\Theta(n^2)$.

Sonraki ders: **12 – Asymptotic Notations #2.**

12 – Asymptotic Notations: Big O, Omega and Theta #2

Video: 1.8.2 Asymptotic Notations - Big Oh - Omega - Theta #2 (10:07)

Bu videonun ana fikri Bu ders Big O, Big Omega ve Theta tanımlarını daha karmaşık fonksiyonlara uygular. Temel amaç yalnızca herhangi bir bound bulmak değil, fonksiyonun gerçek büyümesini mümkün olduğunca yakın ifade eden tight boundu bulmaktır.

Videodaki polynomial ve $n^2 \log n$ örnekleri doğru yöntemi gösterir. factorial bölümünde ise önemli bir matematiksel hata vardır: $n!$ ve $\log(n!)$ için tight bound vardır. Bu notta hem videodaki yaklaşımı hem doğru sonucu ayıracağız.

1. Polynomial örneği

$$f(n) = 2n^2 + 3n + 4$$

Upper bound $n \geq 1$ için:

$$\begin{aligned} 3n &\leq 3n^2 \\ 4 &\leq 4n^2 \end{aligned}$$

Dolayısıyla:

$$\begin{aligned} 2n^2 + 3n + 4 &\leq 2n^2 + 3n^2 + 4n^2 \\ &= 9n^2 \end{aligned}$$

Bu nedenle:

$$f(n) = O(n^2)$$

Geçerli sabitler örneğin $c = 9$, $n_0 = 1$ olabilir.

Lower bound Pozitif terimleri atmak fonksiyonu küçültür:

$$2n^2 + 3n + 4 \geq 2n^2 \geq n^2$$

Dolayısıyla:

$$f(n) = \Omega(n^2)$$

Tight bound Aynı n^2 fonksiyonu hem upper hem lower bound olduğundan:

$$\begin{aligned} n^2 &\leq 2n^2 + 3n + 4 \leq 9n^2 \\ f(n) &= \Theta(n^2) \end{aligned}$$

Genel polynomial kuralı: Leading coefficient pozitifse en yüksek dereceli terim tight bound'u belirler.

2. Mixed function: $n^2 \log n + n$

$$f(n) = n^2 \log n + n$$

$n \geq 2$ için $\log_2 n \geq 1$ ve $n \leq n^2 \log_2 n$ olur. Bu yüzden:

$$n^2 \log n \leq n^2 \log n + n \leq 2n^2 \log n$$

Sonuç:

$$\begin{aligned}f(n) &= O(n^2 \log n) \\f(n) &= \Omega(n^2 \log n) \\f(n) &= \Theta(n^2 \log n)\end{aligned}$$

Bu class, sıralamada n^2 ile n^3 arasındadır:

$$n^2 < n^2 \log n < n^3$$

çünkü $1 < \log n < n$ yeterince büyük n için geçerlidir.

3. Loose bound bulmak neden yeterli değildir? Bir fonksiyon için şu eşitsizlik doğru olabilir:

$$1 \leq f(n) \leq n^n$$

Fakat iki bound arasında muazzam bir boşluk vardır. Bu yalnızca fonksiyonun çok geniş bir bölgede bulunduğunu söyler; gerçek growth rate'i tanımlamaz.

Big O ve Big Omega kullanırken hedef:

- Doğru bir bound bulmak
- Mümkünse bound'u tight hâle getirmek
- Aynı growth function upper ve lower bound olabiliyorsa Theta yazmak

4. Factorial için videodaki bound

$$n! = 1 * 2 * 3 * \dots * n$$

Her çarpan en fazla n olduğu için:

$$n! \leq n * n * \dots * n = n^n$$

Ayrıca $n! \geq 1$:

$$\begin{aligned}n! &= O(n^n) \\n! &= \Omega(1)\end{aligned}$$

Bu ifadeler doğrudur fakat aşırı loostur. Buradan "Theta yoktur" sonucu çıkmaz.

5. Düzeltme: $n!$ için tight bound vardır En tanınmış sonuç Stirling's Approximation'dır:

$$n! \sim \sqrt{2\pi n} * (n/e)^n$$

$\sim$ sembolü oranlarının 1'e yaklaştığını belirtir. Bundan:

$$n! = \Theta(\sqrt{n} * (n/e)^n)$$

elde edilir.

Elbette her fonksiyon kendisiyle aynı class'tadır:

$$n! = \Theta(n!)$$

Ancak bu ifade karşılaştırma açısından yeni bilgi vermez. Stirling formu factorial büyümeyi bilinen temel fonksiyonlarla çok daha açıklayıcı biçimde gösterir.

Factorial'ın sıralamadaki yeri Fixed-base exponential ile n^n arasında büyür:

$$c^n < n! < n^n$$

her sabit $c > 1$ için yeterince büyük n değerlerinde geçerlidir.

6. $\log(n!)$ için videodaki bound

Logarithm product rule:

$$\log(n!) = \log 1 + \log 2 + \dots + \log n$$

Her terim en fazla $\log n$ olduğundan:

$$\log(n!) \leq n \log n$$

Bu, doğru bir upper bound verir:

$$\log(n!) = O(n \log n)$$

Videoda lower bound olarak sabit bir değer kullanılıp tight bound bulunamayacağı söylenir. Bu lower bound doğru fakat gereğinden çok gevşektir.

7. Düzeltme: $\log(n!) = \Theta(n \log n)$

Upper bound zaten bulundu:

$$\log(n!) \leq n \log n$$

Lower bound için toplamın son yarısına bakalım. $k = \lceil n/2 \rceil, \dots, n$ aralığında yaklaşık $n/2$ terim vardır ve her biri en az $\log(n/2)$ dir:

$$\begin{aligned} \log(n!) &= \sum_{k=1}^n \log k \\ &\geq \sum_{k=\lceil n/2 \rceil}^n \log k \\ &\geq (n/2) \log(n/2) \end{aligned}$$

Şimdi:

$$\begin{aligned} (n/2) \log(n/2) &= (n/2) (\log n - \log 2) \\ &= \Theta(n \log n) \end{aligned}$$

Dolayısıyla aynı growth function iki tarafta da vardır:

$$\log(n!) = \Theta(n \log n)$$

Stirling formunun logaritması da aynı sonucu verir:

$$\begin{aligned} \log(n!) &= n \log n - n + \Theta(\log n) \\ &= \Theta(n \log n) \end{aligned}$$

8. Tight bound bulma stratejisi

Bir fonksiyon için:

1. Kolay bir upper bound üret.
2. Kolay ama anlamsız lower bound'la yetinme.
3. Toplam veya çarpımın büyük bölümünü kullanarak güçlü lower bound ara.
4. Upper ve lower bound aynı class'a ulaşıyorsa Theta yaz.

Özellikle sum için bütün terimleri ayrı çözmek gerekmez. Terimlerin sabit oranlı bir bölümünün yeterince büyük olduğunu göstermek çoğu zaman güçlü lower bound üretir.

9. Örneklerin özeti

Function	Tight result
$2n^2 + 3n + 4$	$\theta(n^2)$
$n^2 \log n + n$	$\theta(n^2 \log n)$
$n!$	$\theta(\sqrt{n} (n/e)^n)$
$\log(n!)$	$\theta(n \log n)$

10. Sık yapılan hatalar Aynı bound'u bulamayınca Theta yok sanmak Bu çoğunlukla seçilen bound'ların fazla loose olduğunu gösterir; fonksiyonun tight bound'u olmadığı anlamına gelmez.

1 $\leq n!$ $\leq n^n$ eşitsizliğini tight analiz sanmak Eşitsizlik doğru fakat iki taraf arasındaki boşluk çok büyüktür.

$n!$ ile n^n 'i aynı class sanmak Oranları sabitlerle sınırlanamaz; Stirling formu aradaki exponential farkı gösterir.

$\log(n!)$ ifadesini $\log n$ sanmak $\log(n!)$, n adet $\log$ teriminin toplamıdır ve $\theta(n \log n)$ büyür.

11. Kavram sözlüğü

English term	Türkçe açıklama
Factorial	1den ne kadar pozitif integer'ların çarpımı
Stirling's Approximation	Factorial büyümeyi yaklaşık veren $\sqrt{2\pi n} (n/e)^n$ bağıntısı
Leading Term	Fonksiyonun en hızlı büyüyen terimi
Mixed Function	Polynomial, logarithmic vb. farklı yapıların birlikte bulunduğu fonksiyon
Tight Bound	Aynı class ile sağlanan upper ve lower bound
Loose Bound	Doğru fakat fonksiyonun büyümesine uzak kalan bound

12. Kendini sinama

- $4n^3 + 2n + 8$ için tight bound nedir?
- $n^2 \log n$, neden n^2 ile n^3 arasındadır?
- $n! = O(n^n)$ doğru olduğu hâlde neden yetersiz bir açıklamadır?
- $\log(n!)$ için lower bound üretirken neden son $n/2$ terim kullanılır?
- Stirling's Approximation hangi tight bound'u verir?

Kısa cevaplar

- $\theta(n^3)$.
- Yeterince büyük n için $1 < \log n < n$ dir.
- Bound çok loose'tur ve factorial'ın gerçek büyümesini yakın göstermez.
- Bu terimlerin her biri en az $\log(n/2)$ dir; toplamaları $\Omega(n \log n)$ verir.
- $n! = \theta(\sqrt{n} (n/e)^n)$.

Sonraki ders: **13 – Properties of Asymptotic Notations.**

13 – Properties of Asymptotic Notations

Video: 1.9 Properties of Asymptotic Notations (11:58)

Bu videonun ana fikri Big O, Big Omega ve Theta yalnızca tek tek fonksiyonları sınıflandırmaz; bunlarla cebirsel ve mantıksal işlemler yapmamızı sağlayan özelliklere sahiptir.

Bu derste:

- Constant multiplication
- Reflexivity
- Transitivity
- Symmetry ve transpose symmetry
- Theta eşdeğerliği
- Sum rule
- Product rule

incelenir.

1. Constant Multiplication Property Pozitif sabit a için:

$$\begin{aligned}f(n) = O(g(n)) &\Rightarrow a f(n) = O(g(n)) \\f(n) = \Omega(g(n)) &\Rightarrow a f(n) = \Omega(g(n)) \\f(n) = \Theta(g(n)) &\Rightarrow a f(n) = \Theta(g(n))\end{aligned}$$

Örnek:

$$\begin{aligned}f(n) &= 2n^2 + 5 = \Theta(n^2) \\7f(n) &= 14n^2 + 35 = \Theta(n^2)\end{aligned}$$

Sabit çarpan growth class'ı değiştirmez.

Not: Videonun otomatik anlatımında $7(2n^2+5)$ için yanlış ara katsayı duyulabiliyor. Doğru dağıtım $14n^2+35$ tir.

Sabitin n den bağımsız ve pozitif olması önemlidir. $a=n$ seçilirse artık sabit çarpan değildir:

$$n * n^2 = n^3$$

2. Reflexive Property Her uygun nonnegative function kendisinin bound'udur:

$$\begin{aligned}f(n) &= O(f(n)) \\f(n) &= \Omega(f(n)) \\f(n) &= \Theta(f(n))\end{aligned}$$

Örneğin:

$$n^2 = \Theta(n^2)$$

Definition'da $c_1 = c_2 = 1$ seçilebilir.

3. Transitive Property

$$\begin{aligned}f(n) &= O(g(n)) \text{ and } g(n) = O(h(n)) \\&\Rightarrow f(n) = O(h(n))\end{aligned}$$

Aynı özellik Ω ve Θ için de geçerlidir.

Örnek:

$$\begin{aligned}n &= O(n^2) \\ n^2 &= O(n^3) \\ \Rightarrow n &= O(n^3)\end{aligned}$$

Mantık: f , g ile; g de h ile yukarıdan sınırlanıyorsa sabitler çarpılarak f , h ile sınırlanabilir.

4. Symmetric Property Theta simetriktir:

$$\begin{aligned}f(n) &= \Theta(g(n)) \\ \Rightarrow g(n) &= \Theta(f(n))\end{aligned}$$

Çünkü iki fonksiyon pozitif sabit çarpanlar içinde aynı hızda büyür.

Örnek:

$$\begin{aligned}2n^2 + 3 &= \Theta(n^2) \\ n^2 &= \Theta(2n^2 + 3)\end{aligned}$$

Big O simetrik değildir

$$n = O(n^2)$$

doğrudur; fakat:

$$n^2 = O(n)$$

yanlıştır. Bu nedenle symmetry doğrudan Big O veya Big Omega için kullanılamaz.

5. Transpose Symmetry Upper ve lower bound rolleri ters çevrilebilir:

$$f(n) = O(g(n)) \text{ iff } g(n) = O(f(n))$$

Örnek:

$$\begin{aligned}n &= O(n^2) \\ n^2 &= \Omega(n)\end{aligned}$$

Benzer şekilde:

$$f(n) = \Omega(g(n)) \text{ iff } g(n) = O(f(n))$$

Bu, sıradan symmetry değildir; notation da O ile Ω arasında değişir.

6. Theta Characterization Bir fonksiyon aynı $g(n)$ ile hem upper hem lower bound'a sahipse:

$$\begin{aligned}f(n) &= O(g(n)) \\ \text{and} \\ f(n) &= \Omega(g(n))\end{aligned}$$

o zaman:

$$f(n) = \Theta(g(n))$$

Tersi de doğrudur:

$$f(n) = \theta(g(n))$$

iff

$$f(n) = O(g(n)) \text{ and } f(n) = \Omega(g(n))$$

Bu, Theta ispatının temel yöntemidir.

7. Sum Rule Eğer:

$$f(n) = O(g(n))$$

$$d(n) = O(e(n))$$

ise:

$$f(n) + d(n) = O(g(n) + e(n))$$

Nonnegative asymptotic functions için toplamı baskın terimle sadeleştirebiliriz:

$$f(n) + d(n) = O(\max(g(n), e(n)))$$

Örnekler:

$$O(n) + O(n^2) = O(n^2)$$

$$O(\log n) + O(n) = O(n)$$

$$O(n^2) + O(n^2) = O(n^2)$$

Bu özellik ardışık algorithm phases'in complexity değerlerini birleştirirken kullanılır.

Tight sonuç için dikkat Yalnızca upper bound bilgisi varsa sonuç da upper bound'dur. $f=O(n)$ demek $f(n)$ gerçekten $\theta(n)$ olduğunu garanti etmez. Tight toplama sonucu için daha güçlü bilgiler gerekir.

Örneğin:

$$f(n)=\theta(n), d(n)=\theta(n^2)$$

$$\Rightarrow f(n)+d(n)=\theta(n^2)$$

8. Product Rule Eğer:

$$f(n) = O(g(n))$$

$$d(n) = O(e(n))$$

ise:

$$f(n)d(n) = O(g(n)e(n))$$

Örnek:

$$f(n) = O(n)$$

$$d(n) = O(n^2)$$

$$\Rightarrow f(n)d(n) = O(n^3)$$

Nested loop analizindeki çarpım mantığı bunun tipik uygulamasıdır.

Tight bilgilerle:

$$\theta(n) * \theta(\log n) = \theta(n \log n)$$

9. Algorithm composition örneği

Phase A: Array'i tara $\rightarrow \theta(n)$
 Phase B: Matrix hücrelerini işle $\rightarrow \theta(n^2)$
 Phase C: Her hücrede log işlem yap $\rightarrow \theta(n^2 \log n)$

Ardışık çalışırlarsa:

$$T(n) = \theta(n) + \theta(n^2) + \theta(n^2 \log n) \\ = \theta(n^2 \log n)$$

Phase C, Phase B'nin içinde nested olsaydı ilgili maliyetler çarpılabilirdi. Composition yapısını doğru okumak gerekir.

10. Özelliklerin özeti

Property	Big O	Omega	Theta
Constant multiplication	✓	✓	✓
Reflexive	✓	✓	✓
Transitive	✓	✓	✓
Symmetric	✗	✗	✓
Transpose symmetry	$0 \leftrightarrow \Omega$	$\Omega \leftrightarrow 0$	Kendi içinde simetrik
Sum rule	✓	✓*	✓*
Product rule	✓	✓*	✓*

* İşaretili kurallarda fonksiyonların eventual nonnegative olması ve bound koşullarının uygunluğu varsayılır.

11. Sık yapılan hatalar Big O'yu simetrik kullanmak $f=O(g)$ ifadesinden $g=O(f)$ çıkmaz; yalnızca $g=\Omega(f)$ çıkar.

Sum rule'da küçük terimleri exact eşitlik gibi silmek $n+n^2$ tam olarak n^2 değildir; asymptotically $\theta(n^2)$ dir.

Constant multiplication'da n'e bağlı ifadeyi sabit sanmak $\log n$, n veya input'tan türetilen başka değer sabit değildir.

Upper bound bilgilerinden doğrudan Theta üretmek 0 bilgisi tek başına lower bound sağlamaz.

12. Kavram sözlüğü

English term	Türkçe açıklama
Reflexive	Her öğenin kendisiyle ilişki içinde olması
Transitive	$f \rightarrow g$ ve $g \rightarrow h$ ilişkilerinden $f \rightarrow h$ çıkarılması
Symmetric	İlişkinin yönü ters çevrildiğinde korunması
Transpose Symmetry	Yön ters çevrilirken 0 ile Ω ın yer değiştirmesi
Sum Rule	Toplanan maliyetlerin baskın growth class ile sadeleştirilmesi
Product Rule	Çarpılan maliyetlerin growth functions'larının çarpılması
Eventual Nonnegative	Yeterince büyük input'lardan sonra negatif olmayan fonksiyon

13. Kendini sinama

1. $n = O(n^2)$ ise ters yönde hangi notation kullanılabilir?
2. $\theta(n) + \theta(n^3)$ sonucu nedir?

3. $O(n) * O(\log n)$ sonucu nedir?
4. Big O neden symmetric değildir?
5. $f=O(g)$ ve $f=\Omega(g)$ birlikte ne verir?

Kısa cevaplar

1. $n^2 = \Omega(n)$.
2. $\Theta(n^3)$.
3. $O(n \log n)$.
4. Daha yavaş büyüyen fonksiyon hızlı büyüyene upper bound olamaz.
5. $f=\Theta(g)$.

Sonraki ders: **14 – Comparison of Functions #1.**

14 – Comparison of Functions #1

Video: 1.10.1 Comparison of Functions #1 (9:28)

Bu videonun ana fikri İki karmaşık growth function'ı karşılaştırmak için biçimlerini sadeleştirir, gerekirse logaritma uygular ve yeterince büyük n değerlerindeki davranışlarına bakarız.

Kullanılabilecek yöntemler:

1. Bilinen class sıralaması
2. Algebraic simplification
3. Logarithmic transformation
4. Ratio ve limit yöntemi
5. Sayısal örnekler – yalnızca sezgi için

1. Basit karşılaştırma: n^2 ve n^3

$$\begin{aligned}f(n) &= n^2 \\g(n) &= n^3\end{aligned}$$

$n > 1$ için:

$$n^2 < n^3$$

Oran:

$$f(n)/g(n) = n^2/n^3 = 1/n \rightarrow 0$$

Bu sonuç:

$$n^2 = o(n^3)$$

demektir; n^2 , n^3 'ten strictly asymptotically daha yavaş büyür.

Sayısal değer denemek ispat değildir $n=2,3,4$ deneyip hep $n^3 > n^2$ görmek doğru sezgi verir, fakat sonlu sayıda örnek genel bir asymptotic iddiayı kanıtlamaz. Piecewise veya dalgalanan fonksiyonlar çok geç davranış değiştirebilir.

Bu örnekte gerçek ispat $n^3 = n \cdot n^2$ ve $n > 1$ olmasıdır.

2. Logarithmic transformation neden işe yarar? Pozitif fonksiyonlarda $\log$ strictly increasing'dir:

$$\begin{aligned}f(n) &< g(n) \\ \text{iff} \\ \log f(n) &< \log g(n)\end{aligned}$$

Logarithm:

- Çarpımı toplama dönüştürür.
- Bölmeyi çıkarma dönüştürür.
- Üssü katsayıya indirir.

Bu nedenle exponential, power ve product içeren fonksiyonlar daha kolay karşılaştırılabilir.

3. Gerekli logaritma kuralları

$$\begin{aligned}
\log(ab) &= \log a + \log b \\
\log(a/b) &= \log a - \log b \\
\log(a^b) &= b \log a \\
\log_a n &= \log_b n / \log_b a \\
a^b = n &\text{ iff } b = \log_a n
\end{aligned}$$

Ayrıca sık kullanılan dönüşüm:

$$a^{(\log_b n)} = n^{(\log_b a)}$$

Özel olarak tabanlar eşitse:

$$2^{(\log_2 n)} = n$$

4. Örnek: $n^2 \log n$ ve $n(\log n)^{10}$

$$\begin{aligned}
f(n) &= n^2 \log n \\
g(n) &= n(\log n)^{10}
\end{aligned}$$

İlk bakışta f daha yüksek n kuvvetine, g daha yüksek logaritma kuvvetine sahiptir. Hangisi baskın?

Ratio yöntemi

$$\begin{aligned}
f(n)/g(n) &= [n^2 \log n] / [n(\log n)^{10}] \\
&= n / (\log n)^9
\end{aligned}$$

Her sabit k için logaritma, lineer fonksiyondan yavaş büyür:

$$(\log n)^9 = o(n)$$

Dolayısıyla:

$$n/(\log n)^9 \rightarrow \text{infinity}$$

Sonuç:

$$\begin{aligned}
f(n) &\text{ grows faster than } g(n) \\
g(n) &= o(f(n))
\end{aligned}$$

Log yöntemi

$$\begin{aligned}
\log f(n) &= 2 \log n + \log \log n \\
\log g(n) &= \log n + 10 \log \log n
\end{aligned}$$

$\log n$, $\log \log n$ den daha hızlı büyüdüğü için ilk ifadede bulunan $2 \log n$, ikinci taraftaki sabit sayıdaki $\log \log n$ terimini sonunda geçer. Yine f daha hızlıdır.

5. Polynomial, polylogarithm'i geçer Genel kural, sabit $a > 0$ ve $b > 0$ için:

$$(\log n)^b = o(n^a)$$

Bu nedenle bir fonksiyonda n in daha büyük sabit kuvveti varsa, diğer taraftaki sabit kuvvetli logaritma çoğunlukla aradaki polynomial farkı kapatamaz.

Örnek:

$n^{1.1}$ grows faster than $n(\log n)^{100}$

yeterince büyük n için geçerlidir; crossover çok büyük olabilir.

6. Örnek: $3n^{(\sqrt{n})}$ ve $2^{(\sqrt{n} \log_2 n)}$ Videodaki ikinci örneği net notation ile yazalım:

$$\begin{aligned}f(n) &= 3 * n^{(\sqrt{n})} \\g(n) &= 2^{(\sqrt{n} * \log_2 n)}\end{aligned}$$

$g(n)$ yi dönüştürelim:

$$\begin{aligned}g(n) &= [2^{(\log_2 n)}]^{(\sqrt{n})} \\&= n^{(\sqrt{n})}\end{aligned}$$

Dolayısıyla:

$$f(n) = 3g(n)$$

Aralarında yalnızca sabit 3 çarpanı vardır:

$$\begin{aligned}f(n) &= \theta(g(n)) \\g(n) &= \theta(f(n))\end{aligned}$$

Exact value olarak f üç kat büyüktür; asymptotic growth class olarak eşittirler.

7. Ratio-limit yöntemi Pozitif f ve g için şu limite bakmak çok kullanışlıdır:

$$L = \lim_{n \rightarrow \infty} f(n)/g(n)$$

Limit sonucu	Yorum
$L = 0$	$f = o(g)$; g daha hızlı büyür
$0 < L < \infty$	$f = \theta(g)$
$L = \infty$	$g = o(f)$; f daha hızlı büyür

Limit yoksa doğrudan sonuç çıkarılamayabilir; başka yöntem veya $\limsup/\liminf$ gerekebilir.

Örnek

$$\begin{aligned}f(n) &= 3n^2 + n \\g(n) &= 7n^2\end{aligned}$$

$$\begin{aligned}\lim f/g &= \lim (3n^2+n)/(7n^2) \\&= \lim (3 + 1/n)/7 \\&= 3/7\end{aligned}$$

Pozitif sonlu sabit elde edildiği için $f=\theta(g)$.

8. Hangi yöntemi ne zaman seçmeli?

Function biçimi	Uygun başlangıç yöntemi
Polynomial	Leading degree karşılaştırması
Product / quotient	Factor cancellation veya ratio
Power / exponential	Logarithm uygulama
Factorial	Stirling veya bilinen bounds
Piecewise	Son eşikten sonraki branch
Sum	Dominant term

9. Sık yapılan hatalar Birkaç sample point'i kanıt kabul etmek Asymptotic davranış çok büyük bir threshold sonrasında değişebilir.

Log uyguladıktan sonra inequality yönünü değiştirmek Pozitif tabanı 1'den büyük logarithm increasing olduğu için yön korunur.

n'in küçük kuvvet farkını log kuvvetiyle karıştırmak Her pozitif polynomial kuvvet, her sabit polylogarithmic kuvvetten sonunda hızlı büyür.

Exact equality ile Theta equality'yi karıştırmak $3n^{(\sqrt{n})}$ ile $n^{(\sqrt{n})}$ exact eşit değildir; aynı Theta class'ındadır.

10. Kavram sözlüğü

English term	Türkçe açıklama
Logarithmic Transformation	Karşılaştırmayı kolaylaştırmak için iki tarafa log uygulama
Ratio Test	Fonksiyon oranının limitinden relative growth çıkarma
Polylogarithmic	Sabit k için $(\log n)^k$ biçimindeki büyüme
Strictly Increasing	Girdi arttığında fonksiyon değerinin kesin artması
Sample Point	Sezgi için seçilen belirli input değeri
Little-o	Bir fonksiyonun diğerinden strict asymptotically yavaş büyümesi

11. Kendini sinama

1. Sonlu sayıda sample point neden asymptotic ispat değildir?
2. $n^2(\log n)^3$ ile $n^{2.1}$ den hangisi daha hızlı büyür?
3. $a^{(\log_b n)}$ nasıl yeniden yazılır?
4. Ratio limit pozitif sonlu sabitse ne sonuç çıkar?
5. $5n^3$ ile n^3 aynı Theta class'ında mıdır?

Kısa cevaplar

1. Fonksiyonlar denenen değerlerden sonra davranış değiştirebilir.
2. $n^{2.1}$; polynomial kuvvet farkı polylogarithm'i sonunda geçer.
3. $n^{(\log_b a)}$.
4. İki fonksiyon aynı Theta class'ındadır.
5. Evet; aralarında sabit 5 çarpanı vardır.

Sonraki ders: **15 – Comparison of Functions #2.**

15 – Comparison of Functions #2

Video: 1.10.2 Comparison of Functions #2 (10:26)

Bu videonun ana fikri Bu ders logarithm'i birden fazla kez uygulamayı, exponential tabanların önemini, piecewise functions'ın asymptotic karşılaştırmasını ve doğru/yanlış notation sorularını ele alır.

Ana ilke:

Fonksiyonları yeterince sade bir biçime dönüşene kadar karşılaştır; fakat dönüşüm sonrasında hangi katsayıların growth bilgisini taşıdığını dikkatsizce silme.

1. Örnek: $n^{(\log n)}$ ve $2^{(\sqrt{n})}$ Logarithm tabanını 2 kabul edelim:

$$\begin{aligned} f(n) &= n^{(\log_2 n)} \\ g(n) &= 2^{(\sqrt{n})} \end{aligned}$$

Bir kez log uygulayalım:

$$\begin{aligned} \log_2 f(n) &= (\log_2 n)^2 \\ \log_2 g(n) &= \sqrt{n} \end{aligned}$$

Şimdi $(\log n)^2$ ile $\sqrt{n}$ karşılaştırılır. Bilinen kural:

$$(\log n)^k = o(n^a)$$

her sabit k ve pozitif a için geçerlidir. Burada $k=2$, $a=1/2$:

$$(\log n)^2 = o(\sqrt{n})$$

Logarithm increasing olduğu için original functions aynı sırayı korur:

$$n^{(\log n)} = o(2^{(\sqrt{n})})$$

Yani $2^{(\sqrt{n})}$ daha hızlı büyür.

2. Log'u tekrar uygulamak İlk dönüşümden sonra karşılaştırma hâlâ açık değilse tekrar log alınabilir:

$$(\log n)^2 \text{ versus } \sqrt{n}$$

Tekrar log:

$$2 \log \log n \text{ versus } (1/2) \log n$$

$\log n$, $\log \log n$ den hızlı büyüdüğü için sağ taraf baskındır.

Bu yöntemin geçerli olması için karşılaştırılan değerlerin eventual positive olması ve kullanılan logarithm'in increasing olması gerekir.

3. Örnek: n ve $\sqrt{n} \log n$

$$\begin{aligned} f(n) &= n \\ g(n) &= \sqrt{n} \log n \end{aligned}$$

Oran:

$$\begin{aligned} f(n)/g(n) &= n / (\sqrt{n} \log n) \\ &= \sqrt{n} / \log n \\ &\rightarrow \text{infinity} \end{aligned}$$

Çünkü herhangi bir pozitif polynomial kuvvet logarithm'den hızlı büyür. Sonuç:

$$\sqrt{n} \log n = o(n)$$

Bu örnek, product içindeki logarithm'in polynomial exponent farkını kapatamadığını tekrar gösterir.

4. Exponential taban neden atılamaz?

$$\begin{aligned} f(n) &= 2^n \\ g(n) &= 2^{(2n)} = 4^n \end{aligned}$$

Log uygulayınca:

$$\begin{aligned} \log_2 f(n) &= n \\ \log_2 g(n) &= 2n \end{aligned}$$

Burada "2 sabit katsayıdır, atılır; eşitler" demek yanlıştır. Log dönüşümündeki katsayı, original exponential fonksiyonun taban farkını temsil eder:

$$g(n)/f(n) = 2^{(2n)}/2^n = 2^n \rightarrow \text{infinity}$$

Dolayısıyla:

$$\begin{aligned} 2^n &= o(4^n) \\ 2^n &\neq \theta(4^n) \end{aligned}$$

Karşılaştırma yaparken sabit katsayılar ancak original functions üzerinde multiplicative constants olduklarında Theta içinde yok sayılabilir:

$$2 * 2^n = \theta(2^n)$$

Ama:

$$2^{(2n)} \neq \theta(2^n)$$

5. Piecewise functions Şu iki şematik fonksiyonu düşün:

$$\begin{aligned} g_1(n) &= n^3, \quad n < 100 \\ &\quad n^2, \quad n \geq 100 \\ g_2(n) &= n^2, \quad n < 10000 \\ &\quad n^3, \quad n \geq 10000 \end{aligned}$$

Farklı bölgelerde sıralama değişir:

- Küçük bölgede g_1 daha büyük olabilir.
- Orta bölgede eşit class veya farklı değerler görülebilir.
- $n \geq 10000$ sonrasında $g_1=n^2$, $g_2=n^3$ ve g_2 kalıcı olarak daha hızlı büyür.

Asymptotic sonuç yalnızca final tail'i inceler:

$$\begin{aligned}g_1(n) &= \Theta(n^2) \\g_2(n) &= \Theta(n^3) \\g_1(n) &= o(g_2(n))\end{aligned}$$

Sonlu sayıdaki erken branch asymptotic class'ı değiştirmez.

6. True/False örnekleri $(n + 1)^2 = \Theta(n^2)$ Binomial expansion:

$$(n+1)^2 = n^2 + 2n + 1$$

Leading term n^2 olduğundan ifade doğrudur.

$2^{(n+1)} = \Theta(2^n)$

$$2^{(n+1)} = 2 * 2^n$$

Aradaki 2 multiplicative constant'tır; ifade doğrudur.

$2^{(2n)} = O(2^n)$

$$2^{(2n)}/2^n = 2^n \rightarrow \text{infinity}$$

2^n , $2^{(2n)}$ için upper bound olamaz; ifade yanlıştır.

$\sqrt{\log n} = O(\log \log n)$ $x = \log n$ yazalım. Soru $\sqrt{x}$ ile $\log x$ karşılaştırmasına dönüşür. Pozitif polynomial power, logarithm'den hızlı büyür:

$$\log \log n = o(\sqrt{\log n})$$

Dolayısıyla verilen ifade yanlıştır.

$n^{(\log n)} = O(2^n)$ İki tarafa log uygulayalım:

$$\begin{aligned}\log[n^{(\log n)}] &= (\log n)^2 \\ \log[2^n] &= \Theta(n)\end{aligned}$$

$(\log n)^2 = o(n)$ olduğundan ifade doğrudur; hatta:

$$n^{(\log n)} = o(2^n)$$

7. Dönüşüm sırasında coefficient kuralı Şu iki durumu ayır:

Original function'da multiplicative coefficient

$$3f(n) = \Theta(f(n))$$

Pozitif sabit atılabilir.

Exponent içinde coefficient

$$2^{(3n)} = 8^n$$

Buradaki 3 atılamaz; growth base'i değiştirir:

$$2^{(3n)} / 2^n = 2^{(2n)} \rightarrow \text{infinity}$$

Logarithm sonucunda görünen coefficient Log aldıktan sonra oluşan coefficient'ı silmeden önce original ifadede neyi temsil ettiğini düşün. Exponent farkı olabilir.

8. Güvenilir comparison iş akışı

1. Fonksiyonların eventual positive olduğunu doğrula.
2. Ortak factor'ları ratio içinde sadeleştir.
3. Power/exponential varsa log uygula.
4. Gerekirse log'u tekrar uygula.
5. Bilinen hierarchy'yi kullan:

$\log \log n \ll \log n \ll n^a \ll c^n$

6. Piecewise ise son threshold'dan sonraki ifadeyi incele.
7. Sonucu $0, \Omega, \Theta$ veya o ile açıkça yaz.

9. Sık yapılan hatalar Exponent coefficient'ı multiplicative constant sanmak $2^{(2n)}$, $2 \cdot 2^n$ değildir; 4^n dir.

Piecewise function'ın ilk branch'ine takılmak Asymptotic class, sonlu threshold'lardan sonraki kalıcı branch ile belirlenir.

Bir kez log alınca mutlaka karar verileceğini sanmak Gerekirse ikinci log veya ratio yöntemi kullanılabilir.

O ile Theta'yı birbirinin yerine kullanmak Bir true/false sorusunda istenen bound yönü dikkatle kontrol edilmelidir.

10. Kavram sözlüğü

English term	Türkçe açıklama
Piecewise Function	Farklı input bölgelerinde farklı formüller kullanan fonksiyon
Final Tail	Bütün sonlu threshold'lardan sonraki sınırsız input bölgesi
Exponent Coefficient	Üs içinde bulunan ve exponential rate'i etkileyen katsayı
Multiplicative Constant	Fonksiyonun tamamını çarpan ve Theta class'ını değiştirmeyen sabit
Binomial Expansion	$(a+b)^k$ ifadesini terimlerine açma
Eventual Positivity	Fonksiyonun yeterince büyük input'larda pozitif olması

11. Kendini sına

1. 3^n ile $3^{(2n)}$ aynı Theta class'ında mıdır?
2. $n^{(\log n)}$ ile $2^{(\sqrt{n})}$ den hangisi daha hızlı büyür?
3. Piecewise function'ın sonlu ilk bölümleri neden class'ı değiştirmez?
4. $5 \cdot 2^n$ ile 2^n aynı Theta class'ında mıdır?
5. $\log \log n$ ile $\sqrt{(\log n)}$ den hangisi hızlı büyür?

Kısa cevaplar

1. Hayır; oranları 3^n olarak sınırsız büyür.
2. $2^{(\sqrt{n})}$ daha hızlı büyür.
3. Asymptotic tanım yeterince büyük tüm n'leri inceler ve bir n_0 seçmeye izin verir.
4. Evet; aralarında multiplicative constant vardır.
5. $\sqrt{(\log n)}$ daha hızlı büyür.

Sonraki ders: **16 – Best, Worst and Average Case Analysis.**

16 – Best, Worst and Average Case Analysis

Video: 1.11 Best Worst and Average Case Analysis (18:56)

Bu videonun ana fikri Aynı input size değerine sahip farklı girdiler, bir algoritmanın farklı miktarda iş yapmasına neden olabilir. Best Case, Worst Case ve Average Case, bu değişimi tanımlar.

Bu kavramlar Big O, Omega ve Theta ile aynı şey değildir:

- Case analysis: Hangi girdi durumunu incelediğimizi seçer.
- Asymptotic notation: Seçilen durumda elde edilen fonksiyonun growth bound'unu ifade eder.

Videoda iki örnek kullanılır:

1. Linear Search
2. Binary Search Tree üzerinde arama

1. Case tanımları Belirli bir input size n için bütün geçerli girdiler kümesi I_n olsun. Bir girdi x üzerindeki maliyet $T(x)$ olsun.

Best Case

$$T_{\text{best}}(n) = \min \{ T(x) : x \in I_n \}$$

Boyutu n olan girdiler arasında mümkün olan en düşük maliyet.

Worst Case

$$T_{\text{worst}}(n) = \max \{ T(x) : x \in I_n \}$$

Boyutu n olan girdiler arasında mümkün olan en yüksek maliyet.

Average Case Girdiler üzerinde bir probability distribution $P(x)$ tanımlanmalıdır:

$$\begin{aligned} T_{\text{avg}}(n) &= E[T(x)] \\ &= \sum \text{over } x \in I_n \text{ of } P(x) * T(x) \end{aligned}$$

Yalnızca maliyetleri toplayıp durum sayısına bölmek, bütün durumların eşit olasılıklı olduğu varsayımını yapar. Bu varsayım her problem için doğru değildir.

2. Linear Search

```
Algorithm LinearSearch(A, n, key)
  for i ← 0 to n - 1
    if A[i] = key
      return i
  return -1
```

Algoritma soldan sağa ilerler. Aranan key bulunursa durur; bulunamazsa bütün array'i kontrol eder.

Örnek:

$A = [8, 3, 11, 6, 4, 7, 18]$

- key=7: Altı comparison sonra bulunur.
- key=20: Bütün yedi eleman kontrol edilir ve -1 döner.

3. Linear Search – Best Case Aranan değer ilk konumdaysa:

$A[0] = \text{key}$

Yalnızca bir comparison gerekir:

```
T_best(n) = 1  
T_best(n) =  $\Theta(1)$ 
```

n büyüse bile ilk elemanı kontrol etmenin maliyeti sabittir.

Şunların üçü de teknik olarak doğrudur:

```
T_best(n) =  $O(1)$   
T_best(n) =  $\Omega(1)$   
T_best(n) =  $\Theta(1)$ 
```

Theta tight bilgiyi verdiği için en açıklayıcı olanıdır.

4. Linear Search – Worst Case İki durum n comparison gerektirir:

- key son elemandadır.
- key array'de yoktur.

```
T_worst(n) = n  
T_worst(n) =  $\Theta(n)$ 
```

Yine şu notation'lar mümkündür:

```
T_worst(n) =  $O(n)$   
T_worst(n) =  $\Omega(n)$   
T_worst(n) =  $\Theta(n)$ 
```

Bu, Big O = worst case eşleştirmesinin yanlışlığını açıkça gösterir: Worst-case fonksiyonunu Theta ile ifade edebiliriz.

5. Linear Search – Average Successful Case Önce yalnızca **successful search** ele alalım. keynin her pozisyonda bulunma olasılığı eşit, yani $1/n$ kabul edilsin.

Pozisyona göre comparison sayıları:

```
1, 2, 3, ..., n
```

Expected comparisons:

```
T_avg_success(n)  
= (1/n)(1 + 2 + ... + n)  
= (1/n) * n(n+1)/2  
= (n+1)/2  
=  $\Theta(n)$ 
```

Average değer worst-case'in yaklaşık yarısıdır; fakat sabit $1/2$ atıldığı için ikisi de aynı asymptotic class'tadır.

6. Unsuccessful search average'a nasıl eklenir? Aramanın başarısız olma olasılığı q olsun. Başarılı olduğunda pozisyonların eşit olasılıklı olduğunu varsayalım:

```
E[comparisons]  
= q * n + (1-q) * (n+1)/2
```

Sabit $0 \leq q \leq 1$ için sonuç yine:

```
T_avg(n) =  $\Theta(n)$ 
```

Ancak olasılıkların bilinmediği bir problemde $(n+1)/2$ 'yi koşulsuz average diye sunmak doğru değildir. Distribution varsayımı açıkça yazılmalıdır.

7. Neden Worst Case sık kullanılır?

- Güvenli bir üst performans garantisi verir.
- Çoğu zaman average-case dağılımını tanımlamak zordur.
- Gerçek kullanıcı girdileri eşit olasılıklı olmayabilir.
- Güvenlik ve gerçek zamanlı sistemlerde en kötü gecikme önemlidir.

Fakat worst-case her zaman tek ilgi alanı değildir. Hash table gibi yapılarda expected performance, randomized algorithms'te expectation ve amortized analysis de kritik olabilir.

8. Binary Search Tree özelliği

Bir Binary Search Tree (BST) node'u için:

- Sol subtree'deki keys daha küçüktür.
- Sağ subtree'deki keys daha büyüktür.

Arama:

```
Algorithm BSTSearch(root, key)
  current ← root
  while current != null
    if key = current.key
      return current
    else if key < current.key
      current ← current.left
    else
      current ← current.right
  return null
```

Her comparison bizi yalnızca bir child'a götürür. Maliyet, izlenen root-to-node path uzunluğuna bağlıdır.

9. BST – Best Case

Aranan key root'taysa tek comparison yeterlidir:

$$T_{\text{best}}(n) = \theta(1)$$

Bu sonuç tree balanced veya skewed olsa da değişmez; root araması sabittir.

10. BST – Bir tree için Worst Case

En derin node'u veya mevcut olmayan bir key'i aramak tree height kadar ilerlemeyi gerektirir.

$$T_{\text{search}} = \theta(h + 1) = \theta(h)$$

Burada h , kullanılan height tanımına göre edge veya level sayısı olabilir. $+1$ asymptotic sonucu değiştirmez.

Doğru temel ifade:

BST search complexity doğrudan n veya $\log n$ değil, $\theta(h)$ dir.

11. Balanced BST

Balanced bir BST'de height:

$$h = \theta(\log n)$$

Her level'da bulunabilecek node sayısı yaklaşık iki katına çıktığı için n node yalnızca logarithmic sayıda level'a sığar.

Arama:

$$T_{\text{worst}}(n) = \theta(\log n)$$

Örneğin 7 node'lu perfect tree'de root'tan leaf'e en fazla üç node comparison yapılır.

12. Skewed BST Keys sıralı biçimde eklenirse ordinary BST linked list gibi tek tarafa yatabilir:

```
40
/
30
/
25
/
20
...
```

Bu durumda:

```
h =  $\theta(n)$ 
T_worst(n) =  $\theta(n)$ 
```

BST kuralını karşılamak tek başına logarithmic search garantisi vermez. AVL Tree veya Red-Black Tree gibi self-balancing yapılar height'ı logarithmic tutar.

13. Minimum ve maximum worst-case time Bu ifadeler ilk bakışta çelişkili görünür. İki farklı tree shape üzerinde worst-case maliyetini karşılaştırırız:

- Minimum possible worst-case: n node mümkün olduğunca balanced yerleştirilir.
- Maximum possible worst-case: Tree tamamen skewed olur.

```
Minimum worst-case =  $\theta(\log n)$ 
Maximum worst-case =  $\theta(n)$ 
```

Yani "worst case" bir tree içindeki en pahalı aramayı; "minimum/maximum" ise aynı n için mümkün tree shapes'i karşılaştırır.

14. BST Average Case için gereken varsayım Average search time otomatik olarak $\theta(\log n)$ değildir. Sonuç şunlara bağlıdır:

- Keys'in insertion order dağılımı
- Başarılı ve başarısız search olasılıkları
- Hangi key'lerin daha sık arandığı
- Tree'nin self-balancing olup olmadığı

Random insertion order gibi uygun varsayımlarda expected height ve search time $\theta(\log n)$ olabilir. Adversarial sorted insertion ordinary BST'yi $\theta(n)$ height'a dönüştürür.

15. Case ile notation'ı ayırma Doğru yazım örnekleri:

```
Linear Search best-case    =  $\theta(1)$ 
Linear Search worst-case   =  $\theta(n)$ 
Linear Search average-case =  $\theta(n)$  // uniform successful-position assumption

Balanced BST worst-case    =  $\theta(\log n)$ 
Skewed BST worst-case      =  $\theta(n)$ 
```

Yanlış ezber:

```
Best = Omega
Average = Theta
Worst = Big O
```

Her case için elde edilen function'a üç notation da tanımları ölçüsünde uygulanabilir. Genellikle tight sonucu Theta ile yazmak en bilgilendiricidir.

16. Özet tablo

Problem / durum	Best	Average	Worst
Linear Search	$\theta(1)$	$\theta(n)^*$	$\theta(n)$
Balanced BST Search	$\theta(1)$	$\theta(\log n)^{**}$	$\theta(\log n)$
Skewed BST Search	$\theta(1)$	Dağılıma bağlı	$\theta(n)$

* Successful positions uniform veya sabit başarısızlık olasılığı varsayımıyla.

** Search distribution'ın path uzunluklarını uygun biçimde temsil ettiği varsayımıyla.

17. Sık yapılan hatalar Average için olasılık belirtmemek Expected value ancak distribution tanımlandığında anlamlıdır.

BST search'ü her zaman $O(\log n)$ sanmak Ordinary BST skewed olabilir; genel maliyet $\theta(h)$ dir.

Big O'yu worst case'in adı sanmak Big O bir bound notation'ıdır; case türü değildir.

Unsuccessful Linear Search'ü unutmak Bulunmayan key bütün n elemanın kontrol edilmesini gerektirir.

Minimum worst-case ifadesini çelişki sanmak Minimum, farklı data structure shapes arasındaki worst-case değerlerini karşılaştırır.

18. Kavram sözlüğü

English term	Türkçe açıklama
Best Case	Aynı input size içindeki en düşük maliyetli durum
Worst Case	Aynı input size içindeki en yüksek maliyetli durum
Average Case	Belirli distribution altında expected cost
Expected Value	Olasılıklarla ağırlıklandırılmış ortalama değer
Successful Search	Key'in data structure içinde bulunduğu arama
Unsuccessful Search	Key'in bulunmadığı arama
Tree Height	Root'tan en derin node'a olan maksimum path uzunluğu
Balanced BST	Height'ı kontrollü, yaklaşık logarithmic kalan BST
Skewed BST	Node'ların büyük ölçüde tek yönde zincir oluşturduğu BST

19. Kendini sinama

1. Linear Search best-case neden $\theta(1)$ dir?
2. Uniform successful search varsayımıyla expected comparison sayısı nedir?
3. Average-case hesabında distribution neden gereklidir?
4. BST search için en genel complexity ifadesi nedir?
5. Ordinary BST hangi durumda $\theta(n)$ search süresine düşebilir?
6. Worst-case fonksiyonu Theta ile ifade edilebilir mi?

Kısa cevaplar

1. Key ilk elemandaysa yalnızca bir comparison yapılır.
2. $(n+1)/2$, dolayısıyla $\theta(n)$.
3. Farklı girdilerin gerçekleşme olasılıkları expected cost'u değiştirir.

4. $\theta(h)$; h tree height'tır.
5. Tree sorted insertion gibi bir nedenle tamamen skewed olduğunda.
6. Evet; örneğin Linear Search worst-case $\theta(n)$ dir.

Sonraki ders: **17 – Disjoint Sets: Weighted Union and Collapsing Find.**

17 – Disjoint Sets: Weighted Union and Collapsing Find

Video: 1.12 Disjoint Sets Data Structure - Weighted Union and Collapsing Find (26:04)

Bu videonun ana fikri Disjoint Set Union (DSU), diğer adlarıyla Union-Find, elemanları birbirleriyle kesişmeyen component'ler hâlinde tutar. İki temel operation sağlar:

- **Find(x)**: x elemanının bulunduğu set'in representative/root değerini bulur.
- **Union(a,b)**: Farklı iki set'i tek set hâline getirir.

Bu yapı özellikle undirected graph üzerinde edge eklerken cycle oluşup oluşmadığını hızlıca anlamak için kullanılır. Kruskal's Minimum Spanning Tree Algorithm bunun klasik uygulamasıdır.

1. Disjoint ne demektir? İki set ortak eleman taşııyorsa disjointtir:

```
S1 = {1,2,3,4}
S2 = {5,6,7,8}

S1 ∩ S2 = ∅
```

Graph açısından her set bir connected componentı temsil edebilir. Aynı component içindeki vertex'ler birbirlerine bir path ile bağlıdır.

DSU genel set algebra işlemlerini yapmak için tasarlanmaz. Intersection veya difference yerine yalnızca component membership ve merging işlemlerine odaklanır.

2. MakeSet, Find ve Union Pratikte üç operation düşünülür:

```
MakeSet(x)  -> yalnızca x'i içeren yeni set oluştur
Find(x)     -> x'in set representative'ını döndür
Union(x,y)  -> x ve y'nin set'lerini birleştir
```

Başlangıçta her vertex kendi set'indedir:

```
{1} {2} {3} {4} {5} {6} {7} {8}
```

Union(1,2) sonrası:

```
{1,2} {3} {4} {5} {6} {7} {8}
```

Find(1) ve Find(2) artık aynı representative'ı verir.

3. Representative neden gerekir? Set'in bütün elemanlarını karşılaştırmak yerine her set için bir root seçilir:

```
  1      <- representative/root
 /  \
2    3
```

Find(2) parent pointer'ları takip edip root 1e ulaşır. Aynı root'u veren elemanlar aynı set'tedir:

```
Find(x) = Find(y) => x ve y aynı component'ta
Find(x) != Find(y) => farklı component'larda
```

Representative'in hangi vertex olduğu matematiksel sonucu değiştirmez; önemli olan set içinde tutarlı olmasıdır.

4. Undirected graph'ta cycle detection Bir undirected edge (u,v) işlenirken:

```

rootU <- Find(u)
rootV <- Find(v)

if rootU = rootV
    edge bir cycle oluşturur
else
    Union(rootU, rootV)

```

Neden? u ve v zaten aynı component'taysa aralarında mevcut bir path vardır. Yeni edge bu path'e ikinci bir bağlantı ekleyerek cycle kapatır.

Bu yöntem directed graph cycle detection için doğrudan kullanılamaz; yön bilgisi DSU set'lerinde korunmaz.

5. Küçük örnek

Edge'leri sırayla işleyelim:

(1,2), (3,4), (2,4), (1,3)

Edge	Önceki roots	İşlem	Component'lar
(1,2)	farklı	Union	{1,2}, {3}, {4}
(3,4)	farklı	Union	{1,2}, {3,4}
(2,4)	farklı	Union	{1,2,3,4}
(1,3)	aynı	Cycle	Değişmez

Son edge eklenmezse yapı forest olarak kalır.

6. Kruskal bağlantısı

Kruskal's Algorithm edge'leri ağırlığa göre küçükten büyüğe işler:

```

sort edges by weight
for each edge (u,v)
    if Find(u) != Find(v)
        select edge
        Union(u,v)
    else
        reject edge // cycle oluşturur

```

DSU, "bu edge cycle oluşturuyor mu?" sorusunu hızlı cevaplar. Edge ağırlıklarını sıralamak DSU'nun değil Kruskal'ın başka bir aşamasıdır.

7. Forest representation

Her set bir rooted tree olarak tutulur; bütün set'ler birlikte bir forest oluşturur:

```

Set A          Set B
  1              5
 / \           / \
2   3         6   7

```

Her node parent'ını gösterir. Root kendi set'inin representative'idir ve özel bir değerle işaretlenir.

Naive Union, bir root'u diğer root'un child'ı yapar:

```
parent[rootB] <- rootA
```

Yanlışlıkla root yerine iç node bağlanırsa gereksiz derinlik veya bozuk yapı oluşabilir. Union root'lar arasında yapılmalıdır.

8. Array representation

Video tek bir parent array kullanır. Index vertex'i temsil eder:

```
index:  1  2  3  4
parent: -2  1 -2  3
```

Yorum:

- Negatif değer: Bu index root'tur.
- Negatif değerın mutlak değeri: Set size.
- Pozitif değer: Parent index.

Yukarıdaki durumda:

```
Set 1: root=1, size=2, child=2
Set 2: root=3, size=2, child=4
```

Find(4) yolu:

```
4 -> 3 -> parent[3] negatif, root=3
```

9. Union by Size – Weighted Union Naive birleşimler uzun chain üretebilir. Union by Size, küçük tree'nin root'unu büyük tree'nin root'una bağlar:

```
Algorithm UnionBySize(rootA, rootB)
    if size[rootA] < size[rootB]
        swap(rootA, rootB)

    parent[rootA] <- parent[rootA] + parent[rootB]
    parent[rootB] <- rootA
```

Burada sizes negatif saklandığı için gerçek uygulamadaki karşılaştırma işaret yönüne dikkat etmelidir. Kavramsal kural değişmez:

Smaller tree, larger tree'nin altına bağlanır.

Bu sayede height kontrol altında tutulur. Yalnızca Union by Size/Rank kullanıldığında tree height $O(\log n)$ olur.

10. Union by Rank ile Union by Size farkı

- Union by Size: Root'ta node sayısı tutulur.
- Union by Rank: Tree height için bir upper-bound benzeri rank tutulur.

İkisi aynı optimizasyon fikrini kullanır ama stored metadata aynı değildir. Video "weight/rank" ifadelerini yakın anlamda kullanır; implementation'da hangisinin tutulduğu açık olmalıdır.

Rank yalnızca eşit rank'li iki tree birleştiğinde artar. Path compression sonrasında rank gerçek height olmak zorunda değildir; yapısal bir bound olarak kalır.

11. Collapsing Find – Path Compression Video Collapsing Find adını kullanır; yaygın modern terim Path Compression'dır.

Normal path:

```
8 -> 7 -> 5 -> 1
```

Find(8) root 1'i bulduktan sonra path üzerindeki node'ları doğrudan root'a bağlar:

```
8 -> 1
7 -> 1
5 -> 1
```

Recursive pseudocode:

```

Algorithm Find(x)
    if parent[x] < 0
        return x
    parent[x] <- Find(parent[x])
    return parent[x]

```

Kritik line:

```
parent[x] <- Find(parent[x])
```

Hem root'u bulur hem dönüş sırasında path'i sıkıştırır.

12. Tam DSU pseudocode

```

Algorithm MakeSet(n)
    for x <- 1 to n
        parent[x] <- -1

Algorithm Find(x)
    if parent[x] < 0
        return x
    parent[x] <- Find(parent[x])
    return parent[x]

Algorithm Union(a, b)
    rootA <- Find(a)
    rootB <- Find(b)

    if rootA = rootB
        return false

    // negatif değer daha büyük set için sayısal olarak daha küçüktür
    if parent[rootA] > parent[rootB]
        swap(rootA, rootB)

    parent[rootA] <- parent[rootA] + parent[rootB]
    parent[rootB] <- rootA
    return true

```

13. Complexity Optimization olmadan Tree chain hâline gelebilir:

```
Find = O(n)
```

Yalnızca Union by Size/Rank Height logarithmic tutulur:

```

Find = O(log n)
Union = O(log n) // iki Find nedeniyle

```

Union by Rank/Size + Path Compression Bir operation sequence üzerindeki amortized maliyet:

```
O(α(n)) per operation
```

$\alpha(n)$, inverse Ackermann function'dır ve pratik input boyutlarında 5'ten küçüktür. Bu yüzden "neredeyse constant" denir; matematiksel olarak doğrudan $O(1)$ demek tam doğru değildir.

m operation ve n element için klasik toplam bound:

```
O(m α(n))
```

14. Sık yapılan hatalar Union'ı original node'lar arasında yapmak Önce roots bulunmalı, sonra root'lar birleştirilmelidir.

Directed graph cycle detection'da aynı yöntemi kullanmak DSU'nun bu cycle kuralı undirected graph içindir.

Negative size karşılaştırmasını ters yapmak -8, -2den daha büyük set temsil eder fakat sayısal olarak daha küçüktür.

Path Compression sonrası rank'i gerçek height sanmak Rank bir optimization metadata'sıdır; compression sonrası exact height değildir.

Her Find kesin $O(1)$ demek Bir tek operation daha pahalı olabilir; güçlü bound amortized $O(\alpha(n))$ dir.

15. Kavram sözlüğü

English term	Türkçe açıklama
Disjoint Set Union	Kesişmeyen component'ları birleştirip sorgulayan data structure
Representative	Bir set'i tanımlayan root eleman
Find	Elemanın representative'ını bulma operation'ı
Union	İki farklı set'i birleştirme operation'ı
Union by Size	Küçük tree'yi büyük tree'nin altına bağlama optimization'ı
Union by Rank	Daha düşük rank'li tree'yi yüksek rank'linin altına bağlama
Path Compression	Find sırasında path node'larını doğrudan root'a bağlama
Amortized Analysis	Bir operation sequence'in toplam maliyetini operation başına dağıtma
Inverse Ackermann Function	Çok yavaş büyüyen, DSU complexity'sinde görülen $\alpha(n)$ fonksiyonu

16. Kendini sınama

1. $\text{Find}(u)=\text{Find}(v)$ yeni undirected edge için ne anlama gelir?
2. Negative parent değeri hangi iki bilgiyi taşır?
3. Union by Size neden tree height'ı azaltır?
4. Path Compression hangi pointer'ları değiştirir?
5. İki optimization birlikteken amortized complexity nedir?

Kısa cevaplar

1. Vertex'ler zaten aynı component'tadır; edge cycle oluşturur.
2. Node'un root olduğunu ve set size'ı.
3. Küçük tree büyük tree altına bağlandığı için büyük miktarda node'un depth'i artmaz.
4. Find path'i üzerindeki node'ların parent'larını doğrudan root yapar.
5. Operation başına amortized $O(\alpha(n))$.

Sonraki ders: **18 – Divide and Conquer.**

18 – Divide and Conquer

Video: 2 Divide And Conquer (7:04)

Bu videonun ana fikri Divide and Conquer, büyük bir problemi aynı türden daha küçük subproblem'lara ayıran, bunları çözen ve sonuçlarını birleştiren algorithm design strategy'dir.

Üç temel aşama:

1. Divide
2. Conquer
3. Combine

1. Divide Boyutu n olan problem P , daha küçük problemlere ayrılır:

```
P(n) -> P_1(n_1), P_2(n_2), ..., P_k(n_k)
```

Genellikle $n_i < n$ olmalıdır; aksi hâlde recursion ilerlemez.

Örnekler:

- Merge Sort: Array iki yarıya bölünür.
- Binary Search: Arama aralığının yalnızca bir yarısı seçilir.
- Quick Sort: Elements pivot'a göre iki partition'a ayrılır.
- Strassen: Matrix'ler block'lara ayrılır.

2. Conquer Her subproblem aynı strategy ile çözülür:

```
solution_i <- DivideAndConquer(P_i)
```

Subproblem yeterince küçükse doğrudan çözülür. Buna Base Case denir.

```
if problem is small  
  solve directly
```

Base case yoksa veya size küçülmüyorsa infinite recursion oluşabilir.

3. Combine Subproblem solutions, original problem'ın solution'ına dönüştürülür:

```
solution <- Combine(solution_1, ..., solution_k)
```

Combine maliyeti algorithmadan algorithmaya değişir:

- Merge Sort: İki sorted half linear zamanda merge edilir.
- Binary Search: Tek subproblem seçildiğinden combine hemen hemen constant'tır.
- Quick Sort: Partition işi divide aşamasında yapılır; recursive sonuçlar array içinde zaten konumlanır.

Bir problemi ayırabiliyor olmak yeterli değildir; çözümleri doğru şekilde birleştirebilmek gerekir.

4. Aynı türden subproblem şartı Videonun vurgusu: Subproblem'lar original problem ile aynı problem türünü taşımalıdır.

```
Sort problem -> daha küçük Sort problems  
Search problem -> daha küçük Search problem
```

Bir etkinlik düzenlemeyi poster, davet, ulaşım gibi farklı görevlere bölmek genel decomposition'dur; recursive olarak aynı problem türü oluşmadığı için klasik Divide and Conquer örneği değildir.

Teknik açıdan her Divide and Conquer uygulamasında subproblem'lar birebir aynı görünüme sahip olmayabilir; esas özellik çözüm yapısının recursive/self-similar olmasıdır.

5. General template

```
Algorithm DivideAndConquer(P)
  if IsSmall(P)
    return SolveDirectly(P)

  subproblems <- Divide(P)

  solutions <- empty list
  for each subproblem in subproblems
    solutions.append(DivideAndConquer(subproblem))

  return Combine(solutions)
```

Bu template dört tasarım sorusu doğurur:

1. Base case nedir?
2. Kaç subproblem oluşturuluyor?
3. Her subproblem'in size'ı nedir?
4. Divide ve Combine maliyeti nedir?

Bu sorular recurrence relation'ı belirler.

6. Recurrence Relation bağlantısı Bir algoritma a adet, her biri yaklaşık n/b boyutunda subproblem üretiliyor ve recursive işler dışında $f(n)$ kadar çalışma yapıyorsa:

$$T(n) = aT(n/b) + f(n)$$

Örnekler:

Binary Search

$$T(n) = T(n/2) + \theta(1)$$

Yalnızca bir yarı recursive çözülür.

Merge Sort

$$T(n) = 2T(n/2) + \theta(n)$$

İki yarı çözülür ve linear merge yapılır.

Decrease-and-Conquer örneği

$$T(n) = T(n-1) + \theta(1)$$

Bu yapı sonraki videolarda aynı recurrence çözme araçları içinde ele alınsa da sınıflandırma açısından çoğu kaynak buna Decrease and Conquer der; problem sabit miktarda küçülür.

7. Divide and Conquer ile Dynamic Programming farkı İkisinde de subproblem vardır, fakat tipik ayrım:

- Divide and Conquer: Subproblem'lar bağımsız veya çok az overlap eder.
- Dynamic Programming: Aynı subproblem'lar tekrar tekrar ortaya çıkar; sonuçlar saklanır.

Örneğin naive Fibonacci recursion overlapping subproblem üretir ve memoization ile Dynamic Programming'e dönüşür.

8. Avantajlar ve maliyetler Avantajlar

- Büyük problemi kavramsal olarak sadeleştirir.

- Recursion ile doğal ifade edilir.
- Bağımsız subproblem'lar paralel çalıştırılabilir.
- Cache locality bazı algoritmalarda iyileşebilir.

Maliyetler

- Recursive call stack ek space kullanır.
- Combine pahalı olabilir.
- Dengesiz partition worst-case performansı bozabilir.
- Çok küçük subproblem'larda call overhead gereksiz olabilir.

9. Uygulanabilirlik kontrolü

Bir problem için şu soruları sor:

- Daha küçük, aynı türden subproblem'lara ayrılabilir mi?
- Her recursive call kesin biçimde base case'e yaklaşıyor mu?
- Subproblem solutions'tan original solution üretilebiliyor mu?
- Aynı subproblem tekrar ediyorsa memoization gerekli mi?
- Bölme dengeli mi; worst-case'te ne oluyor?

10. Sık yapılan hatalar Her iş bölümünü Divide and Conquer sanmak

Klasik strategy recursive ve self-similar subproblem yapısı gerektirir. **Combine aşamasını yok saymak** Birleştirme maliyeti çoğu zaman total complexity'nin belirleyici parçasıdır.

İki parçaya bölünüyorsa ikisini de recursive çözmek zorunda sanmak Binary Search yalnızca ilgili yarıyı çözer; recurrence katsayısı 1'dir.

Recursion varsa mutlaka Divide and Conquer demek Recursion bir implementation tekniğidir; her recursive algoritma bu strategy'yi izlemez.

11. Kavram sözlüğü

English term	Türkçe açıklama
Divide and Conquer	Problemi küçük benzer problemlere ayırıp sonuçları birleştiren strategy
Subproblem	Original problem'ın daha küçük örneği
Base Case	Recursion'ın doğrudan çözüldüğü en küçük durum
Divide	Problemi subproblem'lara ayırma aşaması
Conquer	Subproblem'ları recursive çözme aşaması
Combine	Küçük solutions'tan original solution üretme aşaması
Self-similar	Küçük ölçekte aynı problem yapısının tekrarlanması
Recurrence Relation	Recursive çalışma süresini daha küçük input süreleriyle tanımlayan denklem

12. Kendini sına

1. Divide and Conquer'ın üç aşaması nedir?
2. Base case neden zorunludur?
3. Binary Search recurrence'ında neden $2T(n/2)$ bulunmaz?
4. Merge Sort combine aşaması ne yapar?
5. Dynamic Programming ile temel fark nedir?

Kısa cevaplar

1. Divide, Conquer, Combine.

2. Recursion'ı durdurur ve en küçük problemi doğrudan çözer.
3. İki yarıdan yalnızca target'ın bulunabileceği biri recursive aranır.
4. İki sorted half'ı tek sorted sequence hâline getirir.
5. Dynamic Programming tipik olarak overlapping subproblem sonuçlarını saklar.

Sonraki ders: **19 – Recurrence Relation: $T(n)=T(n-1)+1$.**

19 – Recurrence Relation: $T(n) = T(n-1) + 1$

Video: 2.1.1 Recurrence Relation ($T(n)=T(n-1)+1$) #1 (13:43)

Bu videonun ana fikri Recursive bir function'ın çalışma süresi, daha küçük input üzerindeki kendi çalışma süresiyle ifade edilebilir. Böyle bir denkleme Recurrence Relation denir.

Bu dersteki temel recurrence:

$$\begin{aligned}T(n) &= T(n-1) + 1, \quad n > 0 \\T(0) &= 1\end{aligned}$$

Çözümü:

$$T(n) = n + 1 = \theta(n)$$

1. Örnek recursive function

```
Algorithm Test(n)
  if n > 0
    print(n)
    Test(n - 1)
```

Test(3) çağırısı:

```
Test(3): print 3 -> Test(2)
Test(2): print 2 -> Test(1)
Test(1): print 1 -> Test(0)
Test(0): stop
```

Output:

3 2 1

Toplam $n+1$ function call yapılır; bunların n tanesinde print çalışır, sonuncusu base case'tir.

2. Call count ile work count farkı $n=3$ için:

- Function call sayısı: 4 (3,2,1,0)
- print sayısı: 3

Her call condition kontrolü yaptığı için base-case call da sabit maliyet taşır. Kullanılan cost model'e göre exact function n , $n+1$ veya başka linear form olabilir. Hepsisi aynı growth class'tadır:

$$\theta(n)$$

3. Recurrence nasıl yazılır? Input $n>0$ iken function iki iş yapar:

1. Constant local work: condition/print gibi $\theta(1)$
2. $n-1$ üzerinde bir recursive call: $T(n-1)$

Bu nedenle:

$$T(n) = T(n-1) + c$$

Sabit c growth açısından 1 ile temsil edilebilir:

$$T(n) = T(n-1) + 1$$

Base case de sıfır süre değil constant süre kullanır:

$$T(0) = 1$$

Program hiçbir print yapmasa bile call, condition ve return maliyeti vardır.

4. Back Substitution Başlangıç:

$$T(n) = T(n-1) + 1$$

Bir önceki terim:

$$T(n-1) = T(n-2) + 1$$

Yerine koy:

$$\begin{aligned} T(n) &= [T(n-2)+1] + 1 \\ &= T(n-2) + 2 \end{aligned}$$

Bir adım daha:

$$T(n) = T(n-3) + 3$$

Pattern:

$$T(n) = T(n-k) + k$$

5. Base case'e ulaşma Recurrence'ın bildiğimiz değeri $T(0)$ dır. Bu nedenle:

$$\begin{aligned} n - k &= 0 \\ k &= n \end{aligned}$$

Pattern'e yaz:

$$\begin{aligned} T(n) &= T(n-n) + n \\ &= T(0) + n \\ &= 1 + n \end{aligned}$$

Sonuç:

$$\begin{aligned} T(n) &= n + 1 \\ T(n) &= \theta(n) \end{aligned}$$

6. Recursion Tree görünümü Bu recurrence dallanan bir tree değil, tek chain üretir:

```
T(n)      local cost 1
|
T(n-1)    local cost 1
|
T(n-2)    local cost 1
|
...
|
T(0)      local cost 1
```

- Level sayısı: $n+1$
- Her level maliyeti: constant

$$\text{Total} = (n+1) * \theta(1) = \theta(n)$$

7. General constant form

$$\begin{aligned} T(n) &= T(n-1) + c \\ T(0) &= d \end{aligned}$$

Açılım:

$$\begin{aligned} T(n) &= T(0) + nc \\ &= d + nc \\ &= \theta(n) \end{aligned}$$

Pozitif c ve d exact sonucu değiştirir, class'ı değiştirmez.

8. Space Complexity Her recursive call call stack üzerinde bir frame tutar. Recursion depth $n+1$ olduğundan:

$$\text{Auxiliary Space} = \theta(n)$$

Time $\theta(n)$ ve stack space $\theta(n)$ dir. Aynı işlem iterative loop ile yazılırsa auxiliary space $\theta(1)$ olabilir.

Tail recursion compiler tarafından optimize edilirse stack azalabilir; fakat C/C++/Java gibi ortamlarda bu optimizasyon garanti edilmez.

9. Correctness ve termination n nonnegative integer ise her call input'u 1 azaltır:

$$n, n-1, \dots, 1, 0$$

Bu sequence base case'e ulaşır. Negative input kabul edilip yalnızca $n=0$ base case kullanılsaydı termination sorunu doğabilirdi; input domain açık olmalıdır.

10. Sık yapılan hatalar $T(0)=0$ yazmak zorunda sanmak Base case'in return ve condition maliyeti constant'tır; asymptotic analizde genellikle $\theta(1)$ alınır.

Recursive call'u constant saymak Test($n-1$) kendi içinde tekrar çalıştığı için maliyeti $T(n-1)$ dir.

n calls ile $n+1$ calls farkına takılmak Exact sayımda fark vardır; asymptotic class ikisinde de linear'dır.

Time ile stack space'i aynı kavram sanmak İkisi bu örnekte aynı class'a tesadüfen sahiptir; ayrı analiz edilmelidir.

11. Kavram sözlüğü

English term	Türkçe açıklama
Recurrence Relation	$T(n)$ 'i daha küçük input süreleriyle tanımlayan denklem
Base Case	Recursion'ın durduğu ve doğrudan çözülen durum
Recursive Case	Function'ın kendisini daha küçük input ile çağırdığı durum
Back Substitution	Küçük recurrence terimlerini tekrar tekrar yerine koyma yöntemi
Recursion Depth	Aynı anda aktif olabilecek recursive call sayısı
Call Stack	Active function frames'in tutulduğu bellek yapısı
Local Work	Recursive call dışındaki current call maliyeti

12. Kendini sına

1. $T(n)$ neden $n+1$ call üretir?
2. Recurrence'daki $+1$ neyi temsil eder?
3. k açılım sonra neden $T(n-k)+k$ elde edilir?
4. $k=n$ seçilmesinin nedeni nedir?
5. Stack space neden $\theta(n)$ dir?

Kısa cevaplar

1. n positive call'a ek olarak bir $T(0)$ base-case call vardır.
2. Her call'daki constant local work'ü.
3. Her açılım input'u bir azaltıp toplam maliyete bir ekler.
4. Bilinen base case $T(0)$ a ulaşmak için $n-k=0$ yapılır.
5. Recursion depth linear'dır ve her active call constant-size frame tutar.

Sonraki ders: **20 – Recurrence Relation: $T(n)=T(n-1)+n$.**

20 – Recurrence Relation: $T(n) = T(n-1) + n$

Video: 2.1.2 Recurrence Relation ($T(n)=T(n-1)+n$) #2 (16:00)

Bu videonun ana fikri Recursive call input'ü bir azaltırken her call'daki local work current input ile linear büyüyorsa toplam çalışma üçgensel toplam oluşturur:

$$\begin{aligned}T(n) &= T(n-1) + n \\T(0) &= \theta(1)\end{aligned}$$

Çözüm:

$$T(n) = \theta(n^2)$$

1. Örnek function yapısı

```
Algorithm Test(n)
  if n > 0
    for i <- 0 to n - 1
      print(i)
    Test(n - 1)
```

n seviyesinde loop n kez; sonraki call'da n-1 kez; sonra n-2 kez çalışır.

2. Ayrıntılı recurrence Cost model'e göre:

- Condition: 1
- Loop control: yaklaşık n+1
- Loop body: n
- Recursive call: $T(n-1)$

Exact forma benzer bir denklem:

$$T(n) = T(n-1) + 2n + 2$$

Local kısmın tight class'ı:

$$2n+2 = \theta(n)$$

Bu yüzden asymptotic recurrence:

$$T(n) = T(n-1) + \theta(n)$$

Videonun sade yazımı:

$$\begin{aligned}T(n) &= T(n-1) + n \\T(0) &= 1\end{aligned}$$

3. Recursion Tree / Chain yöntemi

Level 0: $T(n)$	local cost n
Level 1: $T(n-1)$	local cost n-1
Level 2: $T(n-2)$	local cost n-2
...	
Level n-1: $T(1)$	local cost 1
Level n: $T(0)$	local cost constant

Toplam local work:

$$n + (n-1) + (n-2) + \dots + 2 + 1 \\ = n(n+1)/2$$

Base-case constant eklenir:

$$T(n) = T(0) + n(n+1)/2$$

Leading term $n^2/2$:

$$T(n) = \theta(n^2)$$

4. Back Substitution Başlangıç:

$$T(n) = T(n-1) + n$$

Bir açılım:

$$T(n-1) = T(n-2) + (n-1)$$

$$T(n) = T(n-2) + (n-1) + n$$

İkinci açılım:

$$T(n) = T(n-3) + (n-2) + (n-1) + n$$

k açılım sonra doğru pattern:

$$T(n) = T(n-k) + (n-k+1) + \dots + (n-1) + n$$

Base case için:

$$n-k = 0 \Rightarrow k=n$$

Sonuç:

$$T(n) = T(0) + 1 + 2 + \dots + n \\ = 1 + n(n+1)/2 \\ = \theta(n^2)$$

5. Neden terimleri erken toplamıyoruz? İlk birkaç açılımı ayrı tutmak pattern'i görmeyi kolaylaştırır:

$$n \\ (n-1)+n \\ (n-2)+(n-1)+n$$

Her adımda algebraic olarak birleştirmek yanlış değildir; fakat k adım pattern'ini gizleyebilir. Önce sequence'i tanı, sonra summation formülünü uygula.

6. Summation notation Recurrence doğrudan şöyle yazılabilir:

$$T(n) = T(0) + \text{sum}(i=1 \text{ to } n) \ i$$

Bilinen toplam:

$$\text{sum}(i=1 \text{ to } n) \ i = n(n+1)/2 = \theta(n^2)$$

Bu bakış genelleştirilebilir:

$$\begin{aligned} T(n) &= T(n-1) + f(n) \\ \Rightarrow T(n) &= T(0) + \text{sum}(i=1 \text{ to } n) \ f(i) \end{aligned}$$

Asıl çözüm “f(n)’i n ile çarpmak” değil, f(i) değerlerini toplamaktır.

7. Exact ve asymptotic recurrence ayrımı

$$T(n)=T(n-1)+2n+2$$

ile

$$T(n)=T(n-1)+n$$

exact olarak aynı değildir. Fakat local work her ikisinde $\theta(n)$ olduğu için çözüm class’ları aynıdır:

$$\theta(n^2)$$

Exact bir recurrence çözülüyorsa katsayıları keyfi değiştiremeyiz. Yalnızca asymptotic hedef açık olduğunda $\theta(n)$ ile sadeleştiririz.

8. Space Complexity

Recursion depth yine $n+1$ dir. Her frame loop variable ve sabit metadata tutar:

$$\text{Auxiliary Stack Space} = \theta(n)$$

Loop’un n kez çalışması aynı anda n adet variable saklandığı anlamına gelmez. Time toplamı $\theta(n^2)$, active stack depth $\theta(n)$ dir.

9. Generalization

Sabit $p \geq 0$ için:

$$T(n) = T(n-1) + n^p$$

Power-sum sonucu:

$$\text{sum}(i=1 \text{ to } n) \ i^p = \theta(n^{(p+1)})$$

Dolayısıyla:

$$T(n) = \theta(n^{(p+1)})$$

Örnekler:

$$\begin{aligned} +1 &\rightarrow \theta(n) \\ +n &\rightarrow \theta(n^2) \\ +n^2 &\rightarrow \theta(n^3) \end{aligned}$$

Bu kural polynomial local work için geçerlidir; her f(n) için körlemesine kullanılamaz.

10. Sık yapılan hatalar

Yalnızca en üst call’ın n işini saymak Alt seviyelerdeki $n-1$, $n-2$, ... maliyetleri de eklenmelidir.

Nested loop olmadan quadratic olamayacağını sanmak Recursive seviyelerdeki linear işler toplandığında quadratic sonuç oluşabilir.

Time n^2 ise stack de n^2 sanmak Stack aynı anda active call sayısına bağlıdır; burada $\theta(n)$ dir.

Asymptotic sadeleştirmeyi exact equality sanmak $2n+2$ exact olarak n değildir; yalnızca aynı Theta class'ındadır.

11. Kavram sözlüğü

English term	Türkçe açıklama
Decreasing Recurrence	Input'u her recursive adımda sabit miktarda azaltan recurrence
Recursion Tree	Recursive calls ve level maliyetlerini gösteren yapı
Arithmetic Series	Sabit farkla ilerleyen sayıların toplamı
Triangular Sum	$1+2+\dots+n = n(n+1)/2$ toplamı
Summation	Bir sequence'in terimlerini sigma mantığıyla toplama
Exact Recurrence	Bütün katsayı ve sabit maliyetleri koruyan denklem
Asymptotic Recurrence	Maliyetleri growth class düzeyinde ifade eden denklem

12. Kendini sinama

1. Her recursive level'daki local work değerleri nelerdir?
2. Toplam neden arithmetic series oluşturur?
3. k açılım sonraki doğru pattern nedir?
4. Time $\theta(n^2)$ iken stack neden $\theta(n)$ dir?
5. $T(n)=T(n-1)+n^2$ için sonuç nedir?

Kısa cevaplar

1. $n, n-1, n-2, \dots, 1$.
2. Input her level'da bir azaldığı için maliyetler ardışık integer'lardır.
3. $T(n-k)+(n-k+1)+\dots+n$.
4. Aynı anda yalnızca recursion chain'deki $n+1$ frame aktiftir.
5. $\theta(n^3)$.

Sonraki ders: **21 – Recurrence Relation: $T(n)=T(n-1)+\log n$**

21 – Recurrence Relation: $T(n) = T(n-1) + \log n$

Video: 2.1.3 Recurrence Relation ($T(n)=T(n-1)+\log n$) #3 (12:25)

Bu videonun ana fikri Recursive input her adımda bir azalırken current call'daki loop logarithmic çalışıyorsa:

$$\begin{aligned}T(n) &= T(n-1) + \log n \\T(0) &= \theta(1)\end{aligned}$$

Local costs toplanınca $\log(n!)$ elde edilir:

$$T(n) = \theta(\log(n!)) = \theta(n \log n)$$

1. Örnek function yapısı

```
Algorithm Test(n)
  if n > 0
    for i <- 1; i < n; i <- i * 2
      print(i)
    Test(n - 1)
```

İç loop'ta i her adımda iki katına çıkar:

1, 2, 4, 8, ...

Current input n için loop:

$$\theta(\log n)$$

çalışır. Recursive call ise $T(n-1)$ maliyetlidir.

2. Recurrence oluşturma

$$\begin{aligned}T(n) &= T(n-1) + \theta(\log n), n > 0 \\T(0) &= \theta(1)\end{aligned}$$

Sade notation:

$$\begin{aligned}T(n) &= T(n-1) + \log n \\T(0) &= 1\end{aligned}$$

Logarithm base'i yalnızca sabit katsayı farkı oluşturduğu için asymptotic sonuçta önemsizdir.

3. Recursion Tree / Chain Her level'in local cost'u:

Level 0: $\log n$
Level 1: $\log(n-1)$
Level 2: $\log(n-2)$
...
Level $n-2$: $\log 2$
Level $n-1$: $\log 1 = 0$
Base: constant

Toplam:

$$T(n) = T(0) + \log n + \log(n-1) + \dots + \log 2 + \log 1$$

4. Log product dönüşümü

Logarithm identity:

$$\log a + \log b = \log(ab)$$

Bu nedenle:

$$\begin{aligned}\log 1 + \log 2 + \dots + \log n \\ &= \log(1 * 2 * \dots * n) \\ &= \log(n!)\end{aligned}$$

Dolayısıyla exact toplamsal biçim:

$$T(n) = \theta(1) + \log(n!)$$

5. Back Substitution

$$T(n) = T(n-1) + \log n$$

Bir açılım:

$$\begin{aligned}T(n-1) &= T(n-2) + \log(n-1) \\ T(n) &= T(n-2) + \log(n-1) + \log n\end{aligned}$$

İkinci açılım:

$$T(n) = T(n-3) + \log(n-2) + \log(n-1) + \log n$$

k açılım sonra:

$$\begin{aligned}T(n) &= T(n-k) \\ &\quad + \log(n-k+1) + \dots + \log(n-1) + \log n\end{aligned}$$

Base case'e ulaşmak için:

$$n-k = 0 \Rightarrow k=n$$

Sonuç:

$$\begin{aligned}T(n) &= T(0) + \log 1 + \log 2 + \dots + \log n \\ &= T(0) + \log(n!)\end{aligned}$$

6. Düzeltme: $\log(n!)$ için tight bound Video, önceki dersteki gevşek yaklaşıma dayanarak $\log(n!)$ için yalnızca upper bound bulunduğunu söyler. Doğru tight sonuç:

$$\log(n!) = \theta(n \log n)$$

Upper bound Her $k \leq n$ için $\log k \leq \log n$:

$$\begin{aligned}\log(n!) &= \sum_{k=1}^n \log k \\ &\leq n \log n\end{aligned}$$

Dolayısıyla:

$$\log(n!) = \theta(n \log n)$$

Lower bound Son $n/2$ terimin her biri en az $\log(n/2)$:

$$\begin{aligned}\log(n!) &\geq (n/2) \log(n/2) \\ &= (n/2)(\log n - \log 2) \\ &= \Omega(n \log n)\end{aligned}$$

Upper ve lower eşleşir:

$$\log(n!) = \theta(n \log n)$$

Bu nedenle recurrence'ın en doğru tight sonucu:

$$T(n) = \theta(n \log n)$$

7. Stirling ile daha ayrıntılı sonuç Stirling's Approximation:

$$\ln(n!) = n \ln n - n + (1/2) \ln(2\pi n) + o(1)$$

Dolayısıyla leading term $n \log n$ dir. Log base değişirse bütün ifade sabit katsayıyla değişir, Theta aynı kalır.

8. İlk üç decreasing recurrence karşılaştırması

Recurrence	Toplam	Sonuç
$T(n)=T(n-1)+1$	sum 1	$\theta(n)$
$T(n)=T(n-1)+\log n$	sum $\log i = \log(n!)$	$\theta(n \log n)$
$T(n)=T(n-1)+n$	sum i	$\theta(n^2)$
$T(n)=T(n-1)+n^2$	sum i^2	$\theta(n^3)$

Bu tablo local work büyüdükçe total time'ın nasıl değiştiğini gösterir.

9. Genel çözüm: summation

$$T(n) = T(n-1) + f(n)$$

için temel dönüşüm:

$$T(n) = T(0) + \sum_{i=1}^n f(i)$$

Doğru genel yöntem budur. $f(n)$ 'i doğrudan n ile çarpmak her zaman geçerli değildir.

Ne zaman $n*f(n)$ kestirmesi çalışır? f nondecreasing ve son sabit oranlı bölümde aynı order'da kalıyorsa, yani örneğin:

$$f(n/2) = \theta(f(n))$$

ise:

$$\sum_{i=1}^n f(i) = \theta(n f(n))$$

Bu şu örneklerde çalışır:

$$\begin{aligned}f(n)=1 &\rightarrow \theta(n) \\ f(n)=\log n &\rightarrow \theta(n \log n) \\ f(n)=n^p &\rightarrow \theta(n^{p+1})\end{aligned}$$

Çalışmadığı örnek

$$f(n)=1/n$$

Körlemesine $n*f(n)=1$ denirse yanlış olur. Gerçek toplam harmonic series'tir:

$$\text{sum}(i=1 \text{ to } n) 1/i = \theta(\log n)$$

Bu nedenle kestirmeden önce function davranışını kontrol et.

10. T(n-c) biçimi Sabit pozitif c için:

$$T(n) = T(n-c) + f(n)$$

Yaklaşık n/c recursive level vardır. Toplam, şu arithmetic subsequence üzerindedir:

$$f(n) + f(n-c) + f(n-2c) + \dots$$

c sabit olduğundan birçok regular polynomial/logarithmic f için class, $c=1$ durumuyla aynı kalır:

$$T(n)=T(n-100)+n \rightarrow \theta(n^2)$$

Fakat base conditions, input domain ve $n \bmod c$ durumları exact çözümde tanımlanmalıdır.

11. Space Complexity Recursive input birer azaldığı için depth:

$$\theta(n)$$

Her call'daki logarithmic loop sequential çalışır; aynı anda $\log n$ frame üretmez. Bu nedenle:

$$\text{Auxiliary Stack Space} = \theta(n)$$

Time $\theta(n \log n)$, stack space $\theta(n)$ dir.

12. Sık yapılan hatalar $\log(n!)$ için yalnızca $O(n \log n)$ yazmak Upper bound doğrudur fakat tight sonuç $\theta(n \log n)$ dir.

$\log n + \log(n-1)$ toplamını $\log(n+(n-1))$ yapmak Log toplamı iç değerlerin product'ına dönüşür, sum'ına değil.

Her decreasing recurrence'da $n*f(n)$ kullanmak Genel çözüm summation'dır; kestirme regularity koşulu ister.

Loop'un $\log n$ süresini recursion depth sanmak Loop local work'tür; depth input'un $n-1$ ile azalmasından dolayı linear'dır.

13. Kavram sözlüğü

English term	Türkçe açıklama
Log-Factorial	$\log(n!)$, yani $\log 1 + \dots + \log n$
Regular Growth	Sabit oranlı input değişiminde aynı order'ı koruyan büyüme davranışı
Harmonic Series	$1 + 1/2 + \dots + 1/n = \theta(\log n)$ toplamı
Arithmetic Subsequence	Sabit farkla seçilen $n, n-c, n-2c, \dots$ değerleri
Local Cost	Bir recursive call içinde alt call dışında yapılan iş
Tight Solution	Recurrence için matching upper/lower bound veren sonuç

14. Kendini sinama

1. Her level'in local cost'u nedir?
2. Log terimlerinin toplamı neden $\log(n!)$ olur?
3. $\log(n!)$ lower bound'u nasıl elde edilir?
4. $T(n)=T(n-1)+f(n)$ için gerçek genel çözüm nedir?
5. $T(n)=T(n-1)+1/n$ neden $\theta(1)$ değildir?

Kısa cevaplar

1. Current input m için $\theta(\log m)$.
2. $\log a + \log b = \log(ab)$ identity'si nedeniyle.
3. Son $n/2$ log teriminin her birini $\log(n/2)$ ile aşağıdan sınırlayarak.
4. $T(0)+\sum(i=1 \text{ to } n)f(i)$.
5. Local costs harmonic series oluşturur ve toplam $\theta(\log n)$ dir.

Sonraki ders: **22 – Recurrence Relation: $T(n)=2T(n-1)+1$.**

22 – Recurrence Relation: $T(n) = 2T(n-1) + 1$

Video: 2.1.4 Recurrence Relation $T(n)=2T(n-1)+1$ #4 (10:42)

Bu videonun ana fikri Her non-base call aynı $n-1$ inputuyla **iki** recursive call yaparsa call tree her level'da iki kat genişler:

$$\begin{aligned}T(n) &= 2T(n-1) + 1 \\T(0) &= 1\end{aligned}$$

Exact çözüm:

$$T(n) = 2^{(n+1)} - 1$$

Tight complexity:

$$T(n) = \Theta(2^n)$$

1. Örnek function yapısı

```
Algorithm Test(n)
  if n > 0
    print(n)
    Test(n - 1)
    Test(n - 1)
```

Bir call:

- Constant local work yapar.
- Aynı boyutta iki child call üretir.

Bu iki call birbirinin sonucunu paylaşmaz; aynı subproblem tekrar çözülür.

2. Recurrence oluşturma Her recursive call $T(n-1)$ maliyetlidir ve iki tane vardır:

$$\begin{aligned}T(n) &= T(n-1) + T(n-1) + \Theta(1) \\&= 2T(n-1) + \Theta(1)\end{aligned}$$

Sade exact model:

$$\begin{aligned}T(n) &= 2T(n-1) + 1, \quad n > 0 \\T(0) &= 1\end{aligned}$$

Baştaki 2, "local work iki kat" değil, iki recursive subproblem demektir.

3. Recursion Tree

Level 0:	$T(n)$	1 node
Level 1:	$T(n-1) \quad T(n-1)$	2 nodes
Level 2:	$T(n-2) \quad \dots \quad T(n-2)$	4 nodes
...		
Level i:		2^i nodes

Input her level'da bir azalır. Base case'e n adımda ulaşılır:

$$\text{Tree height} = n$$

Level i'deki local cost:

$$2^i * 1 = 2^i$$

4. Geometric Series toplamı

Bütün levels:

$$T(n) = 1 + 2 + 2^2 + \dots + 2^n$$

Geometric series formülü:

$$1 + r + r^2 + \dots + r^n = (r^{n+1} - 1) / (r - 1)$$

r=2:

$$T(n) = 2^{n+1} - 1$$

Sabit factor ve düşük terim atılır:

$$T(n) = \theta(2^n)$$

5. Back Substitution

$$T(n) = 2T(n-1) + 1$$

Bir açılım:

$$T(n-1) = 2T(n-2) + 1$$

$$\begin{aligned} T(n) &= 2[2T(n-2)+1] + 1 \\ &= 2^2T(n-2) + 2 + 1 \end{aligned}$$

İkinci açılım:

$$T(n) = 2^3T(n-3) + 2^2 + 2 + 1$$

k açılım sonra:

$$T(n) = 2^k T(n-k) + 2^{k-1} + \dots + 2 + 1$$

Base case:

$$n-k=0 \Rightarrow k=n$$

Yerine koy:

$$T(n) = 2^n T(0) + (2^n - 1)$$

T(0)=1:

$$\begin{aligned} T(n) &= 2^n + 2^n - 1 \\ &= 2^{n+1} - 1 \\ &= \theta(2^n) \end{aligned}$$

6. Call count sezgisi

Full binary tree'de:

- Internal calls: $2^n - 1$
- Base-case leaves: 2^n

- Total calls: $2^{(n+1)}-1$

Örneğin:

n	Total calls
0	1
1	3
2	7
3	15
4	31

Input'a 1 eklemek call sayısını yaklaşık ikiye katlar.

7. Overlapping Subproblems İki child da Test (n-1) i tekrar hesaplar. Bu overlapın belirgin örneğidir.

Eğer function'ın sonucu yalnızca ne bağlıysa, sonucu cache'lemek tekrarları önleyebilir:

Memoized time $\approx \theta(n)$

Ancak örnekte her call'ın print side effect'i isteniyorsa call'ları kaldırmak program davranışını değiştirir. Memoization yalnızca aynı subproblem sonucunu tekrar hesaplayan pure/uygun işlerde geçerlidir.

8. Space Complexity Call sayısı exponential olsa da aynı anda active olan calls yalnızca root-to-leaf path üzerindedir:

Recursion depth = n+1
Auxiliary stack space = $\theta(n)$

Time ile stack space'in neden farklı olabileceğinin iyi örneğidir.

9. $T(n)=aT(n-1)+1$ genellemesi Sabit $a>1$ için:

$T(n) = aT(n-1) + 1$

Call tree level ide a^i node üretir:

$T(n) = \theta(a^n)$

$a=1$ olduğunda branching ortadan kalkar ve sonuç $\theta(n)$ e düşer. $a=1$ ile $a>1$ arasındaki ayrım dramatiktir.

10. Sık yapılan hatalar iki recursive call'u toplamak yerine birini saymak İki call da çalışıyorsa recurrence katsayısı 2 olmalıdır.

Tree height ile node sayısını karıştırmak Height n, total nodes $\theta(2^n)$ dir.

Exponential time ise exponential stack sanmak Sibling calls sırayla tamamlanır; aynı anda yalnızca bir path aktiftir.

$2T(n-1)$ ile $T(n-2)$ 'yi aynı sanmak İlki iki subproblem, ikincisi tek ve daha küçük subproblem ifade eder.

11. Kavram sözlüğü

English term	Türkçe açıklama
Branching Factor	Her non-leaf call'ın ürettiği child call sayısı
Full Binary Recursion Tree	Her internal node'un iki child ürettiği call tree
Geometric Series	Ardışık terimleri sabit oranla değişen toplam
Leaf Call	Base case'e ulaşan ve child üretmeyen call
Overlapping Subproblems	Aynı küçük problemin birden çok kez hesaplanması
Memoization	Hesaplanan subproblem sonuçlarını cache'leme

12. Kendini sına

1. Recursion tree level ide kaç node vardır?
2. Tree height neden ndir?
3. $1+2+\dots+2^n$ toplamı nedir?
4. Time exponential iken stack neden linear'dır?
5. $T(n)=3T(n-1)+1$ için tight complexity nedir?

Kısa cevaplar

1. 2^i .
2. Input her edge'de bir azalır ve nden 0'a gelir.
3. $2^{(n+1)}-1$.
4. Aynı anda yalnızca tek root-to-leaf path'in frames'i aktiftir.
5. $\theta(3^n)$.

Sonraki ders: **23 – Master Theorem for Decreasing Functions.**

23 – “Master Theorem” for Decreasing Functions

Video: 2.2 Masters Theorem Decreasing Function (8:10)

Bu videonun ana fikri Video, aşağıdaki recurrence ailesi için doğrudan sonuç üretmeye çalışan bir kural sunar:

$$T(n) = aT(n-b) + f(n)$$

Burada $a > 0$, $b > 0$ sabit ve çoğunlukla $f(n) = \theta(n^k)$ kabul edilir.

Terminoloji uyarısı: Bu kural, algoritma literatüründeki standart Master Theorem değildir. Standart theorem $T(n) = aT(n/b) + f(n)$ biçimindeki **dividing recurrences** içindir. Buradaki yapı decrease-by-b recurrence'dır.

Ayrıca videodaki $a > 1$ formülü tight sonuç açısından hatalı/geniştir. Bu not doğru çözümü unrolling ile çıkarır.

1. General recurrence

$$T(n) = aT(n-b) + \theta(n^k)$$

Varsayımlar:

- $a > 0$ ve $b > 0$ constant
- $k \geq 0$ constant
- Uygun base cases $n \leq n_0$ için $\theta(1)$
- $f(n)$ eventual nonnegative

Algorithm interpretation'da recursive call sayısı integer olduğundan tipik olarak $a \geq 1$ integer'dır. $0 < a < 1$ durumu daha çok matematiksel recurrence olarak incelenir.

2. Kaç recursive level vardır? Input her level'da b azalır:

$$n, n-b, n-2b, \dots, n-mb$$

Base threshold'a yaklaşıp:

$$m \approx n/b$$

level sonra ulaşılır. b constant olduğu için:

$$m = \theta(n)$$

3. General unrolling Bir açılım:

$$T(n) = aT(n-b) + f(n)$$

İki açılım:

$$T(n) = a^2T(n-2b) + a f(n-b) + f(n)$$

m açılım sonra:

$$T(n) = a^m T(n-mb) + \sum_{j=0}^{m-1} a^j f(n-jb)$$

Bu formül üç case'in gerçek kaynağıdır.

4. Case 1 – a = 1 Branching yoktur; tek recursion chain vardır:

$$T(n) = T(n-b) + \theta(n^k)$$

Yaklaşık n/b polynomial terim toplanır:

$$T(n) = \theta(n^{(k+1)})$$

Örnekler:

$$\begin{aligned} T(n) &= T(n-1) + 1 && \rightarrow \theta(n) \\ T(n) &= T(n-1) + \log n && \rightarrow \theta(n \log n) \quad // \text{ polynomial form dışı ama toplamla çözülür} \\ T(n) &= T(n-1) + n && \rightarrow \theta(n^2) \\ T(n) &= T(n-1) + n^2 && \rightarrow \theta(n^3) \end{aligned}$$

Sabit b , class'ı değiştirmez:

$$T(n) = T(n-100) + n^2 \rightarrow \theta(n^3)$$

5. Case 2 – $a > 1$ Recursive calls her level'da a kat büyür. Depth yaklaşık n/b :

$$\text{leaf count} \approx a^{(n/b)}$$

Polynomial local work için exponential branching baskındır:

$$T(n) = \theta(a^{(n/b)})$$

Önemli video düzeltmesi Video $\theta(n^k * a^{(n/b)})$ benzeri bir sonuç önerir. Bu genellikle tight değildir.

Örnek:

$$T(n) = 2T(n-1) + n$$

Video kuralı $\theta(n2^n)$ gibi loose bir bound verir. Gerçek tight sonuç:

$$T(n) = \theta(2^n)$$

Neden? Unrolling toplamını leaf tarafına göre yeniden indekslersek:

$$\begin{aligned} T(n) / 2^n \\ = \text{base constant} + \sum_{r=1 \text{ to } n} \theta(r^k / 2^r) \end{aligned}$$

$\sum r^k / 2^r$ yakınsak, yani constant'tır. Bu yüzden extra n^k factor kalmaz.

6. Case 3 – $0 < a < 1$ Recursive contribution geometrically küçülür; current/top-level polynomial work baskındır:

$$T(n) = \theta(n^k)$$

Örnek:

$$\begin{aligned} T(n) &= (1/2)T(n-1) + n^2 \\ \Rightarrow T(n) &= \theta(n^2) \end{aligned}$$

Bu case çoğu recursive algorithm için literal "yarım recursive call" anlamına gelmez; mathematical recurrence olarak yararlıdır.

7. Düzeltilmiş case tablosu $T(n) = aT(n-b) + \theta(n^k)$ için:

Koşul	Tight solution
$0 < a < 1$	$\theta(n^k)$
$a = 1$	$\theta(n^{k+1})$
$a > 1$	$\theta(a^{n/b})$

Bu tablo $f(n)=\theta(n^k)$ ve constant a, b, k varsayımları altındadır.

8. Neden standard Master Theorem değil? Standart form:

$$T(n) = aT(n/b) + f(n)$$

Burada depth logarithmic'tir çünkü input bölünür.

Bu videodaki form:

$$T(n) = aT(n-b) + f(n)$$

Burada depth linear'dır çünkü input sabit miktarda azalır.

Form	Depth
$n-b$	$\theta(n)$
n/b	$\theta(\log n)$

Benzer semboller tamamen farklı sonuçlar üretir.

9. Örnekler $T(n)=3T(n-1)+1$

$$a=3>1, b=1, k=0$$

$$T(n)=\theta(3^n)$$

$T(n)=2T(n-2)+n^5$

$$a=2>1, b=2$$

$$T(n)=\theta(2^{n/2})$$

Polynomial factor exponential branching karşısında tight sonucu değiştirmez.

$T(n)=T(n-3)+n$

$$a=1, b=3, k=1$$

$$T(n)=\theta(n^2)$$

10. Sık yapılan hatalar Bu kuralı standard Master Theorem sanmak $n-b$ ve n/b recurrence'ları farklıdır.

$a>1$ durumunda polynomial factor eklemek Bu çoğunlukla doğru ama loose upper bound verir; tight sonuç yalnızca exponential term'dir.

b 'nin exponent denominator etkisini unutmak Depth n/b olduğu için leaf count $a^{n/b}$ dir.

Algorithm'da $0 < a < 1$ recursive call yorumlamak Recursive call sayısı kesirli olamaz; bu case mathematical recurrence bağlamındadır.

11. Kavram sözlüğü

English term	Türkçe açıklama
Decrease-by-b Recurrence	Input'u her adımda constant b kadar azaltan recurrence
Unrolling	Recurrence'ı tekrar tekrar açarak genel toplam elde etme
Branching Factor	Her call'ın ürettiği recursive child sayısı a
Convergent Series	Partial sums'i finite bir limite yaklaşan series
Tight Solution	Matching asymptotic upper ve lower bound
Standard Master Theorem	$aT(n/b)+f(n)$ formunu çözen theorem

12. Kendini sına

1. $n-b$ recurrence'ının depth'i neden linear'dır?
2. $a=1$, $f(n)=n^k$ durumunda sonuç nedir?
3. $a>1$ durumunda neden exponential term baskındır?
4. $T(n)=2T(n-1)+n^2$ için tight sonuç nedir?
5. Standard Master Theorem hangi form içindir?

Kısa cevaplar

1. Base case'e ulaşmak için yaklaşık n/b sabit azaltma gerekir.
2. $\theta(n^{k+1})$.
3. Call sayısı $a^{(n/b)}$ büyür ve polynomial local costs bunun yanında daha yavaş kalır.
4. $\theta(2^n)$.
5. $T(n)=aT(n/b)+f(n)$.

Sonraki ders: **24 – Recurrence Relation: $T(n)=T(n/2)+1$.**

24 – Recurrence Relation: $T(n) = T(n/2) + 1$

Video: 2.3.1 Recurrence Relation Dividing Function $T(n)=T(n/2)+1$ #1 (8:41)

Kaynak notu: Bu video için YouTube İngilizce altyazı sağlamadı. Ders, videonun resmi başlık/açıklamasında belirtilen Recursion Tree ve Back Substitution kapsamı ile recurrence'ın standart çözümü esas alınarak hazırlanmıştır.

Bu videonun ana fikri Recursive input her adımda yarıya iniyor ve her call constant local work yapıyorsa recursion depth logarithmic olur:

$$\begin{aligned}T(n) &= T(n/2) + 1 \\T(1) &= 1\end{aligned}$$

Çözüm:

$$T(n) = \theta(\log n)$$

1. Dividing recurrence ne demektir? Önceki derslerde:

$$n \rightarrow n-1 \rightarrow n-2 \rightarrow \dots$$

şeklinde linear sayıda level vardı.

Şimdi:

$$n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow \dots \rightarrow 1$$

Her step problem size'in sabit oranını kaldırır. Bu nedenle base case'e çok daha hızlı ulaşılır.

2. Algorithm kalıbı

```
Algorithm Test(n)
  if n <= 1
    return

  constantWork()
  Test(n / 2)
```

Her call:

- Bir adet smaller recursive call yapar.
- $\theta(1)$ local work yapar.

Recurrence:

$$T(n) = T(n/2) + \theta(1)$$

3. Recursion Tree / Chain Branching factor 1 olduğu için yapı chain'dir:

$T(n)$	cost 1
$T(n/2)$	cost 1
$T(n/4)$	cost 1
$T(n/8)$	cost 1

```
|
...
|
T(1)      cost 1
```

k level sonra problem size:

$$n / 2^k$$

Base case:

$$\begin{aligned} n / 2^k &= 1 \\ n &= 2^k \\ k &= \log_2 n \end{aligned}$$

Her level constant olduğundan:

$$\begin{aligned} \text{Total} &= \theta(\log n) * \theta(1) \\ &= \theta(\log n) \end{aligned}$$

4. Back Substitution

$$T(n) = T(n/2) + 1$$

Bir açılım:

$$\begin{aligned} T(n/2) &= T(n/4) + 1 \\ T(n) &= T(n/4) + 2 \end{aligned}$$

İkinci açılım:

$$T(n) = T(n/8) + 3$$

k açılım sonra:

$$T(n) = T(n/2^k) + k$$

$n/2^k=1$ ve $k=\log_2 n$:

$$\begin{aligned} T(n) &= T(1) + \log_2 n \\ &= 1 + \log_2 n \\ &= \theta(\log n) \end{aligned}$$

5. n power of 2 değilse Gerçek programda integer division kullanılır ve n tam 2 kuvveti olmayabilir:

$$10 \rightarrow 5 \rightarrow 2 \rightarrow 1$$

Level sayısı floor/ceiling ile değişir:

$$\text{floor}(\log_2 n) \text{ veya } \text{ceil}(\log_2 n) + \text{constant}$$

Bu rounding farkı asymptotic class'ı değiştirmez:

$$\theta(\log n)$$

Formal recurrence bazen şöyle yazılır:

$$T(n) = T(\text{floor}(n/2)) + 1$$

6. Binary Search bağlantısı Binary Search her comparison sonrasında arama aralığının yalnızca yarısında devam eder:

$$T(n) = T(n/2) + \theta(1)$$

Bu nedenle recursive veya iterative Binary Search:

$$\text{Time} = \theta(\log n)$$

Worst-case'te target bulunana veya aralık boşalana kadar logarithmic sayıda adım gerekir.

7. Logarithm base Problem her adımda $b > 1$ sabitine bölünüyorsa:

$$T(n) = T(n/b) + 1$$

Depth:

$$k = \log_b n$$

Change-of-base nedeniyle bütün constant bases aynı asymptotic class'ı verir:

$$\theta(\log_2 n) = \theta(\log_3 n) = \theta(\log n)$$

8. Space Complexity Recursive implementation'da call stack depth logarithmic'tir:

$$\text{Auxiliary Stack Space} = \theta(\log n)$$

Aynı process iterative loop ile yazılırsa:

$$\text{Auxiliary Space} = \theta(1)$$

Time her iki implementation'da da $\theta(\log n)$ kalabilir.

9. n-1 ile n/2 karşılaştırması

Recurrence	Depth	Constant level cost ile total
$T(n)=T(n-1)+1$	$\theta(n)$	$\theta(n)$
$T(n)=T(n/2)+1$	$\theta(\log n)$	$\theta(\log n)$

Tek karakterlik operator farkı (- yerine /) çözümü kökten değiştirir.

10. Sık yapılan hatalar n/2 görünce n/2 level sanmak Input her seferinde yeniden yarıya iner; level sayısı logarithmic'tir.

İki yarı oluştuğu için 2T(n/2) yazmak Algorithm yalnızca bir yarıyı recursive çözüyorsa katsayı 1'dir.

Base case'i T(0) seçip division ile 0'a ulaşmaya çalışmak Matematiksel çözüm için doğal threshold T(1)dir.

Recursive ve iterative space'i aynı sanmak Recursive sürüm call stack kullanır; iterative sürüm kullanmayabilir.

11. Kavram sözlüğü

English term	Türkçe açıklama
Dividing Recurrence	Input size'ı her step'te sabit orana bölen recurrence
Problem Shrinkage	Recursive input'un küçülme biçimi
Rounding	Non-integer division sonucunu floor veya ceiling ile integer yapma
Logarithmic Depth	Recursion level sayısının $\theta(\log n)$ olması
Branching Factor	Her call'daki recursive child sayısı; burada 1

12. Kendini sına

1. k level sonraki problem size nedir?
2. $n/2^k=1$ denklemini ne verir?
3. $n/2$ kuvveti değilse sonuç neden değişmez?
4. $T(n)=T(n/3)+1$ için sonuç nedir?
5. Recursive implementation'ın stack space'i nedir?

Kısa cevaplar

1. $n/2^k$.
2. $k=\log_2 n$.
3. Floor/ceiling yalnızca constant kadar level farkı yaratır.
4. $\theta(\log_3 n)=\theta(\log n)$.
5. $\theta(\log n)$.

Sonraki ders: **25 – Recurrence Relation: $T(n)=T(n/2)+n$.**

25 – Recurrence Relation: $T(n) = T(n/2) + n$

Video: 2.3.2 Recurrence Relation Dividing $T(n)=T(n/2)+n$ #2 (7:26)

Bu videonun ana fikri Problem size her recursive call'da yarıya iner fakat current level'da problem size kadar local work yapılır:

$$\begin{aligned}T(n) &= T(n/2) + n \\T(1) &= 1\end{aligned}$$

Level costs geometric olarak küçülür ve root maliyeti baskın olur:

$$T(n) = \theta(n)$$

1. Recursion chain

Level 0: $T(n)$	local cost n
Level 1: $T(n/2)$	local cost $n/2$
Level 2: $T(n/4)$	local cost $n/4$
Level 3: $T(n/8)$	local cost $n/8$
...	

Depth $\log_2 n$ olsa da her level'ın maliyeti aynı değildir; yarıya iner.

2. Geometric Series

 Toplam local work:

$$n + n/2 + n/4 + n/8 + \dots + 1$$

n common factor:

$$n(1 + 1/2 + 1/4 + 1/8 + \dots)$$

Infinite geometric series:

$$1 + 1/2 + 1/4 + \dots = 1/(1-1/2) = 2$$

Bu nedenle finite toplam 2 'den küçüktür ve n den büyüktür:

$$n \leq T(n) < 2n + \text{constant}$$

Sonuç:

$$T(n) = \theta(n)$$

Video düzeltmesi Videoda tüm $1+1/2+1/4+\dots$ serisinin yaklaşık 1 olduğu söylenen bir an vardır. Doğru toplam 2 'dir. Yalnızca $1/2+1/4+\dots$ tail'i 1 'e eşittir. Complexity sonucu yine $\theta(n)$ dir.

3. Back Substitution

$$T(n) = T(n/2) + n$$

Bir açılım:

$$T(n/2) = T(n/4) + n/2$$

$$T(n) = T(n/4) + n/2 + n$$

İkinci açılım:

$$T(n) = T(n/8) + n/4 + n/2 + n$$

k açılım sonra:

$$T(n) = T(n/2^k) + n/2^{(k-1)} + \dots + n/2 + n$$

Base case:

$$\begin{aligned} n/2^k &= 1 \\ k &= \log_2 n \end{aligned}$$

Yerine koyduğumuzda geometric sum elde edilir ve $\theta(n)$ sonucu çıkar.

4. Exact çözüm – n power of 2 $n=2^m$, $T(1)=1$ olsun:

$$T(n) = n + n/2 + \dots + 2 + T(1)$$

Power-of-two toplamı:

$$1 + 2 + 4 + \dots + n = 2n - 1$$

Dolayısıyla bu cost model'de:

$$T(n) = 2n - 1$$

ve tight class $\theta(n)$ dir.

5. Neden log n level, n log n yapmıyor? $\log n$ adet level bulunması tek başına $n \log n$ sonucu vermez. Bunun için her level'ın toplam maliyeti $\theta(n)$ olmalıdır.

Burada:

$$n, n/2, n/4, \dots$$

şeklinde maliyet azalır. Average level cost n değildir; toplam geometric series olarak linear kalır.

Karşılaştır:

$$\begin{aligned} T(n) &= T(n/2) + n && \rightarrow \theta(n) \\ T(n) &= 2T(n/2) + n && \rightarrow \theta(n \log n) \end{aligned}$$

İkinci recurrence'ta node sayısı iki katlandığı için her level'ın toplamı n kalır.

6. Root-dominated tree Bu recurrence'ta en büyük tek level root'tur:

$$\begin{aligned} \text{Root cost} &= n \\ \text{All lower levels combined} &< n \end{aligned}$$

Bu nedenle tree root-dominated olarak düşünülebilir. Standart Master Theorem'de bu, $f(n)$ in recursive contribution'dan polynomial olarak büyük olduğu case ile ilişkilidir.

7. Space Complexity Tek recursive child vardır ve input yarıya iner:

$$\begin{aligned} \text{Recursion depth} &= \theta(\log n) \\ \text{Auxiliary Stack Space} &= \theta(\log n) \end{aligned}$$

Local n work, n adet memory kullanıldığı anlamına gelmez. Work loop şeklindeyse extra space constant olabilir.

8. General ratio

$$T(n) = T(n/b) + n$$

Sabit $b > 1$ için level costs:

$$n + n/b + n/b^2 + \dots$$

Geometric ratio $1/b < 1$:

$$\text{sum} = n / (1 - 1/b) = \theta(n)$$

Dolayısıyla:

$$T(n) = \theta(n)$$

9. Sık yapılan hatalar Geometric sum'ı log n ile çarpmak Level costs eşit değildir; doğrudan toplanmalıdır.

Infinite series toplamını 1 sanmak $1 + 1/2 + \dots = 2$; $1/2 + 1/4 + \dots = 1$.

Recurrence'daki +n'i space sanmak +n local time'i temsil eder; space ayrıca analiz edilir.

Tek child ile iki child recurrence'ını karıştırmak Baştaki katsayı recursion tree genişliğini belirler.

10. Kavram sözlüğü

English term	Türkçe açıklama
Geometric Decay	Level maliyetinin sabit oranla küçülmesi
Root-dominated	Total cost'un asymptotically root/local üst seviye iş tarafından belirlenmesi
Finite Geometric Series	Sınırlı sayıda sabit oranlı terimin toplamı
Common Ratio	Ardışık terimler arasındaki sabit çarpan; burada 1/2
Level Cost	Recursion tree'nin bir level'ındaki bütün node'ların toplam local work'ü

11. Kendini sınam

1. Level 1'deki local cost nedir?
2. $1 + 1/2 + 1/4 + \dots$ toplamı nedir?
3. Recursion depth nedir?
4. Neden total $\theta(n \log n)$ değildir?
5. $T(n) = T(n/3) + n$ için sonuç nedir?

Kısa cevaplar

1. $n/2^i$.
2. 2.
3. $\theta(\log n)$.
4. Level costs geometric olarak azalır ve toplamı $\theta(n)$ dir.
5. $\theta(n)$.

Sonraki ders: **26 – Recurrence Relation: $T(n) = 2T(n/2) + n$.**

26 – Recurrence Relation: $T(n) = 2T(n/2) + n$

Video: 2.3.3 Recurrence Relation $T(n)=2T(n/2)+n$ #3 (11:20)

Bu videonun ana fikri Problem iki yarı boyutlu subproblem'a ayrılıyor ve her level'da toplam linear local work yapılıyor:

$$\begin{aligned}T(n) &= 2T(n/2) + n \\T(1) &= 1\end{aligned}$$

Recursion tree'nin her level'ı $\theta(n)$ maliyetlidir ve $\theta(\log n)$ level vardır:

$$T(n) = \theta(n \log n)$$

Bu, Merge Sort recurrence'ının temel biçimidir.

1. Algorithm kalıbı

```
Algorithm Test(problem of size n)
  if n <= 1
    solve directly
    return

  linearWork(n)
  Test(left half of size n/2)
  Test(right half of size n/2)
```

Her call:

- İki recursive child üretir: $2T(n/2)$
- Current size kadar divide/combine/local work yapar: $+n$

2. Recursion Tree – ilk levels

```
Level 0:           n
                /   \
Level 1:       n/2   n/2
              / \   / \
Level 2:     n/4 n/4 n/4 n/4
```

Level ide:

- Node sayısı: 2^i
- Her node'un problem size'i: $n/2^i$
- Her node'un local cost'u: $n/2^i$

Level total:

$$2^i * (n/2^i) = n$$

Yani node sayısı artarken node başına iş aynı oranda azalır; her level n kalır.

3. Tree height

Leaf size 1 olduğunda:

$$\begin{aligned}n/2^k &= 1 \\n &= 2^k \\k &= \log_2 n\end{aligned}$$

Dolayısıyla internal level sayısı yaklaşık $\log_2 n$ dir.

Her internal level n maliyetli:

$$\text{Internal cost} = n \log_2 n$$

4. Leaf cost Son level'da node sayısı:

$$2^k = 2^{(\log_2 n)} = n$$

Her leaf constant maliyetliyse:

$$\text{Leaf cost} = n * \theta(1) = \theta(n)$$

Toplam:

$$\begin{aligned} T(n) &= n \log_2 n + \theta(n) \\ &= \theta(n \log n) \end{aligned}$$

Linear leaf term, $n \log n$ yanında lower order kalır.

5. Back Substitution Başlangıç:

$$T(n) = 2T(n/2) + n$$

Bir açılım:

$$\begin{aligned} T(n/2) &= 2T(n/4) + n/2 \\ T(n) &= 2[2T(n/4) + n/2] + n \\ &= 2^2 T(n/4) + 2n \end{aligned}$$

İkinci açılım:

$$\begin{aligned} T(n) &= 2^2 [2T(n/8) + n/4] + 2n \\ &= 2^3 T(n/8) + 3n \end{aligned}$$

k açılım sonra pattern:

$$T(n) = 2^k T(n/2^k) + kn$$

Base case:

$$\begin{aligned} n/2^k &= 1 \\ k &= \log_2 n \\ 2^k &= n \end{aligned}$$

Yerine koy:

$$T(n) = nT(1) + n \log_2 n$$

$T(1)=1$:

$$\begin{aligned} T(n) &= n + n \log_2 n \\ &= \theta(n \log n) \end{aligned}$$

6. $T(n/2)+n$ ile kritik karşılaştırma Tek child

$$T(n) = T(n/2) + n$$

Level totals:

$n, n/2, n/4, \dots$

Geometric toplam:

$\theta(n)$

İki child

$T(n) = 2T(n/2) + n$

Level totals:

$n, n, n, \dots, n$

$\log n$ level:

$\theta(n \log n)$

Recurrence başındaki 2, total work'e bir $\log n$ factor eklenmesine neden olur.

7. Merge Sort bağlantısı Merge Sort:

1. Array'i iki yarıya böler.
2. Her yarıyı recursively sort eder.
3. İki sorted half'ı linear zamanda merge eder.

$T(n) = 2T(n/2) + \theta(n)$
 $= \theta(n \log n)$

Buradaki $+n$, merge aşamasındaki toplam comparison/copy işini temsil eder.

8. Work distribution sezgisi Toplam input volume her level'da korunur:

$1 * n$
 $2 * (n/2)$
 $4 * (n/4)$
 $8 * (n/8)$
 $\dots$

Hepsi ndir. Bu, balanced divide-and-conquer recurrence'larında sık görülen bir invariant'tır.

9. Space Complexity Call stack Calls ikiye dallansa da depth logarithmic'tir. Sibling branches çoğu normal sequential execution'da aynı anda active değildir:

Stack depth = $\theta(\log n)$

Algorithm-specific buffers Recurrence yalnızca time'i tanımlar. Örneğin Merge Sort merge buffer nede niyle:

Auxiliary data space = $\theta(n)$

kullanabilir. Toplam auxiliary space implementation'a göre $\theta(n)$ olur; yalnızca stack'i sorarsak $\theta(\log n)$ dir.

10. Standard Master Theorem görünümü

$T(n) = aT(n/b) + f(n)$

Bu recurrence için:

```
a=2
b=2
f(n)=n
n^(log_b a)=n^(log_2 2)=n
```

Recursive tree contribution ile local work aynı class'tadır. Standard Master Theorem'in balanced case'i:

$T(n)=\theta(n \log n)$

Bu theorem sonraki videolarda ayrıntılı ele alınacaktır.

11. n power of 2 varsayımı Analizi sadeleştirmek için $n=2^m$ kabul edilir. Gerçek input'ta floors/ceilings kullanılır:

$T(n)=T(\text{floor}(n/2))+T(\text{ceil}(n/2))+\theta(n)$

Parçalar dengeli kaldığı sürece tight class değişmez:

$\theta(n \log n)$

12. Sık yapılan hatalar Level i maliyetini yalnızca $n/2^i$ sanmak Bu node başına maliyettir; 2^i node ile çarpınca level total n olur.

Leaf level'ı tamamen yok saymak n leaf vardır ve toplam $\theta(n)$ maliyet ekler; sonucu değiştirmese de doğru analizde gösterilir.

İki child nedeniyle stack'i $\Theta(n)$ sanmak Sequential recursion'da stack depth tree width'e değil height'a bağlıdır.

Time recurrence'dan doğrudan bütün space'i çıkarmak Merge buffer gibi algorithm-specific memory ayrıca incelenmelidir.

13. Kavram sözlüğü

English term	Türkçe açıklama
Balanced Recursion Tree	Subproblem sizes'ın benzer kaldığı recursion tree
Level Invariant	Her level'da korunan toplam; burada total problem volume n
Leaf Cost	Base-case nodes'in toplam maliyeti
Merge Cost	Sorted subresults'ı birleştirme maliyeti
Width	Belirli level'daki node sayısı
Height	Root'tan leaf'e maximum edge/level sayısı

14. Kendini sına

1. Level ide kaç node vardır?
2. Her node'un size'ı nedir?
3. Level total neden ndir?
4. Kaç level vardır?
5. $T(n)=T(n/2)+n$ ile temel fark nedir?
6. Merge Sort neden bu recurrence'ı üretir?

Kısa cevaplar

1. 2^i .

2. $n/2^i$.
3. $2^i * n/2^i = n$.
4. $\theta(\log n)$.
5. Tek-child recurrence'ta level totals küçülür; iki-child recurrence'ta n kalır.
6. İki half recursively sort edilir ve merge linear sürer.

Sonraki ders: **27 – Master Theorem for Dividing Functions #1**.

27 – Master’s Theorem for Dividing Functions

Video: 2.4.1 Master’s Theorem for Dividing Functions #1 (16:50)

Bu videonun ana fikri Divide and Conquer algoritmalarında sık görülen recurrence biçimi şudur:

$$T(n) = aT(n/b) + f(n)$$

Burada recursive work ile bir call içinde yapılan non-recursive work karşılaştırılır. Karşılaştırmanın merkezindeki ifade:

$$n^{(\log_b a)}$$

Video, daha ayrıntılı bir polylogarithmic biçim kullanır:

$$f(n) = \theta(n^k \log^p n)$$

Temel soru şudur:

$$c = \log_b a$$

c, k'den büyük mü, eşit mi, küçük mü?

Bu karşılaştırma recursion tree’de için leaves tarafında mı, bütün levels’a dengeli mi, yoksa root tarafında mı yoğunlaştığını söyler.

1. Parametrelerin anlamı

$$T(n) = aT(n/b) + f(n)$$

Sembol	Anlamı
a	Her call’ın ürettiği subproblem sayısı
b	Her subproblem’in size küçülme oranı
n/b	Bir child problem’in size’ı
f(n)	Divide, combine ve diğer local work
$\log_b a$	Recursion tree’nin leaves büyümesini belirleyen exponent

Theorem’in temel varsayımları:

a ≥ 1
b > 1
f(n) asymptotically nonnegative

Genellikle floors ve ceilings sonucu değıştirmediği için analizde yazılmaz.

2. Neden $n^{(\log_b a)}$? Level ide yaklaşık:

Node sayısı = a^i
Node size = n/b^i

Tree height:

$$n/b^h = 1$$
$$h = \log_b n$$

Leaf sayısı:

$$a^h = a^{(\log_b n)} = n^{(\log_b a)}$$

Her leaf constant iş yapıyorsa toplam leaf work:

$$\theta(n^{(\log_b a)})$$

Bu nedenle $f(n)$ ile karşılaştırılacak doğal benchmark budur.

3. Genişletilmiş üç ana case Kısaltma yapalım:

$$\begin{aligned} c &= \log_b a \\ f(n) &= \theta(n^k \log^p n) \end{aligned}$$

Case 1 – $c > k$ Recursive expansion polynomial olarak daha hızlı büyür:

$$T(n) = \theta(n^c)$$

İş tree'nin altına, yani leaves tarafına yoğunlaşır.

Case 2 – $c = k$ Recursive benchmark ile $f(n)$ aynı polynomial power'a sahiptir. $\log^p n$ belirleyici olur:

$$\begin{aligned} p > -1 &\Rightarrow T(n) = \theta(n^k \log^{(p+1)} n) \\ p = -1 &\Rightarrow T(n) = \theta(n^k \log \log n) \\ p < -1 &\Rightarrow T(n) = \theta(n^k) \end{aligned}$$

En sık karşılaşılan $p=0$ biçimi:

$$\begin{aligned} f(n) &= \theta(n^k) \\ T(n) &= \theta(n^k \log n) \end{aligned}$$

Case 3 – $c < k$ Local work polynomial olarak daha hızlı büyür. Uygun regularity condition altında:

$$\begin{aligned} T(n) &= \theta(f(n)) \\ &= \theta(n^k \log^p n) \end{aligned}$$

İş root tarafına yoğunlaşır.

4. Standard Master Theorem'in kısa biçimi Standard biçim çoğunlukla şu üç case ile öğretilir:

1. $f(n) = O(n^{(c-\epsilon)})$
 $T(n) = \theta(n^c)$
2. $f(n) = \theta(n^c \log^p n)$, $p \geq 0$
 $T(n) = \theta(n^c \log^{(p+1)} n)$
3. $f(n) = \Omega(n^{(c+\epsilon)})$
ve regularity condition sağlanıyorsa
 $T(n) = \theta(f(n))$

Buradaki epsilon, pozitif sabittir. Yalnızca bir log factor farkı, polynomial gap değildir.

5. Case 1 örnekleri Örnek A

$$T(n) = 2T(n/2) + 1$$

$$\begin{aligned} a &= 2, \quad b = 2 \\ c &= \log_2 2 = 1 \\ f(n) &= 1 = n^0 \Rightarrow k = 0 \end{aligned}$$

$$c > k$$

$$1 > 0$$

$$T(n) = \theta(n)$$

Örnek B

$$T(n) = 4T(n/2) + n$$

$$c = \log_2 4 = 2$$

$$k = 1$$

$$T(n) = \theta(n^2)$$

Örnek C

$$T(n) = 8T(n/2) + n^2$$

$$c = \log_2 8 = 3$$

$$k = 2$$

$$T(n) = \theta(n^3)$$

$\log^p n$ factor'ları polynomial exponent farkını kapatamaz.

6. Case 2 örnekleri Örnek A – klasik balanced case

$$T(n) = 2T(n/2) + n$$

$$c = 1, k = 1, p = 0$$

$$T(n) = \theta(n \log n)$$

Örnek B – local work'te log factor var

$$T(n) = 2T(n/2) + n \log^2 n$$

$$c = 1, k = 1, p = 2$$

$$T(n) = \theta(n \log^3 n)$$

Eşit polynomial power durumunda log exponent'i bir artar.

Örnek C – boundary $p = -1$

$$T(n) = 2T(n/2) + n/\log n$$

$$c = 1, k = 1, p = -1$$

$$T(n) = \theta(n \log \log n)$$

Burada doğrudan "bir log n ile çarp" kuralı uygulanmaz. Bu özel boundary, log log n üretir.

Örnek D – $p < -1$

$$T(n) = 2T(n/2) + n/\log^2 n$$

$$c=1, k=1, p=-2$$

$$T(n)=\theta(n)$$

Bu kez levels boyunca gelen normalized series yakınsar.

7. Case 3 örnekleri

$$T(n) = T(n/2) + n^2$$

$$\begin{aligned} a=1, b=2 \\ c=\log_2 1=0 \\ k=2 \end{aligned}$$

$$\begin{aligned} c < k \\ T(n)=\theta(n^2) \end{aligned}$$

Recursion tree level costs:

$$\begin{aligned} n^2 + (n/2)^2 + (n/4)^2 + \dots \\ = n^2(1 + 1/4 + 1/16 + \dots) \\ = \theta(n^2) \end{aligned}$$

8. Videodaki $p < 0$ kısayoluna önemli düzeltme Videoda $c < k$ ve $p < 0$ olduğunda denominator'daki logarithmic factor'ın atılabileceği söyleniyor. Bu genel olarak tight değildir.

Örneğin:

$$T(n) = T(n/2) + n^2/\log n$$

Burada:

$$c=0, k=2, p=-1$$

Doğru tight sonuç:

$$T(n)=\theta(n^2/\log n)$$

$\theta(n^2)$ yalnızca bir upper bound olabilir; tight bound değildir. Polynomial üstünlük bulunduğu ve regularity condition sağlandığı için Case 3, $f(n)$ 'i denominator dahil korur.

9. Regularity Condition nedir? Standard Case 3 için genellikle şu istenir:

$$a f(n/b) \leq q f(n)$$

Yeterince büyük n'ler ve bir $q < 1$ sabiti için eşitsizlik sağlanmalıdır.

Amaç, child levels'in maliyetinin root cost'a göre geometrically küçüldüğünü garanti etmektir. Yalnızca $f(n)$ daha büyük görünüyor diye Case 3 otomatik uygulanmaz.

10. Uygulama algoritması Bir recurrence gördüğünde şu sırayı izle:

1. a, b ve $f(n)$ 'i belirle.
2. $c = \log_b a$ hesapla.
3. $f(n)$ 'i mümkünse $n^k \log^p n$ biçiminde yaz.
4. c ile k 'yi karşılaştır.

5. Eşitlik varsa p değerini incele.
6. Case 3 kullanıyorsan regularity condition'ı kontrol et.
7. Sonucu θ ile, tight bound olarak yaz.

11. Master Theorem'in doğrudan uygulanamadığı durumlar Şu recurrence'lar standard biçime uymaz:

$T(n) = T(n-1) + n$
 $T(n) = T(\sqrt{n}) + 1$
 $T(n) = T(n/3) + T(2n/3) + n$
 $T(n) = 2T(n/2) + n \log \log n$

Bazıları başka theorem'larla veya doğrudan substitution/recursion tree ile çözülebilir. Formülü zorla uygulamak doğru değildir.

12. Time ve Space hakkında sınır Master Theorem yalnızca recurrence ile tanımlanan Time Complexity'i çözer. Space Complexity için ayrıca şunlar incelenir:

- Recursion depth
- Aynı anda active call sayısı
- Local arrays veya buffers
- Input'un kopyalanıp kopyalanmadığı

Time sonucu $\theta(n \log n)$ çıktı diye space de aynı olmak zorunda değildir.

13. Sık yapılan hatalar $\log_b$ a yerine $\log_a b$ hesaplamak Doğru sıra:

log base b of a

Yalnızca a ile $f(n)$ 'i karşılaştırmak Karşılaştırılacak expression $n^{(\log_b a)}$ dir.

Big O ile tight sonucu aynı sanmak Master Theorem'in case sonuçları burada θ olarak yazılır. θ , yalnızca upper bound söyler.

Logarithmic farkı polynomial fark sanmak $n \log n$, $n^{(1+\epsilon)}$ ile aynı değildir. log factor için balanced/polylog case dikkatle seçilmelidir.

Case 3'te regularity condition'ı unutmak Local term baskın görünse bile gerekli düzenlilik garanti edilmelidir.

14. Kavram sözlüğü

English term	Türkçe açıklama
Master Theorem	Belirli divide-and-conquer recurrence'larını doğrudan sınıflandıran theorem
Dividing Function	Problem size'in n/b biçiminde küçüldüğü recurrence ailesi
Critical Exponent	$c = \log_b a$; recursive growth'ün polynomial exponent'i
Polynomial Gap	İki function arasında n^ϵ büyüklüğünde fark
Polylogarithmic Factor	$\log^p n$ biçimindeki factor
Regularity Condition	Case 3'te lower levels'in geometrically küçülmesini sağlayan koşul
Tight Bound	Hem upper hem lower bound veren θ sınıfı

15. Kendini sinama

1. a ve b , recursion tree'de hangi iki özelliği belirler?
2. $n^{\log_b a}$ neden önemlidir?
3. $T(n)=9T(n/3)+n$ hangi case'e girer?
4. $T(n)=4T(n/2)+n^2$ sonucu nedir?
5. $T(n)=2T(n/2)+n/\log n$ neden n değildir?
6. Case 3'te hangi ek koşul kontrol edilmelidir?

Kısa cevaplar

1. Branching factor'ı ve child size küçülmesini.
2. Toplam leaf growth'ü ve recursive benchmark'ı verir.
3. $c=2>k=1$; Case 1, $\theta(n^2)$.
4. $c=k=2$, $p=0$; $\theta(n^2 \log n)$.
5. $p=-1$ boundary case'i $\theta(n \log \log n)$ üretir.
6. Regularity condition.

Sonraki ders: **28 – Examples for Master Theorem.**

28 – Examples for Master Theorem

Video: 2.4.2 Examples for Master Theorem #2 (5:41)

Kaynak notu: Bu videoda YouTube English captions bulunmuyor. Resmî açıklama videonun “Examples for all cases of Master Theorem” içerdiğini belirtiyor. Bu ders, bu kapsam ve önceki videodaki theorem esas alınarak; örneklerin matematiksel sonuçları ayrıca doğrulanarak hazırlanmıştır.

Bu videonun ana fikri Master Theorem ezberlenecek bir sonuç tablosundan çok, recurrence içindeki iki büyüme kaynağını karşılaştırma yöntemidir:

$$T(n) = aT(n/b) + f(n)$$

Karşılaştır:

$$\begin{aligned} \text{Recursive benchmark} &= n^{(\log_b a)} \\ \text{Local work} &= f(n) \end{aligned}$$

Bu derste farklı recurrence’ları sınıflandırıp sonucu sistemli biçimde çıkaracağız.

1. Her örnekte kullanılacak şablon

1. $a = ?$
2. $b = ?$
3. $f(n) = ?$
4. $c = \log_b a = ?$
5. $f(n)$ ile n^c arasında nasıl bir ilişki var?
6. Hangi case?
7. Tight bound nedir?

Sonucu yalnızca yazmak yerine case seçiminin gerekçesini de göstermelisin.

2. Case 1 – Recursive work baskın Örnek 1

$$T(n) = 4T(n/2) + n$$

Parametreler:

$$\begin{aligned} a &= 4 \\ b &= 2 \\ f(n) &= n \\ c &= \log_2 4 = 2 \\ n^c &= n^2 \end{aligned}$$

Karşılaştırma:

$$f(n)=n = O(n^{(2-1)})$$

Polynomial olarak recursive benchmark daha büyük:

$$T(n)=\Theta(n^2)$$

Recursion tree sezgisi:

$$\begin{aligned} \text{Level 0 cost: } &n \\ \text{Level 1 cost: } &4(n/2) = 2n \end{aligned}$$

Level 2 cost: $4^2(n/2^2) = 4n$

...

Level costs aşağı doğru büyür; leaves tarafı baskındır.

Örnek 2

$$T(n) = 8T(n/2) + n^2 \log n$$

$$c = \log_2 8 = 3$$

$$f(n) = n^2 \log n$$

$n^2 \log n$, n^3 'ten polynomial olarak küçüktür. Örneğin herhangi bir sabit $0 < \epsilon < 1$ için yeterince büyük n 'de:

$$n^2 \log n = O(n^{3-\epsilon})$$

Sonuç:

$$T(n) = \Theta(n^3)$$

Bir $\log n$ factor, bir tam polynomial power farkını aşamaz.

3. Case 2 – İki taraf dengeli Örnek 3

$$T(n) = 2T(n/2) + n$$

$$c = \log_2 2 = 1$$

$$n^c = n$$

$$f(n) = \Theta(n)$$

Her level'in total cost'u $\Theta(n)$ ve yaklaşık $\log n$ level vardır:

$$T(n) = \Theta(n \log n)$$

Örnek 4

$$T(n) = 4T(n/2) + n^2 \log^2 n$$

$$c = \log_2 4 = 2$$

$$f(n) = \Theta(n^2 \log^2 n)$$

$$p = 2$$

Extended balanced case:

$$T(n) = \Theta(n^2 \log^3 n)$$

Polynomial powers eşitse existing $\log$ exponent'i bir artar.

Örnek 5 – $p = -1$ boundary

$$T(n) = 2T(n/2) + n/\log n$$

$$c = 1$$

$$k = 1$$

$$p = -1$$

Sonuç:

$$T(n) = \theta(n \log \log n)$$

Nedenini level toplamlarıyla görelim. Level ide:

$$\begin{aligned} 2^i * [(n/2^i) / \log(n/2^i)] \\ = n / (\log n - i) \end{aligned}$$

Levels boyunca harmonic-type bir toplam oluşur ve ek factor $\log \log n$ olur.

Örnek 6 – $p < -1$

$$T(n) = 2T(n/2) + n/\log^2 n$$

$$c=1, k=1, p=-2$$

Extended case sonucu:

$$T(n) = \theta(n)$$

$p < -1$ olduğunda ilgili series yakınsar; leaf contribution $\theta(n)$ belirleyici kalır.

4. Case 3 – Local work baskın Örnek 7

$$T(n) = T(n/2) + n^2$$

$$\begin{aligned} a &= 1 \\ b &= 2 \\ c &= \log_2 1 = 0 \\ f(n) &= n^2 \end{aligned}$$

Local work polynomial olarak büyüktür:

$$T(n) = \theta(n^2)$$

Regularity kontrolü:

$$a f(n/b) = (n/2)^2 = f(n)/4$$

$q = 1/4 < 1$ seçilebilir; koşul sağlanır.

Örnek 8

$$T(n) = 3T(n/4) + n$$

$$\begin{aligned} c &= \log_4 3 \approx 0.792 \\ f(n) &= n \end{aligned}$$

$1 > \log_4 3$ ve arada polynomial gap vardır:

$$T(n) = \theta(n)$$

Regularity:

$$3f(n/4) = 3n/4 \leq qn$$

$q = 3/4 < 1$ uygundur.

Örnek 9 – denominator korunur

$$T(n) = T(n/2) + n^2/\log n$$

$$\begin{aligned}c &= 0 \\k &= 2 \\p &= -1\end{aligned}$$

Case 3'te polynomial gap vardır ve regularity sağlanır:

$$T(n) = \theta(n^2/\log n)$$

$\log n$ denominator'ını atıp $\theta(n^2)$ demek tight değildir.

5. Birbirine benzeyen ama farklı üç recurrence

- A. $T(n) = 4T(n/2) + n$
- B. $T(n) = 4T(n/2) + n^2$
- C. $T(n) = 4T(n/2) + n^3$

Hepsinde:

$$c = \log_2 4 = 2$$

Recurrence	Karşılaştırma	Case	Sonuç
A	$n < n^2$	Recursive work baskın	$\theta(n^2)$
B	$n^2 = n^2$	Balanced	$\theta(n^2 \log n)$
C	$n^3 > n^2$	Local work baskın	$\theta(n^3)$

Bu tablo theorem'in özünü özetler.

6. Değişik base ile logarithm hesaplama

$$T(n) = 9T(n/3) + n^2$$

$$\begin{aligned}c &= \log_3 9 = 2 \\f(n) &= n^2\end{aligned}$$

Balanced case:

$$T(n) = \theta(n^2 \log n)$$

Logarithm base'i final asymptotic expression'da önemli değildir:

$$\log_3 n = \log_2 n / \log_2 3 = \theta(\log n)$$

Fakat $c = \log_b a$ hesaplanırken base doğru kullanılmalıdır.

7. Master Theorem uygulanamazsa ne yapılır? Unequal subproblem sizes

$$T(n) = T(n/3) + T(2n/3) + n$$

Tek bir $aT(n/b)$ biçimi yoktur.

Decrementing recurrence

$$T(n)=T(n-1)+n$$

Size division değil subtraction ile küçülür.

Root recurrence

$$T(n)=T(\sqrt{n})+1$$

Child size n/b değildir.

Bu örneklerde Recursion Tree, Substitution, summation veya Akra-Bazzi Method gibi başka araçlar gerekir.

8. Sınavda hızlı kontrol

Önce yalnızca şu satırı yaz:

$$c = \log_b a$$

Ardından $f(n)$ 'in polynomial exponent'ini bul. Çoğu hata, case ezberinden değil a, b, k değerlerinin yanlış okunmasından çıkar.

Örnek:

$$T(n)=16T(n/4)+n \log n$$

$$\begin{aligned} c &= \log_4 16 = 2 \\ k &= 1 \\ c &> k \\ T(n) &= \theta(n^2) \end{aligned}$$

$f(n)$ 'de $\log n$ bulunması otomatik olarak balanced case anlamına gelmez.

9. Sık yapılan hatalar

ayrı child size ile karıştırmak a , child sayısıdır; b , child size'in divisor'üdür.

$f(n)$ içindeki coefficient'a önem vermek

$$7n^2 = \theta(n^2)$$

Constant coefficient case seçimini değiştirmez.

Equal case'te mevcut log power'ı unutmak $n^c \log^2 n$ varsa sonuç $n^c \log^3 n$ olur; yalnızca $n^c \log n$ değil.

Case 3'ü regularity kontrolü olmadan kullanmak Theorem'in koşulları sonuç kadar önemlidir.

Uymayan recurrence'a theorem'i zorla uygulamak Önce syntax değil structure kontrol edilmelidir.

10. Kavram sözlüğü

English term	Türkçe açıklama
Recursive Benchmark	$n^{(\log_b a)}$ ile ölçülen recursive growth
Dominant Work	Total cost'u asymptotically belirleyen bölüm
Balanced Case	Recursive work ile local work'ün aynı polynomial class'ta olması
Boundary Case	Genel kısa kuralın değiştiği sınır değer; burada $p=-1$
Change of Base	Logarithm base'ini constant factor üzerinden değiştirme kuralı
Applicable	Theorem'in yapısal koşullarına uygun

11. Kendini sına

1. $T(n)=8T(n/2)+n^2$ sonucu nedir?
2. $T(n)=8T(n/2)+n^3$ sonucu nedir?
3. $T(n)=8T(n/2)+n^4$ sonucu nedir?
4. $T(n)=2T(n/2)+n \log^4 n$ sonucu nedir?
5. $T(n)=T(n/2)+n/\log n$ sonucu nedir?
6. $T(n)=T(n/3)+T(2n/3)+n$ neden doğrudan uygun değildir?

Kısa cevaplar

1. $c=3>2$; $\theta(n^3)$.
2. Balanced; $\theta(n^3 \log n)$.
3. Case 3; $\theta(n^4)$.
4. $\theta(n \log^5 n)$.
5. Case 3 ve denominator korunur; $\theta(n/\log n)$.
6. Subproblem sizes aynı n/b biçiminde değildir.

Sonraki ders: **29 – Root Function Recurrence Relation.**

29 – Root Function Recurrence Relation

Video: 2.5 Root Function (Recurrence Relation) (5:37)

Kaynak notu: Bu videoda YouTube English captions bulunmuyor. Resmî açıklama Root Function Recurrence Relationın Substitution Method ile çözüldüğünü belirtir. Aşağıdaki anlatım, videonun bu kapsamı ve recurrence'ın doğrulanmış matematiksel çözümü üzerinden hazırlanmıştır.

Bu videonun ana fikri Bu derste problem size'ı sabit bir sayıya bölünmez; her recursive call'da square root alınarak küçülür:

$$T(n) = T(\sqrt{n}) + 1$$

$\sqrt{n}$, büyük nlerde $n/2$ den çok daha küçüktür. Bu nedenle recursion depth $\log n$ değil:

$$\theta(\log \log n)$$

olur.

1. Neden Master Theorem kullanılamaz? Standard Master Theorem şu biçimi ister:

$$T(n) = aT(n/b) + f(n)$$

Fakat burada child size:

$$\sqrt{n} = n^{(1/2)}$$

ve herhangi bir sabit b için n/b değildir. Bu yüzden theorem'i zorla uygulamak yerine variable substitution kullanılır.

2. Direct Back Substitution Başlangıç:

$$T(n) = T(n^{(1/2)}) + 1$$

Bir kez daha aç:

$$\begin{aligned} T(n^{(1/2)}) &= T((n^{(1/2)})^{(1/2)}) + 1 \\ &= T(n^{(1/4)}) + 1 \end{aligned}$$

Dolayısıyla:

$$T(n) = T(n^{(1/4)}) + 2$$

Devam edersek:

$$\begin{aligned} T(n) &= T(n^{(1/8)}) + 3 \\ T(n) &= T(n^{(1/16)}) + 4 \\ &\dots \end{aligned}$$

k expansion sonra pattern:

$$T(n) = T(n^{(1/2^k)}) + k$$

3. Base case'e ne zaman ulaşılır? Base threshold'ı 2 alalım:

$$n^{(1/2^k)} \leq 2$$

İki tarafın base-2 logarithm'ini al:

$$(\log_2 n)/2^k \leq 1$$

Buradan:

$$\begin{aligned} 2^k &\geq \log_2 n \\ k &\geq \log_2(\log_2 n) \end{aligned}$$

Yani recursion depth:

$$k = \Theta(\log \log n)$$

Her level'da constant local work yapıldığı için:

$$T(n) = \Theta(\log \log n)$$

4. Variable Substitution ile daha temiz çözüm Şu dönüşümü yap:

$$\begin{aligned} n &= 2^m \\ m &= \log_2 n \end{aligned}$$

Square root:

$$\text{sqrt}(n) = \text{sqrt}(2^m) = 2^{(m/2)}$$

Yeni function tanımlayalım:

$$S(m) = T(2^m)$$

Original recurrence:

$$T(n) = T(\text{sqrt}(n)) + 1$$

şuna dönüşür:

$$S(m) = S(m/2) + 1$$

Bu, daha önce çözdüğümüz dividing recurrence'dır:

$$S(m) = \Theta(\log m)$$

$m = \log n$ geri koyulursa:

$$T(n) = \Theta(\log \log n)$$

5. Sayısal örnek

$$n = 65,536 = 2^{16}$$

Recursive sizes:

$$\begin{aligned} 65,536 &= 2^{16} \\ 256 &= 2^8 \\ 16 &= 2^4 \end{aligned}$$

$$\begin{aligned} 4 &= 2^2 \\ 2 &= 2^1 \rightarrow \text{base case} \end{aligned}$$

Exponent her adımda yarıya iner:

$$16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$$

Gerekli adım sayısı:

$$\begin{aligned} \log_2 16 &= 4 \\ &= \log_2 (\log_2 65536) \end{aligned}$$

Bu büyüme olağanüstü yavaştır.

6. log n ile log log n farkı

n	$\log_2 n$	$\log_2 \log_2 n$
16	4	2
256	8	3
65,536	16	4
2^{32}	32	5
2^{64}	64	6

Input size çok büyük artsa bile $\log \log n$ yavaş büyür.

7. Recursion Tree görünümü Bu recurrence yalnızca bir child üretir:

```
n
|
sqrt(n)
|
n^(1/4)
|
n^(1/8)
|
...

```

Tree aslında tek bir path'tir. Her node'un local cost'u 1 ise total cost, node sayısına yani depth'e eşittir:

$$\theta(\log \log n)$$

8. Daha genel root recurrence Sabit $r > 1$ için:

$$T(n) = T(n^{1/r}) + 1$$

k step sonra size:

$$n^{1/r^k}$$

Base case koşulu:

$$\begin{aligned} r^k &= \theta(\log n) \\ k &= \theta(\log_r \log n) \end{aligned}$$

Constant log base'leri asymptotic class'ı değiştirmez:

$$T(n) = \theta(\log \log n)$$

9. Local work constant değilse Recurrence'ın sonucu yalnızca size küçülmesine bağlı değildir. Örneğin:

$$T(n) = T(\sqrt{n}) + \log n$$

Level costs:

$\log n$
 $(1/2)\log n$
 $(1/4)\log n$
...

Geometric sum:

$$T(n) = \theta(\log n)$$

Depth hâlâ $\theta(\log \log n)$ dir; fakat her level constant olmadığı için total time farklıdır.

10. Time ve Space Complexity Original recurrence:

$$T(n) = T(\sqrt{n}) + \theta(1)$$

Time:

$$\theta(\log \log n)$$

Recursive implementation'ın call stack depth'i:

$$\theta(\log \log n)$$

Her frame constant memory kullanıyorsa auxiliary stack space de:

$$\theta(\log \log n)$$

Iterative olarak aynı size sequence takip edilirse additional space $\theta(1)$ olabilir.

11. Computation model uyarısı Recurrence, $\sqrt{n}$ hesaplamasını ve local operation'ı $\theta(1)$ kabul eder. Arbitrary-precision çok büyük integers üzerinde gerçek square-root cost'u input'un bit length'ine bağlı olabilir.

Dolayısıyla sonuç şu model içindir:

$$\text{Her level'in local cost'u } \theta(1)$$

12. Sık yapılan hatalar Depth'i log n sanmak $n \rightarrow n/2$ değil, $n \rightarrow \sqrt{n}$ dönüşümü vardır. Doğru depth $\log \log n$ dir.

sqrt işlemini tekrar tekrar yanlış yazmak

$$\text{sqrt}(\text{sqrt}(n)) = n^{1/4}$$

$n/4$ değildir.

k step sonra exponent'i $1/(2k)$ sanmak Doğru exponent:

$$1/2^k$$

Master Theorem'i zorla uygulamak Child size n/b biçiminde olmadığı için önce variable substitution gerekir.

log log n'i (log n)^2 sanmak

$$\begin{aligned}\log \log n &= \log(\log n) \\ \log^2 n &= (\log n)^2\end{aligned}$$

Bunlar tamamen farklı functions'dır.

13. Kavram sözlüğü

English term	Türkçe açıklama
Root Function	Input size'in root alınarak küçültüldüğü function
Square Root	$\text{sqrt}(n)=n^{(1/2)}$
Variable Substitution	Recurrence'ı bilinen bir biçime dönüştürmek için değişken değiştirme
Double Logarithm	$\log \log n$; logarithm'in yeniden logarithm'i
Recursion Depth	Base case'e kadar active call chain uzunluğu
Computation Model	Hangi primitive işlemlerin constant cost sayıldığını belirleyen varsayım

14. Kendini sına

1. İki kez square root alındığında size ne olur?
2. k kez sonra size nedir?
3. $n=2^m$ substitution'ı recurrence'ı hangi biçime dönüştürür?
4. $T(n)=T(\text{sqrt}(n))+1$ sonucu nedir?
5. Recursive auxiliary space nedir?
6. $T(n)=T(\text{sqrt}(n))+\log n$ neden $\theta(\log \log n)$ değildir?

Kısa cevaplar

1. $n^{(1/4)}$.
2. $n^{(1/2^k)}$.
3. $S(m)=S(m/2)+1$.
4. $\theta(\log \log n)$.
5. Her frame constant ise $\theta(\log \log n)$.
6. Level costs geometric olarak $\log n + \log n/2 + \dots$ toplamını verir; sonuç $\theta(\log n)$ dir.

Sonraki ders: **30 – Binary Search: Iterative Method.**

30 – Binary Search: Iterative Method

Video: 2.6.1 Binary Search: Iterative Method (19:36)

Bu videonun ana fikri Binary Search, sorted bir search space'in ortasındaki değeri kontrol eder ve her comparison'dan sonra olası alanın yarısını eler.

Remaining size:
 $n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow \dots \rightarrow 1$

Bu nedenle worst-case search time:

$\theta(\log n)$

Iterative Method, bu işlemi loop ile gerçekleştirir ve constant auxiliary space kullanır.

1. En önemli prerequisite: sorted order Binary Search'in doğru karar verebilmesi için array monotonik sırada olmalıdır. Ascending order varsayalım:

$A[mid] < key \Rightarrow$ key varsa sağ taraftadır
 $A[mid] > key \Rightarrow$ key varsa sol taraftadır

Array sorted değilse bu çıkarımlar geçersizdir. Algoritma bir değer döndürse bile sonuç güvenilir olmaz.

2. Linear Search ile karşılaştırma Linear Search sırayla kontrol eder:

$A[0], A[1], A[2], \dots$

Worst case:

$\theta(n)$ comparisons

Binary Search her adımda yarıyı eler:

$\theta(\log n)$ comparisons

Ancak Binary Search için:

- Data sorted olmalıdır.
- Efficient random access gerekir; array bu açıdan uygundur.
- Tek bir arama için önce sorting yapmak gerekiyorsa sorting cost ayrıca hesaba katılmalıdır.

3. Üç index: low, high, mid 0-based indexing kullanalım:

low = current search interval'ın ilk index'i
high = current search interval'ın son index'i
mid = interval'ın orta index'i

Search interval inclusive'dir:

[low, high]

Loop yalnızca interval boş değilken çalışır:

low <= high

4. Güvenli midpoint hesabı Matematiksel olarak:

```
mid = floor((low+high)/2)
```

Fakat fixed-width integer kullanan dillerde `low+high` overflow yapabilir. Güvenli biçim:

```
mid = low + floor((high-low)/2)
```

Bu iki ifade matematiksel olarak eşdeğerdir; ikincisi implementation açısından daha güvenlidir.

5. Iterative pseudocode

```
Algorithm BinarySearch(A, key)
  low  <- 0
  high <- length(A) - 1

  while low <= high
    mid <- low + floor((high-low)/2)

    if A[mid] == key
      return mid
    else if key < A[mid]
      high <- mid - 1
    else
      low <- mid + 1

  return NOT_FOUND
```

NOT_FOUND için dile göre -1, null, None veya özel bir result type kullanılabilir.

6. Başarılı search örneği

 Sorted array:

Index:	0	1	2	3	4	5	6	7	8
Value:	3	8	12	17	21	29	35	42	50

Aranan:

```
key = 42
```

Iteration 1

```
low=0, high=8
mid=4
A[4]=21
```

```
42 > 21
```

Sol yarı ve mid elenir:

```
low=mid+1=5
```

Iteration 2

```
low=5, high=8
mid=6
A[6]=35
```

```
42 > 35
low=7
```


Iteration 3

```
low=7, high=8  
mid=7  
A[7]=42
```

Match bulundu:

```
return 7
```

Toplam üç midpoint comparison yapılmıştır.

7. Başarısız search örneği Aynı array'de:

```
key=30
```

Trace:

Iteration	low	high	mid	A[mid]	Karar
1	0	8	4	21	low=5
2	5	8	6	35	high=5
3	5	5	5	29	low=6

Son durumda:

```
low=6  
high=5  
low>high
```

Search interval boşalmıştır:

```
return NOT_FOUND
```

low ile high'in birbirini geçmesi algoritmanın hata yaptığı anlamına değil, key'in bulunmadığına dair termination certificate anlamına gelir.

8. Loop Invariant ile correctness Güçlü invariant:

key array'de bulunuyorsa her iteration başında $A[\text{low}..\text{high}]$ aralığındadır.

Initialization Başlangıçta interval bütün array'dir:

```
[0, n-1]
```

Key varsa bu aralıktadır.

Maintenance

- $\text{key} < A[\text{mid}]$ ise sorted order nedeniyle mid ve sağındaki bütün değerler elenir.
- $\text{key} > A[\text{mid}]$ ise mid ve solundaki bütün değerler elenir.
- Eşitse doğru index döndürülür.

Bu güncellemeler key bulunabiliyorsa onu current interval içinde tutar.

Termination $\text{low} > \text{high}$ olduğunda interval boştur. Invariant'a göre key array'de olamaz.

Bu üç bölüm algoritmanın correctness proof'unu verir.

9. Neden mid yeniden kontrol edilmez? Eşitlik önce test edilir. Eşit değilse:

```
key < A[mid] => high=mid-1
key > A[mid] => low=mid+1
```

mid'i tekrar interval'a almak ilerlemeyi durdurabilir. Özellikle iki elemanlı interval'larda $low=mid$ veya $high=mid$ kullanmak infinite loop üretebilir.

10. Time Complexity analizi Her iteration'dan sonra remaining size en fazla yaklaşık yarıya iner:

$$n/2^k \leq 1$$

Çözüm:

$$2^k \geq n$$
$$k \geq \log_2 n$$

Dolayısıyla:

$$\text{Worst Case Time} = \theta(\log n)$$

Best Case İlk midpoint key'e eşitse:

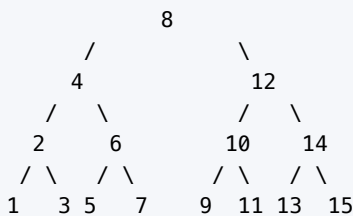
$$\text{Best Case Time} = \theta(1)$$

Average Case Belirli bir probability distribution tanımlanmadan kesin average case hesabı yapılamaz. Successful keys'in uygun biçimde dağıldığı standard modelde:

$$\text{Average Case Time} = \theta(\log n)$$

Video da tree levels üzerindeki comparison sayılarını ortalayarak logarithmic average sonucuna ulaşır.

11. Comparison tree görünümü 15 elemanlı dengeli bir örnekte önce ortadaki index kontrol edilir; sonra iki yarının ortaları, ardından quarters'ın ortaları gelir:



Bu çizim array'in memory representation'ı değildir. Possible comparison sequence'leri gösteren conceptual decision tree'dir.

15 element için maximum successful comparisons:

$$4 = \log_2(15+1)$$

Genel worst-case comparison sayısı $\theta(\log n)$ dir.

12. Divide and Conquer mı, Decrease and Conquer mı? Video Binary Search'i Divide and Conquer başlığı altında anlatır: Search space ikiye ayrılır.

Daha hassas terminolojide Binary Search çoğu zaman Decrease and Conquer olarak sınıflandırılır; çünkü iki yarı oluşturulsa da recursive/iterative olarak yalnızca bir yarı çözülür ve iki solution combine edilmez.

İki adlandırmada da temel fikir aynıdır:

Her step'te problem size yaklaşık yarıya düşer.

13. Space Complexity

Iterative version yalnızca birkaç index variable tutar:

```
low, high, mid
```

Bu nedenle:

```
Auxiliary Space =  $\Theta(1)$ 
```

Input array space'i auxiliary space hesabına eklenmez; algoritmaya zaten verilmiştir.

14. Duplicate değerler

Array'de aynı key birden fazla kez varsa standard Binary Search bunlardan herhangi birini döndürebilir.

```
[2, 5, 5, 5, 9]
```

İlk veya son occurrence isteniyorsa match bulunduğunda hemen durmak yerine boundary aramaya devam eden variant gerekir:

- Lower Bound / first occurrence
- Upper Bound / last occurrence'tan sonraki konum

Bu variant'lar da $\Theta(\log n)$ zamanda çalışabilir.

15. Sorting maliyetini hesaba katmak

Unsorted data için yalnızca bir kez search yapılacaktır:

```
Sort:  $\Theta(n \log n)$   
Search:  $\Theta(\log n)$   
Total:  $\Theta(n \log n)$ 
```

Bu durumda doğrudan Linear Searchin $\Theta(n)$ olması daha ucuz olabilir.

Fakat aynı static collection üzerinde çok sayıda search yapılacaktır sorting cost amortize edilir ve Binary Search avantajlı hâle gelir.

16. Sık yapılan hatalar

Unsorted array üzerinde kullanmak Yarıyı eleme kararı sorted order'a dayanır.

Loop condition'ı low < high yazmak Inclusive interval kullanılıyorsa low==high durumundaki son aday da kontrol edilmelidir:

```
while low <= high
```

Boundary'yi mid yapmak Eşitlik başarısız olduktan sonra mid elenmiştir. mid-1 veya mid+1 kullanılmalıdır.

Overflow-prone midpoint Large indexes için low+(high-low)/2 daha güvenlidir.

Duplicate durumda ilk occurrence garantisi beklemek Standard version yalnızca bir matching index garanti eder.

Worst Case'i $O(\log n)$ deyip tight bound'u kaçırmak Worst case yalnızca upper bound olarak $O(\log n)$ değil, tight olarak $\Theta(\log n)$ dir.

17. Kavram sözlüğü

English term	Türkçe açıklama
Binary Search	Sorted search space'i her comparison'da yarıya indiren search algorithm
Iterative Method	Recursion yerine loop kullanan uygulama
Search Interval	Key'in bulunabileceği current [low, high] aralığı
Midpoint	Current interval'ın orta index'i
Loop Invariant	Her iteration öncesi doğru kalan ve correctness'i kanıtlayan özellik
Successful Search	Key'in bulunduğu search
Unsuccessful Search	Interval boşalınca key'in bulunamadığı search
Random Access	Bir index'teki elemana constant zamanda erişilme

18. Kendini sına

1. Binary Search için temel prerequisite nedir?
2. $low > high$ ne ifade eder?
3. Güvenli midpoint formülü nedir?
4. Worst-case time neden logarithmic'tir?
5. Iterative auxiliary space nedir?
6. Standard algorithm duplicate değerlerden hangisini döndürür?
7. Neden $low = mid$ güncellemesi tehlikeli olabilir?

Kısa cevaplar

1. Search space'in sorted olması.
2. Search interval'ın boşaldığını ve key'in bulunmadığını.
3. $low + \text{floor}((high - low) / 2)$.
4. Remaining candidate sayısı her iteration'da yaklaşık yarıya iner.
5. $O(1)$.
6. Herhangi bir matching occurrence'ı; first/last garantisi yoktur.
7. mid elenmediği için interval küçülmeyebilir ve infinite loop oluşabilir.

Sonraki ders: **31 – Binary Search: Recursive Method.**

31 – Binary Search: Recursive Method

Video: 2.6.2 Binary Search: Recursive Method (7:11)

Kaynak notu: Bu videoda YouTube English captions bulunmuyor. Resmî açıklama kapsamı Divide and Conquer, recursive Binary Search ve analysis olarak verir. Bu ders, söz konusu kapsam ve algoritmanın doğrulanmış recurrence/correctness analizi kullanılarak hazırlanmıştır.

Bu videonun ana fikri Recursive Binary Search, iterative version ile aynı yarıya indirme kararını verir. Fark, updated search interval'ı loop içinde sürdürmek yerine function'ın kendisini daha küçük interval ile çağırmasıdır.

$$T(n) = T(n/2) + \theta(1)$$

Bu recurrence'ın çözümü:

$$\text{Time Complexity} = \theta(\log n)$$

Recursive calls stack'te tutulduğu için:

$$\text{Auxiliary Space} = \theta(\log n)$$

1. Function contract Function'a yalnızca array ve key değil, current search boundaries de verilir:

```
BinarySearch(A, low, high, key)
```

Contract:

- A ascending sorted'dır.
- Search yalnızca inclusive $A[\text{low}..\text{high}]$ aralığında yapılır.
- Key bulunursa index döner.
- Aralık boşsa NOT_FOUND döner.

Initial call:

```
BinarySearch(A, 0, length(A)-1, key)
```

2. Recursive pseudocode

```
Algorithm BinarySearchRecursive(A, low, high, key)
    if low > high
        return NOT_FOUND

    mid ← low + floor((high-low)/2)

    if A[mid] == key
        return mid

    if key < A[mid]
        return BinarySearchRecursive(A, low, mid-1, key)

    return BinarySearchRecursive(A, mid+1, high, key)
```

Her recursive branch'in önündeki return önemlidir. Child call'ın bulduğu index caller'a, oradan da initial caller'a taşınır.

3. Base cases İki farklı stopping condition vardır.

Successful base case

```
A[mid] == key
```

Key bulunmuştur; mid döndürülür.

Unsuccessful base case

```
low > high
```

Search interval boştur; NOT_FOUND döndürülür.

İkinci case unutulursa missing key için recursion yanlış boundaries ile devam eder ve termination garanti edilmez.

4. Başarılı örnek

```
A = [3, 8, 12, 17, 21, 29, 35, 42, 50]  
key = 42
```

Initial call:

```
Search(A, 0, 8, 42)
```

Call 1

```
low=0, high=8, mid=4  
A[4]=21  
42>21
```

Right half:

```
Search(A, 5, 8, 42)
```

Call 2

```
low=5, high=8, mid=6  
A[6]=35  
42>35
```

Right half:

```
Search(A, 7, 8, 42)
```

Call 3

```
low=7, high=8, mid=7  
A[7]=42
```

```
return 7
```

Bu result önce Call 2'ye, sonra Call 1'e, en sonunda caller'a döner.

5. Başarısız örnek

```
key=30
```

Call chain:

```
Search(0,8) -> mid=4, A[mid]=21 -> Search(5,8)
Search(5,8) -> mid=6, A[mid]=35 -> Search(5,5)
Search(5,5) -> mid=5, A[mid]=29 -> Search(6,5)
Search(6,5) -> low>high          -> NOT_FOUND
```

Return chain:

```
NOT_FOUND <- NOT_FOUND <- NOT_FOUND <- NOT_FOUND
```

Bu nedenle recursive call result'larının mutlaka return edilmesi gerekir.

6. Recurrence nasıl çıkarılır? Size n olan current interval'da:

- Midpoint hesaplama constant work'tür.
- Bir comparison yapılır.
- Yalnızca bir half üzerinde recursion sürer.

Bu nedenle:

```
T(n)=T(n/2)+c
```

Asymptotic notation ile:

```
T(n)=T(n/2)+ $\theta(1)$ 
```

İki yarıdan yalnızca biri aranır. Recurrence şu değildir:

```
2T(n/2)+1 // yanlış
```

7. Substitution ile Time Complexity

```
T(n)=T(n/2)+1
```

Açılım:

```
T(n)=T(n/4)+2
T(n)=T(n/8)+3
...
T(n)=T(n/2^k)+k
```

Base size 1:

```
n/2^k=1
k=log2n
```

Dolayısıyla:

```
T(n)=T(1)+log2n
    = $\theta(\log n)$ 
```

8. Best, Worst ve Average Case Best Case İlk midpoint match olur:

```
 $\theta(1)$ 
```

Worst Case Key deepest candidate'tadır veya yoktur:

$\theta(\log n)$

Average Case Probability model belirtilmeden tek bir sayısal average tanımlanamaz. Standard successful-search modelinde:

$\theta(\log n)$

Recursive ve iterative implementations aynı midpoint policy'yi izliyorsa comparison sayıları aynı asymptotic class'tadır.

9. Space Complexity – temel fark Her recursive call kendi frame'ini call stack'e koyar. Bir frame tipik olarak şunları taşır:

- low, high, mid
- Parameters veya references
- Return address

Maximum call depth:

$\theta(\log n)$

Her frame constant size ise:

Auxiliary Space = $\theta(\log n)$

Iterative version ise birkaç variable ile:

Auxiliary Space = $\theta(1)$

Bu nedenle yalnızca efficiency düşünülüyorsa iterative implementation çoğu dilde daha pratiktir.

10. Array'i recursive call'da kopyalama İyi implementation array'in tamamını veya slice'ını kopyalamaz; aynı array reference'ını ve yalnızca boundaries'i geçirir:

Search(A, low, high, key)

Şöyle bir yaklaşım:

Search(copyOfLeftHalf, key)

her level'da copying cost ve ek memory üretebilir. Bu durumda basit $T(n)=T(n/2)+1$ analizi implementation'ı doğru temsil etmez.

11. Correctness – inductive düşünce Statement:

Sorted A[low..high] içinde key bulunuyorsa function matching bir index döndürür; bulunmuyorsa NOT_FOUND döndürür.

Base case Empty interval için low>high; key yoktur ve NOT_FOUND doğrudur.

Inductive step

- A[mid]==key: Doğru index bulunur.
- key<A[mid]: Sorted order nedeniyle key yalnızca left half'ta olabilir. Daha küçük interval için induction hypothesis uygulanır.
- key>A[mid]: Aynı mantık right half için geçerlidir.

Her recursive interval strictly küçüldüğü için algorithm terminate eder.

12. Divide and Conquer nüansı Video konuyu Divide and Conquer içinde işler. Search interval gerçekten ikiye ayrılır; fakat yalnızca bir taraf çözülür.

Bu yüzden daha ayrıntılı algorithm taxonomy içinde şu ad da kullanılır:

Decrease and Conquer by a constant factor

Merge Sort ile fark:

Binary Search: $T(n)=T(n/2)+\theta(1)$
Merge Sort: $T(n)=2T(n/2)+\theta(n)$

13. Tail Recursion meselesi Recursive call her branch'te function'ın son yaptığı işlem olduğundan yapı tail recursive biçime yakındır.

Ancak bütün programming languages Tail Call Optimization garantilemez. Optimization yoksa stack space yine:

$\theta(\log n)$

Algoritmanın tail-recursive yazılması otomatik olarak constant space olduğu anlamına gelmez.

14. Duplicate değerler Standard recursive version bir matching occurrence bulunca durur. Duplicate key'lerde first veya last index'i garanti etmez.

First occurrence için match bulunduğunda left tarafta başka match olup olmadığı aranabilir; last occurrence için right taraf araştırılır. Boundary dikkatle tutulursa time yine:

$\theta(\log n)$

olabilir.

15. Sık yapılan hatalar Empty interval base case'ini yazmamak Missing key senaryosunda termination bozulabilir.

Recursive result'ı return etmemek

BinarySearchRecursive(...)

çağrısı tek başına yapılırsa bulunan index üst frame'lere iletilmez. Doğrusu:

return BinarySearchRecursive(...)

İki yarıyı da recursively aramak Sorted order hangi half'ın mümkün olduğunu söyler. İkisini de aramak logarithmic avantajı yok eder.

Recurrence'ı $2T(n/2)+1$ yazmak Bir call yalnızca bir recursive child izler; doğru recurrence $T(n/2)+1$ dir.

Slice kopyalama maliyetini yok saymak Boundaries geçirmek ile subarray kopyalamak aynı değildir.

Recursive ve iterative space'i aynı sanmak Time class aynıdır; auxiliary space farklıdır.

16. Kavram sözlüğü

English term	Türkçe açıklama
Recursive Method	Function'ın daha küçük input/boundary ile kendisini çağırdığı yöntem
Base Case	Daha fazla recursive call yapılmadan doğrudan sonuç döndürülen durum
Recursive Case	Problemin daha küçük instance'ına çağrı yapılan durum
Call Stack	Active function calls ve local state'i tutan stack
Return Propagation	Child result'ın caller frames üzerinden yukarı taşınması
Tail Recursion	Recursive call'ın function'daki son işlem olduğu yapı
Stack Frame	Bir active call'a ait parameters, locals ve return bilgisini tutan kayıt

17. Kendini sına

1. Recursive function neden low ve high alır?
2. Unsuccessful base case nedir?
3. Correct recurrence neden $2T(n/2)+1$ değildir?
4. Recursive time ve auxiliary space nedir?
5. Child call'ın önüne neden return yazılır?
6. Subarray kopyalamak analizi nasıl etkileyebilir?
7. Tail recursion neden her dilde $\theta(1)$ stack anlamına gelmez?

Kısa cevaplar

1. Her call'ın current search interval'ını tanımlamak için.
2. $low > high$.
3. İki half'tan yalnızca biri üzerinde recursion devam eder.
4. Time $\theta(\log n)$, stack space $\theta(\log n)$.
5. Bulunan index veya NOT_FOUND sonucunu üst frames'e taşımak için.
6. Her level'a copying time ve additional memory ekler.
7. Dil/runtime Tail Call Optimization garanti etmeyebilir.

Sonraki ders: **32 – Heap, Heap Sort, Heapify, Priority Queues.**

32 – Heap, Heap Sort, Heapify and Priority Queues

Video: 2.7 Heap, Heap Sort, Heapify, Priority Queues (51:08)

Bu videonun ana fikri Heap, array içinde çok verimli temsil edilebilen bir Complete Binary Treedir. İki temel çeşidi vardır:

```
Max-Heap: Her parent >= children
Min-Heap: Her parent <= children
```

Bu düzen sayesinde:

- Maximum veya minimum element root'ta bulunur.
- Insert ve root deletion $\Theta(\log n)$ sürer.
- Bottom-up Build-Heap işlemi $\Theta(n)$ sürer.
- Heap Sort, worst case dahil $\Theta(n \log n)$ zamanda çalışır.
- Heap, Priority Queue için temel implementation'lardan biridir.

1. Binary Tree'nin array representation'ı Complete Binary Tree level-order sırasıyla array'e gaps olmadan yerleştirilir.

1-based indexing Node index'i i ise:

```
Left child = 2i
Right child = 2i+1
Parent      = floor(i/2)
```

Örnek:

```
Index:  1  2  3  4  5  6  7
Value:  A  B  C  D  E  F  G
```

B, index 2'dedir:

```
Left(B)  = index 4 = D
Right(B) = index 5 = E
```

0-based indexing Gerçek programlarda sık kullanılan biçim:

```
Left child = 2i+1
Right child = 2i+2
Parent      = floor((i-1)/2), i>0
```

Formülleri karıştırmamak için implementation boyunca tek indexing convention kullanılmalıdır.

2. Complete Binary Tree Bir Complete Binary Treede:

- Son level dışındaki bütün levels tamamen doludur.
- Son level soldan sağa, gaps olmadan doldurulur.

Array açısından eşdeğer tanım:

Level-order representation'da root'tan son node'a kadar boş index yoktur.

Bu yapı n node için minimum yüksekliğe sahiptir:

```
Height = floor(log2n)
```

Burada height, root-to-leaf edge sayısıdır.

3. Terminoloji düzeltmesi: Full ve Perfect Videoda bütün levels'ı tamamen dolu tree için Full Binary Tree ifadesi kullanılıyor. Yaygın standard terminolojide:

- Perfect Binary Tree: Bütün internal nodes iki child'a sahiptir ve bütün leaves aynı level'dadır.
- Full/Proper/Strict Binary Tree: Her node ya sıfır ya iki child'a sahiptir; leaves aynı level'da olmak zorunda değildir.
- Complete Binary Tree: Son level hariç dolu, son level soldan sağa doldurulmuş tree'dir.

Height h olan Perfect Binary Tree'de:

$$\text{Node count} = 2^{(h+1)} - 1$$

Her perfect tree complete'tir; her complete tree perfect değildir.

4. Heap Property Max-Heap Her node için:

$$A[\text{parent}(i)] \geq A[i]$$

Sonuç olarak root maximum elementtir.

Min-Heap Her node için:

$$A[\text{parent}(i)] \leq A[i]$$

Sonuç olarak root minimum elementtir.

Heap property yalnızca parent-child ilişkilerinde kontrol edilir. Transitivity nedeniyle parent, bütün descendants'a göre de uygun yönde sıralıdır.

5. Heap neyi garanti etmez? Heap tamamen sorted değildir.

Bir Max-Heap'te:

$$\text{Parent} \geq \text{child}$$

garantisi vardır; fakat sibling nodes veya farklı branches arasında total order yoktur.

Bu yüzden:

- Maximum'u bulmak $\theta(1)$ dir.
- Arbitrary bir value aramak worst case $\theta(n)$ dir.
- Heap, Binary Search Tree değildir.

Duplicate values kullanılabilir; property $\geq$ veya $\leq$ ile tanımlanır.

6. Max-Heap insertion Yeni element önce complete-tree property'yi koruyacak next free index'e eklenir. Ardından parent'ından büyük olduğu sürece yukarı taşınır.

Bu işleme şu adlar verilir:

- Sift Up
- Bubble Up
- Percolate Up

Örnek Başlangıç Max-Heap:

[50, 30, 20, 15, 10, 8, 16]

60 ekle:

[50, 30, 20, 15, 10, 8, 16, 60]

Parent chain:

```
60 > 15 -> swap
60 > 30 -> swap
60 > 50 -> swap
```

Sonuç:

```
[60, 50, 20, 30, 10, 8, 16, 15]
```

Pseudocode

```
Algorithm PushMaxHeap(H, x)
  append x to H
  i ← size(H)-1

  while i > 0
    p ← floor((i-1)/2)
    if H[p] >= H[i]
      break
    swap H[p], H[i]
    i ← p
```

Complexity

```
Best Case =  $\Theta(1)$ 
Worst Case =  $\Theta(\log n)$ 
```

Maximum hareket sayısı tree height kadardır.

7. Max-Heap root deletion Heap ADT'deki doğal deletion, root'taki highest-priority element'i çıkarmaktır.

Adımlar:

1. Root value'yu result olarak sakla.
2. Last element'i root'a taşı.
3. Heap size'ı bir azalt.
4. Yeni root'u daha büyük child ile gerektiği kadar aşağı taşı.

Bu işleme:

- Sift Down
- Bubble Down
- Percolate Down

denir.

Neden daha büyük child seçilir? Max-Heap'te parent iki child'dan da büyük olmalıdır. Küçük child ile swap yapmak, büyük child'ın yeni parent'tan büyük kalmasına yol açabilir.

Pseudocode

```
Algorithm PopMax(H)
  if H is empty
    error

  result ← H[0]
  H[0] ← H[last]
  remove last element
```

```
SiftDown(H, 0, size(H))
return result
```

```
Algorithm SiftDown(H, i, heapSize)
while true
    left  <- 2i+1
    right <- 2i+2
    largest <- i

    if left < heapSize and H[left] > H[largest]
        largest <- left
    if right < heapSize and H[right] > H[largest]
        largest <- right

    if largest == i
        break

    swap H[i], H[largest]
    i <- largest
```

Complexity:

```
Peek maximum =  $\Theta(1)$ 
Pop maximum  =  $\Theta(\log n)$  worst case
```

8. Insertion ve deletion yönleri

```
Insertion: Leaf → Root (Sift Up)
Deletion:  Root → Leaf (Sift Down)
```

İki operation da complete-tree shape'i önce korur; value order daha sonra onarılır.

9. Heap oluşturma'nın iki yolu Yöntem A – Repeated Insertion Empty heap'e bütün elements'ı tek tek insert et:

```
for each x in A
    PushMaxHeap(H, x)
```

Her insert worst case $\Theta(\log n)$:

```
Build cost =  $O(n \log n)$ 
```

Tight worst-case bound da $\Theta(n \log n)$ olabilir.

Yöntem B – Bottom-Up Heapify Array'i önce complete tree olarak kabul et. Leaves zaten tek-node heap'tir. Son non-leaf node'dan root'a doğru her node'a SiftDown uygula:

```
Algorithm BuildMaxHeap(A)
n <- length(A)
for i <- floor(n/2)-1 downto 0
    SiftDown(A, i, n)
```

Son non-leaf index:

```
floor(n/2)-1 // 0-based
floor(n/2)   // 1-based
```

10. Bottom-Up Heapify neden $\Theta(n)$? Her node $\log n$ kadar hareket etmez.

- Yaklaşık $n/2$ leaf: sıfır hareket
- Yaklaşık $n/4$ node: en fazla bir level
- Yaklaşık $n/8$ node: en fazla iki levels
- ...

Toplam work:

$$\begin{aligned} & n/4 * 1 + n/8 * 2 + n/16 * 3 + \dots \\ & = n * \sum (h/2^h) \\ & = \theta(n) \end{aligned}$$

Bu yüzden:

$$\text{Bottom-Up Build-Heap} = \theta(n)$$

"n node ve her biri $\log n$, öyleyse $n \log n$ " analizi fazla gevşektir.

11. Heap Sort Ascending sort için Max-Heap kullanılır.

Aşama 1

BuildMaxHeap(A)

Aşama 2 Root'taki maximum'u active heap'in sonuna taşı, heap size'ı küçült ve root'u düzelt:

```
Algorithm HeapSort(A)
  BuildMaxHeap(A)

  for end <- length(A)-1 downto 1
    swap A[0], A[end]
    SiftDown(A, 0, end)
```

Array iki logical bölgeye ayrılır:

[active max-heap | sorted suffix]

Her iteration sonunda sorted suffix bir element büyür.

12. Heap Sort correctness invariant Loop başında:

- $A[0..end]$ valid Max-Heap'tir.
- $A[end+1..n-1]$ sorted'dır.
- Suffix'teki bütün elements active heap'teki elements'dan büyük veya eşittir.

Root ile $A[end]$ swap edilince active set'in maximum'u final position'a gider. SiftDown, küçülen prefix'te heap property'yi yeniden kurar.

13. Heap Sort Complexity Bottom-up implementation:

Build-Heap: $\theta(n)$
 $n-1$ extraction/reheapification: $\theta(n \log n)$
 Total: $\theta(n \log n)$

Best, average ve worst asymptotic time:

$$\theta(n \log n)$$

Array üzerinde in-place implementation:

Auxiliary Space = $\Theta(1)$

Recursive SiftDown kullanılırsa call stack $\Theta(\log n)$ olabilir; iterative version bunu önler.

14. Heap Sort stable mı? Standard in-place Heap Sort stable değildir. Uzak mesafeli swaps, equal keys'in relative order'ını değiştirebilir.

Stability gerekiyorsa original index gibi ek metadata kullanılabilir; bu da space ve comparison modelini etkiler.

15. Priority Queue Priority Queue, removal sırasını arrival time'ın değil priority'nin belirlediği ADT'dir.

Max-Priority Queue Larger key, higher priority:

Max-Heap kullanılır.

Min-Priority Queue Smaller key, higher priority:

Min-Heap kullanılır.

Temel operations:

Operation	Binary Heap
Peek-Min/Peek-Max	$\Theta(1)$
Insert	$\Theta(\log n)$
Extract-Min/Extract-Max	$\Theta(\log n)$
Build-Heap	$\Theta(n)$
Arbitrary search	$\Theta(n)$

Priority Queue ile normal FIFO Queue aynı abstraction değildir.

16. Min-Heap'e dönüştürme Bütün procedures aynıdır; comparison yönleri ters çevrilir:

Sift Up: child < parent ise swap
Sift Down: daha küçük child ile karşılaştı

Min-Heap'ten repeated extraction ascending priority order verir; in-place sorting düzeni kullanılan swap yönüne göre değerlendirilmelidir.

17. Shape property ve order property Heap iki bağımsız invariant taşır:

Shape property: Complete Binary Tree
Order property: Parent-child heap relation

Insertion ve deletion önce shape'i koruyacak physical position'ı seçer, sonra order property'yi onarır. Bu ayırım operation'ları anlamının en temiz yoludur.

18. Sık yapılan hatalar Yeni maximum'u doğrudan root'a eklemek Bu, complete-tree shape'i bozabilir. Önce last position'a eklenmeli, sonra Sift Up yapılmalıdır.

Root silinince uygun child'ı doğrudan root yapmak Last element root'a taşınmazsa complete-tree shape bozulabilir.

Sift Down'da rastgele child seçmek Max-Heap için daha büyük, Min-Heap için daha küçük child seçilmelidir.

Heap'i tamamen sorted sanmak Yalnızca parent-child partial order garanti edilir.

Build-Heap'i $\Theta(n \log n)$ sanmak Repeated insertion $\Theta(n \log n)$; bottom-up heapify $\Theta(n)$ dir.

Complete ile Perfect'i karıştırmak Complete tree'nin last level'i kısmen dolu olabilir; fakat soldan sağa doldurulmalıdır.

Priority Queue'da FIFO beklemek Eşit priorities için ayrı tie-breaking policy tanımlanmadıkça arrival order garanti edilmez.

19. Kavram sözlüğü

English term	Türkçe açıklama
Heap	Complete binary tree shape'i ve heap order property'yi sağlayan data structure
Max-Heap	Maximum key'in root'ta olduğu heap
Min-Heap	Minimum key'in root'ta olduğu heap
Complete Binary Tree	Son level soldan sağa doldurulan minimum-height binary tree
Perfect Binary Tree	Bütün levels'ı tamamen dolu binary tree
Sift Up	Insert edilen node'u parent yönünde taşıyarak property'yi onarma
Sift Down	Root/subtree node'unu child yönünde taşıyarak property'yi onarma
Heapify	Bir node'un subtree'sini veya bütün array'i heap'e dönüştürme işlemi
Build-Heap	Unordered array'den heap oluşturma
Priority Queue	Element removal'ın priority'ye göre yapıldığı ADT
In-Place	Constant additional storage ile input üzerinde çalışma

20. Kendini sinama

- 0-based array'de index i nin children formülleri nedir?
- Heap'in iki temel invariant'ı nelerdir?
- Max-Heap insertion hangi yönde adjustment yapar?
- Root deletion'da neden last element root'a taşınır?
- Bottom-up Build-Heap neden $\Theta(n)$ dir?
- Heap Sort'un time, auxiliary space ve stability özellikleri nelerdir?
- Heap'te arbitrary value aramak neden $O(\log n)$ değildir?
- Min-Priority Queue için hangi heap kullanılır?

Kısa cevaplar

- $2i+1$ ve $2i+2$.
- Complete-tree shape ve parent-child order property.
- Leaf'ten root'a, yani Sift Up.
- Complete-tree shape'i gaps olmadan korumak için.
- Çoğu node leaf'e yakındır ve çok az hareket eder; ağırlıklı height toplamı lineardır.
- $\Theta(n \log n)$, iterative implementation'da $\Theta(1)$ auxiliary space, standard biçimde unstable.
- Sibling/subtree'lar arasında total ordering yoktur.
- Min-Heap.

Sonraki ders: **33 – Two-Way Merge Sort: Iterative Method.**

33 – Two-Way Merge Sort: Iterative Method

Video: 2.8.1 Two-Way Merge Sort: Iterative Method (20:19)

Bu videonun ana fikri Merging, iki sorted list'i tek bir sorted list hâline getirme işlemidir. Two-Way Merge Sort, başlangıçta her element'i uzunluğu 1 olan sorted run kabul eder ve adjacent run'ları ikiye ikiye merge eder:

Run length: 1 → 2 → 4 → 8 → ...

Her pass'te bütün n elements işlenir ve $\Theta(\log n)$ pass vardır:

Time Complexity = $\Theta(n \log n)$

Bu sürüm bottom-up ve iterative'dir; recursion kullanmaz.

1. Merging nedir? İki input list:

A = [3, 8, 12, 18]
B = [5, 9, 15, 17]

İkisi de sorted'dır. Başlarında iki pointer tutulur:

i → A'nın current element'i
j → B'nin current element'i
k → output'taki next free position

Her adımda A[i] ile B[j] karşılaştırılır; küçük olan output'a yazılır ve yalnızca o list'in pointer'ı ilerletilir.

Bir list bittiğinde diğer list'in remaining suffix'i doğrudan output'a kopyalanır.

2. Merge örneği

A = [3, 8, 12, 18]
B = [5, 9, 15, 17]

Kararlar:

3 < 5 → 3
8 > 5 → 5
8 < 9 → 8
12 > 9 → 9
12 < 15 → 12
18 > 15 → 15
18 > 17 → 17
B bitti → 18'i kopyala

Sonuç:

C = [3, 5, 8, 9, 12, 15, 17, 18]

3. Merge correctness invariant Main loop'un her iteration'ında:

- C[0..k-1], iki input'un şimdiye kadar tüketilmiş elements'ının sorted birleşimidir.
- A[i] ve B[j], kendi list'lerindeki smallest unconsumed elements'dır.
- Bu ikisinden küçük olan, global olarak next smallest elementtir.

Bu invariant nedeniyle seçilen element output'un next position'ına güvenle yazılabilir.

4. Merge pseudocode

```
Algorithm MergeLists(A, B)
  i ← 0
  j ← 0
  C ← empty array of size length(A)+length(B)

  while i < length(A) and j < length(B)
    if A[i] ≤ B[j]
      append A[i] to C
      i ← i+1
    else
      append B[j] to C
      j ← j+1

  while i < length(A)
    append A[i] to C
    i ← i+1

  while j < length(B)
    append B[j] to C
    j ← j+1

  return C
```

Equal keys durumunda left list'ten seçmek ($\leq$) stability'yi korur.

5. Merge Complexity

List lengths m ve n ise bütün elements bir kez output'a yazılır:

Time = $\theta(m+n)$

Maximum key comparisons:

$m+n-1$

Array implementation'da output buffer:

Auxiliary Space = $\theta(m+n)$

Linked lists, node links değiştirilerek farklı bir memory profile ile merge edilebilir.

6. Two-Way ve M-Way Merging

Two-Way Merging Aynı anda iki sorted list merge edilir.

M-Way Merging Aynı anda m sorted lists'ten smallest front element seçilir.

Naive biçimde her seferinde bütün fronts karşılaştırılırsa maliyet yüksek olur. Min-Heap ile k lists'in front values'ı yönetilirse toplam N elements için:

$\theta(N \log k)$

Bu bağlantı external sorting'de önemlidir.

7. Sorting'e dönüşüm

Unsorted array:

[9, 3, 7, 5, 6, 4, 8, 2]

Her element tek başına sorted bir run'dır:

[9] [3] [7] [5] [6] [4] [8] [2]

Pass 1 – run width 1 Adjacent singletons merge edilir:

[3,9] [5,7] [4,6] [2,8]

Pass 2 – run width 2

[3,5,7,9] [2,4,6,8]

Pass 3 – run width 4

[2,3,4,5,6,7,8,9]

Artık tek sorted run vardır.

8. Bottom-Up iterative pseudocode

```
Algorithm BottomUpMergeSort(A)
  n ← length(A)
  B ← new array of size n
  width ← 1

  while width < n
    left ← 0

    while left < n
      mid ← min(left+width, n)
      right ← min(left+2*width, n)

      MergeRange(A, B, left, mid, right)
      left ← left + 2*width

    swap references A and B
    width ← 2*width

  if final data is not in original array
    copy it back
```

Ranges half-open'dır:

Left run: [left, mid)
Right run: [mid, right)

Half-open convention boundary hatalarını azaltır.

9. Odd sayıda run veya power-of-two olmayan n n power of two olmak zorunda değildir.

Bir pass'in sonunda pair bulamayan run:

- Output'a aynen kopyalanabilir.
- Bir sonraki pass'te daha büyük bir run ile merge edilir.

min(...) boundaries, incomplete final run'ları güvenli biçimde yönetir.

Örneğin 9 elements için ilk pass:

4 çift merge + 1 unpaired singleton

Asymptotic sonuç değişmez.

10. Pass sayısı Her pass sonunda run width iki katına çıkar:

$$1 * 2^p \geq n$$

Bu yüzden:

$$p = \lceil \log_2 n \rceil$$

Her pass bütün n elements'ı read/write eder:

$$\theta(n) \text{ per pass}$$

Toplam:

$$\theta(n) * \theta(\log n) = \theta(n \log n)$$

11. Best, Average ve Worst Case Standard implementation her pass'te bütün ranges'i işler:

$$\begin{aligned} \text{Best Case} &= \theta(n \log n) \\ \text{Average Case} &= \theta(n \log n) \\ \text{Worst Case} &= \theta(n \log n) \end{aligned}$$

Already sorted adjacent runs için $A[\text{mid}-1] \leq A[\text{mid}]$ testi eklenerek bazı merges atlanabilir; fakat genel asymptotic bound çoğu modelde aynı kalır.

12. Space Complexity Array-based standard merge için reusable buffer gerekir:

$$\text{Auxiliary Space} = \theta(n)$$

Bottom-up version recursion kullanmadığı için recursive stack yoktur:

$$\text{Call-stack overhead} = \theta(1)$$

Buffer ile source/destination arrays'in roles'i pass'ler arasında değiştirilerek gereksiz copy-back sayısı azaltılabilir.

13. Stability Merge Sort doğru tie rule ile stable'dir:

```
if leftValue <= rightValue
    take leftValue first
```

Equal keys'te right run önce seçilirse relative order bozulabilir.

Stability özellikle records birden fazla key ile art arda sıralandığında önemlidir.

14. Iterative ile recursive Merge Sort farkı

Özellik	Bottom-Up Two-Way	Top-Down Recursive
Başlangıç	Length-1 runs	Bütün array
Hareket	Küçük runs'i büyütür	Büyük range'i böler
Kontrol	Loops	Recursive calls
Merge düzeni	Pass-by-pass	Recursion dönüşünde
Time	$\theta(n \log n)$	$\theta(n \log n)$
Array buffer	$\theta(n)$	$\theta(n)$
Call stack	$\theta(1)$	$\theta(\log n)$

İki yöntem aynı temel merge operation'ını kullanır.

15. External sorting bağlantısı Data RAM'e sığmıyorsa sorted runs disk üzerinde oluşturulup daha sonra multi-way merge edilebilir. Sequential reading/writing nedeniyle merge tabanlı yöntemler external memory için uygundur.

Bu konu 35. derste daha ayrıntılı ele alınacaktır.

16. Sık yapılan hatalar Input runs sorted değilken merge uygulamak Linear merge'in doğru-luğu her iki input run'ın sorted olmasına bağlıdır.

Bir list bitince diğer suffix'i unutmak Main comparison loop'tan sonra remaining elements kopyalanmalıdır.

Aynı array'e kontrolsüz biçimde yazmak Unread elements overwrite edilebilir. Standard array implementation separate buffer kullanır.

Pass sayısını n sanmak Run width her pass iki katına çıkar; pass sayısı $\lceil \log_2 n \rceil$ dir.

Equal values'te right'ı önce almak Bu tie rule stability'yi bozabilir.

n power of two değilse algoritmanın çalışmayacağını sanmak Boundary clipping ve unpaired run handling yeterlidir.

17. Kavram sözlüğü

English term	Türkçe açıklama
Merge	Sorted lists'i tek sorted list hâline getirme
Run	Kendi içinde sorted contiguous sequence
Two-Way Merge	Aynı anda iki sorted run'ı birleştirme
M-Way Merge	Aynı anda birden fazla sorted run'ı birleştirme
Bottom-Up	Küçük solved pieces'tan başlayıp daha büyük solution oluşturma
Pass	Bütün input'un bir kez işlendiği merge round'u
Stable Sort	Equal keys'in relative order'ını koruyan sort
Auxiliary Buffer	Merge result'ını geçici olarak tutan ek array

18. Kendini sına

1. İki list'in lengths'i m ve n ise merge time nedir?
2. Main merge loop ne zaman biter?
3. Remaining suffix neden doğrudan kopyalanabilir?
4. Run width her pass nasıl değişir?
5. $n=13$ için kaç pass gerekir?
6. Array-based iterative Merge Sort'un time ve space'i nedir?
7. Stability için equal values'te hangi run tercih edilmelidir?

Kısa cevaplar

1. $\Theta(m+n)$.
2. İki run'dan en az biri tükendiğinde.
3. Kendi run'ında zaten sorted'dır ve diğer run bitmiştir.
4. İki katına çıkar: 1, 2, 4, 8, ...
5. $\lceil \log_2 13 \rceil = 4$.
6. Time $\Theta(n \log n)$, auxiliary space $\Theta(n)$.
7. Left run; böylece original relative order korunur.

Sonraki ders: **34 – Merge Sort Algorithm.**

34 – Merge Sort Algorithm

Video: 2.8.2 Merge Sort Algorithm (20:23)

Bu videonun ana fikri Top-down Merge Sort, Divide and Conquer stratejisini doğrudan uygular:

1. Divide: Range'i iki half'a böl.
2. Conquer: İki half'ı recursively sort et.
3. Combine: Sorted halves'ı linear zamanda merge et.

Base case tek elementli veya empty range'dir. Recurrence:

$$T(n) = 2T(n/2) + \theta(n)$$

Sonuç:

$$T(n) = \theta(n \log n)$$

1. Problem ne zaman "small"dır? Bir range içinde sıfır veya bir element varsa zaten sorted'dır:

$$\text{low} \geq \text{high}$$

İki veya daha fazla element varsa bölmek gerekir:

$$\text{low} < \text{high}$$

Base case'te ayrıca sorting operation yapılmaz.

2. Recursive structure Inclusive boundaries ile:

```
Algorithm MergeSort(A, low, high)
  if low >= high
    return

  mid <- low + floor((high-low)/2)

  MergeSort(A, low, mid)
  MergeSort(A, mid+1, high)
  Merge(A, low, mid, high)
```

İlk recursive call dönmeden ikinci call ve merge başlamaz. Execution order depth-first'tür.

3. Merge procedure Merge çağrısından önce şu precondition sağlanır:

```
A[low..mid]    sorted
A[mid+1..high] sorted
```

Merge bu iki run'ı temporary buffer'a yazar ve ilgili range'e geri kopyalar.

```
Algorithm Merge(A, low, mid, high, buffer)
  i <- low
  j <- mid+1
  k <- low

  while i <= mid and j <= high
    if A[i] <= A[j]
      buffer[k] <- A[i]
      i <- i+1
    else
```

```

        buffer[k] <- A[j]
        j <- j+1
        k <- k+1

    while i <= mid
        buffer[k] <- A[i]
        i <- i+1
        k <- k+1

    while j <= high
        buffer[k] <- A[j]
        j <- j+1
        k <- k+1

    copy buffer[low..high] into A[low..high]

```

Tek buffer initial call'da allocate edilip bütün recursive calls tarafından reuse edilebilir.

4. Adım adım örnek

A = [9, 3, 7, 5, 6, 4, 8, 2]

Divide phase

```

[9,3,7,5,6,4,8,2]
 /      \
[9,3,7,5]  [6,4,8,2]
 /  \      /  \
[9,3] [7,5] [6,4] [8,2]
 / \  / \  / \  / \
[9] [3] [7] [5] [6] [4] [8] [2]

```

Combine phase

```

[9] + [3] -> [3,9]
[7] + [5] -> [5,7]
[6] + [4] -> [4,6]
[8] + [2] -> [2,8]

```

Sonraki level:

```

[3,9] + [5,7] -> [3,5,7,9]
[4,6] + [2,8] -> [2,4,6,8]

```

Root merge:

```

[3,5,7,9] + [2,4,6,8]
-> [2,3,4,5,6,7,8,9]

```

5. Calls ve merges'in execution sırası

Algorithm önce left subtree'yi tamamen bitirir:

```

Left recursive call
Right recursive call
Merge current node

```

Bu sıra binary recursion tree açısından Postorder düzenine benzer:

Left -> Right -> Root operation

“Aynı level’deki bütün merges aynı anda yapılır” çizimsel analiz için kullanışlıdır; sequential recursive execution gerçekte depth-first ilerler.

6. Recursion Tree analizi

Balanced split varsayımıyla level ide:

Subproblem sayısı = 2^i
Her size = $n/2^i$

Her level’daki toplam merge work:

$$2^i * \theta(n/2^i) = \theta(n)$$

Tree height:

$$n/2^h = 1$$
$$h = \log_2 n$$

Her level $\theta(n)$, toplam $\theta(\log n)$ merge level:

$$T(n) = \theta(n \log n)$$

7. Recurrence Relation

Bir size-n call:

- İki size- $n/2$ recursive problem çözer.
- Merge sırasında $\theta(n)$ work yapar.

$$T(n) = 2T(n/2) + cn$$
$$T(1) = d$$

Master Theorem:

$$a=2$$
$$b=2$$
$$f(n) = \theta(n)$$
$$n^{\log_b a} = n$$

Balanced case:

$$T(n) = \theta(n \log n)$$

8. n power of two değilse

Exact halves zorunlu değildir:

$$T(n) = T(\text{floor}(n/2)) + T(\text{ceil}(n/2)) + \theta(n)$$

Tree dengeli kalır ve height $\theta(\log n)$ olur. Sonuç değişmez:

$$\theta(n \log n)$$

9. Merge sayısı

n singleton leaves’i tek run hâline getirmek için binary merge tree’de:

$$n-1 \text{ merge operations}$$

vardır. Ancak merge operations eşit cost’ta değildir: alt levels küçük, üst levels büyük ranges işler. Bu nedenle yalnızca $n-1$ sayısını bilmek total cost’u açıklamaz; levels boyunca toplam element work incelenmelidir.

10. Correctness – induction

Claim:

MergeSort(A, low, high), ilgili range’i sorted hâle getirir ve aynı multiset’i korur.

Base case Length 0 veya 1 range zaten sorted'dır.

Inductive step Induction hypothesis'e göre iki smaller half recursive calls sonrası sorted'dır. Merge, bu iki sorted half'ın bütün elements'ını exactly once seçerek sorted birleşimi üretir.

Dolayısıyla current range de sorted ve permutation-preserving olur.

11. Time Complexity cases Standard Merge Sort input order'dan bağımsız olarak aynı splitting yapısını izler ve her level'da data'yı işler:

```
Best Case    =  $\theta(n \log n)$ 
Average Case =  $\theta(n \log n)$ 
Worst Case   =  $\theta(n \log n)$ 
```

Bu predictable worst-case bound, Merge Sort'un önemli avantajıdır.

12. Space Complexity Array buffer Standard array merging:

```
 $\theta(n)$ 
```

temporary storage ister.

Call stack Maximum active recursion depth:

```
 $\theta(\log n)$ 
```

Total auxiliary space:

```
 $\theta(n) + \theta(\log n) = \theta(n)$ 
```

Her recursive call içinde size'a bağlı yeni arrays allocate etmek unnecessary allocation overhead yaratabilir. Reusable single buffer tercih edilir.

13. Stability Merge sırasında equal keys için left half'taki record önce seçilirse algorithm stable'dır:

```
if A[i] <= A[j]
    take A[i]
```

Yalnızca < kullanıp equality'de right'ı seçmek stability'yi bozabilir.

14. Recursive ve iterative sürümlerin ilişkisi İki sürüm de aynı conceptual merge tree'yi işler:

- Top-down recursive version tree'yi root'tan leaves'e kurar, dönüşte merge eder.
- Bottom-up iterative version singleton runs'tan başlayarak yukarı çıkar.

Time ve array buffer class'ları aynıdır; kontrol akışı ve stack kullanımı farklıdır.

15. Optimization fikirleri Already ordered boundary Recursive calls döndükten sonra:

```
if A[mid] <= A[mid+1]
    merge'i atla
```

Current halves birlikte zaten sorted'dır.

Small-run cutoff Çok küçük ranges için Insertion Sort kullanmak function-call ve merge overhead'ini azaltabilir. Cutoff deneysel olarak seçilir; evrensel sabit değildir.

Buffer reuse Tek $\theta(n)$ buffer allocate edilerek allocation sayısı düşürülür.

16. Sık yapılan hatalar Merge'i recursive calls'dan önce yapmak Merge precondition'ı iki half'ın sorted olmasıdır; bu ancak calls döndükten sonra sağlanır.

Base case'i low == high ile sınırlamak Robust condition $low \geq high$ dir; empty range'i de kapsar.

Midpoint ranges'inde gap veya overlap oluşturmak Inclusive convention:

[low,mid] ve [mid+1,high]

Temporary arrays'i her call'da gereksiz allocate etmek Toplam peak space değişmese de allocation overhead artar.

Merge Sort'u in-place sanmak Standard array implementation $\theta(n)$ auxiliary buffer kullanır.

Tree width'i stack depth sanmak Sequential execution'da maximum active chain $\theta(\log n)$ dir.

17. Kavram sözlüğü

English term	Türkçe açıklama
Merge Sort	Recursively bölüp sorted halves'ı merge eden stable sorting algorithm
Top-Down	Bütün problemden başlayarak smaller subproblems'a inme
Divide	Range'i iki half'a ayırma
Conquer	Halves'ı recursively çözme
Combine	Sorted halves'ı merge etme
Postorder	Left ve right'tan sonra current node operation'ının yapıldığı sıra
Reusable Buffer	Bütün merge calls tarafından paylaşılan temporary array
Cutoff	Small subproblem'da başka algorithm'a geçiş threshold'ı

18. Kendini sinama

1. Merge Sort'un base case'i nedir?
2. Merge çağrısının precondition'ı nedir?
3. Recurrence neden $2T(n/2) + \theta(n)$ dir?
4. Her recursion-tree level'inin total merge cost'u nedir?
5. Array implementation'ın auxiliary space'i nedir?
6. Merge Sort nasıl stable tutulur?
7. n power of two değilse complexity değişir mi?

Kısa cevaplar

1. Range length sıfır veya bir; $low \geq high$.
2. İki adjacent half'ın ayrı ayrı sorted olması.
3. İki recursive half ve linear merge bulunduğu için.
4. $\theta(n)$.
5. Buffer nedeniyle $\theta(n)$; stack ayrıca $\theta(\log n)$ dir ama total yine $\theta(n)$.
6. Equal keys'te left half'taki element önce seçilerek.
7. Hayır; balanced floors/ceilings ile $\theta(n \log n)$ kalır.

Sonraki ders: **35 – Merge Sort In-Depth Analysis.**

35 – Merge Sort In-Depth Analysis

Video: 2.8.3 Merge Sort In-Depth Analysis (13:28)

Bu videonun ana fikri Bu ders Merge Sort'un yalnızca $\theta(n \log n)$ sonucunu değil, hangi ortamlarda neden güçlü veya zayıf olduğunu inceler.

Başlıca avantajlar:

- Predictable $\theta(n \log n)$ time
- Stable Sort olabilmesi
- Linked lists'e uygunluk
- External Sorting için uygun sequential merge yapısı

Başlıca maliyetler:

- Array implementation'da $\theta(n)$ auxiliary buffer
- Recursive version'da $\theta(\log n)$ call stack
- Small ranges'te recursion/merge overhead

1. Large data için neden uygundur? Merge Sort'un worst-case time'i:

$\theta(n \log n)$

Input'un already sorted, reverse-sorted veya adversarial olması bound'u quadratic yapmaz.

Ayrıca merge operation sequential access yapar. Sequential memory ve disk access, random access'e göre storage hierarchy üzerinde daha verimli olabilir.

Videodaki "başka hiçbir sort büyük listeleri desteklemez" ifadesi fazla geneldir. Başka $\theta(n \log n)$ algorithms da vardır; Merge Sort'un özel gücü predictable time ve external/sequential I/O uyumudur.

2. Linked Lists için uygunluk İki sorted linked list, node values'ı yeni array'e kopyalamadan links değiştirilerek merge edilebilir.

L1: 2 -> 8 -> 10 -> 12
L2: 3 -> 9 -> 11

Dummy-head yaklaşımı:

```
Algorithm MergeLinkedLists(a, b)
    dummy <- new node
    tail <- dummy

    while a != null and b != null
        if a.key <= b.key
            tail.next <- a
            a <- a.next
        else
            tail.next <- b
            b <- b.next
        tail <- tail.next

    if a != null
        tail.next <- a
    else
        tail.next <- b

    return dummy.next
```

Node relinking dışındaki merge auxiliary work:

$\theta(1)$ extra pointers

ve time:

$\theta(m+n)$

olur.

3. Linked List Merge Sort toplam space'i "Üçüncü list oluşturulmuyor" demek bütün algorithm'ın kesin $\theta(1)$ space olduğu anlamına gelmez.

- Top-down recursive version: call stack $\theta(\log n)$
- Bottom-up iterative linked-list version: $\theta(1)$ auxiliary pointers ile tasarlanabilir

Array'deki $\theta(n)$ element buffer'a ihtiyaç duyulmaması temel avantajdır.

4. External Sorting nedir? Data RAM'e sığmadığında sorting'in bir bölümü SSD/disk gibi external storage üzerinde gerçekleştirilir.

Örneğin:

Input data: 100 GB
Available RAM: 4 GB

Bütün data aynı anda memory'ye alınamaz.

5. External Merge Sort aşamaları Phase 1 – Run generation

1. RAM'e sığan bir chunk oku.
2. Chunk'ı internal-memory sort ile sırala.
3. Sorted run'ı diske yaz.
4. Input bitene kadar tekrarla.

Phase 2 – Multi-Way Merge Birçok sorted run'dan küçük buffers RAM'e alınır. Her run'ın front element'i arasında minimum seçilir ve output buffer'a yazılır.

Min-Heap kullanılırsa k runs için her selection:

$O(\log k)$

olabilir.

Asıl optimization hedefi yalnızca CPU comparisons değil, disk I/O pass sayısını azaltmaktır.

6. Neden sequential I/O önemlidir? Merge, sorted runs'i baştan sona sequential okur ve output'u sequential yazar. Bu access pattern:

- Disk seek sayısını azaltır.
- Read-ahead ve buffering'e uygundur.
- Büyük blocks ile etkili çalışır.

Bu nedenle Merge Sort, database systems ve large-file sorting'de temel bir fikirdir.

7. Stability Bir sorting algorithm, equal keys'in original relative order'ını koruyorsa stable'dir.

Örnek records:

(Ravi, 8) önce
(Sunil, 8) sonra

Score'a göre stable sorting sonrası da:

(Ravi, 8) önce
(Sunil, 8) sonra

kalır.

Merge sırasında kural:

Equal key $\rightarrow$ left run'daki record önce

Yani stability otomatik değil, tie-handling implementation'ına bağlıdır.

8. Stability neden kullanışlıdır? Records önce secondary key'e, ardından stable biçimde primary key'e göre sıralanabilir.

Örneğin:

1. Önce name'e göre sırala.
2. Sonra stable olarak grade'e göre sırala.

Same grade records kendi aralarında name order'ını korur.

9. Array üzerinde neden standard olarak in-place değildir? İki adjacent sorted ranges'i linear zamanda merge ederken unread elements'in overwrite edilmemesi gerekir. Standard çözüm temporary buffer kullanır:

Auxiliary buffer = $\theta(n)$

Bu nedenle standard array Merge Sort:

Not in-place

olarak sınıflandırılır.

Advanced in-place merge algorithms vardır; fakat implementation'ları daha karmaşık olabilir ve constants/practical performance bakımından farklı trade-offs taşır. "In-place merge imkânsızdır" demek mutlak anlamda doğru değildir; standard basit implementation in-place değildir.

10. Total auxiliary space Top-down array Merge Sort:

Merge buffer: $\theta(n)$
Call stack: $\theta(\log n)$
Total: $\theta(n)$

Çünkü:

$\theta(n + \log n) = \theta(n)$

Buffer her call'da ayrı ayrı allocate edilmemeli; tek buffer reuse edilmelidir.

11. Small subproblem overhead Asymptotic olarak:

Merge Sort = $\theta(n \log n)$
Insertion Sort = $\theta(n^2)$ worst case

Ancak çok küçük nlerde:

- Recursive call overhead
- Boundary calculations
- Temporary-buffer operations
- Cache effects

nedeniyle Insertion Sort daha hızlı olabilir.

Bu, asymptotic notation ile çelişmez; constants ve input range önemlidir.

12. Hybrid Merge Sort Small range'e gelince recursion'ı sürdürmek yerine Insertion Sort kullanılabilir:

```
Algorithm HybridMergeSort(A, low, high)
  if high-low+1 <= CUTOFF
    InsertionSort(A, low, high)
    return

  mid <- ...
  HybridMergeSort(A, low, mid)
  HybridMergeSort(A, mid+1, high)
  Merge(A, low, mid, high)
```

Cutoff değeri:

- Programming language
- Hardware/cache
- Element size
- Comparator cost
- Implementation details

üzerinden benchmark ile seçilir. Videodaki yaklaşık 15 değeri evrensel theorem değildir.

13. Neden Insertion Sort? Insertion Sort:

- Small arrays üzerinde düşük overhead'e sahiptir.
- Nearly sorted data'da hızlıdır.
- Basit implementation ile stable yapılabilir.
- Linked lists için de uygulanabilir.

Stable Merge Sort pipeline'ında small-run sorter'ın da stable seçilmesi, bütün algorithm'ın stability'sini korumaya yardımcı olur.

14. Bubble Sort da kullanılabilir mi? Stable Bubble Sort kullanılabilir; fakat pratik hybrid algorithms genellikle daha iyi cache/locality ve düşük operation count nedeniyle Insertion Sort'u tercih eder.

"İkisi de $\Theta(n^2)$, o hâlde aynıdır" çıkarımı doğru değildir. Asymptotic class, constants ve performed operations hakkında bütün ayrıntıları vermez.

15. Recursive stack ayrıntısı Recursion tree'de toplam $\Theta(n)$ calls oluşabilir; fakat hepsi aynı anda active değildir.

Maximum active root-to-leaf chain:

$\Theta(\log n)$

Bu yüzden call-stack peak:

$\Theta(\log n)$

Total call sayısı ile maximum simultaneous stack frames birbirine karıştırılmamalıdır.

16. Pros ve Cons özeti

Özellik	Değerlendirme
Worst-case time	$\theta(n \log n)$ – güçlü
Stability	Doğru tie rule ile stable
Array auxiliary space	$\theta(n)$ – maliyet
Linked-list merge	Link değiştirerek verimli
External sorting	Çok uygun
Small arrays	Hybrid cutoff faydalı olabilir
Random access gereksinimi	Zorunlu değil
Parallelization	Independent halves nedeniyle uygundur

17. Ne zaman tercih edilir? Merge Sort güçlü bir seçimdir:

- Stability gerekiyorsa
- Predictable worst-case time isteniyorsa
- Data linked list'teyse
- Data RAM'e sığmıyorsa
- Sequential access önemliyse
- Parallel sorting planlanıyorsa

Ek $\theta(n)$ array buffer kabul edilemiyorsa in-place alternatives değerlendirilebilir.

18. Sık yapılan hatalar Stable olmayı yalnızca algorithm adına bağlamak Merge tie rule right element'i önce alırsa stability bozulabilir.

Linked list merge buffer istemiyor diye stack'i yok saymak Recursive control hâlâ $\theta(\log n)$ stack kullanabilir.

External sorting'i bütün file'ı RAM'e almak sanmak Chunks/runs ve bounded buffers kullanılır.

Cutoff'u sabit matematiksel gerçek sanmak Uygun threshold benchmark'a bağlıdır.

In-place merge'i teorik olarak imkânsız sanmak Advanced algorithms vardır; standard straightforward merge $\theta(n)$ buffer kullanır.

Toplam recursive call sayısını peak space sanmak Space, aynı anda active frames ile ölçülür.

19. Kavram sözlüğü

English term	Türkçe açıklama
External Sorting	RAM'e sığmayan data'yı external storage kullanarak sıralama
Sorted Run	Kendi içinde sorted disk/memory segment'i
Run Generation	Input chunks'ından initial sorted runs üretme
Multi-Way Merge	Birden fazla sorted run'ı aynı aşamada birleştirme
Stable Sort	Equal keys'in relative order'ını koruyan sort
In-Place Sort	Constant veya çok sınırlı auxiliary storage ile sorting
Hybrid Algorithm	Farklı input sizes/conditions için birden fazla algorithm birleştirme
Cutoff Threshold	Başka bir algorithm'a geçilen small-size sınırı
Sequential I/O	Data'yı storage üzerinde ardışık blocks hâlinde okuma/yazma

20. Kendini sinama

1. Merge Sort external sorting'e neden uygundur?
2. Linked lists hangi ek avantajı sağlar?
3. Stability hangi tie rule'a bağlıdır?
4. Standard array Merge Sort'un total auxiliary space'i nedir?
5. Small ranges'te neden Insertion Sort kullanılabilir?
6. Cutoff neden evrensel değildir?
7. Total recursive calls $\theta(n)$ olabilirken stack neden $\theta(\log n)$ dir?

Kısa cevaplar

1. Sorted runs'i bounded buffers ile sequential okuyup yazabildiği için.
2. Nodes kopyalanmadan links değiştirilerek merge yapılabilir.
3. Equal keys'te left run'ın önce seçilmesine.
4. Buffer $\theta(n)$ ve stack $\theta(\log n)$; total $\theta(n)$.
5. Düşük overhead ve small/nearly-sorted input'taki iyi practical performance nedeniyle.
6. Hardware, language, comparator ve implementation'a bağlıdır.
7. Sequential depth-first execution'da aynı anda yalnızca bir root-to-leaf chain active'dir.

Sonraki ders: **36 – Quick Sort Algorithm.**

36 – Quick Sort Algorithm

Video: 2.9.1 Quick Sort Algorithm (13:43)

Bu videonun ana fikri Quick Sort, bir pivot seçer ve range'i şu koşulu sağlayacak biçimde partition eder:

```
Pivot'un solundaki elements <= pivot
Pivot'un sağındaki elements > pivot
```

Pivot final sorted position'ına yerleştirildikten sonra sol ve sağ ranges recursively sort edilir.

```
Partition -> left Quick Sort -> right Quick Sort
```

Bu videonun odağı algorithm ve partition procedure'dür; ayrıntılı time analysis sonraki videodur.

1. "Bir element sorted position'da" ne demektir? Bir element'in kendi tarafındaki elements'in tamamen sorted olması gerekmez.

Pivot p için:

```
Her left element <= p
Her right element > p
```

ise pivot'un final rank'i bellidir ve bir daha hareket etmesi gerekmez.

Örnek:

```
[4, 6, 7, 10, 16, 12, 13, 14]
```

10'dan önceki values küçük, sonrakiler büyükse 10 doğru final position'dadır; iki taraf kendi içinde henüz sorted olmayabilir.

2. Divide and Conquer yapısı

```
Divide: Partition ile pivot boundary'sini bul.
Conquer: Left ve right ranges'i recursively sort et.
Combine: Ayrı merge işlemi yoktur.
```

Merge Sort'tan temel fark:

- Merge Sort önce recursively çözer, sonra linear merge yapar.
- Quick Sort önce linear partition yapar, sonra recursive calls'a geçer.
- Quick Sort'ta combine phase genellikle $\theta(1)$ dir.

3. Pivot seçimi Videodaki procedure first element'i pivot seçer:

```
pivot = A[low]
```

Bu seçim algorithm'ı açıklamak için basittir; fakat already sorted veya adversarial input'ta çok dengesiz partitions üretebilir.

Alternatives:

- Random pivot
- Median-of-three
- Last element
- Sampling tabanlı seçim

Pivot selection, correctness'ten çok partition balance ve performance'ı etkiler.

4. Two-pointer partition fikri Videoda:

- i, soldan pivot'tan büyük bir element arar.
- j, sağdan pivot'tan küçük veya eşit bir element arar.
- $i < j$ iken yanlış taraftaki bu iki element swap edilir.
- Pointers kesiştiğinde pivot, j ile swap edilir.

Son durumda j, pivot'un final position'ıdır.

5. Adım adım partition örneği

```
A = [10, 16, 8, 12, 15, 6, 3, 9, 5]
pivot = 10
```

İlk tarama Soldan:

```
i -> 16    // pivot'tan büyük, dur
```

Sağdan:

```
j -> 5     // pivot'tan küçük, dur
```

Swap:

```
[10, 5, 8, 12, 15, 6, 3, 9, 16]
```

İkinci tarama

```
i -> 12
j -> 9
```

Swap:

```
[10, 5, 8, 9, 15, 6, 3, 12, 16]
```

Üçüncü tarama

```
i -> 15
j -> 3
```

Swap:

```
[10, 5, 8, 9, 3, 6, 15, 12, 16]
```

Pointers devam edip kesiştiğinde pivot ile A[j] swap edilir. Örneğin j value 6 üzerindeyse:

```
[6, 5, 8, 9, 3, 10, 15, 12, 16]
```

Artık:

```
Left  <= 10
Right > 10
```

Pivot final position'dadır.

6. Sentinel yerine güvenli boundary kontrolleri Video right end'e Infinity sentinel ekleyerek inin range dışına çıkmasını önler. Bu klasik öğretim tekniğidir; gerçek implementation'da:

- Extra sentinel slot her zaman bulunmayabilir.
- Numeric type için gerçek infinity olmayabilir.
- Maximum possible key ile sentinel equality sorunu oluşabilir.

Bounds'ı explicit kontrol etmek daha güvenlidir.

7. Güvenli partition pseudocode

Aşağıdaki contract inclusive [low, high] range kullanır:

```
Algorithm PartitionFirstPivot(A, low, high)
  pivot ← A[low]
  i ← low+1
  j ← high

  while true
    while i ≤ high and A[i] ≤ pivot
      i ← i+1

    while j ≥ low+1 and A[j] > pivot
      j ← j-1

    if i ≥ j
      break

    swap A[i], A[j]
    i ← i+1
    j ← j-1

  swap A[low], A[j]
  return j
```

Postcondition:

```
A[j] = pivot
A[low..j-1] ≤ pivot
A[j+1..high] > pivot
```

8. Quick Sort pseudocode

```
Algorithm QuickSort(A, low, high)
  if low ≥ high
    return

  p ← PartitionFirstPivot(A, low, high)

  QuickSort(A, low, p-1)
  QuickSort(A, p+1, high)
```

Initial call:

```
QuickSort(A, 0, length(A)-1)
```

Pivot p final position'da olduğundan recursive ranges'e tekrar dahil edilmez.

9. Videodaki range convention nüansı

Videodaki sentinel-based pseudocode'da high, kimi zaman actual last element'tan sonraki sentinel boundary gibi kullanılır ve left call low..j şeklinde gösterilir.

Farklı partition contracts farklı recursive boundaries gerektirir. İki convention karıştırılırsa:

- Pivot tekrar tekrar range'e girebilir.
- Empty/single ranges yanlış ele alınabilir.
- Infinite recursion oluşabilir.

Bu notta daha açık olan inclusive-range contract kullanılmıştır; bu contract için doğru calls:

```
[low, p-1]
[p+1, high]
```

10. Partition invariant

Tarama sırasında conceptual regions:

```
[ pivot | <= pivot bölgesi | bilinmeyen | > pivot bölgesi ]
```

i ve j unknown region'ı daraltır.

- i , left side'a uygun values üzerinden geçer.
- j , right side'a uygun values üzerinden geçer.
- Yanlış tarafta bulunan pair swap edilir.
- Pointers kesişince bütün non-pivot values doğru side'dadır.

Final pivot swap'ı partition'ı tamamlar.

11. Partition Time ve Space

Pointers range boyunca toplam linear mesafe ilerler:

```
Partition Time =  $\theta(n)$ 
```

In-place swaps kullanılır:

```
Partition Auxiliary Space =  $\theta(1)$ 
```

Quick Sort'un total time'ı, partitions'ın ne kadar dengeli olduğuna bağlıdır; sonraki derste ayrıntılı çözülecektir.

12. Recurrence'in genel biçimi

Pivot final position'a geldikten sonra left side size k , right side size $n-k-1$ olsun:

```
 $T(n) = T(k) + T(n-k-1) + \theta(n)$ 
```

Balanced partition:

```
 $k \approx (n-1)/2$ 
```

Unbalanced partition:

```
 $k=0$  veya  $k=n-1$ 
```

Bu iki durum Quick Sort'un average/worst behavior farkını oluşturur.

13. Duplicate keys

Two-way partition'da equality'nin hangi side'a gönderildiği önemlidir. Çok sayıda equal key:

- Dengesiz partitions üretebilir.
- Gereksiz recursive work yaratabilir.

Three-Way Partitioning input'u:

```
< pivot | == pivot | > pivot
```

olarak ayırır ve duplicate-heavy data'da daha iyi olabilir.

14. Stability ve in-place özellik

Standard Quick Sort:

Stable: Hayır

In-place partition: Evet, $\theta(1)$ auxiliary array space

Ancak recursion stack ayrıca vardır. Total auxiliary space pivot balance'a göre değişir:

- Balanced calls: $\theta(\log n)$ stack
- Worst-case chain: $\theta(n)$ stack

15. Quick Sort neden "quick" olabilir? İsim worst-case guarantee anlamına gelmez. Practical speed'in nedenleri arasında:

- Contiguous arrays üzerinde iyi cache locality
- Inner loop'un basit olması
- Extra merge buffer gerektirmemesi
- Average partitions'ın yeterince dengeli olması

bulunur.

"Quick Sort her zaman en hızlı sorting algorithm'dır" sonucu çıkarılamaz.

16. Correctness – induction özeti Partition sonrası:

1. Pivot doğru final position'dadır.
2. Left range'deki her key pivot'tan küçük/eşittir.
3. Right range'deki her key pivot'tan büyüktür.

Induction hypothesis'e göre recursive calls iki ranges'i sort eder. Left sorted range, pivot ve right sorted range birlikte bütün array'i sorted yapar.

17. Sık yapılan hatalar Pivot'u recursive range'e yanlışlıkla tekrar katmak Boundary convention'a uygun $p-1$ ve $p+1$ kullanılmalıdır.

Pointer'lara bounds kontrolü koymamak Sentinel yoksa out-of-bounds access oluşabilir.

i ve j kesiştikten sonra swap yapmak Pointers crossing partition scan'ın bittiğini gösterir.

Partition sonrası iki tarafı sorted sanmak Yalnızca side relation sağlanır; sides recursively sort edilmez.

Pivot seçiminin sonucu etkilemediğini sanmak Correctness değişmez; partition balance ve time ciddi biçimde değişebilir.

Equal keys'i yok saymak Duplicates hem termination koşullarını hem balance'ı etkiler.

Quick Sort'u tamamen constant-space sanmak Partition constant auxiliary storage kullansa da recursion stack vardır.

18. Kavram sözlüğü

English term	Türkçe açıklama
Quick Sort	Pivot etrafında partition edip subranges'i recursively sıralayan algorithm
Pivot	Partition kararlarının referans aldığı selected element
Partition	Elements'ı pivot'a göre sides'a ayırma procedure'ü
Partition Position	Pivot'un final sorted index'i
Sentinel	Scan termination'ı kolaylaştıran boundary marker
Two-Pointer Scan	Range'in iki yönünden ilerleyen indices yöntemi
Three-Way Partitioning	<, =, > olmak üzere üç region üreten partition
In-Place Partition	Separate linear buffer kullanmadan swap ile partition
Unbalanced Partition	Subproblem sizes'ın birbirinden çok farklı olduğu split

19. Kendini sinama

1. Pivot'un final position'da olması hangi iki side koşulunu gerektirir?
2. i ve j ne arar?
3. Pointers kesişince ne yapılır?
4. Partition time ve auxiliary space nedir?
5. Partition sonucu neden bütün range sorted değildir?
6. Pivot neden recursive ranges'den çıkarılır?
7. Çok fazla duplicate için hangi variant yararlıdır?
8. General Quick Sort recurrence nedir?

Kısa cevaplar

1. Left values $\leq$ pivot, right values $>$ pivot.
2. i soldan büyük, j sağdan küçük/eşit yanlış-taraf elements'ı arar.
3. Scan durdurulur ve pivot j position'ına yerleştirilir.
4. $\theta(n)$ time, $\theta(1)$ auxiliary partition space.
5. Yalnızca pivot'a göre side separation yapılmıştır; sides kendi içinde unsorted olabilir.
6. Pivot zaten final sorted position'dadır ve yeniden işlenmesi termination'ı bozabilir.
7. Three-Way Partitioning.
8. $T(n)=T(k)+T(n-k-1)+\theta(n)$.

Sonraki ders: **37 – Quick Sort Analysis.**

37 – Quick Sort Analysis

Video: 2.9.2 Quick Sort Analysis (11:37)

Bu videonun ana fikri Quick Sortun performansını partition işleminin kendisi değil, pivot'un input'una ne kadar dengeli iki parçaya ayırdığı belirler.

General recurrence:

$$T(n) = T(k) + T(n-k-1) + \theta(n)$$

- Balanced partitions: $\theta(n \log n)$
- Extremely unbalanced partitions: $\theta(n^2)$
- Randomized pivot: Expected $\theta(n \log n)$

Partition'ın her level'daki linear work'ü aynı olabilir; kritik fark recursion tree'nin height'ıdır.

1. Balanced partition Pivot median'a yakınsa:

$$k \approx (n-1)/2$$

Recurrence:

$$T(n) = 2T(n/2) + \theta(n)$$

Recursion tree'de her level'ın total partition work'ü $\theta(n)$ ve height $\theta(\log n)$ dir:

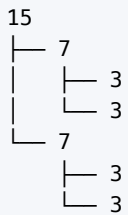
$$T(n) = \theta(n \log n)$$

Bu Quick Sort'un best-case büyüme sınıfıdır.

2. 15 elementli balanced tree İlk pivot final rank 8'e yerleşsin:

Size 15
-> size 7 + pivot + size 7

Sonraki partitions da ortada gerçekleşirse:



Height yaklaşık:

$$\log_2 15 = \theta(\log n)$$

Bir level'daki subarrays ayrı olduğundan total sizes yaklaşık n eder.

3. Median ile bağlantı Sorted order'daki middle-ranked element median'dır. Exact median pivot seçilirse left ve right sizes mümkün olduğunca dengeli olur.

Fakat median'ı seçmek için input'u önceden sort etmek gerekmez; linear-time selection algorithms teorik olarak exact/approximate median bulabilir. Yine de standard Quick Sort'ta bu overhead çoğu zaman tercih edilmez.

Videodaki "best case mümkün değildir" ifadesini şöyle düzeltmek gerekir:

Best case belirli inputs ve pivot choices için mümkündür; fakat her recursive call'da exact median'ı ücretsiz biçimde garanti edemeyiz.

4. Worst-case partition

Pivot her seferinde smallest veya largest element olursa:

```
Size n
-> size 0 + pivot + size n-1
```

Recurrence:

```
T(n)=T(n-1)+θ(n)
```

Expansion:

```
T(n)=n+(n-1)+(n-2)+...+1
      =n(n+1)/2
      =θ(n²)
```

Recursion tree dengeli tree değil, length-n bir chain olur.

5. Sorted input neden sorun olabilir?

Pivot policy her zaman first element ise:

```
[1,2,3,4,5,6,7]
```

input'unda first element her call'da smallest'tır. Partition sizes:

```
0 ve n-1
```

Reverse-sorted input'ta da first pivot çoğunlukla extreme olur.

Sonuç:

```
First-pivot + sorted/reverse-sorted input = θ(n²)
```

Sorun sorted olmanın kendisi değil, input order ile deterministic pivot policy'nin kötü etkileşimidir.

6. Middle index pivot çözüm mü?

Already sorted input'ta middle index'teki value median'dır ve balanced partition üretir. Böylece bu özel kötü case düzelir.

Ancak herhangi bir deterministic policy için, policy'nin sürekli kötü pivot seçmesini sağlayan adversarial arrangements bulunabilir. Bu yüzden:

```
Middle-index pivot worst-case'i genel olarak θ(n log n) yapmaz.
```

Yalnızca bazı yaygın kötü inputs'a karşı koruma sağlar.

7. Randomized Quick Sort

Her call'da pivot uniform random seçilirse input order ile pivot policy arasındaki sistematik ilişki kırılır.

```
Randomized pivot
Expected Time = θ(n log n)
Worst Case    = θ(n²)
```

Worst case hâlâ mümkündür; fakat art arda aşırı kötü pivots seçme olasılığı çok düşüktür.

Expected result, yalnızca informal "genelde dengeli olur" sezgisine değil probability analysis'e dayanır.

8. Expected comparisons

Distinct keys için randomized Quick Sort'un expected comparison sayısı yaklaşık:

$$2n \ln n + O(n)$$

Dolayısıyla:

$$E[T(n)] = \theta(n \log n)$$

Buradaki expectation random pivot choices üzerindendir. Input dağılımı varsaymak zorunda değildir.

9. Average Case ne demektir? İki farklı model karıştırılmamalıdır:

- Deterministic Quick Sort + random permutation input
- Randomized Quick Sort + arbitrary fixed input

Her ikisinde de uygun assumptions altında expected $\theta(n \log n)$ elde edilir; fakat randomness'in kaynağı farklıdır.

"Average case $n \log n$ " derken probability model belirtilmelidir.

10. Best, Expected ve Worst özeti

Durum	Partition pattern	Time
Best	Sürekli dengeli	$\theta(n \log n)$
Expected randomized	Ortalama olarak yeterli balance	$\theta(n \log n)$
Worst	Sürekli 0 ve $n-1$	$\theta(n^2)$

Quick Sort'un best case'i linear değildir; her level'da bütün current elements partition tarafından incelenir.

11. Stack Space Standard recursive implementation'da stack depth recursion tree height'ına bağlıdır.

Balanced

$$\theta(\log n)$$

Worst case

$$\theta(n)$$

Partition array üzerinde in-place olsa da recursive stack göz ardı edilemez.

12. Stack'i worst case $O(\log n)$ tutma İki recursive side'dan smaller olanı recursively çöz, larger olanı loop ile sürdür:

```
Algorithm QuickSortStackSafe(A, low, high)
  while low < high
    p ← Partition(A, low, high)

    if p-low < high-p
      QuickSortStackSafe(A, low, p-1)
      low ← p+1
    else
      QuickSortStackSafe(A, p+1, high)
      high ← p-1
```

Her recursive call size'ı en az yarıya indirir:

$$\text{Worst-case stack} = O(\log n)$$

Time worst case yine $\theta(n^2)$ olabilir; yalnızca stack kontrol altına alınır.

13. Pivot strategies First/Last element Basit, fakat ordered inputs'a hassas.

Middle index Ordered inputs'ı düzeltir; genel adversarial guarantee vermez.

Random pivot Expected $\theta(n \log n)$ ve pratikte güçlü koruma.

Median-of-three First, middle ve last values'ın median'ını seçer. Bazı kötü cases'i azaltır; worst case yine quadratic olabilir.

Guaranteed selection Median of Medians ile sufficiently balanced pivot seçmek teorik worst-case $\theta(n \log n)$ Quick Sort sağlayabilir; constants ve complexity nedeniyle standard implementation'larda daha az yaygındır.

14. Duplicate-heavy input Two-way partition çok sayıda equal key için kötü balance üretebilir.

Three-Way Partitioning:

```
< pivot | == pivot | > pivot
```

Equal region'ı recursion dışında bırakır. Bütün keys eşitse suitable implementation linear time'a yaklaşabilir.

15. Quick Sort ve Merge Sort karşılaştırması

Özellik	Quick Sort	Merge Sort
Expected/Average time	$\theta(n \log n)$	$\theta(n \log n)$
Worst time	$\theta(n^2)$ standard	$\theta(n \log n)$
Array buffer	$\theta(1)$ partition storage	$\theta(n)$ standard merge buffer
Stack	$\theta(\log n)$ expected, $\theta(n)$ naive worst	$\theta(\log n)$
Stability	Standard biçimde unstable	Uygun merge ile stable
Cache locality	Genellikle güçlü	İyi ama buffer traffic var

Seçim yalnızca asymptotic time'a göre yapılmaz.

16. Sık yapılan hatalar Best case'i $\theta(n)$ sanmak Balanced tree'nin $\log n$ levels'ında toplam n partition work vardır.

Sorted input her Quick Sort'u quadratic sanmak Bu, pivot policy'ye bağlıdır. First/last pivot özellikle hassastır.

Random pivot worst-case'i yok eder sanmak Worst case hâlâ $\theta(n^2)$; expected time iyileşir.

Middle pivot'un guarantee verdiğini sanmak Middle index, median value olmak zorunda değildir.

In-place ile $O(1)$ total space'i aynı sanmak Partition buffer kullanmayabilir; recursion stack yine vardır.

Average case'i probability model olmadan söylemek Random input veya random pivot assumption açık olmalıdır.

17. Kavram sözlüğü

English term	Türkçe açıklama
Balanced Partition	Subproblem sizes'in birbirine yakın olduğu parti-tion
Unbalanced Partition	Bir side'in diğerinden çok daha büyük olduğu parti-tion
Median	Sorted order'daki middle-ranked value
Randomized Quick Sort	Pivot'u random seçen Quick Sort variant'ı
Expected Time	Random choices üzerinden beklenen çalışma zama-nı
Adversarial Input	Belirli policy'yi worst behavior'a iten input
Median-of-Three	Üç candidate'in median'ını pivot seçme heuristic'i
Tail-Recursion Elimination	Bir recursive branch'i loop'a çevirerek stack'i azalt-ma

18. Kendini sınamaya

1. Quick Sort'un general recurrence'ı nedir?
2. Balanced partitions hangi time'ı verir?
3. First-pivot policy sorted input'ta neden kötüleşir?
4. Randomized Quick Sort'un expected ve worst time'ı nedir?
5. Middle-index pivot neden genel guarantee değildir?
6. Balanced ve naive worst-case stack depths nedir?
7. Smaller side'ı recursively çözmek neyi iyileştirir?
8. Duplicate-heavy input için hangi partition uygundur?

Kısa cevaplar

1. $T(n) = T(k) + T(n-k-1) + \theta(n)$.
2. $\theta(n \log n)$.
3. First value sürekli extreme pivot olur ve $\theta/(n-1)$ split üretir.
4. Expected $\theta(n \log n)$, worst $\theta(n^2)$.
5. Middle index'teki value arbitrary input'ta median olmak zorunda değildir.
6. $\theta(\log n)$ ve $\theta(n)$.
7. Worst-case stack'i $\theta(\log n)$ tutar; time worst case'ini değiştirmez.
8. Three-Way Partitioning.

Sonraki ders: **38 – Strassen's Matrix Multiplication.**

38 – Strassen's Matrix Multiplication

Video: 2.10 Strassen's Matrix Multiplication (23:40)

Bu videonun ana fikri Classical multiplication of two $n \times n$ matrices:

$$\theta(n^3)$$

zamanda çalışır. Naive divide-and-conquer block multiplication da sekiz recursive matrix multiplication kullandığı için yine $\theta(n^3)$ olur.

Strassen's Algorithm, sekiz yerine yedi recursive multiplication kullanır; daha fazla matrix addition/subtraction karşılığında:

$$T(n) = 7T(n/2) + \theta(n^2)$$

ve:

$$T(n) = \theta(n^{\log_2 7}) \approx \theta(n^{2.8074})$$

elde eder.

1. Matrix multiplication prerequisite $A, m \times k$; $B, k \times n$ ise product tanımlıdır:

$$C = AB$$
$$C \text{ dimension} = m \times n$$

Her output entry:

$$C[i, j] = \sum A[i, t] * B[t, j]$$

Square $n \times n$ case'te her $C[i, j]$ için n multiply-add, toplam n^2 entries vardır:

$$\theta(n^3)$$

2. Classical pseudocode

```
Algorithm MatrixMultiply(A, B)
  n ← number of rows
  C ← n×n zero matrix

  for i ← 0 to n-1
    for j ← 0 to n-1
      for k ← 0 to n-1
        C[i, j] ← C[i, j] + A[i, k]*B[k, j]

  return C
```

Time:

$$\theta(n^3)$$

Loop ordering ve cache blocking practical performance'ı etkileyebilir; asymptotic exponent değişmez.

3. Block matrix representation Matrices dört $n/2 \times n/2$ block'a bölünür:

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

Product:

$$C = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$$

Classical block formulas:

$$\begin{aligned} C_{11} &= A_{11}B_{11} + A_{12}B_{21} \\ C_{12} &= A_{11}B_{12} + A_{12}B_{22} \\ C_{21} &= A_{21}B_{11} + A_{22}B_{21} \\ C_{22} &= A_{21}B_{12} + A_{22}B_{22} \end{aligned}$$

Blocks scalar gibi görünsün de operations matrix multiplication/addition'dır.

4. Naive Divide and Conquer

Dört output block ve her biri için iki product:

8 recursive multiplications of size $n/2$

Matrix additions total $\Theta(n^2)$ work:

$$T(n) = 8T(n/2) + \Theta(n^2)$$

Master Theorem:

$$\begin{aligned} a &= 8, \quad b = 2 \\ n^{\log_2 8} &= n^3 \\ f(n) &= n^2 \end{aligned}$$

Recursive work baskın:

$$T(n) = \Theta(n^3)$$

Yalnızca problemi blocks'a bölmek exponent'i azaltmaz.

5. Strassen'ın temel fikri Matrix multiplication, addition'a göre daha pahalı kabul edilir. Sekiz recursive products yerine yedi carefully chosen products hesaplanır; output blocks bu products'ın sums/differences'ından oluşturulur.

Bir multiplication'ın eksilmesi recursion'ın her level'ında tekrarlandığı için asymptotic exponent düşer.

6. Yedi Strassen products

Yaygın notation:

$$\begin{aligned} M_1 &= (A_{11} + A_{22})(B_{11} + B_{22}) \\ M_2 &= (A_{21} + A_{22})B_{11} \\ M_3 &= A_{11}(B_{12} - B_{22}) \\ M_4 &= A_{22}(B_{21} - B_{11}) \\ M_5 &= (A_{11} + A_{12})B_{22} \\ M_6 &= (A_{21} - A_{11})(B_{11} + B_{12}) \\ M_7 &= (A_{12} - A_{22})(B_{21} + B_{22}) \end{aligned}$$

Her juxtaposition bir size- $n/2$ matrix multiplication'dır.

7. Output blocks

$$\begin{aligned} C_{11} &= M_1 + M_4 - M_5 + M_7 \\ C_{12} &= M_3 + M_5 \\ C_{21} &= M_2 + M_4 \\ C_{22} &= M_1 - M_2 + M_3 + M_6 \end{aligned}$$

Bu identities expand edilerek classical block formulas'a eşit oldukları doğrulanabilir.

8. C11 doğrulama örneği

$$C11 = M1 + M4 - M5 + M7$$

Products açıldığında positive ve negative cross terms birbirini götürür; kalan:

$$A11B11 + A12B21$$

olur. Diğer output blocks için de aynı algebraic cancellation gerçekleşir.

Strassen'ın başarısı yalnızca recursive programming değil, bu algebraic decomposition'dır.

9. Recursive Strassen pseudocode

```
Algorithm Strassen(A, B)
  n ← dimension(A)

  if n ≤ CUTOFF
    return ClassicalMultiply(A, B)

  split A into A11, A12, A21, A22
  split B into B11, B12, B21, B22

  compute M1...M7 recursively
  compute C11...C22 from M1...M7

  return combine(C11, C12, C21, C22)
```

Theoretical analysis cutoff'u 1 veya 2 alabilir; practical implementation daha büyük threshold kullanır.

10. Recurrence ve çözüm Yedi recursive calls:

$$7T(n/2)$$

Block additions, subtractions, splits/combines uygun view/buffer design ile:

$$\theta(n^2)$$

Toplam:

$$T(n) = 7T(n/2) + \theta(n^2)$$

Master Theorem:

$$\log_2 7 \approx 2.8073549 > 2$$

Sonuç:

$$T(n) = \theta(n^{\log_2 7})$$

11. Neden küçük fark önemli? Bir level'da sekiz yerine yedi products az görünür. Fakat depth boyunca:

Naive leaf multiplications: $8^{\log_2 n} = n^3$
Strassen leaf multiplications: $7^{\log_2 n} = n^{\log_2 7}$

Recursive factor'daki değişiklik polynomial exponent'i düşürür.

12. Power-of-two olmayan dimensions Basit anlatım $n=2^k$ varsayar. Diğer dimensions için matrices zero-padding ile next power of two'ya genişletilebilir.

Ancak doğrudan next power of two'ya padding bazı sizes için ciddi overhead yaratabilir. Practical libraries:

- Uneven blocks
- Peeling/fringe handling
- Carefully selected padding
- Classical cutoff

kullanabilir.

13. Rectangular matrices Strassen en temiz biçimde square matrices için anlatılır. Rectangular multiplication için blocking ve shape-aware variants gerekir.

Bir $m \times k$ ve $k \times n$ product'ı körlemesine büyük square matrix'e pad etmek memory/work israfı yaratabilir.

14. Space Complexity Straightforward implementation birçok temporary sum ve product matrix oluşturur. Peak auxiliary memory implementation'a bağlıdır ve naive allocation ciddi olabilir.

Optimized implementations:

- Buffers reuse eder.
- Blocks'ı copies yerine views olarak temsil eder.
- Products'ı uygun sırada hesaplayıp temporaries'i yeniden kullanır.

Time improvement ücretsiz değildir; memory traffic önemli bir trade-off'tur.

15. Practical cutoff Small matrices'te classical multiplication genellikle daha hızlıdır çünkü:

- Strassen daha fazla additions yapar.
- Recursive-call overhead vardır.
- Temporaries ve memory traffic artar.
- Optimized classical kernels cache/vector hardware'ı iyi kullanır.

Bu yüzden base case teoride 1×1 olsa da pratikte benchmark ile seçilen daha büyük CUTOFF kullanılır.

16. Numerical considerations Floating-point arithmetic exact değildir. Strassen'ın ek additions/subtractions'ı rounding error davranışını classical algorithm'dan farklılaştırabilir.

Bu, her durumda kullanılamaz demek değildir; numerical stability, input scale ve required precision değerlendirilmelidir.

Integer/modular arithmetic'te farklı overflow ve intermediate-value sorunları görülebilir.

17. Daha hızlı matrix multiplication var mı? Strassen, $O(n^3)$ barrier'ını kıran tarihsel algorithm'dır. Daha düşük theoretical exponents sağlayan algorithms geliştirilmiştir.

Ancak theoretical exponent tek practical ölçüt değildir:

- Huge constants
- Implementation complexity
- Numerical behavior
- Memory overhead
- Relevant matrix sizes

nedeniyle Strassen ve optimized classical multiplication pratikte hâlâ daha anlamlı olabilir.

18. Sık yapılan hatalar Block'ları scalar sanmak A11B11, recursive matrix multiplication'dır.

Naive divide-and-conquer'ın daha hızlı olduğunu sanmak Sekiz recursive products recurrence'ı yine $\Theta(n^3)$ yapar.

Additions'ı ücretsiz saymak Toplam $\Theta(n^2)$ per level combine work vardır ve practical constants önemlidir.

Strassen'ı küçük matrices'te zorunlu kullanmak Cutoff olmadan overhead faydayı aşabilir.

Padding cost'unu yok saymak Next-power-of-two padding work ve memory'yi artırabilir.

2.81'i exact exponent sanmak Exact exponent:

$$\log_2 7 \approx 2.8073549$$

19. Kavram sözlüğü

English term	Türkçe açıklama
Matrix Multiplication	Rows ile columns'ın dot product'larından output matrix üretme
Block Matrix	Matrix'i submatrix blocks hâlinde ele alma
Strassen's Algorithm	Yedi recursive products ile exponent'i düşüren algorithm
Scalar Multiplication	Tek numbers arasında multiplication
Matrix Addition	Corresponding entries'ın toplanması
Zero Padding	Dimension uyumu için zero rows/columns ekleme
Cutoff	Recursive Strassen'dan classical kernel'a geçiş size'ı
Numerical Stability	Rounding errors'ın computation boyunca nasıl büyüdüğü özelliği

20. Kendini sinama

1. Classical square matrix multiplication time'ı nedir?
2. Naive divide-and-conquer kaç recursive product kullanır?
3. Neden recurrence hâlâ $\theta(n^3)$ olur?
4. Strassen kaç recursive product kullanır?
5. Strassen recurrence'ı ve sonucu nedir?
6. Power-of-two olmayan dimensions nasıl ele alınabilir?
7. Neden practical cutoff gerekir?
8. Strassen'ın iki önemli practical maliyeti nedir?

Kısa cevaplar

1. $\theta(n^3)$.
2. Sekiz.
3. $T(n)=8T(n/2)+\theta(n^2)$ ve $\log_2 8=3$ olduğu için.
4. Yedi.
5. $7T(n/2)+\theta(n^2)$; $\theta(n^{\log_2 7}) \approx \theta(n^{2.8074})$.
6. Padding, uneven blocks veya fringe handling ile.
7. Small sizes'te recursion, addition ve memory overhead classical kernel'dan pahalı olabilir.
8. Extra additions/temporaries-memory traffic ve numerical-error davranışı.

Sonraki ders: **39 – Greedy Method: Introduction.**

39 – Greedy Method: Introduction

Video: 3.1 Greedy Method: Introduction (12:02)

Bu videonun ana fikri Greedy Method, solution'ı stages hâlinde kurar ve her stage'de o anda en iyi görünen feasible choice'ı kalıcı olarak seçer.

Choose locally best feasible option
Commit to it
Continue on the remaining problem

Ama kritik uyarı:

Bir problem optimization problem olduğu için Greedy otomatik olarak doğru değildir.

Greedy'nin global optimum verdiği ayrıca kanıtlanmalıdır.

1. Optimization Problem Bir objective function'ı minimize veya maximize etmeyi isteyen problem:

Minimize cost/time/distance
veya
Maximize profit/value/throughput

şeklinde tanımlanır.

Matematiksel görünüm:

Optimize $f(x)$
subject to constraints on x

Burada x , candidate solution'ı temsil eder.

2. Candidate Solution Problem için düşünülebilecek her possible assignment/selection bir candidate solution'dur.

Örneğin bir yolculukta:

- Walk
- Car
- Train
- Flight

farklı candidate solutions olabilir.

Henüz constraints'i sağlayıp sağlamadıkları bilinmek zorunda değildir.

3. Constraint ve Feasible Solution Constraint, solution'ın uyması gereken koşuldur.

Örnek:

Travel time $\leq$ 12 hours

Bu koşulu sağlayan candidate:

Feasible Solution

olarak adlandırılır.

Feasibility, objective'in iyi olmasıyla aynı değildir. Bir solution valid olabilir fakat pahalı/yavaş olduğu için optimal olmayabilir.

4. Objective Function ve Optimal Solution Objective:

```
Minimize travel cost
```

ise feasible solutions arasında minimum cost'u veren solution optimaldir.

Optimal Solution:

- Önce feasible olmalıdır.
- Feasible set içindeki en iyi objective value'yu sağlamalıdır.

5. Birden fazla optimal solution olabilir Videoda "yalnızca bir optimal solution olabilir" deniyor; bu doğru değildir.

Örneğin iki farklı rota:

```
Route A cost = 100  
Route B cost = 100
```

ve başka hiçbir feasible rota daha ucuz değilse, ikisi de optimaldir.

Doğru ifade:

Optimal objective value tektir; fakat bu value'yu gerçekleştiren birden fazla optimal solution olabilir.

Job Sequencing dersindeki aynı maximum profit'i veren farklı schedules bunun somut örneğidir.

6. Greedy choice nedir? Her stage'de candidate set içinden bir selection rule ile choice yapılır:

```
x <- Select(candidates)
```

Choice partial solution'ı infeasible yapmıyorsa kabul edilir:

```
if Feasible(solution u {x})  
    solution <- solution u {x}
```

Bu choice genellikle geri alınmaz; Greedy'nin irrevocable choice özelliği budur.

7. Generic Greedy schema

```
Algorithm Greedy(candidates)  
    solution <- empty  
  
    while candidates is not empty and solution not complete  
        x <- SelectBestCandidate(candidates)  
        remove x from candidates  
  
        if Feasible(solution, x)  
            Add x to solution  
  
    return solution
```

Asıl problem-specific kısımlar:

- SelectBestCandidate
- Feasible
- Solution'ın complete olma koşulu

8. Local optimum ve Global optimum Locally optimal choice, current stage'de en iyi görünen karardır.

Globally optimal solution, bütün choices birlikte değerlendirildiğinde mümkün olan en iyi final result'tır. Greedy'nin hedefi:

Local choices -> Global optimum

Fakat bu implication her problemde geçerli değildir.

9. Greedy-Choice Property Bir problem Greedy-Choice Property taşıyorsa optimal solution'lardan en az biri greedy choice ile başlayabilir.

Yani ilk local choice gelecekte global optimum'u imkânsız hâle getirmez.

Bu property genellikle Exchange Argument ile kanıtlanır:

1. Herhangi bir optimal solution al.
2. Greedy choice bu solution'da yoksa uygun bir choice ile değiştir.
3. Feasibility ve objective value'nun kötüleşmediğini göster.
4. Greedy choice içeren optimal solution elde et.

10. Optimal Substructure Optimal solution içindeki remaining decisions, choice yapıldıktan sonra kalan subproblem için de optimal olmalıdır.

Optimal solution
= greedy choice
+ remaining subproblem'in optimal solution'ı

Optimal Substructure, Dynamic Programming'de de görülür. Tek başına Greedy için yeterli değildir; greedy-choice property de gerekir.

11. Greedy ne zaman başarısız olur? Coin Change karşı örneği Coins:

[1,3,4]
Target=6

Largest coin first:

4+1+1 = 3 coins

Optimal:

3+3 = 2 coins

Local olarak largest coin seçmek global optimum vermemiştir.

0/1 Knapsack Highest value/weight ratio'yu seçmek fractional version'da optimaldir; indivisible 0/1 version'da optimal olmak zorunda değildir.

12. Greedy ile Dynamic Programming farkı

Greedy	Dynamic Programming
Bir local choice yapıp genellikle geri dönmez	Birçok subproblem alternative'ini karşılaştırır
State genellikle daha azdır	Table/memoization kullanabilir
Çoğu zaman daha hızlı ve düşük space	Daha geniş problem sınıfında optimum bulabilir
Correctness için greedy-choice proof gerekir	Recurrence ve optimal substructure proof gerekir

Bir problemin DP ile çözülebilmesi Greedy ile de çözülebileceği anlamına gelmez.

13. Greedy ile Divide and Conquer farkı Divide and Conquer:

- Problem'i independent subproblems'a böler.
- Birden fazla subproblem'ı çözüp results'ı combine eder.

Greedy:

- Candidate'lar arasından bir choice yapar.
- Partial solution'ı incrementally büyütür.
- Choice genellikle kalıcıdır.

İki strategy farklı design patterns'dır; bir algorithm bazı yönleriyle ikisini birlikte de kullanabilir.

14. Selection rule neden yeterli değildir? "En pahalıyı seç", "en hafifi seç", "en erken biteni seç" gibi rules ancak belirli problem structure'ı içinde anlamlıdır.

Bir heuristic iyi sonuçlar üretebilir fakat optimal guarantee vermeyebilir.

Greedy algorithm = selection rule + feasibility rule + proof

Proof yoksa elimizde çoğunlukla yalnızca heuristic vardır.

15. Greedy correctness proof teknikleri Exchange Argument Optimal solution, greedy choice'ı içerecek şekilde kayıpsız dönüştürülür.

Greedy Stays Ahead Her prefix/stage sonunda greedy partial solution'ın başka bir solution'dan geri kalmadığı gösterilir.

Structural Induction Greedy choice sonrası remaining subproblem'in aynı formda olduğu ve induction hypothesis'in uygulanabildiği gösterilir.

Cut Property Graph algorithms'da, belirli cut üzerinden safe edge seçimi kanıtlanır.

16. Greedy algorithm örnekleri Bu playistte görülecek örnekler:

- Fractional Knapsack
- Job Sequencing with Deadlines
- Optimal Merge Pattern
- Huffman Coding
- Prim's Algorithm
- Kruskal's Algorithm
- Dijkstra's Algorithm – nonnegative edges koşuluyla

Her biri farklı selection ve feasibility rules kullanır.

17. Complexity nereden gelir? Greedy algorithms çoğu zaman candidates'ı bir key'e göre sıralar:

Sorting = $O(n \log n)$
Scan = $O(n)$

Total sıklıkla:

$O(n \log n)$

olur. Fakat heap, disjoint set veya graph traversal gibi data structures complexity'yi değiştirebilir.

18. Sık yapılan hatalar Her optimization problemine Greedy uygulamak Problem-specific proof gerekir.

Feasible ile optimal'i aynı sanmak Feasible yalnızca constraints'i sağlar; en iyi olmak zorunda değildir.

Optimal solution'ın mutlaka unique olduğunu sanmak Multiple optimal solutions aynı objective value'yu paylaşabilir.

Local best'in global best olduğunu varsaymak Greedy-choice property kanıtlanmalıdır.

Example üzerinde çalışmayı proof sanmak Birkaç input'ta doğru sonuç general correctness'i kanıtlamaz.

Selection rule'u problem definition'dan koparmak Aynı "largest first" rule bir problemde doğru, başka problemde yanlış olabilir.

19. Kavram sözlüğü

English term	Türkçe açıklama
Optimization Problem	Objective'i minimize veya maximize eden problem
Constraint	Valid solution'ın uyması gereken koşul
Feasible Solution	Bütün constraints'i sağlayan solution
Optimal Solution	Feasible solutions içinde en iyi objective value'yu veren solution
Objective Function	Optimize edilmek istenen ölçü
Greedy Choice	Current stage'de local olarak en iyi seçilen candidate
Greedy-Choice Property	Optimal solution'ın greedy choice ile başlayabilme özelliği
Optimal Substructure	Optimal solution'ın optimal subsolutions içermesi
Exchange Argument	Optimal solution'ı greedy choice içerecek biçimde kayıpsız dönüştüren proof
Heuristic	İyi sonuç hedefleyen fakat optimum guarantee veremeyebilen yöntem

20. Kendini sınama

1. Feasible solution ile optimal solution arasındaki fark nedir?
2. Birden fazla optimal solution olabilir mi?
3. Greedy choice'ın temel özelliği nedir?
4. Her optimization probleminde Greedy neden çalışmaz?
5. Greedy correctness için sık kullanılan iki property nedir?
6. Exchange Argument neyi gösterir?
7. Coin [1,3,4], target 6 örneği neyi kanıtlar?

Kısa cevaplar

1. Feasible constraints'i sağlar; optimal ayrıca en iyi objective value'yu verir.
2. Evet; aynı optimal value'yu gerçekleştiren farklı solutions olabilir.
3. Current stage'de local best seçilir ve genellikle geri alınmaz.
4. Local optimum global optimum'u garanti etmez.
5. Greedy-Choice Property ve Optimal Substructure.
6. Bir optimal solution'ın greedy choice'ı içerecek şekilde objective bozulmadan dönüştürülebileceğini.
7. Largest-first choice'ın canonical olmayan coin systems'de optimal olmayabileceğini.

Sonraki ders: **40 – Knapsack Problem: Greedy Method.**

40 – Fractional Knapsack: Greedy Method

Video: 3.2 Knapsack Problem: Greedy Method (15:30)

Bu videonun ana fikri Bu videoda çözülen problem Fractional Knapsack'tır. Items bölünebilir; bir item'in tamamı veya herhangi bir fraction'ı alınabilir.

Greedy selection key:

```
Value density = profit / weight
```

Items decreasing value-density order'da alınır. Son item tamamen sığmıyorsa kalan capacity kadar fraction alınır.

```
Time = O(n log n) // sorting nedeniyle
```

Bu greedy rule Fractional Knapsack için optimaldir; 0/1 Knapsack için değildir.

1. Problem definition Her item i için:

```
profit  $p_i$   
weight  $w_i > 0$ 
```

Knapsack capacity:

```
M
```

Decision variable:

```
 $0 \leq x_i \leq 1$ 
```

x_i , item'in alınan fraction'ıdır.

Objective:

```
Maximize  $\sum p_i x_i$ 
```

Constraint:

```
 $\sum w_i x_i \leq M$ 
```

2. Neden yalnız profit'e bakılmaz? İki item düşün:

```
Item A: profit 100, weight 100 -> density 1  
Item B: profit 60, weight 10 -> density 6
```

Capacity sınırlıyken B, her unit weight başına daha fazla profit üretir.

Yalnız maximum profit seçmek, heavy fakat düşük-density item ile capacity'yi verimsiz doldurabilir.

3. Neden yalnız minimum weight'e bakılmaz? Hafif item'in profit'i çok düşük olabilir:

```
Item A: profit 1, weight 1 -> density 1  
Item B: profit 20, weight 2 -> density 10
```

Weight tek başına objective'i temsil etmez. Doğru marginal gain:

```
profit per unit weight
```

4. Greedy algorithm

```
Algorithm FractionalKnapsack(items, capacity)
  for each item
    item.density <- item.profit / item.weight

  sort items by density descending

  totalProfit <- 0
  remaining <- capacity

  for item in sorted items
    if remaining == 0
      break

    if item.weight <= remaining
      take whole item
      remaining <- remaining-item.weight
      totalProfit <- totalProfit+item.profit
    else
      fraction <- remaining/item.weight
      take fraction of item
      totalProfit <- totalProfit+fraction*item.profit
      remaining <- 0

  return totalProfit and selected fractions
```

5. Videodaki örnek Yedi item:

Item	Profit p_i	Weight w_i	Density p_i/w_i
1	10	2	5
2	5	3	$5/3 \approx 1.667$
3	15	5	3
4	7	7	1
5	6	1	6
6	18	4	4.5
7	3	1	3

Capacity:

M=15

Density order:

Item 5, Item 1, Item 6, Item 3/7, Item 7/3, Item 2, Item 4

Items 3 ve 7 aynı density'ye sahip olduğu için kendi aralarındaki order optimal value'yu değiştirmez.

6. Adım adım seçim Bir valid tie order kullanalım:

Seçim	Alınan weight	Kazanılan profit	Remaining capacity
Item 5 tamamı	1	6	14
Item 1 tamamı	2	10	12
Item 6 tamamı	4	18	8
Item 3 tamamı	5	15	3
Item 7 tamamı	1	3	2
Item 2'nin 2/3ü	2	10/3	0

Decision vector, original item order'ında:

$$x = [1, 2/3, 1, 0, 1, 1, 1]$$

7. Weight doğrulaması

$$\begin{aligned} \sum w_i x_i \\ &= 2(1) + 3(2/3) + 5(1) + 7(0) + 1(1) + 4(1) + 1(1) \\ &= 2 + 2 + 5 + 0 + 1 + 4 + 1 \\ &= 15 \end{aligned}$$

Constraint tam capacity ile sağlanır.

8. Profit doğrulaması ve video düzeltmesi

$$\begin{aligned} \sum p_i x_i \\ &= 10(1) + 5(2/3) + 15(1) + 7(0) + 6(1) + 18(1) + 3(1) \\ &= 10 + 10/3 + 15 + 0 + 6 + 18 + 3 \\ &= 166/3 \\ &= 55\frac{1}{3} \approx 55.333 \end{aligned}$$

Videodaki yaklaşık 54.6 sonucu, Item 2'nin density/fraction profit hesabındaki aritmetik hatadan kaynaklanır. Doğru exact optimum:

$$55\frac{1}{3}$$

9. Greedy neden optimal? – Exchange Argument En yüksek remaining density'ye sahip item g olsun. Bir purported optimal solution, capacity'nin bir kısmını lower-density item j'ye ayırmış fakat g'den alınabilecek kısmı almamış olsun.

Equal amount of weight δ için:

$$\begin{aligned} \text{Greedy item gain} &= \delta * \text{density}(g) \\ \text{Other item gain} &= \delta * \text{density}(j) \end{aligned}$$

$$\text{density}(g) \geq \text{density}(j)$$

olduğundan j'den δ weight çıkarıp g'ye aktarmak:

- Feasibility'yi değiştirmez.
- Profit'i azaltmaz.

Bu exchange tekrarlanarak greedy prefix'i içeren optimal solution oluşturulur. Bu nedenle density order optimaldir.

10. Neden en fazla bir fractional item vardır? Sorted density order'da items tamamen alınır. İlk sığmayan item'dan capacity'yi dolduracak fraction alınır ve remaining capacity zero olur.

Dolayısıyla standard solution'da:

At most one item has $0 < x_i < 1$

Equal densities varsa farklı fakat aynı-value optimal representations olabilir.

11. Complexity Density computation:

$\theta(n)$

Sorting:

$O(n \log n)$

Greedy scan:

$O(n)$

Total:

$O(n \log n)$

Items density'ye göre önceden sorted ise scan $\theta(n)$ dir.

Sorting'e bağlı auxiliary space kullanılan algorithm'a göre değişir.

12. Fractional ile 0/1 Knapsack farkı Fractional

$0 \leq x_i \leq 1$

Item bölünebilir. Density-greedy optimaldir.

0/1

$x_i \in \{0,1\}$

Item ya tamamen alınır ya alınmaz. Density-greedy optimal olmak zorunda değildir.

13. 0/1 için greedy karşı örneği Capacity:

50

Items:

(weight=10, profit=60, density=6)
(weight=20, profit=100, density=5)
(weight=30, profit=120, density=4)

Density greedy ilk iki item'ı alır:

weight=30, profit=160

Third item kalan 20'ye sığmaz.

Optimal 0/1 choice:

Item 2 + Item 3
weight=50, profit=220

Fraction alınamadığı için exchange proof bozulur.

14. Capacity her zaman tamamen dolar mı? Positive profits ve divisible items varsa total available weight capacity'yi aşıyorsa optimal solution capacity'yi doldurur.

Şu durumlarda boş capacity kalabilir:

- Bütün items'ın total weight'i capacity'den azsa
- Zero/negative density items almak zorunlu değilse
- Ek constraints varsa

Problem assumptions açık olmalıdır.

15. Floating-point ve ratio comparison p_i/w_i yi floating-point olarak hesaplamak rounding issues oluşturabilir. Positive integer inputs için iki density:

$$p1/w1 > p2/w2$$

cross multiplication ile karşılaştırılabilir:

$$p1*w2 > p2*w1$$

Products integer overflow yapabilecekse wider type/arbitrary precision gerekir.

16. Sık yapılan hatalar Problem'i 0/1 Knapsack sanmak Videodaki problem fractions'a izin verir.

Highest profit'i doğrudan seçmek Capacity consumption dikkate alınmalıdır.

Lowest weight'i doğrudan seçmek Objective profit'tir; weight başına profit önemlidir.

Fraction profit'ini yanlış hesaplamak Item'ın x fraction'ı alınırsa profit $x*p_i$, weight $x*w_i$ dir.

Density'yi rounded value ile erken yuvarlamak Exact comparison/order bozulabilir.

Greedy proof'u 0/1 version'a taşımak Arbitrary fractional exchange yapılamadığı için proof geçersizdir.

17. Kavram sözlüğü

English term	Türkçe açıklama
Knapsack	Capacity'si sınırlı taşıma container'ı
Fractional Knapsack	Items'ın fraction olarak alınabildiği optimization problem
0/1 Knapsack	Her item'ın tamamen alındığı veya bırakıldığı problem
Value Density	Unit weight başına profit; p_i/w_i
Capacity Constraint	Total selected weight'in capacity'yi aşmaması
Decision Variable	Item'dan alınan fraction'ı gösteren x_i
Exchange Argument	Lower-density allocation'ı higher-density ile değiştirerek optimality kanıtı
Marginal Gain	Bir unit resource eklemenin objective'e katkısı

18. Kendini sına

1. Fractional Knapsack'ın greedy key'i nedir?
2. x_i hangi aralıkta değer alır?
3. Videodaki örneğin doğru total profit'i nedir?
4. Neden en fazla bir item fractional olur?
5. Sorting dahil time complexity nedir?
6. Density greedy neden 0/1 Knapsack'te başarısız olabilir?

7. Ratios floating-point division yapmadan nasıl karşılaştırılabilir?

Kısa cevaplar

1. p_i/w_i , yani value density.
2. $0 \leq x_i \leq 1$.
3. $166/3 = 55\frac{1}{3} \approx 55.333$.
4. İlk kısmen alınan item remaining capacity'yi tamamen doldurur.
5. $O(n \log n)$; presorted ise scan $O(n)$.
6. Items bölünemediği için local density choice future combinations'ı engelleyebilir.
7. $p1*w2$ ile $p2*w1$ karşılaştırılarak.

Sonraki ders: **41 – Job Sequencing with Deadlines.**

41 – Job Sequencing with Deadlines

Video: 3.3 Job Sequencing with Deadlines (13:29)

Bu videonun ana fikri Tek bir machine üzerinde, her biri exactly one unit time alan jobs verilir. Job i , deadline d_i ye kadar tamamlanırsa profit p_i kazanılır.

Amaç:

Deadline'ları kaçırmadan total profit'i maximize et

Greedy strategy:

1. Jobs'u decreasing profit order'da incele.
2. Her job'u deadline'ını aşmayan en geç empty slot'a yerleştir.
3. Uygun slot yoksa job'u reddet.

1. Problem assumptions Bu greedy algorithm şu özel model içindir:

- Tek machine vardır.
- Bütün jobs aynı anda available'dır.
- Her job exactly one unit processing time alır.
- Job preempt edilmez.
- Deadlines integer slot boundaries'dir.
- Profit, job deadline'a kadar biterse kazanılır.
- Genellikle profits nonnegative kabul edilir.

Assumptions değişirse algorithm'ın correctness'i değişebilir.

2. Feasible schedule Job i , slot t ye yerleştirilsin. Slot numbering $1..D$ olsun:

$t \leq d_i$

olmalıdır.

Her slot'ta en fazla bir job bulunabilir.

$D = \max_i d_i$

Fakat n job'dan fazla slot kullanılamayacağı için implementation:

number of slots = $\min(n, \text{maxDeadline})$

alabilir.

3. Neden profit descending? Bir slot için birden fazla job yarışiyorsa yüksek-profit job'u korumak total objective açısından daha değerlidir.

Bu tek başına yeterli değildir; job'u hangi slot'a koyduğumuz da önemlidir.

4. Neden latest available slot? Deadline d olan job, $1..d$ slots'ın herhangi birinde yapılabilir. Onu mümkün olan en geç slot'a koymak:

- Job'un kendi feasibility'sini korur.
- Daha erken deadline'lı future jobs için early slots'ı açık bırakır.

Örneğin deadline 3 olan job slot 1'e gereksiz yere konursa, yalnız slot 1'e sığabilen deadline-1 job engellenir.

5. Greedy pseudocode – basit sürüm

```

Algorithm JobSequencing(jobs)
    sort jobs by profit descending

    D <- min(number of jobs, maximum deadline)
    slots[1..D] <- EMPTY
    totalProfit <- 0

    for each job in sorted jobs
        t <- min(job.deadline, D)

        while t >= 1 and slots[t] is not EMPTY
            t <- t-1

        if t >= 1
            slots[t] <- job
            totalProfit <- totalProfit+job.profit

    return slots, totalProfit

```

6. Birinci örnek Videodaki yapı:

Job	Deadline	Profit
J1	2	20
J2	2	15
J3	1	10
J4	3	5
J5	3	daha düşük

Maximum deadline:

D=3

Slots:

[1] [2] [3]

J1 Deadline 2; latest free slot 2:

[_ , J1, _]

J2 Deadline 2; slot 2 dolu, slot 1 free:

[J2, J1, _]

J3 Deadline 1; slot 1 dolu. Reject.

J4 Deadline 3; slot 3 free:

[J2, J1, J4]

Total profit:

15+20+5=40

7. Multiple optimal schedules J1 ve J2nin deadlines'i 2 olduğundan iki valid execution order bulunabilir:

[J2, J1, J4]
[J1, J2, J4]

İkisi de same selected set ve profit 40 verir.

Bu, farklı optimal solutions'ın bulunabileceğini gösterir. Optimal value unique olabilir; schedule unique olmak zorunda değildir.

8. İkinci örnek Jobs decreasing profit order'da olsun:

Job	Deadline	Profit
J1	3	35
J2	4	30
J3	4	25
J4	2	20
J5	3	15
J6	1	12
J7	2	5

Four slots vardır.

Selections:

J1 -> slot 3
J2 -> slot 4
J3 -> slot 2 // slot 4 ve 3 dolu
J4 -> slot 1 // slot 2 dolu

Schedule:

[J4, J3, J1, J2]

Profit:

20+25+35+30=110

Slots dolduğu için lower-profit remaining jobs seçilmez.

9. Feasibility kontrolü Schedule [J4, J3, J1, J2] için completion slots:

J4 completes at 1 <= deadline 2
J3 completes at 2 <= deadline 4
J1 completes at 3 <= deadline 3
J2 completes at 4 <= deadline 4

Bütün deadlines sağlanır.

10. Correctness – Greedy ve matroid yapısı Unit-time jobs için schedulable job sets bir matroid oluşturur. Bir set S şu koşulla feasible'dir:

Her t için, deadline'ı t veya daha küçük olan selected job sayısı <= t

Bu family:

- Empty set'i içerir.
- Feasible set'in her subset'i de feasible'dir.

- Daha büyük feasible set'ten daha küçük feasible set'e eklenebilecek uygun bir job bulunduğunu söyleyen exchange propertyyi sağlar.

Weighted Matroid Greedy Theorem gereği elements decreasing weight/profit order'da ele alınıp feasibility'yi koruyanlar seçildiğinde maximum-weight feasible set elde edilir.

Burada latest-free-slot işlemi, selected set'in feasible kaldığını hem test eder hem de somut bir schedule üretir. Job'u olabildiğince geç yerleştirmek earlier slots'ı future tight-deadline jobs için korur.

11. Naive Complexity Sorting:

$O(n \log n)$

Her job için deadline'dan geriye linear slot scan:

$O(D)$ per job

Toplam:

$O(n \log n + nD)$

$D \leq n$ sınırlandırmasıyla worst case:

$O(n^2)$

Space:

$O(D)$

12. Disjoint Set Union optimization En geç available slot'ı hızlı bulmak için Disjoint Set Union (DSU) kullanılabilir.

Interpretation:

$\text{find}(t) = t$ veya t 'den önceki latest free slot

Slot t kullanıldığında:

$\text{parent}[t] \leftarrow \text{find}(t-1)$

Path compression ile scheduling operations total yaklaşık:

$O(n \alpha(n))$

Sorting dahil:

$O(n \log n)$

13. DSU pseudocode

```
Algorithm JobSequencingDSU(jobs)
  sort jobs by profit descending
  D ← min(n, maximum deadline)

  for t ← 0 to D
    parent[t] ← t

  for job in jobs
```



```
t <- Find(min(job.deadline, D))

if t > 0
  schedule job at slot t
  parent[t] <- Find(t-1)
```

Slot 0, "available positive slot yok" sentinel'idir.

14. Why unit processing time matters Bir job iki veya daha fazla units sürerse tek slot assignment modeli bozulur. Profit descending ve latest single slot kuralı artık yeterli olmayabilir.

Variable processing times ile problem:

- Weighted scheduling
- Knapsack-like selection
- Daha zor scheduling variants

hâline gelebilir.

Videodaki greedy guarantee yalnız unit-time jobs içindir.

15. Negative veya zero profits Bir job'u yapmak zorunlu değilse negative-profit job seçilmemelidir; empty slot bırakmak daha iyidir.

Zero-profit jobs objective'i değiştirmez; yalnızca ayrıca bir secondary objective varsa anlamlı olabilir.

Implementation, $profit \leq 0$ jobs'u skip edebilir.

16. Release times ve dependencies Jobs'ın farklı release times'ı veya precedence constraints'i varsa:

```
"deadline'dan önce latest free slot"
```

tek başına feasibility'yi temsil etmez. Problem modelinin adı benzer olsa bile aynı greedy proof kullanılamaz.

17. Sık yapılan hatalar Job'u earliest slot'a koymak Future tight-deadline jobs için gerekli early slots gereksiz yere kapanır.

Deadline slot'u doluysa hemen reddetmek Deadline'dan önceki daha erken free slots aranmalıdır.

Deadline'dan sonraki slot'a koymak Bu solution infeasible'dır; late completion profit getirmez.

Sorting key olarak deadline kullanmak Bu problemde greedy order decreasing profit'tir; earliest-deadline farklı scheduling objectives için kullanılır.

Processing times'ın unit olduğunu unutmak Algorithm'ın correctness assumption'ı bozulur.

Optimal schedule'ın unique olduğunu sanmak Same selected jobs farklı feasible orders ile aynı profit'i verebilir.

Naive scan'i $O(n \log n)$ sanmak Sorting dışında slot scans worst case quadratic olabilir; DSU optimization gerekir.

18. Kavram sözlüğü

English term	Türkçe açıklama
Job Sequencing with Deadlines	Unit jobs arasından deadline'larla maximum profit schedule seçme problem
Deadline	Job'un profit kazanmak için bitmesi gereken en geç time
Time Slot	Bir unit-time job'ın çalıştırılabildiği interval
Schedule	Jobs'ın machine üzerindeki execution order ve slots'ı
Latest Available Slot	Deadline'ı aşmayan en geç empty slot
Unit Processing Time	Her job'ın exactly bir time unit sürmesi
Disjoint Set Union	Latest free slot queries'ini hızlandıran data structure
Path Compression	DSU Find paths'ini kısaltan optimization
Non-preemptive	Job başladıktan sonra bölünmeden tamamlanması

19. Kendini sınama

- Jobs hangi key'e göre sıralanır?
- Job neden latest available slot'a yerleştirilir?
- Deadline slot'u doluysa ne yapılır?
- Videodaki ilk örneğin selected jobs ve profit'i nedir?
- Naive time complexity nedir?
- DSU neyi hızlandırır?
- Algorithm hangi processing-time assumption'ına dayanır?
- Birden fazla optimal schedule mümkün müdür?

Kısa cevaplar

- Profit descending.
- Earlier slots'ı tighter deadlines için açık tutmak için.
- Deadline'dan geriye doğru latest free slot aranır; yoksa reject edilir.
- J1,J2,J4; total profit 40.
- $O(n \log n + nD)$, $D \leq n$ iken worst $O(n^2)$.
- Latest free slot query ve occupation update'ini.
- Her job exactly one unit sürer.
- Evet; aynı selected set farklı valid orders'da aynı optimal profit'i verebilir.

Sonraki ders: **42 – Optimal Merge Pattern.**

42 – Optimal Merge Pattern

Video: 3.4 Optimal Merge Pattern (9:33)

Bu videonun ana fikri Birden fazla sorted file/list ikişer ikişer merge edilecekse merge sırası total cost'u değiştirir.

Greedy rule:

Her adımda en küçük iki file'ı merge et.

Yeni oluşan file tekrar candidate set'e eklenir. Bu yöntem Optimal Merge Patternı üretir ve Huffman Coding ile aynı tree-building yapısını kullanır.

1. Cost model Lengths m ve n olan iki sorted list'i merge etme time'ı:

$\theta(m+n)$

Optimal Merge Pattern'da birleşimin cost'u basitleştirilerek:

$\text{mergeCost} = m+n$

alınır.

Yeni merged file'ın size'ı da $m+n$ olur; bu file gelecekte tekrar merge edilirse elements yeniden işlenir.

2. Merge order neden önemlidir? File sizes:

[6, 5, 2, 3]

Kötü sequential pattern

```
6+5 =11
11+2 =13
13+3 =16
Total=40
```

Pairwise pattern

```
6+5 =11
2+3 =5
11+5 =16
Total=32
```

Greedy optimal pattern

```
2+3 =5
5+5 =10
6+10 =16
Total=31
```

Small files erken merge edilince tekrar işlenmeleri ucuz; büyük files tree'nin üstünde daha az kez işlenir.

3. Min-Heap algorithm

```
Algorithm OptimalMergePattern(sizes)
  H <- MinHeap(sizes)
  totalCost <- 0
```

```

while size(H) > 1
  x <- ExtractMin(H)
  y <- ExtractMin(H)
  z <- x+y

  totalCost <- totalCost+z
  Insert(H, z)

return totalCost

```

İstenirse her merge bir internal tree node olarak saklanarak pattern'ın kendisi de reconstruct edilir.

4. Beş file örneği Sizes:

[20,30,10,5,30]

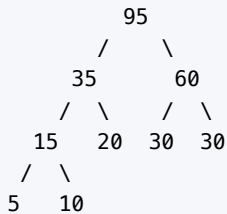
Min-Heap order mantığıyla:

5+10 =15
 15+20 =35
 30+30 =60
 35+60 =95

Total:

15+35+60+95 = 205

5. Merge tree



Leaves original files, internal nodes merge results'dır.

6. İki eşdeğer cost hesabı Internal nodes toplamı

15+35+60+95=205

Weighted external path length Her original file size, tree depth'i kadar merge edilir:

$5*3 + 10*3 + 20*2 + 30*2 + 30*2$
 $=15+30+40+60+60$
 $=205$

Formül:

Total cost = $\sum \text{size}_i * \text{depth}_i$

Bu eşitlik Huffman Coding'e doğrudan bağlantıdır.

7. Greedy neden optimal? En küçük iki weight x ve y olsun. Her full binary merge tree'de en derin sibling leaves seçilebilir.

Bir optimal tree al:

1. En derin sibling positions'da başka weights varsa x ve y ile swap et.
2. En küçük weights daha derine taşındığında weighted path length artmaz.
3. x ve y sibling hâle gelir.
4. Bu siblings'i weight $x+y$ olan tek pseudo-file olarak contract et.
5. Kalan problem aynı formda daha küçük Optimal Merge Pattern problemidir.

Bu greedy-choice ve optimal-substructure proof'udur.

8. Complexity n files için heap construction:

$\Theta(n)$

$n-1$ iterations, her birinde iki extract ve bir insert:

$O(\log n)$ per iteration

Toplam:

$O(n \log n)$

Heap/tree storage:

$O(n)$

Input already sorted olsa bile her yeni sum'ın doğru position'a eklenmesi gerekir. İki queue kullanarak sorted initial weights üzerinde $O(n)$ özel implementation da mümkündür.

9. Equal sizes ve multiple optimal patterns

Equal minimum sizes arasında farklı tie choices yapılabilir. Trees farklı görünse de same minimum total cost üretilebilir.

Optimal value unique olabilir;
optimal merge tree unique olmak zorunda değildir.

Deterministic output istenirse file id gibi secondary tie-breaker tanımlanır.

10. Merging comparisons ile cost model nüansı

İki sorted list'in exact comparison sayısı data order'a bağlıdır ve maximum $m+n-1$ dir. Optimal Merge Pattern genellikle merge cost'u $m+n$ olarak modeller; bu copied/processed elements miktarına karşılık gelir.

Gerçek I/O cost modelinde:

- Read/write blocks
- Buffer sizes
- Number of merge passes
- Storage bandwidth

de önemli olabilir.

11. Huffman Coding bağlantısı

Optimal Merge Pattern:

weights = file sizes
objective = total merge work

Huffman Coding:

weights = symbol frequencies
objective = total encoded bits

ikisinde de:

```
Extract two minimum weights
Combine them
Reinsert their sum
```

12. Sık yapılan hatalar **Yalnız başlangıç listesini bir kez sort etmek** Yeni sums candidate set'e tekrar doğru sırada eklenmelidir.

En küçük iki original file'ı seçip merged results'ı yok saymak Yeni file gelecekteki choices'a katılır.

Total cost'u yalnız final size sanmak Final size her pattern'da aynıdır; intermediate merge costs de toplanmalıdır.

En büyükleri önce merge etmek Large combined file sonraki levels'da tekrar tekrar yüksek cost yaratır.

Optimal pattern'in unique olduğunu sanmak Equal weights multiple optimal trees üretebilir.

13. Kavram sözlüğü

English term	Türkçe açıklama
Optimal Merge Pattern	Total pairwise merge cost'u minimize eden merge sırası
Merge Cost	İki list için size toplamı olarak modellenen iş
Min-Heap	Minimum iki weight'ı hızlı seçen data structure
Merge Tree	Leaves'in original files, internal nodes'in merges olduğu tree
Weighted Path Length	$\sum \text{weight}_i * \text{depth}_i$
Greedy-Choice Property	En küçük iki weight'in optimal tree'de siblings seçilebilmesi
Pseudo-File	Merge sonrası sum weight ile yeniden candidate yapılan file

14. Kendini sına

1. Greedy selection rule nedir?
2. [6,5,2,3] için minimum total cost nedir?
3. Neden merged file heap'e geri eklenir?
4. Internal-node toplamı ile leaf-depth formülü nasıl ilişkilidir?
5. Min-Heap implementation time'ı nedir?
6. Equal weights neye yol açabilir?
7. Huffman Coding ile ortak yapı nedir?

Kısa cevaplar

1. Her adımda en küçük iki current file'ı merge etmek.
2. 31.
3. Sonraki merges'te candidate olacağı için.
4. İkisi aynı total merge cost'u verir: $\sum \text{size}_i * \text{depth}_i$.
5. $O(n \log n)$.
6. Farklı fakat aynı-cost optimal merge trees'e.
7. İki minimum weight'ı birleştirip sum'ı yeniden eklemek.

Sonraki ders: **43 – Huffman Coding**.

43 – Huffman Coding

Video: 3.5 Huffman Coding (17:33)

Bu videonun ana fikri Huffman Coding, sık geçen symbols'a kısa, seyrek geçen symbols'a uzun binary codes vererek message'ın weighted encoded length'ini minimize eder.

Greedy rule:

En düşük frequency'li iki node'u birleştir.

Sonuç bir binary prefix-code tree'dir:

- Left edge genellikle 0
- Right edge genellikle 1
- Root-to-leaf path symbol code'udur.

1. Fixed-Length Coding k distinct symbols için fixed-length code width:

$\text{ceil}(\log_2 k)$

Beş symbols için:

$\text{ceil}(\log_2 5) = 3 \text{ bits/symbol}$

20-symbol message:

$20 \times 3 = 60 \text{ bits}$

Bu, bütün symbols'a frequency'den bağımsız aynı length'i verir.

2. ASCII uyarısı Video her character'ı 8-bit ASCII olarak modeller. Standard ASCII aslında 7-bit character set'tir; pratikte characters çoğunlukla bytes veya Unicode encodings ile saklanır.

Video karşılaştırmasındaki 8-bit assumption pedagojik cost model'dir:

$20 \text{ characters} \times 8 \text{ bits} = 160 \text{ bits}$

Gerçek compression ratio source encoding ve format overhead'ine bağlıdır.

3. Örnek frequencies

Symbol	Frequency
A	3
B	5
C	6
D	4
E	2

Total:

$3+5+6+4+2=20$

Increasing order:

E:2, A:3, D:4, B:5, C:6

4. Huffman tree construction Step 1

E:2 + A:3 → node 5

Current weights:

4,5,5,6

Step 2

D:4 + node:5 → node 9

Current:

B:5, C:6, node:9

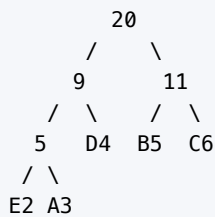
Step 3

B:5 + C:6 → node 11

Step 4

node 9 + node 11 → root 20

Tree:



5. Code assignment Left=0, right=1 seçersek:

Symbol	Code	Length
E	000	3
A	001	3
D	01	2
B	10	2
C	11	2

Left/right labels ters çevrilebilir; codes değişir fakat lengths ve total cost aynı kalır.

6. Encoded message length

$L = \sum \text{frequency}(\text{symbol}) * \text{codeLength}(\text{symbol})$

Örnek:

A: $3*3 = 9$
B: $5*2 = 10$
C: $6*2 = 12$
D: $4*2 = 8$


```
E: 2*3 = 6
Total    =45 bits
```

Bu aynı zamanda tree'nin Weighted External Path Lengthidir.

Fixed 3-bit representation 60 bits iken payload 45 bits olur.

7. Prefix-Free Property Hiçbir symbol code'u başka bir symbol code'unun prefix'i değildir.

Örneğin:

```
01, 10, 11, 000, 001
```

set'inde 01 ile başlayan daha uzun başka valid code yoktur.

Neden? Symbols yalnız leaves'tedir; bir leaf başka leaf'e giden path'in internal point'i olamaz.

Bu property delimiters olmadan unambiguous decoding sağlar.

8. Decoding Bitstream root'tan okunur:

```
0 -> left
1 -> right
```

Leaf'e ulaşınca symbol output edilir ve root'a dönülür.

Örnek:

```
10 11 001 01 ...
B  C  A  D
```

Invalid/truncated bitstream internal node'da bitebilir; decoder format errors'ı ele almalıdır.

9. Huffman neden optimal? Known frequencies ve binary prefix-free codes arasında Huffman minimum weighted code length üretir.

Proof yapısı:

1. En düşük iki frequency'nin bir optimal tree'de deepest sibling leaves seçilebileceği gösterilir.
2. Bu iki leaves bir pseudo-symbol olarak birleştirilir.
3. Kalan daha küçük problem de optimal çözülmelidir.
4. Induction ile greedy construction optimaldir.

Bu, Optimal Merge Pattern proof'uyla aynıdır.

10. Min-Heap implementation

```
Algorithm BuildHuffman(frequencies)
  H <- MinHeap(all symbol nodes)

  while size(H) > 1
    x <- ExtractMin(H)
    y <- ExtractMin(H)

    z <- new internal node
    z.frequency <- x.frequency+y.frequency
    z.left <- x
    z.right <- y

    Insert(H, z)

  return ExtractMin(H)
```

k distinct symbols için:

```
Tree construction =  $O(k \log k)$   
Encoding N symbols =  $O(N)$  with code table  
Decoding =  $O(\text{number of encoded bits})$ 
```

11. Code table/tree overhead düzeltmesi Encoded payload tek başına yeterli değildir; decoder codebook'u bilmelidir. Fakat videodaki gibi variable-length codes'i yalnızca arka arkaya yazıp "12 bits table" demek self-delimiting değildir. Decoder code boundaries/tree topology bilgisini de bilmelidir.

Header için seçenekler:

- Tree shape + leaf symbols
- Symbol-frequency table
- Symbol-code-length table
- Canonical Huffman Code lengths
- Önceden agreed static codebook

Gerçek total size:

```
header + encoded payload + format metadata/padding
```

olur.

12. Small message problemi Payload 45 bits olsa bile tree/codebook overhead'i küçük messages'ta savings'i aşabilir.

Compression'ın anlamlı olması için:

- Message yeterince büyük olmalı
- Frequency distribution skewed olmalı
- Header compact olmalı veya codebook paylaşılmalı

Bu nedenle yalnız payload bit count compression ratio değildir.

13. Canonical Huffman Coding Canonical form, yalnız code lengths ve deterministic symbol order ile codes'i yeniden üretir.

Avantajlar:

- Header daha compact olabilir.
- Decoder table kurulumu kolaylaşır.
- Same lengths için unique standardized assignment elde edilir.

Compression ratio code lengths aynı kaldığı için değişmez.

14. Equal frequencies Equal minimum frequencies için multiple valid tie choices vardır. Farklı Huffman trees ve bit patterns oluşabilir; optimal weighted length aynı kalabilir.

Encoder ile decoder aynı tree/codebook'u kullanmalıdır. Deterministic tie-breaking reproducibility sağlar.

15. Single-symbol edge case Alfabet yalnız bir symbol içeriyorsa tree tek leaf'tir. Empty code teorik olarak yeterli görünse de stream length/boundaries açısından pratik format genellikle symbol'a 0 gibi one-bit code verir ve count metadata taşır.

16. Huffman'ın sınırları

- Her symbol'a integer number of bits verir.
- Symbol probabilities bağımsız/sabit varsayımına yakın model kullanır.
- Context'i tek başına modellemez.
- Arithmetic/range coding bazı distributions'da entropy'ye daha fazla yaklaşabilir.
- Compression öncesi modeling kalitesi sonucu belirler.

Huffman yine de hızlı, prefix-free ve yaygın bir building block'tur.

17. Sık yapılan hatalar En büyük iki frequency'yi birleştirmek Greedy rule en küçük ikiyi seçer.

Internal nodes'a symbol koymak Symbols leaves'te olmalıdır; prefix-free property böyle korunur.

Equal frequency'de tek doğru tree beklemek Multiple optimal trees olabilir.

Yalnız payload size'ı total compressed size sanmak Codebook/header overhead'i de vardır.

Variable codes'i boundaries olmadan table'a yazmak Tree shape veya lengths gibi self-delimiting metadata gerekir.

ASCII'yi evrensel 8-bit character representation sanmak Modern text çoğunlukla Unicode encoding kullanır; cost model format'a bağlıdır.

18. Kavram sözlüğü

English term	Türkçe açıklama
Huffman Coding	Frequency tabanlı optimal binary prefix coding yöntemi
Frequency	Symbol'ın message'daki occurrence sayısı
Fixed-Length Code	Bütün symbols için aynı bit length kullanan code
Variable-Length Code	Symbols'a farklı bit lengths veren code
Prefix-Free Code	Hiçbir codeword'ün diğerinin prefix'i olmadığı code set
Codeword	Bir symbol'a atanmış bit sequence
Canonical Huffman Code	Code lengths'ten deterministic biçimde üretilen Huffman representation
Payload	Header hariç encoded message bits'i
Codebook	Symbols ile codes/lengths eşlemesi

19. Kendini sinama

1. Huffman'ın greedy rule'u nedir?
2. Prefix-free property neden önemlidir?
3. Örnekte payload neden 45 bits'tir?
4. Left/right labels değişirse optimal cost değişir mi?
5. Codebook neden gereklidir?
6. Huffman tree construction complexity nedir?
7. Equal frequencies neye yol açabilir?
8. Canonical Huffman neyi kolaylaştırır?

Kısa cevaplar

1. En düşük frequency'li iki current node'u birleştirmek.
2. Delimiter olmadan unique decoding sağladığı için.
3. $\sum \text{frequency} \times \text{codeLength} = 45$.
4. Hayır; bit patterns değişir, lengths/cost değişmez.
5. Decoder'ın bit paths'i symbols'a eşlemesi için.
6. k symbols için $O(k \log k)$.
7. Farklı fakat aynı-cost optimal trees'e.
8. Code lengths'ten deterministic codebook üretmeyi ve compact header'ı.

Sonraki ders: **44 – Prim's and Kruskal's Algorithms.**

44 – Prim's and Kruskal's Algorithms

Video: 3.6 Prim's and Kruskal's Algorithms (20:12)

Bu videonun ana fikri Minimum Spanning Tree (MST), connected undirected weighted graph'ın bütün vertices'ini minimum total edge weight ile bağlayan spanning tree'dir.

İki greedy algorithm:

- Prim's Algorithm: Tek connected tree'yi bir vertex'ten büyütür.
- Kruskal's Algorithm: Global olarak light edges seçip components'i birleştirir.

İkisi de Cut Property ile doğrulanır; çalışma biçimleri ve data structures farklıdır.

1. Spanning Tree tanımı Graph:

$$G=(V,E)$$

Bir spanning tree T:

- Bütün vertices'i içerir.
- Connected'dır.
- Cycle içermez.
- $|V|-1$ edges içerir.

Bu özelliklerden uygun herhangi ikisi/üçü diğerlerini tree bağlamında getirir. Yalnız "bütün vertices ve $V-1$ edges" demek connectedness olmadan yeterli değildir.

2. Minimum Spanning Tree Edge weight function:

$$w:E \rightarrow \text{real numbers}$$

MST objective:

$$\text{Minimize } \sum w(e), e \text{ in } T$$

MST shortest paths tree değildir. Total tree weight minimize edilir; belirli source'tan vertex distances minimize edilmez.

Negative edge weights MST algorithms için problem değildir; connected undirected graph'ta safe-edge properties çalışmaya devam eder.

3. Spanning tree sayısı düzeltmesi Videoda genel spanning-tree sayısı için:

$$C(E,V-1) = \text{numberOfCycles}$$

benzeri bir formül öneriliyor. Bu genel olarak doğru değildir:

- Bir $V-1$ edge subset birden fazla cycle/disconnected yapı içerebilir.
- Cycles birbirleriyle overlap edebilir.
- Basit subtraction double-counting/under-counting yapar.

Genel weighted/unweighted graph'ta spanning-tree sayısını hesaplamak için Kirchhoff's Matrix-Tree Theorem kullanılır: graph Laplacian'ın herhangi bir cofactor determinant'ı spanning-tree sayısını verir.

Özel olarak cycle graph C_m için spanning-tree sayısı m dir; çünkü herhangi bir tek edge silinebilir.

4. Cut Property Vertices'i iki nonempty gruba ayıran bir cut düşün. Cut'ı geçen minimum-weight edge, uygun tie koşullarıyla bir MST için safe edge'dir.

Daha genel ifade:

Current chosen forest ile uyumlu bir cut'ı geçen light edge, bazı MST'ye güvenle eklenebilir.

Prim ve Kruskal farklı cuts seçer fakat aynı property'ye dayanır.

5. Cycle Property Bir cycle'daki strictly heaviest edge hiçbir MST'de bulunmak zorunda değildir; unique heaviest ise hiçbir MST'de bulunmaz.

Kruskal cycle oluşturan edge'ı reddederken bu yapıyla uyumlu hareket eder.

Equal weights varsa edge choices ve MST uniqueness daha dikkatli değerlendirilir.

6. Prim's Algorithm – fikir Visited tree set S olsun. Her step'te:

S ile $V-S$ cut'ını geçen minimum-weight edge'i seç.

Bu edge yeni bir vertex'i tree'ye bağlar. $|V|-1$ edge seçildiğinde spanning tree tamamlanır.

7. Prim başlangıcı düzeltmesi Video önce graph'taki global minimum edge'ı seçer. Bu safe olabilir fakat Prim için gerekli değildir.

Standard Prim:

Arbitrary bir start vertex seç.

Sonra current tree'den dışarı çıkan minimum edge'ı seç. Start vertex MST total cost'unu değiştirmez; equal weights varsa bulunan MST'nin şekli değişebilir.

8. Prim pseudocode – Min-Priority Queue

```
Algorithm Prim(G, start)
  for each vertex v
    key[v] <- infinity
    parent[v] <- NIL

  key[start] <- 0
  Q <- MinPriorityQueue(all vertices by key)

  while Q not empty
    u <- ExtractMin(Q)

    for each edge (u,v,w) with v still in Q
      if w < key[v]
        key[v] <- w
        parent[v] <- u
        DecreaseKey(Q, v, w)

  return parent edges
```

$key[v]$, v'yi current tree'ye bağlayan bilinen cheapest edge weight'tir.

9. Prim invariant Her iteration'da:

- Extracted vertices bir connected tree oluşturur.
- Parent edges bazı MST'nin subset'i olarak seçilebilir.
- Queue'daki her vertex için key, current tree'den gelen cheapest known crossing edge'dir.

Extract-min edge current cut'ın light edge'ı olduğu için safe'dir.

10. Prim Complexity Adjacency matrix + linear min selection

$\Theta(V^2)$

Dense graphs için uygundur.

Adjacency list + binary heap

$O((V+E) \log V)$

Connected graph'ta $E \geq V-1$, sıklıkla:

$O(E \log V)$

olarak yazılır.

Fibonacci heap teorik bound

$O(E + V \log V)$

Practical choice implementation/constants'a bağlıdır.

11. Kruskal's Algorithm – fikir Başlangıçta her vertex ayrı component'tir. Edges nondecreasing weight order'da incelenir:

Edge farklı components'i bağlıyorsa seç.
Same component içindeyse cycle oluşturur; reddet.

$V-1$ selected edges olduğunda connected graph için MST tamamlanır.

12. Kruskal pseudocode

```
Algorithm Kruskal(G)
  A ← empty set

  for each vertex v
    MakeSet(v)

  sort edges by nondecreasing weight

  for each edge (u,v,w) in sorted edges
    if Find(u) != Find(v)
      add (u,v,w) to A
      Union(u,v)

    if size(A) == |V|-1
      break

  return A
```

Disjoint Set Union cycle detection ve component merging'i yönetir.

13. Kruskal invariant Chosen edges her zaman forest oluşturur. Her accepted edge:

- İki distinct components'i birleştirir.
- Cycle oluşturmaz.
- Bu components'i ayıran cut için current light eligible edge'dir.

Bu nedenle chosen forest bazı MST'ye extend edilebilir.

14. Kruskal Complexity düzeltmesi Videoda complexity $V \cdot E$, ardından heap ile yaklaşık $V \log V$ olarak veriliyor. Standard doğru analiz:

Edge sorting: $O(E \log E)$
DSU calls: $O(E \alpha(V))$ amortized
Total: $O(E \log E)$

Simple graph'ta $E \leq V^2$ olduğu için:

$$\log E = O(\log V)$$

ve bound $O(E \log V)$ olarak da yazılabilir.

Min-Heap kullanılsa bile cycle yapan edges de extract edilip kontrol edilebilir; yalnız $V-1$ extraction garanti değildir. Worst case E edges incelenir.

15. Prim ve Kruskal karşılaştırması

Özellik	Prim	Kruskal
Büyüttüğü yapı	Tek connected tree	Forest/components
Greedy choice	Current tree'den çıkan light edge	Global next light non-cycle edge
Ana structure	Min-Priority Queue	Sorted edges + DSU
Dense graph	Matrix ile $O(V^2)$ iyi olabilir	Edge sorting maliyetli olabilir
Sparse graph	Heap ile $O(E \log V)$	$O(E \log E)$ çok uygun
Disconnected input	Bir start'tan tek component	Minimum spanning forest üretir

16. Disconnected graph Disconnected graph'ın bütün vertices'ini içeren tek spanning tree'si yoktur; dolayısıyla MST de yoktur.

- Prim bir start vertex'ten yalnız o component'ın MST'sini bulur. Her unvisited component'ta yeniden başlatılırsa forest oluşur.
- Kruskal doğal olarak her component için minimum tree üretir.

Sonuç:

Minimum Spanning Forest

17. MST unique olmak zorunda mı? Hayır. Equal edge weights farklı MSTs üretebilir.

Örneğin bir cycle'da iki alternative edge aynı weight'teyse ikisinden biri seçilebilir ve total cost değişmeyebilir.

Minimum total weight tek bir sayı olabilir;
bu weight'i veren tree birden fazla olabilir.

Bütün edge weights distinct ise MST unique olması garantidir. Tersini zorunlu değildir: equal weights varken de graph'ın MST'si unique olabilir.

18. Prim ve Kruskal aynı tree'yi verir mi? Unique MST varsa ikisi aynı edge set'i bulur. Multiple MST varsa:

- Start vertex
- Equal-weight tie order
- Data structure ordering

nedeniyle farklı MSTs bulabilirler. Total cost yine minimumdur.

19. MST ile Shortest Path Tree farkı MST:

Minimize total selected edge weight

Shortest Path Tree:

Bir source'tan her vertex'e distance'i minimize et

Aynı graph'ta iki tree farklı olabilir. Prim/Kruskal, single-source shortest paths çözmez; Dijkstra MST çözmez.

20. Sık yapılan hatalar $V-1$ edges varsa otomatik tree sanmak Connectedness veya acyclicity de doğrulanmalıdır.

Prim'de graph'ın herhangi yerindeki minimum edge'i almak Edge current tree ile unvisited side arasındaki cut'ı geçmelidir.

Kruskal'da cycle kontrolünü görmezden gelmek DSU ile endpoints'in different components'ta olduğu kontrol edilir.

Kruskal'ı $O(V \log V)$ sanmak Sorting ve scan edges üzerindedir: $O(E \log E)$.

Disconnected graph için MST beklemek Yalnız minimum spanning forest vardır.

Minimum cost unique diye MST'yi unique sanmak Multiple trees aynı minimum total cost'u paylaşabilir.

MST'yi shortest path tree sanmak Objectives farklıdır.

21. Kavram sözlüğü

English term	Türkçe açıklama
Spanning Tree	Bütün vertices'i bağlayan acyclic $V-1$ edge subgraph
Minimum Spanning Tree	Minimum total edge weight'li spanning tree
Cut	Vertex set'i iki nonempty parçaya ayırma
Light Edge	Bir cut'ı geçen minimum-weight edge
Cut Property	Compatible cut'ın light edge'inin safe olduğunu söyleyen özellik
Minimum Spanning Forest	Disconnected graph components'larının minimum spanning trees'i
Disjoint Set Union	Components ve cycle detection için kullanılan data structure
Decrease-Key	Priority Queue içindeki key'i azaltma operation'ı
Matrix-Tree Theorem	Spanning-tree sayısını Laplacian determinantiyla hesaplayan theorem

22. Kendini sinama

1. Spanning tree kaç edge içerir?
2. Videodaki spanning-tree counting formülü neden genel değildir?
3. Prim'in greedy choice'ı nedir?
4. Prim mutlaka global minimum edge ile mi başlar?
5. Kruskal cycle'ı nasıl kontrol eder?
6. Kruskal'ın standard complexity'si nedir?
7. Disconnected graph'ta sonuç nedir?
8. Distinct weights neyi garanti eder?
9. MST ile Shortest Path Tree arasındaki fark nedir?

Kısa cevaplar

1. Connected graph'ta $V-1$.

2. Edge subsets cycles/disconnectedness bakımından basit subtraction ile sayılamaz; overlaps vardır.
3. Current tree cut'ını geçen light edge.
4. Hayır; arbitrary start vertex kullanılabilir.
5. DSU $\text{Find}(u) \neq \text{Find}(v)$ testiyle.
6. $O(E \log E)$; DSU kısmı $O(E \alpha(V))$ amortized.
7. MST değil, Minimum Spanning Forest.
8. MST'nin unique olmasını.
9. MST total tree weight'ı, SPT source-to-vertex distances'i minimize eder.

Sonraki ders: **45 – Dijkstra Algorithm: Single Source Shortest Path.**

45 – Dijkstra Algorithm: Single-Source Shortest Path

Video: 3.7 Dijkstra Algorithm: Single Source Shortest Path (18:35)

Bu videonun ana fikri Dijkstra's Algorithm, nonnegative edge weights bulunan weighted graph'ta bir source vertex'ten bütün reachable vertices'e shortest distances'i hesaplar.

Greedy choice:

Tentative distance'i en küçük unsettled vertex'i finalize et.

Ardından bu vertex'in outgoing edges'i relax edilir.

Kritik koşul:

Her edge weight ≥ 0

Negative edge varsa finalization mantığı geçerli değildir.

1. Single-Source Shortest Path Graph:

$G=(V,E)$

Source:

$s \in V$

Amaç her vertex v için:

$\delta(s,v) = s$ 'den v 'ye minimum path weight

hesaplamaktır.

v unreachable ise:

$\delta(s,v)=\text{infinity}$

2. Directed ve Undirected graphs Dijkstra:

- Directed graph'ta outgoing edges üzerinde çalışır.
- Undirected edge'i iki directed edge gibi ele alabilir.

Her iki durumda da weights nonnegative olmalıdır.

Bir undirected negative edge, iki yönde tekrar geçilebiliyorsa negative cycle oluşturur; standard shortest-walk distances well-defined olmayabilir.

3. Tentative distance $\text{dist}[v]$, şu ana kadar bulunan en iyi source-to- v path weight'idir.

Initialization:

```
dist[s]=0
dist[v]=infinity for v!=s
parent[v]=NIL
```

Başlangıçta yalnız empty path ile source distance kesin olarak bilinir.

4. Relaxation Edge:

(u,v,w)

Eğer u üzerinden gitmek daha kısa ise:

```
if dist[u]+w < dist[v]
    dist[v] <- dist[u]+w
    parent[v] <- u
```

Bu operation edge relaxationdır.

Relaxation bir distance'i artırmaz; yalnız daha iyi upper bound bulursa azaltır.

5. Küçük örnek

```
s -> u weight 2
u -> v weight 4
s'den v'ye direct edge yok
```

Başlangıç:

```
dist[s]=0
dist[u]=2
dist[v]=infinity
```

u extract edilince:

```
dist[u]+4 = 6 < infinity
```

Dolayısıyla:

```
dist[v]=6
parent[v]=u
```

6. Standard pseudocode

```
Algorithm Dijkstra( $G, s$ )
  for each vertex  $v$ 
    dist[v] <- infinity
    parent[v] <- NIL

  dist[s] <- 0
   $Q \leftarrow$  MinPriorityQueue containing  $(0, s)$ 

  while  $Q$  not empty
     $(du, u) \leftarrow$  ExtractMin( $Q$ )

    if  $du \neq \text{dist}[u]$ 
      continue // stale entry

    for each outgoing edge  $(u, v, w)$ 
      if  $\text{dist}[u] + w < \text{dist}[v]$ 
        dist[v] <- dist[u] + w
        parent[v] <- u
        Insert( $Q, (\text{dist}[v], v)$ )

  return dist, parent
```

Bu version explicit Decrease-Key yerine improved distance için yeni heap entry ekler ve stale entries'i atlar.

7. Finalization neden güvenli? Min queue'dan u , minimum tentative distance ile çıkarılsın. u ya daha kısa fakat henüz keşfedilmemiş bir path olduğunu varsayalım.

Bu path settled set'ten unsettled set'e ilk geçtiği edge'i içersin. Bütün edge weights nonnegative olduğundan, path'in o boundary vertex'e kadar olan prefix'i total path'ten daha büyük olamaz. O vertex'in tentative distance'i $\text{dist}[u]$ dan küçük olmalıydı ve önce seçilmeliydi; contradiction.

Bu proof negative weights olduğunda bozulur.

8. Settled ve unsettled ayrımı

- Settled/Finalized: Shortest distance'i artık kesin olan vertex.
- Unsettled: Distance'i daha sonra relaxation ile azalabilecek vertex.

Nonnegative condition sayesinde extracted minimum vertex yeniden açılmaz.

Implementation stale queue entries kullansa bile dist equality check doğru current entry'yi ayırt eder.

9. Shortest paths'i reconstruct etme Yalnız distances değil actual path de istenirse relaxation sırasında:

```
parent[v]=u
```

tutulur.

Target t için:

```
t -> parent[t] -> parent[parent[t]] -> ... -> s
```

chain'i ters çevrilir.

Unreachable vertex'in parent'ı NIL, distance'i infinity kalır.

10. Multiple shortest paths Equal distance'li farklı paths olabilir. Strict < comparison ilk bulunan parent'ı korur; <= tie handling başka tree üretebilir.

Shortest distance value aynı kalırken Shortest Path Tree unique olmak zorunda değildir.

Deterministic output için vertex id veya edge id tie-breaker kullanılabilir.

11. Complexity – adjacency matrix Her iteration minimum unsettled vertex linear scan ile bulunur ve row taranır:

```
 $\Theta(V^2)$ 
```

Video bu dense/matrix-style analizi verir. Dense graphs için uygun olabilir.

12. Complexity – adjacency list ve heap Binary heap ile:

```
Extract-Min:  $O(V \log V)$   
Relax/queue updates:  $O(E \log V)$   
Total:  $O((V+E) \log V)$ 
```

Connected/sufficiently nontrivial graph için sıkça:

```
 $O(E \log V)$ 
```

Fibonacci heap teorik bound:

```
 $O(E + V \log V)$ 
```

Graph representation complexity'nin ayrılmaz parçasıdır.

13. Negative-edge counterexample Şu directed edges olsun:

```
s -> a : 3
s -> b : 5
b -> c : 2
c -> a : -10
```

Dijkstra önce a'yı distance 3 ile finalize eder. Daha sonra:

```
s -> b -> c -> a
5 + 2 - 10 = -3
```

path'i bulunur. Gerçek distance -3 iken earlier finalization yanlıştır.

Negative edge her input'ta yanlış sonuç üretmek zorunda değildir; algorithm artık guarantee vermez.

14. Negative edges için ne kullanılır? Bellman-Ford Algorithm:

- Negative edges'i destekler.
- Source'tan reachable negative cycles'i detect edebilir.
- Standard complexity $O(VE)$ dir.

Directed Acyclic Graph için topological-order relaxation daha hızlı olabilir:

```
 $O(V+E)$ 
```

ve negative edges'e izin verir.

15. Negative cycle ne demektir? Source'tan reachable bir cycle'ın total weight'i negative ise cycle tekrar tekrar dolaşarak path cost sınırsız düşürülebilir:

```
 $\delta(s,v) = -\text{infinity}$ 
```

anlamında finite shortest path bulunmaz.

Dijkstra negative cycle detection algorithm'ı değildir.

16. Early stopping Yalnız belirli target t aranıyorsa t min queue'dan current distance ile çıkarıldığında durulabilir. O anda distance finalized'dır.

Target yalnız discovery/first relaxation anında bulundu diye durmak yanlıştır; daha kısa path daha sonra gelebilir.

17. Dijkstra ve BFS ilişkisi Bütün edge weights aynı positive constant ise Dijkstra'nın priority order'ı BFS levels'ıyla uyumludur. Unweighted graph shortest paths için:

```
BFS =  $O(V+E)$ 
```

daha basittir.

Weights yalnız 0 veya 1 ise 0-1 BFS deque kullanarak:

```
 $O(V+E)$ 
```

çalışabilir.

18. Dijkstra ve Prim farkı İkisi de min-priority queue kullanabilir fakat keys farklıdır:

```
Dijkstra key[v] = source'tan total tentative distance
Prim key[v]      = current tree'ye bağlayan tek cheapest edge
```

Dijkstra Shortest Path Tree, Prim Minimum Spanning Tree üretir.

19. Overflow ve infinity Finite large sentinel kullanılıyorsa:

```
if dist[u] != INF and dist[u]+w < dist[v]
```

kontrolü gerekir. $INF+w$ integer overflow yapmamalıdır.

Wider integer type veya checked arithmetic kullanılabilir.

20. Sık yapılan hatalar Negative edges üzerinde guarantee beklemek Algorithm'ın proof'u nonnegative weights'e dayanır.

En küçük edge'i seçmek Dijkstra minimum edge değil, minimum total tentative distance'li vertex'i seçer.

Relaxation'da yalnız edge weight'i karşılaştırmak Karşılaştırma $dist[u]+w(u,v)$ ile $dist[v]$ arasındadır.

Vertex discovery anında distance'i final sanmak Finalization extract-min anında olur.

Stale heap entries'i işlemeye devam etmek Duplicate-entry implementation'da current dist ile eşleşmeyen entries atlanmalıdır.

Complexity'yi her implementation için $\Theta(V^2)$ sanmak Adjacency list + heap sparse graphs'ta daha hızlıdır.

Unreachable vertex'e path üretmek Distance infinity ve parent NIL kalır.

21. Kavram sözlüğü

English term	Türkçe açıklama
Single-Source Shortest Path	Bir source'tan bütün vertices'e shortest paths problemi
Tentative Distance	Şu ana kadar bilinen en iyi distance upper bound'u
Relaxation	Bir edge üzerinden daha kısa path bulunursa distance'i güncelleme
Settled/Finalized Vertex	Shortest distance'i kesinleşmiş vertex
Shortest Path Tree	Parent links'in oluşturduğu source-rooted shortest-path tree
Negative Edge	Weight'i zero'dan küçük edge
Negative Cycle	Total weight'i negative cycle
Stale Entry	Heap'te kalan eski, artık current olmayan distance kaydı
Infinity	Henüz reachable path bilinmediğini gösteren sentinel

22. Kendini sına

1. Dijkstra'nın zorunlu weight condition'ı nedir?
2. Greedy choice nedir?
3. Relaxation formülü nedir?
4. Vertex distance'i ne zaman finalized olur?
5. Matrix ve binary-heap complexities nedir?
6. Negative edge proof'u neden bozar?
7. Actual path nasıl reconstruct edilir?
8. Prim ile temel key farkı nedir?

Kısa cevaplar

1. Bütün edge weights nonnegative olmalıdır.
2. Minimum tentative distance'li unsettled vertex'i seçmek.

3. if $\text{dist}[u] + w < \text{dist}[v]$, $\text{dist}[v] = \text{dist}[u] + w$.
4. Current minimum olarak queue'dan çıkarıldığında.
5. $\theta(V^2)$ ve $\theta((V+E) \log V)$.
6. Daha sonra negative edge üzerinden finalized vertex'e daha kısa path gelebilir.
7. Relaxation sırasında tutulan parent chain ters çevrilerek.
8. Dijkstra source'tan total path distance, Prim tree'ye bağlayan single edge weight kullanır.

Sonraki ders: **46 – Principle of Optimality: Introduction.**

46 – Principle of Optimality: Dynamic Programming Introduction

Video: 4.1 Principle of Optimality: Introduction (14:52)

Bu videonun ana fikri Dynamic Programming (DP), aynı subproblems tekrar ortaya çıktığında results'ı saklayıp yeniden kullanır ve bir state'in optimal value'sunu smaller states'in values'ından kurar.

İki temel structural property:

- Optimal Substructure
- Overlapping Subproblems

İki yaygın evaluation biçimi:

- Memoization: Top-down recursion + cache
- Tabulation: Bottom-up iterative table

Fibonacci optimization problem değildir; yalnız repeated-subproblem ve iki evaluation stilini göstermek için kullanılır.

1. Dynamic Programming nedir? DP'de problem:

1. Statelerle ifade edilir.
2. States arasındaki dependency bir Recurrence ile tanımlanır.
3. Her distinct state genellikle bir kez hesaplanır.
4. Result saklanarak tekrar computation önlenir.

DP yalnız optimization için kullanılmaz. Şunları da çözebilir:

- Counting problems
- Feasibility/decision problems
- Sequence matching
- Probability calculations
- Game states

Optimization, önemli fakat tek kullanım alanı değildir.

2. Bellman's Principle of Optimality Principle of Optimality öz olarak şunu söyler:

Optimal bir decision sequence'in ilk decision'ı ve resulting state ne olursa olsun, kalan decisions o resulting state için optimal bir policy oluşturmalıdır.

Başka deyişle optimal solution'ın relevant subsolutions'ı da optimal olmalıdır.

Bu, Optimal Substructure fikrinin decision-process biçimidir.

3. Optimal Substructure örneği sden tye shortest path, v üzerinden geçsin:

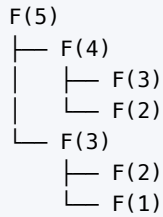
s -> ... -> v -> ... -> t

Bu path optimal ise sden vye kullanılan prefix de shortest olmalıdır. Daha kısa bir prefix olsaydı bütün path daha da kısaltılabilirdi; contradiction.

Bu yapı shortest-path DP recurrences'ını mümkün kılar.

4. Overlapping Subproblems Naive recursion aynı state'i tekrar tekrar çağırıyorsa subproblems overlap eder.

Fibonacci:



$F(3)$, $F(2)$, $F(1)$ birçok kez hesaplanır. DP her distinct $F(i)$ yi saklar.

5. Greedy ve DP farkı Greedy

- Her stage'de local best choice yapar.
- Choice çoğunlukla irrevocable'dır.
- Alternatives'ın future effects'ini full state table ile karşılaştırmaz.
- Correctness için greedy-choice property gerekir.

Dynamic Programming

- Bir state için relevant alternatives'ı recurrence içinde karşılaştırır.
- Subproblem values'ını reuse eder.
- Global optimum'u state transitions üzerinden kurar.
- State space daha fazla time/memory isteyebilir.

Videodaki "Greedy bir kez decision alır, DP her stage decision alır" ayrımı yanıltıcıdır; Greedy de her stage choice yapar. Asıl fark, Greedy'nin local choice'a commit etmesi; DP'nin alternatiflerin optimal continuation values'ını karşılaştırmasıdır.

6. DP bütün solutions'ı tek tek dener mi? Her zaman hayır. DP çoğu zaman bütün relevant states/transitions'ı değerlendirir fakat exponentially çok raw solution sequence'i açıkça enumerate etmez.

Örneğin 0/1 Knapsack'te 2^n subsets yerine yaklaşık:

$n * \text{capacity}$

states hesaplanabilir.

DP'nin kazancı, birçok solution'ın same state/subproblem'a ulaşmasını birleştirmesidir.

7. Fibonacci recurrence

```

F(0)=0
F(1)=1
F(n)=F(n-1)+F(n-2), n>=2

```

Naive recursive implementation:

```

Algorithm Fib(n)
  if n <= 1
    return n
  return Fib(n-1)+Fib(n-2)

```

8. Naive Fibonacci Complexity düzeltmesi

Exact recurrence:

$T(n)=T(n-1)+T(n-2)+\theta(1)$

Bu Fibonacci büyümesine uyar:

$T(n)=\theta(\phi^n)$
 $\phi=(1+\sqrt{5})/2 \approx 1.618$

$O(2^n)$

doğru fakat loose upper bound'dur. Videoda $2T(n-1)$ yaklaşımıyla yalnız bu upper bound çıkarılır; tight result $\Theta(\phi^n)$ dir.

Naive recursion stack depth:

$\Theta(n)$

9. Memoization – Top-Down Doğru terim Memoization'dır; memorization değil.

```
Algorithm FibMemo(n, memo)
  if n <= 1
    return n

  if memo[n] is known
    return memo[n]

  memo[n] <- FibMemo(n-1, memo) + FibMemo(n-2, memo)
  return memo[n]
```

Her distinct state $0..n$ en fazla bir kez fully hesaplanır.

Complexity:

Time: $\Theta(n)$
Memo: $\Theta(n)$
Stack: $\Theta(n)$

10. Memoization evaluation order Top-level needed state'ten başlanır:

$F(n)$

Yalnız recursive dependency'lerin gerektirdiği states hesaplanır. Bazı DP problems'de state space'in yalnız küçük kısmı reachable ise memoization avantajlıdır.

Cache lookup'ın state key hashing/indexing cost'u analysis'e dahil edilmelidir.

11. Tabulation – Bottom-Up Dependencies önce hesaplanacak order seçilir:

```
Algorithm FibTable(n)
  if n <= 1
    return n

  F[0] <- 0
  F[1] <- 1

  for i <- 2 to n
    F[i] <- F[i-1] + F[i-2]

  return F[n]
```

Complexity:

Time: $\Theta(n)$
Space: $\Theta(n)$
Stack: $\Theta(1)$

12. Space optimization Fibonacci recurrence yalnız önceki iki value'ya bağlıdır:

```

Algorithm FibConstantSpace(n)
  if n <= 1
    return n

  prev2 <- 0
  prev1 <- 1

  for i <- 2 to n
    current <- prev1+prev2
    prev2 <- prev1
    prev1 <- current

  return prev1

```

Time = $\theta(n)$
Space = $\theta(1)$

Space compression yalnız future transitions'ın hangi past states'e ihtiyaç duyduğu analiz edilerek yapılır.

13. Memoization ve Tabulation karşılaştırması

Özellik	Memoization	Tabulation
Yön	Top-down	Bottom-up
Kontrol	Recursion	Iteration
Hesaplanan states	Genellikle yalnız reachable/ne-eded	Genellikle chosen table region'ın tamamı
Stack	Recursion depth kadar	Genellikle constant
Evaluation order	Call dependencies belirler	Programmer belirler
Uygulama kolaylığı	Natural recurrence için kolay	Dependency order açık olmalı
Overhead	Calls + cache lookup	Loop/table access

Birinin her zaman daha iyi olduğu söylenemez.

14. DP problem çözme şablonu

1. State'i tanımla.
2. State'in ne anlama geldiğini tek cümleyle yaz.
3. Choices/transitions'ı çıkar.
4. Recurrence'ı kur.
5. Base cases'i belirle.
6. Evaluation order seç.
7. Final answer'ın hangi state'te olduğunu belirt.
8. Gerekirse choices/parents tutarak solution reconstruct et.
9. State sayısı * transition cost ile complexity hesapla.

15. State tanımı neden kritik? State, future decisions için gerekli bütün bilgiyi taşımalı; gereksiz history taşımamalıdır.

Eksik state:

- Farklı future possibilities'i yanlışlıkla birleştirir.
- Yanlış recurrence üretir.

Fazla state:

- Time ve space'i gereksiz büyütür.

DP'nin en yaratıcı kısmı çoğu zaman state design'dır.

16. Value ile solution reconstruction farkı DP table yalnız optimal value'yu tutabilir. Actual choices/path/items isteniyorsa:

- Parent pointer
- Chosen transition
- Table'ı geriye doğru inceleme

gerekir.

Minimum cost = 42 bilmek ile bu cost'u oluşturan solution'ı vermek aynı görev değildir.

17. Pseudo-polynomial time Bazı DP complexities numeric parameter'a bağlıdır:

$O(nW)$

Burada W capacity'nin numeric value'suysa input bit length $\log W$ dir. Bu yüzden $O(nW)$ input encoding size'a göre polynomial olmayabilir; pseudo-polynomial denir.

Bu ayırım ilerde 0/1 Knapsack'te önemlidir.

18. Sık yapılan hatalar DP'yi yalnız optimization sanmak Counting ve feasibility dahil birçok problem türünde kullanılır.

Optimal substructure tek başına yeterli sanmak Overlapping subproblems, DP reuse avantajını sağlar; bazı optimal-substructure problems Divide and Conquer ile daha doğal çözülebilir.

Memoization cache'ini her recursive call'da yeniden oluşturmak Cache calls arasında shared olmalıdır.

Unknown sentinel ile valid result'ı karıştırmak -1 valid value olabiliyorsa separate visited flag veya nullable entry gerekir.

Fibonacci time'ını tight olarak $\Theta(2^n)$ yazmak Tight bound $\Theta(\phi^n)$; $O(2^n)$ upper bound'dur.

Tabulation order'ını dependency'lere ters seçmek Bir state hesaplanırken ihtiyaç duyduğu states hazır olmalıdır.

Yalnız value'yu hesaplayıp requested solution'ı reconstruct edememek Choice/parent bilgisi planlanmalıdır.

19. Kavram sözlüğü

English term	Türkçe açıklama
Dynamic Programming	Overlapping states'ın results'ını saklayıp recurrence ile solution kurma strategy'si
Principle of Optimality	Optimal decision sequence'in remaining suffix'inin resulting state için optimal olması
Optimal Substructure	Optimal solution'ın optimal subsolutions içermesi
Overlapping Subproblems	Aynı state'in farklı computation paths'ten tekrar oluşması
State	Subproblem'ı future için yeterli bilgilerle tanımlayan representation
Transition	Bir state'i smaller/previous states üzerinden hesaplayan choice
Memoization	Top-down recursion results'ını cache'leme
Tabulation	States'i dependency order'da bottom-up table'a doldurma
Pseudo-Polynomial	Numeric value'da polynomial, bit-length'te olmaya-bilen complexity
Solution Reconstruction	DP values'dan actual choices/path'i geri çıkarma

20. Kendini sinama

1. DP'nin iki temel structural property'si nedir?
2. Principle of Optimality ne söyler?
3. Greedy ile DP arasındaki asıl decision farkı nedir?
4. Naive Fibonacci'nin tight time bound'u nedir?
5. Memoized Fibonacci'nin time ve space'i nedir?
6. Tabulation neden bottom-up denir?
7. Fibonacci space nasıl $\theta(1)$ e indirilir?
8. DP value ile actual solution neden farklıdır?

Kısa cevaplar

1. Optimal Substructure ve Overlapping Subproblems.
2. Optimal sequence'in remaining decisions'ının resulting state için optimal olması gerektiğini.
3. Greedy local choice'a commit eder; DP alternatives'ın continuation values'ını karşılaştırır.
4. $\theta(\phi^n)$; $\theta(2^n)$ loose upper bound.
5. $\theta(n)$ time, $\theta(n)$ memo ve $\theta(n)$ recursion stack.
6. Base/small states'ten başlayıp larger states'e ilerlediği için.
7. Yalnız son iki values'ı tutarak.
8. Optimal value yalnız objective'i; actual solution chosen transitions/parents'ı gerektirir.

Sonraki ders: **47 – MultiStage Graph.**

47 – MultiStage Graph

Video: 4.1 MultiStage Graph (21:07)

Bu videonun ana fikri MultiStage Graph, vertices'in ordered stages'a ayrıldığı weighted directed acyclic graph'tır. Edges ileri stages'a gider; videodaki özel biçimde yalnız next stage'e bağlanır.

Amaç source'tan sink'e minimum-cost path bulmaktır.

DP recurrence:

```
cost(sink)=0
cost(v)=min_{(v,u) in E} [w(v,u)+cost(u)]
```

next[v], minimum'u veren successor'ı saklar; path source'tan forward izlenerek reconstruct edilir.

1. Graph yapısı Stages:

```
Stage 1 -> Stage 2 -> ... -> Stage K
```

Genellikle:

- Stage 1'de tek source
- Stage K'da tek sink
- Edges yalnız ileri doğru

bulunur.

Transcriptte "vertices iki stage'e ayrılır" şeklinde duyulan ifade hatalıdır; multi-stage graph iki değil, birden fazla ordered stage içerebilir.

2. Neden DAG? Edges yalnız ileri stages'a gittiği için cycle oluşmaz. Bu topological dependency order sağlar:

```
Bir vertex'in cost'u yalnız daha sonraki stages'ın costs'una bağlıdır.
```

Bu nedenle sink'ten geriye doğru table doldurulabilir.

3. State tanımı

```
cost[v] = v'den sink'e minimum path cost
```

State yalnız current vertex'tir. Stage bilgisi vertex mapping'inden zaten biliniyorsa ayrıca state dimension olmak zorunda değildir.

Videodaki notation:

```
cost(stage, vertex)
```

aynı anlamı açıkça gösterir.

4. Base case

```
cost[sink]=0
```

Sink'ten sink'e empty path'in cost'u zero'dır.

Sink'e ulaşamayan vertex için:

```
cost[v]=infinity
```

kalmalıdır.

5. Transition Current vertex v den successor u ya gidildiğinde total cost:

$$w(v, u) + \text{cost}[u]$$

Bütün outgoing edges denenir:

$\text{cost}[v] = \text{minimum candidate}$
 $\text{next}[v] = \text{argmin successor}$

cost optimal value'yu, next actual decision'ı tutar.

6. Principle of Optimality Optimal source-to-sink path $v \rightarrow u$ edge'yle başlıyorsa, u dan sink'e kalan suffix de optimal olmalıdır.

Daha ucuz bir u -to-sink suffix olsaydı original path'in suffix'i değiştirilip daha düşük total cost elde edilirdi; contradiction.

Bu, recurrence'ın correctness temelidir.

7. Videodaki cost table Stage 5

Vertex	Cost	Next
12	0	–

Stage 4

Vertex	Cost	Next
9	4	12
10	2	12
11	5	12

Stage 3

Vertex	Cost	Next
6	7	10
7	5	10
8	7	10

Stage 2

Vertex	Cost	Next
2	7	7
3	9	6
4	18	8
5	15	8

Stage 1

Vertex	Cost	Next
1	16	2 veya 3

Equal candidates nedeniyle two optimal first decisions vardır.

8. Bir state hesabı

Vertex 6'nın outgoing edges'i 9 ve 10'a olsun:

```
cost[6]=min(
    w(6,9)+cost[9],
    w(6,10)+cost[10]
)
```

Videodaki values:

```
=min(6+4, 5+2)
=min(10,7)
=7
```

```
next[6]=10
```

cost[9] ve cost[10] farklı predecessors tarafından reuse edilir; overlapping dependency görünür.

9. Source state hesabı

```
cost[1]=min(
    9+cost[2],
    7+cost[3],
    3+cost[4],
    2+cost[5]
)
```

```
=min(9+7, 7+9, 3+18, 2+15)
=min(16,16,21,17)
=16
```

İlk iki candidates tie'dır:

```
next[1] = 2 veya 3
```

10. Path reconstruction

Bir tie choice 2 seçilirse:

```
1 -> 2 -> 7 -> 10 -> 12
```

Diğer optimal path:

```
1 -> 3 -> 6 -> 10 -> 12
```

İkisinin cost'u 16'dır.

Bu örnek, optimal value unique olsa bile multiple optimal paths olabileceğini yeniden gösterir.

11. Forward ve backward adlandırması Video costs'i sink'ten geriye doğru hesaplar fakat decisions source'tan sink'e izlendiği için yöntemi Forward Method olarak adlandırır.

Literatürde adlandırma değişebilir:

- Cost-to-go: sink'ten backward table fill
- Path reconstruction: source'tan forward

Önemli olan dependency order ve state anlamıdır.

12. Generic bottom-up algorithm


```

Algorithm MultiStageShortestPath(G, source, sink, reverseTopologicalOrder)
  for each vertex v
    cost[v] <- infinity
    next[v] <- NIL

  cost[sink] <- 0

  for v in reverseTopologicalOrder excluding sink
    for each outgoing edge (v,u,w)
      if cost[u] != infinity and w+cost[u] < cost[v]
        cost[v] <- w+cost[u]
        next[v] <- u

  return cost[source], next

```

13. Complexity

Adjacency list ile her vertex ve edge bir kez işlenir:

```

Time =  $\theta(V+E)$ 
Space =  $\theta(V)$ 

```

Path output ek olarak path length kadar sürer.

Adjacency matrix ile bütün possible successor pairs taranır:

```

Time =  $\theta(V^2)$ 

```

14. Negative edges

Graph acyclic olduğu için negative edge weights kullanılabilir; negative cycle mümkün değildir.

DP topological order'da bütün relevant successors'ın exact costs'unu bildiğinden Dijkstra'nın nonnegative restriction'ına ihtiyaç duymaz.

15. Edges yalnız next stage'e gitmek zorunda mı?

Videodaki model next-stage edges kullanır. Daha genel DAG'de edge birkaç stage atlayabilir. Recurrence değişmez; yalnız topological order korunmalıdır.

16. Sık yapılan hatalar Costs'i source'tan rastgele order'da doldurmak

Bir vertex hesaplanırken successor costs hazır olmalıdır.

next table tutmadan yalnız minimum cost bulmak Value bulunur fakat actual path reconstruct edilemez.

Greedy olarak her vertex'te minimum edge'i seçmek $w(v,u)$ tek başına değil, $w(v,u)+cost[u]$ minimize edilmelidir.

Tie durumunda tek optimal path varsaymak Multiple successors aynı minimum'u verebilir.

Unreachable successor'da infinity ile toplama yapmak Overflow/invalid arithmetic önlenmelidir.

17. Kavram sözlüğü

English term	Türkçe açıklama
MultiStage Graph	Vertices'i ordered stages'a ayrılmış directed acyclic graph
Source	Path'in başladığı vertex
Sink	Path'in bittiği vertex
Cost-to-Go	Current vertex'ten sink'e minimum remaining cost
Successor	Outgoing edge ile gidilen next vertex
Argmin	Minimum value'yu üreten choice/index
Reverse Topological Order	Successors önce işlenecek ters dependency sırası
Path Reconstruction	Stored next decisions ile actual path'i çıkarma

18. Kendini sına

1. $cost[v]$ state'i neyi ifade eder?
2. Base case nedir?
3. Transition formula nedir?
4. Table neden sink'ten geriye doldurulur?
5. Videodaki minimum source-to-sink cost nedir?
6. İki optimal path hangileridir?
7. Adjacency-list complexity nedir?
8. Negative edges neden kullanılabilir?

Kısa cevaplar

1. vden sink'e minimum path cost'u.
2. $cost[sink]=0$.
3. $\min(w(v,u)+cost[u])$.
4. Current state successor states'e bağlı olduğu için.
5. 16.
6. 1-2-7-10-12 ve 1-3-6-10-12.
7. $\theta(V+E)$ time, $\theta(V)$ DP storage.
8. Graph DAG olduğu ve negative cycle bulunmadığı için.

Sonraki ders: **48 – MultiStage Graph Program.**

48 – MultiStage Graph Program

Video: 4.1.1 MultiStage Graph Program (14:26)

Kaynak notu: Bu videoda YouTube English captions bulunmuyor. Resmî açıklama kapsamı "Multistage graph Program and explanation" olarak verir. Aşağıdaki ders, önceki videonun recurrence'ı ve bu program kapsamı kullanılarak; implementation boundaries ve complexity ayrıca doğrulanarak hazırlanmıştır.

Bu videonun ana fikri Önceki recurrence programda iki arrays ile gerçekleştirilir:

```
cost[v] = v'den sink'e minimum cost
next[v] = minimum'u veren successor
```

Vertices reverse topological/stage order'da işlenir. Her outgoing edge için candidate hesaplanır:

```
candidate = edgeWeight + cost[successor]
```

Minimum candidate hem cost hem next tables'ını günceller.

1. Program input contract Programın güvenli çalışması için:

- Graph directed ve acyclic olmalıdır.
- Source ve sink belirtilmelidir.
- Vertex ordering topological olmalı veya ayrıca hesaplanmalıdır.
- Edge weights finite numeric type'a sığmalıdır.
- Unreachable vertices desteklenmelidir.

Yalnız vertex labels'ın küçükten büyüğe olması topological order garantisi değildir; input contract açık olmalıdır.

2. Data structures Adjacency list:

```
adj[v] = v'den çıkan (to, weight) edges
```

DP arrays:

```
cost[V]
next[V]
```

Ordering:

```
topologicalOrder[V]
```

3. Pseudocode

```
Algorithm MultiStageDP(adj, source, sink, topologicalOrder)
  for each vertex v
    cost[v] <- infinity
    next[v] <- NIL

  cost[sink] <- 0

  for v in reverse(topologicalOrder)
    if v == sink
      continue

    for each edge (v,u,w) in adj[v]
```

```

        if cost[u] == infinity
            continue

        candidate <- w+cost[u]

        if candidate < cost[v]
            cost[v] <- candidate
            next[v] <- u

    return cost, next

```

4. C++ implementation

```

#include <algorithm>
#include <cstdint>
#include <iostream>
#include <limits>
#include <stdexcept>
#include <utility>
#include <vector>

struct Edge {
    int to;
    std::int64_t weight;
};

struct Result {
    std::vector<std::int64_t> cost;
    std::vector<int> next;
};

Result multistageShortestPath(
    const std::vector<std::vector<Edge>>& graph,
    int sink,
    const std::vector<int>& topologicalOrder
) {
    const auto vertexCount = static_cast<int>(graph.size());
    const auto INF = std::numeric_limits<std::int64_t>::max();

    if (sink < 0 || sink >= vertexCount) {
        throw std::out_of_range("invalid sink");
    }

    std::vector<std::int64_t> cost(vertexCount, INF);
    std::vector<int> next(vertexCount, -1);
    cost[sink] = 0;

    for (auto it = topologicalOrder.rbegin();
         it != topologicalOrder.rend(); ++it) {
        const int v = *it;

        if (v == sink) {
            continue;
        }

        for (const Edge& edge : graph[v]) {
            if (cost[edge.to] == INF) {
                continue;
            }

```

```

        if (edge.weight > 0 &&
            cost[edge.to] > INF - edge.weight) {
            throw std::overflow_error("path cost overflow");
        }

        const auto candidate = edge.weight + cost[edge.to];

        if (candidate < cost[v]) {
            cost[v] = candidate;
            next[v] = edge.to;
        }
    }
}

return {std::move(cost), std::move(next)};
}

```

Negative weights için lower-bound overflow da production code'da checked arithmetic ile ele alınmalıdır; örnek positive-overflow kontrolünü gösterir.

5. Path reconstruction

```

std::vector<int> reconstructPath(
    int source,
    int sink,
    const std::vector<int>& next
) {
    std::vector<int> path;

    for (int v = source; v != -1; v = next[v]) {
        path.push_back(v);

        if (v == sink) {
            return path;
        }

        if (path.size() > next.size()) {
            throw std::logic_error("next pointers contain a cycle");
        }
    }

    return {}; // sink unreachable
}

```

Empty result, source'tan sink'e path olmadığını gösterir.

6. Stage-based ordering Input vertices stage groups olarak verilmişse explicit topological sort gerekmez:

```

for stage <- K-1 downto 1
    for each vertex v in stage
        evaluate outgoing edges

```

Videodaki table tam olarak bu order'ı kullanır.

7. Topological sort gerekiyorsa General DAG'de stage labels yoksa:

```

TopologicalSort(G)

```

önce $\Theta(V+E)$ zamanda hesaplanabilir.

Cycle bulunursa input MultiStage/DAG contract'ını ihlal eder; program error raporlamalıdır.

8. Tie handling

Standard update:

```
candidate < cost[v]
```

ilk minimum successor'ı korur.

All optimal paths istenirse:

```
next[v] = list of all successors attaining minimum
```

tutulabilir. Ancak optimal paths sayısı exponential olabilir; hepsini output etmek doğal olarak pahalıdır.

Deterministic single path için equal candidate'da smallest vertex id gibi tie-breaker eklenebilir.

9. Table trace

Sink:

```
cost[12]=0
```

Stage 4:

```
cost[9] =4, next[9] =12  
cost[10]=2, next[10]=12  
cost[11]=5, next[11]=12
```

Sonra stages 3, 2 ve 1 doldurulur. Final:

```
cost[1]=16
```

Reconstruction:

```
1 -> next[1] -> ... -> 12
```

10. Correctness invariant

Reverse-order loop'ta vertex v işlenirken:

vnin bütün outgoing successors'ının cost values'ı exact minimum sink costs'dur.

Bu invariant sayesinde bütün first-edge choices exact continuation cost ile karşılaştırılır. Minimum candidate cost[v]yi doğru verir.

11. Complexity

Topological sort dahil:

```
Time =  $\theta(V+E)$ 
```

Çünkü her vertex ve edge constant sayıda işlenir.

Storage:

```
Graph adjacency list =  $\theta(V+E)$   
DP arrays =  $\theta(V)$ 
```

Graph input storage hariç auxiliary DP space $\theta(V)$ dir.

12. Adjacency matrix variant

Videoya yakın matrix representation:

```
for v from V-1 downto 0  
  for u from 0 to V-1
```

```
if W[v][u] exists
    relax candidate
```

Time:

$\theta(V^2)$

Sparse graph'ta adjacency list daha verimlidir.

13. Source=sink edge case

```
source == sink
```

ise minimum cost 0, path yalnız source vertex'tir.

Program bunu doğal olarak desteklemelidir.

14. Unreachable sink

Source'un bütün outgoing routes'i sink'e ulaşamıyorsa:

```
cost[source]=INF
next[source]=-1
```

INF'yi normal numeric result gibi print etmek yerine "unreachable" gösterilmelidir.

15. Sık yapılan hatalar

Vertex labels'ı topological order sanmak Ordering input contract ile garanti edilmiyorsa ayrıca sort gerekir.

cost array'i zero ile initialize etmek Unknown/unreachable states INF; yalnız sink zero olmalıdır.

Sink'ten başlamadan source'u hesaplamak Successor costs henüz hazır olmaz.

next yerine edge weight saklamak Path reconstruction için successor id gerekir.

INF ile arithmetic yapmak Overflow veya sahte finite candidate üretebilir.

General cyclic graph'a aynı loop'u uygulamak Dependency order yoktur; problem farklı shortest-path algorithm gerektirir.

16. Kavram sözlüğü

English term	Türkçe açıklama
Adjacency List	Her vertex'in outgoing edges'ini saklayan graph representation
Topological Order	Her directed edge'in earlier vertex'ten later vertex'e gittiği ordering
Reverse Iterator	Sequence'i sondan başa dolaşan iterator
Infinity Sentinel	Unreachable/unknown cost'u temsil eden özel value
Checked Arithmetic	Overflow/underflow'u algılayan arithmetic
Tie-Breaker	Equal objective choices arasında deterministic seçim kuralı
Next Pointer	Optimal path'te gidilecek successor'ı saklayan value

17. Kendini sına

1. cost ve next arrays ne tutar?

2. Vertices hangi order'da işlenir?
3. Unknown costs neden zero değildir?
4. Adjacency-list time complexity nedir?
5. Multiple optimal paths nasıl saklanır?
6. Graph cyclic çıkarsa ne yapılmalıdır?
7. $INF + weight$ neden doğrudan hesaplanmamalıdır?

Kısa cevaplar

1. Minimum sink cost'u ve minimum'u veren successor'ı.
2. Reverse topological/stage order.
3. Zero valid cost'tur; unknown/unreachable için INF gerekir.
4. $\theta(V+E)$.
5. Her vertex için bütün minimum-attaining successors listelenerek.
6. Input reddedilmeli veya uygun başka shortest-path yöntemi seçilmelidir.
7. Overflow ve yanlış finite value üretebileceği için.

Sonraki ders: **49 – All-Pairs Shortest Path: Floyd-Warshall.**

49 – All-Pairs Shortest Path: Floyd-Warshall

Video: 4.2 All-Pairs Shortest Path: Floyd-Warshall (14:13)

Bu videonun ana fikri Floyd-Warshall Algorithm, weighted graph'ta her ordered vertex pair'i arasındaki shortest-path distance'ı bulur. Bu nedenle problem All-Pairs Shortest Path olarak adlandırılır.

Dynamic Programming state'i şudur:

$D^k(i, j)$ = yalnız $\{1, 2, \dots, k\}$ vertices'ini intermediate olarak kullanmaya izin verildiğinde i 'den j 'ye minimum cost

Her k için optimal path iki durumdan birindedir:

1. k vertex'ini kullanmaz
2. k vertex'inden geçer: $i \rightarrow k \rightarrow j$

Bu ayrım recurrence'ı verir:

$$D^k(i, j) = \min(\begin{aligned} &D^{k-1}(i, j), \\ &D^{k-1}(i, k) + D^{k-1}(k, j) \end{aligned})$$

1. All-Pairs ne demektir? Single-Source Shortest Path, seçilen bir source'tan bütün vertices'e olan distances'ı bulur. All-Pairs Shortest Path ise her vertex'i sırayla source gibi ele alır ve bütün ordered pairs için sonuç üretir:

$dist[i][j]$ = i 'den j 'ye shortest-path cost

Directed graph'ta genellikle:

$dist[i][j] \neq dist[j][i]$

olabilir.

2. Initial distance matrix Algorithm başlangıçta yalnız direct edges'i ve empty paths'i bilir:

$$D^0(i, j) = \begin{cases} 0 & \text{if } i == j \text{ and no negative self-loop exists} \\ \text{weight}(i, j) & \text{if direct edge } i \rightarrow j \text{ exists} \\ \text{INF} & \text{otherwise} \end{cases}$$

Parallel edges varsa initial value minimum edge weight olmalıdır.

Daha genel ve eksiksiz initialization, diagonal için $\min(0, \text{weight}(i, i))$ kullanır. Negative self-loop doğrudan bir negative cycle'dır; algorithm sonunda negative diagonal testinde görünür.

Videodaki graph'ın initial matrix'i:

	1	2	3	4
1	0	3	INF	7
2	8	0	2	INF
3	5	INF	0	1
4	2	INF	INF	0

Bu matrix henüz multi-edge paths'i içermez.

3. State'teki k neyi sınırlar? k , path'in toplam vertex sayısını veya en fazla kaç edge kullanılacağını söylemez. Yalnız izin verilen **intermediate vertices set**'ini belirtir.

Endpoints i ve j , bu set restriction'dan ayrı düşünülür:

$i \rightarrow \dots \rightarrow j$

path'inin iç kısmında yalnız $1..k$ kullanılabilir.

4. Recurrence neden doğrudur? $1..k$ vertices'ine izin verilen optimal $i \rightarrow j$ path'ini ele alalım.

Case 1: Path k yi kullanmıyor O zaman izin verilen intermediate set aslında $1..k-1$ ile sınırlıdır:

$cost = D^{(k-1)}[i][j]$

Case 2: Path k den geçiyor Negative cycle bulunmadığını varsayarsak shortest path simple seçilebilir ve k üzerinden iki parçaya ayrılabilir:

$i \rightarrow \dots \rightarrow k + k \rightarrow \dots \rightarrow j$

Bu iki parçanın internal vertices'i yalnız $1..k-1$ den gelir:

$cost = D^{(k-1)}[i][k] + D^{(k-1)}[k][j]$

İki case'in minimum'u optimal answer'dır.

5. Videodaki matrix'in final sonucu Bütün vertices intermediate olarak sırayla eklendikten sonra:

	1	2	3	4
1	0	3	5	6
2	5	0	2	3
3	3	6	0	1
4	2	5	7	0

Örnekler:

$dist[1][3] = 5$ via $1 \rightarrow 2 \rightarrow 3$
 $dist[1][4] = 6$ via $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$
 $dist[2][1] = 5$ via $2 \rightarrow 3 \rightarrow 4 \rightarrow 1$
 $dist[4][3] = 7$ via $4 \rightarrow 1 \rightarrow 2 \rightarrow 3$

6. In-place implementation Teorik anlatım ayrı $D^{(k)}$ matrices'i gösterse de yalnız bir $dist$ matrix yeterlidir:

```
for k = 1 to V
  for i = 1 to V
    for j = 1 to V
      if dist[i][k] != INF and dist[k][j] != INF
        dist[i][j] = min(
          dist[i][j],
          dist[i][k] + dist[k][j]
        )
```

Bu in-place update geçerlidir; k iteration'ında kullanılan row/column values recurrence'ın istediği bilgiyle uyumludur.

7. Loop order neden önemlidir? In-place standard algorithm'da k outermost loop olmalıdır:

$k \rightarrow i \rightarrow j$

Çünkü her completed outer iteration şu invariant'ı temsil eder:

```
dist[i][j], intermediate olarak yalnız 1..k kullanıldığındaki optimum'dur.
```

i veya j outermost yapılırsa bu DP invariant bozulabilir ve bazı paths zamanından önce güncellenmiş, bazıları güncellenmemiş values'a dayanır.

8. Path reconstruction

Yalnız distance değil actual path de isteniyorsa next matrix tutulabilir.

Initialization:

```
if edge i -> j exists
    next[i][j] = j
else
    next[i][j] = NIL
```

Improvement bulunduğunda:

```
if dist[i][k] + dist[k][j] < dist[i][j]
    dist[i][j] = dist[i][k] + dist[k][j]
    next[i][j] = next[i][k]
```

Reconstruction:

```
path = [i]
while i != j
    i = next[i][j]
    path.append(i)
```

`next[i][j] == NIL` ise path yoktur.

9. Negative edges ve negative cycles

Floyd-Warshall negative edge weights ile çalışabilir. Fakat reachable bir Negative Cycle varsa onun üzerinden tekrar tekrar dönülerek cost sınırsız küçültülebilir; finite shortest path tanımlı değildir.

Algorithm bittikten sonra:

```
dist[v][v] < 0
```

olan bir v varsa negative cycle vardır.

Daha hassas yorum:

```
i, v'ye ulaşabiliyor ve v, j'ye ulaşabiliyorsa
```

o zaman $i \rightarrow j$ pair'i için cost $-\infty$ yönünde düşürülebilir. Negative cycle'ın varlığı bütün pairs'i otomatik olarak etkilemez; yalnız cycle'a girip çıkabilen pairs etkilenir.

Undirected graph'ta negative edge, iki yönde geçilebildiği için çoğu standart shortest-walk modelinde hemen negative cycle oluşturur.

10. INF ve overflow güvenliği

Şu işlem doğrudan yapılmamalıdır:

```
INF + finiteWeight
```

Çünkü fixed-width integer'da overflow veya sahte küçük value oluşabilir. Önce iki segmentin de reachable olduğu kontrol edilmelidir:

```
if dist[i][k] != INF and dist[k][j] != INF
    candidate = checked_add(dist[i][k], dist[k][j])
```

Production code'da positive overflow ve negative underflow birlikte ele alınmalıdır.

11. Complexity Üç nested loops vardır:

Time = $\Theta(V^3)$
Space = $\Theta(V^2)$

next matrix tutulursa space yine asymptotically $\Theta(V^2)$ dir.

Her k için ayrı matrix saklamak gereksizdir; saklanırsa space $\Theta(V^3)$ e çıkar.

12. Floyd-Warshall mı, repeated Dijkstra mı? Seçim graph yapısına bağlıdır:

Durum	Uygun yaklaşım
Dense graph, bütün pairs gerekli	Floyd-Warshall basit ve öngörülebilir
Negative edges, negative cycle yok	Floyd-Warshall çalışabilir
Sparse graph, nonnegative edges	Her source için Dijkstra daha hızlı olabilir
Yalnız bir source gerekli	Single-Source algorithm tercih edilir

Repeated Dijkstra'nın complexity'si implementation ve graph representation'a bağlıdır; her durumda otomatik olarak $\Theta(V^3)$ değildir.

13. Sık yapılan hatalar kyi innermost loop yapmak Standard in-place DP dependency order bozulur.

Missing edge'i zero yazmak Zero gerçek bir edge weight olabilir. Missing edge INF olmalıdır.

Negative edge görünce algorithm'ı reddetmek Negative edges desteklenir; asıl sorun negative cycle'dir.

Negative cycle'ı kontrol etmemek $\text{dist}[v][v] < 0$ test edilmelidir.

Distance bulunca path'in de hazır olduğunu sanmak Actual route için next veya predecessor information ayrıca saklanır.

INF ile arithmetic yapmak Reachability guard ve checked arithmetic gerekir.

14. Kavram sözlüğü

English term	Türkçe açıklama
All-Pairs Shortest Path	Her ordered vertex pair için shortest distance bulma problemi
Intermediate Vertex	Path'in endpoints dışında kalan iç vertex'i
Distance Matrix	Bütün pairs'in costs'unu tutan square matrix
In-place Update	Yeni table yerine aynı memory üzerinde update yapma
Negative Edge	Weight'i zero'dan küçük edge
Negative Cycle	Total weight'i negative olan directed cycle
Path Reconstruction	DP decisions'dan actual vertex sequence'i çıkarma
Infinity Sentinel	Ulaşılamaz pair'i temsil eden özel value

15. Kendini sına

1. $D^k(i, j)$ tam olarak neyi temsil eder?
2. Recurrence'taki iki case nedir?
3. In-place algorithm'da hangi loop outermost olmalıdır?
4. Missing edge initial matrix'te nasıl gösterilir?
5. Negative edge desteklenir mi?

6. Negative cycle nasıl tespit edilir?
7. Actual path için hangi ek structure tutulabilir?
8. Time ve space complexity nedir?

Kısa cevaplar

1. Yalnız 1..k intermediate vertices'ine izin verildiğindeki $i \rightarrow j$ minimum cost'u.
2. kyi kullanmamak veya $i \rightarrow k \rightarrow j$ biçiminde kullanmak.
3. k.
4. INF ile; diagonal normalde zero'dır.
5. Evet, negative cycle yoksa.
6. Final matrix'te herhangi bir $dist[v][v] < 0$ aranır.
7. next matrix.
8. $\theta(V^3)$ time ve $\theta(V^2)$ space.

Sonraki ders: **50 – Matrix Chain Multiplication.**

50 – Matrix Chain Multiplication

Video: 4.3 Matrix Chain Multiplication (23:00)

Bu videonun ana fikri Matrix Chain Multiplication (MCM), compatible matrices chain'ini hangi parenthesization ile çarparsak en az scalar multiplications gerekir sorusunu çözer.

Matrix multiplication associative'dir:

$$(A_1 A_2) A_3 = A_1 (A_2 A_3)$$

Fakat iki parenthesization'ın arithmetic cost'u aynı olmak zorunda değildir. DP, matrix values'ı çarpmaz; yalnız en ucuz **evaluation order**'ı bulur.

1. İki matrix'i çarpmanın cost'u Boyutlar:

A: $p \times q$
B: $q \times r$

ise:

A x B result dimension = $p \times r$
scalar multiplications = $p \times q \times r$

Inner dimensions eşit değilse multiplication compatible değildir.

2. Parenthesization neden cost'u değiştirir? Üç matrix düşünelim:

A1: $p_0 \times p_1$
A2: $p_1 \times p_2$
A3: $p_2 \times p_3$

İki seçenek vardır.

(A1 A2) A3

$$\text{cost} = p_0 \times p_1 \times p_2 + p_0 \times p_2 \times p_3$$

A1 (A2 A3)

$$\text{cost} = p_1 \times p_2 \times p_3 + p_0 \times p_1 \times p_3$$

Intermediate matrix dimensions farklı olduğu için total costs de farklı olabilir.

3. Input'u dimension vector ile gösterme n matrices için n+1 dimensions yeterlidir:

$p = [p_0, p_1, p_2, \dots, p_n]$
A_i has dimension $p[i-1] \times p[i]$

Videodaki örnek:

$$p = [5, 4, 6, 2, 7]$$

Dolayısıyla:

A1: 5×4
A2: 4×6
A3: 6×2
A4: 2×7

4. Kaç farklı parenthesization vardır? n matrices'in full parenthesization sayısı:

Catalan($n-1$)

İlk değerler:

```
n=1 -> 1
n=2 -> 1
n=3 -> 2
n=4 -> 5
n=5 -> 14
```

Brute-force bütün parenthesizations'ı denerse search space exponential büyür. Dynamic Programming aynı subchains'i tekrar çözmeyi önler.

5. DP state

$m[i][j]$ = $A_i A_{i+1} \dots A_j$ subchain'ini çarpmanın minimum scalar cost'u

Tek matrix için multiplication gerekmez:

$m[i][i] = 0$

6. Split point k $A_i \dots A_j$ chain'inin son multiplication'ı chain'i bir yerde ikiye ayırır:

$(A_i \dots A_k) (A_{k+1} \dots A_j)$

Possible split points:

$i \leq k < j$

Sol result dimension:

$p[i-1] \times p[k]$

Sağ result dimension:

$p[k] \times p[j]$

Bu iki result'u çarpmanın ek cost'u:

$p[i-1] * p[k] * p[j]$

7. DP recurrence

```
m[i][i] = 0

m[i][j] = min over i <= k < j of (
    m[i][k]
    + m[k+1][j]
    + p[i-1]*p[k]*p[j]
)
```

Minimum'u veren k , ayrı $s[i][j]$ table'ında saklanır:

$s[i][j] = \operatorname{argmin}_k$

m minimum cost'u, s optimal structure'ı verir.

8. Videodaki örneğin length-2 subchains'i Dimension vector:

[5,4,6,2,7]

İkişerli chains:

$m[1][2] = 5*4*6 = 120$
 $m[2][3] = 4*6*2 = 48$
 $m[3][4] = 6*2*7 = 84$

Split tek mümkün location'dır:

$s[1][2]=1$
 $s[2][3]=2$
 $s[3][4]=3$

9. Length-3 subchains $m[1][3]$ k=1:

$m[1][1] + m[2][3] + 5*4*2$
 $= 0 + 48 + 40$
 $= 88$

k=2:

$m[1][2] + m[3][3] + 5*6*2$
 $= 120 + 0 + 60$
 $= 180$

Sonuç:

$m[1][3]=88$
 $s[1][3]=1$

Yani A1(A2A3) daha ucuzdur.

$m[2][4]$ k=2:

$0 + 84 + 4*6*7 = 252$

k=3:

$48 + 0 + 4*2*7 = 104$

Sonuç:

$m[2][4]=104$
 $s[2][4]=3$

Yani (A2A3)A4 daha ucuzdur.

10. Full chain $m[1][4]$ Üç split denenir.

k=1

$m[1][1] + m[2][4] + 5*4*7$
 $= 0 + 104 + 140$
 $= 244$

k=2

```
m[1][2] + m[3][4] + 5*6*7
= 120 + 84 + 210
= 414
```

k=3

```
m[1][3] + m[4][4] + 5*2*7
= 88 + 0 + 70
= 158
```

Minimum:

```
m[1][4]=158
s[1][4]=3
```

11. Optimal parenthesization'ı çıkarma $s[1][4]=3$:

```
(A1 A2 A3) | A4
```

Sol taraf için $s[1][3]=1$:

```
A1 | (A2 A3)
```

Tam sonuç:

```
((A1(A2A3))A4)
```

Evaluation order:

```
1. A2*A3
2. A1*(A2*A3)
3. [önceki result]*A4
```

Total scalar multiplications 158dir.

12. Table fill order $m[i][j]$, daha kısa subchains'e bağlıdır. Bu nedenle main diagonal'dan başlayıp chain length artırılır:

```
length 1: m[i][i] = 0
length 2: m[1][2], m[2][3], ...
length 3: m[1][3], m[2][4], ...
...
length n: m[1][n]
```

Row-major bütün cells'i rastgele doldurmak dependency order'ı garanti etmez.

13. Bottom-up pseudocode

```
Algorithm MatrixChainOrder(p[0..n])
  for i = 1 to n
    m[i][i] <- 0

  for length = 2 to n
    for i = 1 to n-length+1
      j <- i+length-1
      m[i][j] <- INF
```

```

    for k = i to j-1
        q <- m[i][k]
            + m[k+1][j]
            + p[i-1]*p[k]*p[j]

        if q < m[i][j]
            m[i][j] <- q
            s[i][j] <- k

    return m, s

```

14. Complexity States:

$\theta(n^2)$

Her state için en fazla $\theta(n)$ splits denenir:

Time = $\theta(n^3)$
Space = $\theta(n^2)$

Parenthesization output'u $\theta(n)$ symbols/nodes içerir.

Brute force ise Catalan sayısı nedeniyle exponential'dır.

15. Algorithm neyi optimize eder, neyi yapmaz? MCM:

- En az scalar multiplications gerektiren parenthesization'ı bulur.
- Matrices'in actual entries'ini multiply etmez.
- Memory traffic, cache behavior, parallelism veya specialized kernels'i standard cost model'de hesaba katmaz.
- Compatible chain varsayar.

Real numerical systems'de yalnız scalar-count optimum'u her zaman wall-clock optimum olmayabilir; fakat klasik algorithm analysis probleminde objective budur.

Matematiksel olarak parenthesizations aynı matrix result'ını verir. Floating-point arithmetic'te evaluation order nedeniyle küçük rounding differences oluşabilir.

16. Sık yapılan hatalar Dimension vector'ü yanlış indexlemek A_i boyutu $p[i-1] \times p[i]$ dir.

Split range'e jyi dahil etmek $k < j$ olmalıdır; aksi halde sağ subchain boş kalır.

Final multiplication cost'unu unutmak Yalnız left ve right subproblem costs yeterli değildir; $p[i-1]*p[k]*p[j]$ eklenmelidir.

Table'ı yanlış sırada doldurmak Short subchains önce çözülmelidir.

Cost table'dan parenthesization beklemek Structure için s split table gerekir.

Number of matrix multiplications ile scalar multiplications'ı karıştırmak Her full parenthesization $n-1$ matrix multiplications yapar; fakat bu operations'ın scalar costs'u dimensions'a göre değişir.

17. Kavram sözlüğü

English term	Türkçe açıklama
Matrix Chain	Ardışık dimensions'ı compatible olan matrices dizisi
Parenthesization	Multiplication evaluation order'ını brackets ile gösterme
Scalar Multiplication	Matrix entries'i üzerinde tek sayı çarpımı
Dimension Vector	Chain'i $n+1$ numbers ile tanımlayan p array'i
Split Point	Subchain'i left ve right parts'a ayıran k
Cost Table	Minimum scalar costs'u saklayan m table
Split Table	Optimal split points'i saklayan s table
Catalan Number	Full parenthesizations sayısını veren sequence

18. Kendini sinama

1. $(p \times q)$ ve $(q \times r)$ matrices'ini çarpmanın scalar cost'u nedir?
2. MCM actual matrix product'u hesaplar mı?
3. $m[i][j]$ neyi tutar?
4. $m[i][i]$ neden zero'dır?
5. Recurrence'ın final multiplication term'i nedir?
6. Videodaki örneğin minimum cost'u ve parenthesization'ı nedir?
7. Table hangi order'da doldurulur?
8. Time ve space complexity nedir?

Kısa cevaplar

1. $p*q*r$.
2. Hayır; en ucuz evaluation order'ı bulur.
3. $A_i \dots A_j$ subchain'inin minimum scalar multiplication cost'unu.
4. Tek matrix'i elde etmek için multiplication gerekmediğinden.
5. $p[i-1]*p[k]*p[j]$.
6. 158 ve $((A1(A2A3))A4)$.
7. Artan chain length, yani diagonal-by-diagonal.
8. $\theta(n^3)$ time ve $\theta(n^2)$ space.

Sonraki ders: **51 – Matrix Chain Multiplication Using DP Formula.**

51 – Matrix Chain Multiplication Using DP Formula

Video: 4.3.1 Matrix Chain Multiplication Using DP Formula (52:02)

Bu videonun ana fikri Bu ders, Matrix Chain Multiplication recurrence'ının nereden geldiğini ve formülün Cost Table ile Split Table üzerinde nasıl uygulandığını ayrıntılandırır.

```
C[i][j] = min over i <= k < j of (
    C[i][k]
    + C[k+1][j]
    + d[i-1]*d[k]*d[j]
)
```

Burada:

```
C[i][j] = Ai..Aj için minimum scalar multiplication cost
K[i][j] = minimum'u veren split point k
```

Videoda K table denilen structure, literatürde sıkça S table veya Split Table olarak da yazılır.

1. Problem neden yalnız order problemidir? Matrix multiplication associative olduğu için legal parenthesizations matematiksel olarak aynı product'u verir:

A1 A2 A3 A4

Fakat intermediate matrix dimensions değiştiğinden scalar operation count ciddi biçimde değişebilir. Amaç:

hangi pair önce multiply edilmeli?

sorusunu bütün chain için optimal cevaplamaktır.

2. Üç matrix üzerinden formülün türetilmesi

A1: d₀ x d₁
A2: d₁ x d₂
A3: d₂ x d₃

Split k=1

A1 | (A2 A3)

Cost:

C[1][1] + C[2][3] + d₀*d₁*d₃

Son term neden d₀*d₁*d₃?

- Left result A1: d₀ x d₁
- Right result A2A3: d₁ x d₃
- Bu ikisini multiply etme cost'u: d₀*d₁*d₃

Split k=2

(A1 A2) | A3

Cost:

```
C[1][2] + C[3][3] + d0*d2*d3
```

Genelleştirildiğinde final multiplication term'i:

```
d[i-1]*d[k]*d[j]
```

olur.

3. Principle of Optimality $A_i..A_j$ için optimal parenthesization son adımda kde ayrılıyorsa:

```
(A_i..A_k)(A_{k+1}..A_j)
```

left ve right parenthesizations da kendi subchains'i için optimal olmak zorundadır.

Örneğin left taraf daha ucuz bir order ile değiştirilebilseydi total solution da ucuzlardı; original solution optimal olamazdı. Aynı argument right taraf için geçerlidir.

4. Naive recursion neden pahalıdır? Recursive definition bütün split points'i dener:

```
MCM(i,j):
  if i == j return 0
  answer = INF
  for k = i to j-1
    answer = min(answer,
                  MCM(i,k)
                  + MCM(k+1,j)
                  + d[i-1]*d[k]*d[j])
```

Aynı (i, j) subproblems farklı recursion branches'de tekrar hesaplanır. Full parenthesization sayısı Catalan growth gösterdiğinden naive exploration exponential'dır.

İki çözüm vardır:

- Memoization: recursive function sonucunu cache'lemek
- Tabulation: küçük subchains'den büyük subchains'e bottom-up doldurmak

İkisi de standard formulation'da $\Theta(n^3)$ time ve $\Theta(n^2)$ space'e iner.

5. Base case ve table geometrisi Tek matrix için:

```
C[i][i]=0
```

Yalnız $i \leq j$ cells anlamlıdır; cost table'ın upper triangle kısmı kullanılır:

	j=1	j=2	j=3	j=4
i=1	0	?	?	?
i=2		0	?	?
i=3			0	?
i=4				0

Lower triangle gerekmez çünkü $i > j$ bir forward matrix subchain tanımlamaz.

6. Neden diagonal-by-diagonal doldurulur? $C[i][j]$, $C[i][k]$ ve $C[k+1][j]$ gibi daha kısa intervals'a bağlıdır. Bu yüzden:

```
gap 0 -> single matrices
gap 1 -> chains of length 2
gap 2 -> chains of length 3
...
```

sırası izlenir.

Equivalently:

$$\text{length} = j - i + 1$$

artan sırada işlenir.

7. Videodaki ayrıntılı örnek Dimension vector:

$$d = [3, 2, 4, 2, 5]$$

Matrices:

A1: 3 x 2
A2: 2 x 4
A3: 4 x 2
A4: 2 x 5

Başlangıç:

$$C[1][1] = C[2][2] = C[3][3] = C[4][4] = 0$$

8. Gap 1: length-2 chains Her cell'de yalnız bir possible split vardır.

C[1][2]

$$\begin{aligned} k &= 1 \\ C[1][2] &= 0 + 0 + 3 \cdot 2 \cdot 4 = 24 \\ K[1][2] &= 1 \end{aligned}$$

C[2][3]

$$\begin{aligned} k &= 2 \\ C[2][3] &= 0 + 0 + 2 \cdot 4 \cdot 2 = 16 \\ K[2][3] &= 2 \end{aligned}$$

C[3][4]

$$\begin{aligned} k &= 3 \\ C[3][4] &= 0 + 0 + 4 \cdot 2 \cdot 5 = 40 \\ K[3][4] &= 3 \end{aligned}$$

9. Gap 2: length-3 chains **C[1][3]** k=1:

$$\begin{aligned} C[1][1] + C[2][3] + d_0 \cdot d_1 \cdot d_3 \\ &= 0 + 16 + 3 \cdot 2 \cdot 2 \\ &= 28 \end{aligned}$$

k=2:

$$\begin{aligned} C[1][2] + C[3][3] + d_0 \cdot d_2 \cdot d_3 \\ &= 24 + 0 + 3 \cdot 4 \cdot 2 \\ &= 48 \end{aligned}$$

Minimum:

$$\begin{aligned} C[1][3] &= 28 \\ K[1][3] &= 1 \end{aligned}$$

C[2][4] k=2:

$$\begin{aligned}
 &C[2][2] + C[3][4] + d1*d2*d4 \\
 &= 0 + 40 + 2*4*5 \\
 &= 80
 \end{aligned}$$

k=3:

$$\begin{aligned}
 &C[2][3] + C[4][4] + d1*d3*d4 \\
 &= 16 + 0 + 2*2*5 \\
 &= 36
 \end{aligned}$$

Minimum:

$$\begin{aligned}
 &C[2][4]=36 \\
 &K[2][4]=3
 \end{aligned}$$

10. Gap 3: full chain C[1][4] için k=1,2,3 denenir.

k=1

$$\begin{aligned}
 &C[1][1] + C[2][4] + d0*d1*d4 \\
 &= 0 + 36 + 3*2*5 \\
 &= 66
 \end{aligned}$$

k=2

$$\begin{aligned}
 &C[1][2] + C[3][4] + d0*d2*d4 \\
 &= 24 + 40 + 3*4*5 \\
 &= 124
 \end{aligned}$$

k=3

$$\begin{aligned}
 &C[1][3] + C[4][4] + d0*d3*d4 \\
 &= 28 + 0 + 3*2*5 \\
 &= 58
 \end{aligned}$$

Minimum:

$$\begin{aligned}
 &C[1][4]=58 \\
 &K[1][4]=3
 \end{aligned}$$

Videonun otomatik English captions'ında bazı ara sayılar yanlış duyulmuştur; yukarıdaki values recurrence ve dimensions kullanılarak yeniden doğrulanmıştır. Özellikle k=1 candidate 66dır; final optimum değişmez.

11. Tamamlanmış tables Cost table:

C[i][j]	1	2	3	4
1	0	24	28	58
2	–	0	16	36
3	–	–	0	40
4	–	–	–	0

Split table:

K[i][j]	1	2	3	4
1	–	1	1	3
2	–	–	2	3
3	–	–	–	3
4	–	–	–	–

12. Split table'dan parenthesization çıkarma Full chain ile başlanır:

K[1][4]=3

İlk split:

(A1 A2 A3) | A4

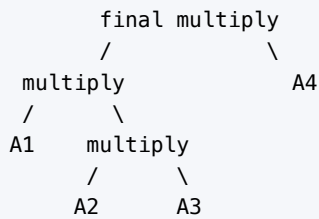
Sol subchain:

K[1][3]=1
A1 | (A2 A3)

Sonuç:

((A1(A2A3))A4)

Execution tree:



İlk A2*A3, sonra A1 ile sonucu, en son A4 ile yeni sonucu multiply edilir.

13. Parenthesization reconstruction pseudocode

```

PrintOptimal(i,j,K)
  if i == j
    print "A" + i
    return

  print "("
  PrintOptimal(i, K[i][j], K)
  PrintOptimal(K[i][j]+1, j, K)
  print ")"

```

Call:

PrintOptimal(1,n,K)

Her matrix leaf bir kez output edildiği için reconstruction $\theta(n)$ time kullanır; cost table'ı hesaplama $\theta(n^3)$ kalır.

14. Bottom-up formula implementation


```

Algorithm MatrixChainDP(d[0..n])
  create C[1..n][1..n]
  create K[1..n][1..n]

  for i = 1 to n
    C[i][i] <- 0

  for gap = 1 to n-1
    for i = 1 to n-gap
      j <- i+gap
      C[i][j] <- INF

      for k = i to j-1
        candidate <- C[i][k]
                      + C[k+1][j]
                      + d[i-1]*d[k]*d[j]

        if candidate < C[i][j]
          C[i][j] <- candidate
          K[i][j] <- k

  return C[1][n], K

```

gap loop'u yerine length=2..n loop'u da aynı dependency order'ı verir.

15. Complexity'nin daha kesin açıklaması

Upper-triangular meaningful states sayısı:

$$n(n+1)/2 = \theta(n^2)$$

Fakat her nontrivial state için tek işlem yapılmaz; bütün split points denenir. Total split trials:

$$\text{sum over chain lengths and starting positions of (length-1)} \\ = \theta(n^3)$$

Bu yüzden tight bound:

$$\text{Time} = \theta(n^3) \\ \text{Space} = \theta(n^2)$$

Videoda $\theta(n^3)$ upper bound olarak söylenir; standard triple-loop implementation için daha kesin ifade $\theta(n^3)$ tür.

16. Tie durumları

İki farklı split aynı minimum cost'u verebilir.

Yalnız bir optimal parenthesization isteniyorsa deterministic tie-break kullanılabilir:

ilk minimum k'yi sakla

Bütün optimal parenthesizations gerekiyorsa:

$K[i][j]$ = minimum'u veren bütün k values

şeklinde list saklanmalıdır. Bu durumda output sayısı exponential olabileceğinden bütün solutions'ı yazdır-mak da exponential sürebilir.

17. Numeric safety

Dimensions büyükse:

$$d[i-1]*d[k]*d[j]$$

32-bit integer'ı aşabilir. Cost type en azından uygun genişlikte seçilmeli ve multiplication/addition overflow kontrolü yapılmalıdır.

INF sentinel seçildiğinde candidate arithmetic öncesi operands kontrol edilmelidir.

18. Sık yapılan hatalar $d[i]*d[k]*d[j]$ yazmak Left result'ın row count'u $d[i-1]$ dir; doğru term $d[i-1]*d[k]*d[j]$.

$k \leq j$ kullanmak Doğru range $i \leq k < j$ dir.

Diagonal'ı INF bırakmak Base case $C[i][i]=0$ olmalıdır.

Büyük intervals'ı önce çözmek Required subproblems henüz hazır olmaz; artan gap/length gerekir.

Yalnız minimum cost'u saklamak Parenthesization gerekiyorsa optimal k de tutulmalıdır.

$\theta(n^2)$ cells var diye time'ı $\theta(n^2)$ sanmak Her cell birden çok split dener; total time $\theta(n^3)$ tür.

Caption değerlerini kontrol etmeden kopyalamak Formül ve dimensions kullanılarak her candidate yeniden hesaplanmalıdır.

19. Kavram sözlüğü

English term	Türkçe açıklama
Recurrence Relation	Büyük problem value'sunu subproblem values ile tanımlayan denklem
Cost Table	Her interval'ın minimum cost'unu saklayan C matrix
Split Table	Minimum'u veren k values'ını saklayan table
Gap	Table cell için $j-i$, yani diagonal uzaklığı
Chain Length	$j-i+1$ matrices içeren interval uzunluğu
Memoization	Recursive results'ı cache'leyerek tekrar hesabı önleme
Tabulation	States'i dependency order'da bottom-up hesaplama
Evaluation Tree	Multiplication order'ını binary tree olarak gösterme
Tie	Birden çok split'in aynı minimum cost'u vermesi

20. Kendini sına

1. $C[i][j]$ ve $K[i][j]$ ne tutar?
2. Final multiplication cost neden $d[i-1]*d[k]*d[j]$ dir?
3. Possible k range nedir?
4. Table neden diagonal-by-diagonal doldurulur?
5. $[3,2,4,2,5]$ örneğinin minimum cost'u nedir?
6. Bu örneğin optimal parenthesization'ı nedir?
7. $C[1][3]$ için optimal split hangisidir?
8. Neden $\theta(n^2)$ states olmasına rağmen time $\theta(n^3)$ tür?
9. Multiple optimal splits nasıl korunur?

Kısa cevaplar

1. Minimum scalar cost'u ve minimum'u veren split point'i.
2. Left result $d[i-1] \times d[k]$, right result $d[k] \times d[j]$ olduğu için.
3. $i \leq k < j$.
4. Her state daha kısa subchains'e bağlı olduğu için.
5. 58.
6. $((A1(A2A3))A4)$.

7. $k=1$.
8. Her state için $\theta(n)$ e kadar split denendiği için.
9. Her cell'de bütün minimum-attaining k values listelenerek.

Sonraki ders: **52 – Matrix Chain Multiplication Program.**

52 – Matrix Chain Multiplication Program

Video: 4.3.1 Matrix Chain Multiplication Program (18:40)

Kaynak notu: Bu videoda YouTube English captions bulunmuyor. Resmî başlık program implementation'ını kapsadığını gösterir. Aşağıdaki ders, 50 ve 51. videolardaki recurrence ile bu program kapsamı temel alınarak; indexing, overflow, reconstruction ve complexity ayrıca doğrulanarak hazırlanmıştır.

Bu videonun ana fikri Matrix Chain Multiplication programı iki tables doldurur:

```
cost[i][j] = Ai..Aj subchain'inin minimum scalar multiplication cost'u
split[i][j] = minimum'u veren k
```

Her cell için bütün legal split points denenir:

```
candidate = cost[i][k]
            + cost[k+1][j]
            + dimensions[i-1]*dimensions[k]*dimensions[j]
```

Table artan chainLength sırasıyla doldurulur. En sonda cost[1][n] optimum'u, split table ise optimal parenthesization'ı verir.

1. Program input contract n matrices için input dimension vector uzunluğu n+1 olmalıdır:

```
dimensions = [d0,d1,...,dn]
Ai has dimension d[i-1] x d[i]
```

Örnek:

```
dimensions = [3,2,4,2,5]

A1: 3x2
A2: 2x4
A3: 4x2
A4: 2x5
```

Bu representation, neighboring matrices'in compatibility bilgisini zaten içerir.

Program için dimensions positive integers olmalıdır. dimensions.size() < 2 ise ortada multiply edilecek matrix chain yoktur.

2. Neden iki table gerekir? Yalnız optimum numeric cost isteniyorsa cost yeterlidir. Fakat hangi multiplication order'ın bu sonucu verdiğini bulmak için decisions da korunmalıdır:

```
split[i][j] = optimal last split point
```

Value ile solution structure aynı şey değildir.

3. Dependency order cost[i][j], daha kısa intervals'a bağlıdır:

```
cost[i][k]
cost[k+1][j]
```

Bu nedenle loop order:

```
chainLength -> i -> k
```

olmalıdır. $j, i + \text{chainLength} - 1$ ile hesaplanır.

4. Core pseudocode

```
MatrixChainOrder(d[0..n])
  for i = 1 to n
    cost[i][i] <- 0

  for chainLength = 2 to n
    for i = 1 to n-chainLength+1
      j <- i+chainLength-1
      cost[i][j] <- INF

      for k = i to j-1
        candidate <- cost[i][k]
                      + cost[k+1][j]
                      + d[i-1]*d[k]*d[j]

        if candidate < cost[i][j]
          cost[i][j] <- candidate
          split[i][j] <- k

  return cost, split
```

5. Güvenli C++ implementation

```
#include <stdint>
#include <limits>
#include <stdexcept>
#include <string>
#include <utility>
#include <vector>

struct MCMResult {
  std::int64_t minimumCost;
  std::vector<std::vector<std::int64_t>>> cost;
  std::vector<std::vector<int>>> split;
};

bool checkedCandidate(
  std::int64_t left,
  std::int64_t right,
  std::int64_t a,
  std::int64_t b,
  std::int64_t c,
  std::int64_t& result) {

  const std::int64_t MAX = std::numeric_limits<std::int64_t>::max();

  if (a > MAX / b) return false;
  const std::int64_t ab = a * b;
  if (ab > MAX / c) return false;
  const std::int64_t product = ab * c;

  if (left > MAX - right) return false;
  const std::int64_t partial = left + right;
  if (partial > MAX - product) return false;

  result = partial + product;
  return true;
}
```

```

}

MCMResult matrixChainOrder(const std::vector<std::int64_t>& d) {
    if (d.size() < 2) {
        throw std::invalid_argument("at least one matrix is required");
    }
    for (std::int64_t x : d) {
        if (x <= 0) {
            throw std::invalid_argument("dimensions must be positive");
        }
    }

    const int n = static_cast<int>(d.size()) - 1;
    const std::int64_t INF = std::numeric_limits<std::int64_t>::max();

    std::vector<std::vector<std::int64_t>> cost(
        n + 1, std::vector<std::int64_t>(n + 1, INF));
    std::vector<std::vector<int>> split(
        n + 1, std::vector<int>(n + 1, -1));

    for (int i = 1; i <= n; ++i) {
        cost[i][i] = 0;
    }

    for (int length = 2; length <= n; ++length) {
        for (int i = 1; i <= n - length + 1; ++i) {
            const int j = i + length - 1;

            for (int k = i; k < j; ++k) {
                std::int64_t candidate;
                if (!checkedCandidate(
                    cost[i][k], cost[k + 1][j],
                    d[i - 1], d[k], d[j], candidate)) {
                    continue;
                }

                if (candidate < cost[i][j]) {
                    cost[i][j] = candidate;
                    split[i][j] = k;
                }
            }

            if (split[i][j] == -1) {
                throw std::overflow_error("matrix-chain cost overflow");
            }
        }
    }

    return {cost[1][n], std::move(cost), std::move(split)};
}

```

checkedCandidate, bütün dimensions positive olduğu için multiplication ve additions'ı işlemden önce upper bound ile denetler. Böylece signed integer overflow gerçekleşmeden candidate reddedilir. Alternatif olarak multiprecision integer kullanılabilir.

6. Parenthesization function

```

std::string buildParenthesization(
    int i,
    int j,

```

```

const std::vector<std::vector<int>>& split) {

    if (i == j) {
        return "A" + std::to_string(i);
    }

    const int k = split[i][j];
    if (k < i || k >= j) {
        throw std::logic_error("invalid split table");
    }

    return "(" + buildParenthesization(i, k, split)
        + buildParenthesization(k + 1, j, split) + ")";
}

```

Kullanım:

```

std::vector<std::int64_t> d{3, 2, 4, 2, 5};
MCMResult result = matrixChainOrder(d);

// result.minimumCost == 58
// buildParenthesization(1, 4, result.split)
// -> ((A1(A2A3))A4)

```

7. Örneğin table values'

```

cost[1][2]=24
cost[2][3]=16
cost[3][4]=40

cost[1][3]=28, split=1
cost[2][4]=36, split=3

cost[1][4]=58, split=3

```

Full chain candidates:

```

k=1 -> 0 + 36 + 3*2*5 = 66
k=2 -> 24 + 40 + 3*4*5 = 124
k=3 -> 28 + 0 + 3*2*5 = 58

```

8. move neden kullanıldı? Function result oluşturulurken large two-dimensional vectors'ın gereksiz kopyalanmasını önlemek için:

```

std::move(cost)
std::move(split)

```

kullanılır. Algorithm mantığı için zorunlu değildir; implementation efficiency ayrıntısıdır.

9. Tie handling Condition:

```

candidate < cost[i][j]

```

olduğunda first minimum split korunur.

$\leq$ kullanılırsa equal cost durumunda last split korunur. İki seçim de minimum cost açısından doğrudur; deterministic behavior dokümente edilmelidir.

Bütün optimal orders gerekiyorsa tek integer yerine minimum'u veren bütün k values saklanmalıdır.

10. Complexity Üç nested logical loops:

```
length, i, k
```

nedeniyle:

```
Time =  $\theta(n^3)$   
Space =  $\theta(n^2)$ 
```

Parenthesization reconstruction:

```
Time =  $\theta(n)$   
recursion stack =  $\theta(n)$ 
```

11. Sık yapılan program hataları Matrix sayısını dimension sayısı sanmak

```
n = d.size()-1
```

olmalıdır.

d[i]*d[k]*d[j] kullanmak Doğru term:

```
d[i-1]*d[k]*d[j]
```

k <= j loop'u yazmak Legal splits $k=i..j-1$ aralığındadır.

Table'ı row-major doldurmak Subproblems hazır olmayabilir; increasing chain length gerekir.

int overflow'u görmezden gelmek Dimension products çok hızlı büyür; wide/checked arithmetic gerekir.

Split table'ı güncellemek Minimum cost bulunur fakat optimal order reconstruct edilemez.

Empty veya nonpositive dimension kabul etmek Input contract doğrulanmalıdır.

12. Kavram sözlüğü

English term	Türkçe açıklama
Dimension Vector	Matrix chain'i $n+1$ boyutla temsil eden array
Cost Table	Minimum scalar costs'u saklayan DP table
Split Table	Optimum'u veren split points'i saklayan table
Chain Length	Bir subproblem'deki matrix sayısı
Checked Arithmetic	Overflow oluşmadan önce sınırı denetleyen arithmetic
Tie-Breaking	Equal-cost decisions arasında tutarlı seçim kuralı
Reconstruction	Split decisions'dan parenthesization üretme

13. Kendini sına

1. n matrices için dimension vector uzunluğu kaçtır?
2. Program neden increasing chainLength kullanır?
3. Innermost loop'ta k hangi aralıktadır?
4. cost ve split tables ne tutar?
5. $[3,2,4,2,5]$ için minimum cost nedir?
6. Bu örneğin parenthesization'ı nedir?
7. int kullanmak neden risklidir?

8. Time ve space complexity nedir?

Kısa cevaplar

1. $n+1$.
2. Her state daha kısa subchains'e bağlı olduğu için.
3. $i \leq k < j$.
4. Minimum cost'u ve minimum'u veren split point'i.
5. 58.
6. $((A1(A2A3))A4)$.
7. Dimension products ve accumulated cost overflow yapabileceği için.
8. $\theta(n^3)$ time ve $\theta(n^2)$ space.

Sonraki ders: **53 – Bellman-Ford: Single Source Shortest Path.**

53 – Bellman-Ford: Single Source Shortest Path

Video: 4.4 Bellman-Ford Algorithm (17:12)

Bu videonun ana fikri Bellman-Ford Algorithm, weighted directed graph'ta bir source'tan bütün reachable vertices'e shortest-path distances bulur.

Temel operation Relaxationdır:

```
if dist[u] + weight(u,v) < dist[v]
    dist[v] = dist[u] + weight(u,v)
    parent[v] = u
```

Bütün edges en fazla $V-1$ passes boyunca relax edilir. Bir ek pass'te hâlâ improvement varsa source'tan reachable bir Negative-Weight Cycle vardır.

1. Neden Dijkstra yerine Bellman-Ford? Dijkstra nonnegative edge weights ister. Negative edge olduğunda finalized sandığı bir distance daha sonra küçülebilir.

Bellman-Ford:

- Negative edges'i destekler.
- Reachable negative cycle'ı detect eder.
- Genel edge-list üzerinde çalışır.
- Standard formda $\Theta(VE)$ time kullanır.

2. Initialization Source s için:

```
dist[s]=0
dist[v]=INF for every  $v \neq s$ 
parent[v]=NIL
```

INF, source'tan henüz ulaşılmadığını temsil eder.

3. Relaxation ne demektir? Bir edge $u \rightarrow v$ ve weight w olsun. Source'tan uya bilinen bir path varsa, bu edge eklenerek vye candidate path elde edilir:

```
candidate = dist[u]+w
```

Candidate daha ucuzsa $dist[v]$ güncellenir.

Güvenli condition:

```
if dist[u] != INF and dist[u]+w < dist[v]
```

$dist[u] == INF$ iken arithmetic yapılmamalıdır.

4. Neden $V-1$ passes yeterlidir? Negative cycle yoksa reachable bir vertex'e shortest path simple seçilebilir. V vertices içeren simple path en fazla:

```
 $V-1$  edges
```

içerir.

Her full pass, edge ordering kötü olsa bile bir edge daha uzun shortest paths'in doğru biçimde propagate edilmesini garanti eder:

```
pass 1 -> en fazla 1-edge shortest paths
pass 2 -> en fazla 2-edge shortest paths
```

```
...  
pass V-1 -> en fazla V-1-edge shortest paths
```

İyi edge ordering ile bir pass'te birden fazla hop propagate olabilir; bu yalnız daha erken bitmeye yardım eder.

5. Videodaki ana örnek

Source vertex 1; edges:

```
1->2 (6)    1->3 (5)    1->4 (5)  
2->5 (-1)  
3->2 (-2)    3->5 (1)  
4->3 (-2)    4->6 (-1)  
5->7 (3)  
6->7 (3)
```

Initial distances:

```
[0, INF, INF, INF, INF, INF, INF]
```

Videodaki edge order ile successive full passes sonunda:

```
Vertex:  1  2  3  4  5  6  7  
dist:    0  1  3  5  0  4  3
```

Örnek paths:

```
1 -> 4 -> 3 -> 2 -> 5  
cost = 5-2-2-1 = 0  
  
1 -> 4 -> 3 -> 2 -> 5 -> 7  
cost = 3
```

6. Early stopping

Bir full pass boyunca hiçbir dist değişmezse sonraki passes de değişiklik yaratmaz:

```
for pass = 1 to V-1  
    changed = false  
    relax every edge  
    if an update occurs  
        changed = true  
    if not changed  
        break
```

Bu optimization best cases'i hızlandırır. Worst-case bound yine $\Theta(V E)$ dir.

7. Negative cycle örneği

Videodaki ikinci graph'a şu cycle eklenir:

```
2 -> 4 weight 5  
4 -> 3 weight 3  
3 -> 2 weight -10
```

Cycle total weight:

```
5+3-10 = -2
```

Cycle her dolaşıldığında path cost 2 daha azalır. Bu yüzden cycle'dan reachable vertices için finite shortest path yoktur.

8. Negative cycle detection

Normal $V-1$ passes bittikten sonra bütün edges bir kez daha kontrol edilir:

```
for each edge (u,v,w)
  if dist[u] != INF and dist[u]+w < dist[v]
    reachable negative cycle exists
```

$\text{dist}[u] \neq \text{INF}$ şartı önemlidir. Source'tan ulaşamayan başka bir component'teki negative cycle, bu single-source run'ın distances'ını etkilemez ve standard check tarafından raporlanmaz.

9. Pseudocode

```
BellmanFord(vertices, edges, source)
  for each vertex v
    dist[v] <- INF
    parent[v] <- NIL
  dist[source] <- 0

  for pass = 1 to V-1
    changed <- false

    for each edge (u,v,w)
      if dist[u] != INF and dist[u]+w < dist[v]
        dist[v] <- dist[u]+w
        parent[v] <- u
        changed <- true

    if not changed
      break

  for each edge (u,v,w)
    if dist[u] != INF and dist[u]+w < dist[v]
      return NEGATIVE_CYCLE

  return dist, parent
```

10. Path reconstruction $\text{parent}[v]$, last improvement'ı veren predecessor'ı saklar.

Negative cycle yoksa target'tan source'a doğru izlenir:

```
target -> parent[target] -> ... -> source
```

Sequence reverse edilerek source-to-target path elde edilir.

$\text{dist}[\text{target}] == \text{INF}$ ise target unreachable'dır.

11. Negative cycle'ın etkilediği vertices Yalnız "cycle var" bilgisi yerine hangi distances'ın -infinity olduğunu belirlemek istenebilir:

1. Extra pass'te relax olabilen vertices'i işaretle.
2. Bu vertices'ten graph üzerinde reachable olan bütün vertices'i traverse et.
3. Bunların distances'ını $-\text{INF}$ /undefined olarak raporla.

Cycle'a ulaşamayan veya cycle'dan sonra ulaşamayan vertices'in finite answers'ı korunabilir.

12. Complexity Her pass bütün E edges'i tarar; en fazla $V-1$ normal pass ve bir detection pass vardır:

```
Time =  $\Theta(V \cdot E)$ 
Space =  $\Theta(V)$ 
```

Edge list input storage dahil edilirse total auxiliary+input memory $\Theta(V+E)$ dir.

Sparse connected/reachable graph için $E=\Theta(V)$ ise:

$$\theta(V^2)$$

Dense directed graph için $E=\theta(V^2)$ ise:

$$\theta(V^3)$$

Videoda complete graph edge sayısı için söylenen $V(V-1)/2$ undirected graph formülüdür. Directed complete graph'ta self-loops hariç $V(V-1)$ directed edges olabilir. Asymptotic sonuç iki durumda da $\theta(V^2)$ edges'tir.

13. Dynamic Programming yorumu Bellman-Ford şu state fikriyle yazılabilir:

$D[k][v]$ = source'tan v 'ye en fazla k edge kullanan minimum path cost

Recurrence:

```
D[k][v] = min(  
    D[k-1][v],  
    min over incoming edges (u,v,w) of D[k-1][u]+w  
)
```

Standard in-place edge-relaxation implementation aynı pass içinde yeni values'ı da kullanabilir. Bu, convergence'i hızlandırabilir; correctness ve $V-1$ upper bound korunur.

14. Bellman-Ford ile Floyd-Warshall farkı

Özellik	Bellman-Ford	Floyd-Warshall
Problem	Single-source	All-pairs
Time	$\theta(VE)$	$\theta(V^3)$
Main storage	$\theta(V)$	$\theta(V^2)$
Negative edges	Evet	Evet
Negative cycle detect	Extra relaxation	Negative diagonal

15. Sık yapılan hatalar **Yalnız bir kez bütün edges'i relax etmek** Kötü edge ordering'de improvement uzak vertices'e henüz ulaşmayabilir.

V normal passes yapmak Shortest simple paths için $V-1$ yeterlidir; bir extra pass detection içindir.

$INF+w$ hesaplamak Reachability guard ve overflow-safe addition gerekir.

Her negative edge'i negative cycle sanmak Negative edge tek başına sorun değildir; total weight'i negative cycle finite optimum'u bozar.

Ulaşılamayan negative cycle'ı source problemiyle karıştırmak Standard Bellman-Ford yalnız source'tan reachable cycles'i detect eder.

Parent array tutmadan actual path beklemek Distances tek başına route'u doğrudan vermez.

16. Kavram sözlüğü

English term	Türkçe açıklama
Relaxation	Bir edge üzerinden daha ucuz path bulunursa distance'ı güncelleme
Edge List	Graph'ın edges'ini (u, v, w) records olarak saklama
Pass	Bütün edges'in bir kez taranması
Negative Edge	Weight'i zero'dan küçük edge
Negative-Weight Cycle	Total edge weight'i negative olan cycle
Early Stopping	Update olmayan pass'ten sonra loop'u bitirme
Predecessor	Reconstructed path'te vertex'ten önce gelen vertex
Unreachable	Source'tan hiçbir directed path bulunmaması

17. Kendini sinama

1. Relaxation condition nedir?
2. Bütün edges neden $V-1$ kez relax edilir?
3. Bellman-Ford negative edges ile çalışır mı?
4. Reachable negative cycle nasıl detect edilir?
5. $\text{dist}[u] \neq \text{INF}$ guard neden gerekir?
6. Main örnekte $\text{dist}[7]$ kaçtır?
7. Time ve auxiliary space complexity nedir?
8. Dense directed graph'ta neden $\Theta(V^3)$ olabilir?

Kısa cevaplar

1. $\text{dist}[u] + w < \text{dist}[v]$ ise $\text{dist}[v]$ güncellenir.
2. Simple shortest path en fazla $V-1$ edge içerdiği için.
3. Evet.
4. $V-1$ passes sonrası extra pass'te hâlâ relaxation mümkünse.
5. Unreachable state ile arithmetic ve overflow yapılmaması için.
6. 3.
7. $\Theta(VE)$ time ve $\Theta(V)$ auxiliary space.
8. $E = \Theta(V^2)$ olduğu için $VE = \Theta(V^3)$.

Sonraki ders: **54 – 0/1 Knapsack: Two Methods.**

54 – 0/1 Knapsack: Two Methods

Video: 4.5 0/1 Knapsack: Two Methods (28:24)

Bu videonun ana fikri 0/1 Knapsack Probleminde her item ya tamamen seçilir (1) ya da hiç seçilmez (0). Amaç total weight capacity'yi aşmadan total profit'i maximize etmektir.

Video iki çözüm gösterir:

1. Tabulation Method: item index ve capacity üzerine DP table
2. Set Method: (profit, weight) states üretilip infeasible ve dominated states'i eleme

İlk yöntem integer capacity için $O(nW)$ time kullanır. İkinci yöntem pruning yapabilse de worst case'de exponential sayıda nondominated state tutabilir.

1. Mathematical formulation Her item i için:

```
profit = p[i]
weight = w[i]
decision x[i] in {0,1}
```

Objective:

```
maximize sum(p[i]*x[i])
```

Constraint:

```
sum(w[i]*x[i]) <= W
```

W , bag capacity'dir.

2. Neden 0/1 denir? Decision yalnız iki value alabilir:

```
x[i]=0 -> item alınmaz
x[i]=1 -> item tamamen alınır
```

Fractional item alınamaz. Bu nedenle problem, önceki Fractional Knapsack probleminden farklıdır. profit/weight ratio'ya göre greedy seçim 0/1 Knapsack için genel olarak optimal değildir.

3. Videodaki örnek

Item	Profit	Weight
1	1	2
2	2	3
3	5	4
4	6	5

Capacity $W=8$

Optimal selection:

```
x = [0,1,0,1]
items = {2,4}
total weight = 3+5 = 8
total profit = 2+6 = 8
```

4. Brute force neden pahalıdır? Her item için include/exclude kararı olduğundan n items için:

2^n subsets

vardır. Bütün subsets'i doğrudan denemek exponential time gerektirir.

DP, farklı decision sequences'in ulaştığı aynı (itemCount, remainingCapacity) subproblems'ini bir kez çözer.

5. Tabulation state

$V[i][c]$ = yalnız ilk i item kullanılabiliyorken
capacity c ile elde edilebilen maximum profit

Rows:

$i=0..n$

Columns:

$c=0..W$

Table size:

$(n+1)*(W+1)$

6. Base cases

Hiç item yoksa profit zero:

$V[0][c]=0$

Capacity zero ise standard positive-weight items'da profit zero:

$V[i][0]=0$

Zero-weight positive-profit items destekleniyorsa $V[i][0]$ her zaman zero değildir; generic recurrence bunları ayrıca ele almalıdır. Klasik table genellikle strictly positive weights varsayar.

7. Include/exclude recurrence

Current item weight'i capacity'yi aşıyorsa alınamaz:

```
if w[i] > c:  
    V[i][c] = V[i-1][c]
```

Siğiyorsa iki decision karşılaştırılır:

```
exclude = V[i-1][c]  
include = p[i] + V[i-1][c-w[i]]  
  
V[i][c] = max(exclude, include)
```

include case'in previous row'a bakması aynı item'ın bir kez daha kullanılmasını engeller. Aynı row'a bakılsaydı problem Unbounded Knapsack'a dönüşürdü.

8. Table'ın doldurulması

Videodaki tamamlanmış table:

Items / Capacity	0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0	0
1	0	0	1	1	1	1	1	1	1
2	0	0	1	2	2	3	3	3	3
3	0	0	1	2	5	5	6	7	7
4	0	0	1	2	5	6	6	7	8

Final answer:

$V[4][8]=8$

9. Örnek cell hesapları $V[3][6]$ Item 3: profit 5, weight 4:

$exclude = V[2][6] = 3$
 $include = 5 + V[2][2] = 5+1 = 6$
 $V[3][6] = 6$

$V[4][8]$ Item 4: profit 6, weight 5:

$exclude = V[3][8] = 7$
 $include = 6 + V[3][3] = 6+2 = 8$
 $V[4][8] = 8$

10. Selected items'ı reconstruct etme Bottom-right cell'den başlanır:

$i=n, c=W$

Eğer:

$V[i][c] == V[i-1][c]$

ise item i alınmadan da aynı optimum elde edilir; bir reconstruction policy olarak item atlanabilir.

Farklıysa item alınmıştır:

$select\ i$
 $c = c-w[i]$
 $i = i-1$

Örnekte:

$V[4][8]=8 \neq V[3][8]=7 \rightarrow$ item 4 seç
 $c=8-5=3$
 $V[3][3]=2 == V[2][3]=2 \rightarrow$ item 3 seçme
 $V[2][3]=2 \neq V[1][3]=1 \rightarrow$ item 2 seç
 $c=3-3=0$

Sonuç $\{2,4\}$.

11. Tie durumunda reconstruction Şu equality:

$V[i][c] == V[i-1][c]$

item'ın hiçbir optimal solution'da bulunmadığını kanıtlamaz. Yalnız item'ı almayan en az bir optimum bulunduğunu gösterir.

Include ve exclude equal ise multiple optimal solutions vardır. Bütün solutions gerekiyorsa iki branch de izlenmelidir; output sayısı exponential olabilir.

12. Tabulation complexity ve Pseudo-Polynomial ayrıntısı

Her table cell constant work yapar:

```
Time =  $\theta(nW)$   
Space =  $\theta(nW)$ 
```

Bu bound numeric capacity wye bağlıdır. W binary yazıldığında input'ta yalnız $\log W$ bits kaplar. Dolayısıyla $\theta(nW)$, formal input length açısından polynomial değil Pseudo-Polynomial'dır.

Bu ayrıntı önemlidir; 0/1 Knapsack bir NP-hard optimization problemidir.

13. Space optimization

Yalnız maximum profit gerekiyorsa one-dimensional array yeterlidir:

```
dp[0..W] = 0  
  
for each item i  
    for c = W down to w[i]  
        dp[c] = max(dp[c], p[i]+dp[c-w[i]])
```

Capacity'nin descending dolaşılması zorunludur. Ascending dolaşılırsa current item aynı iteration'da tekrar kullanılabilir ve Unbounded Knapsack sonucu çıkar.

```
Space =  $\theta(W)$ 
```

Ancak straightforward item reconstruction için decisions ayrıca saklanmalı veya yeniden hesaplama yapılmalıdır.

14. İkinci yöntem: Set Method

Her partial solution bir ordered pair ile temsil edilir:

```
(profit, weight)
```

Başlangıç frontier'i:

```
S0 = {(0, 0)}
```

Item (p, w) için mevcut her pair'den item'ı içeren yeni pair üretilir:

```
include(S) = {(profit+p, weight+w) for each pair in S}
```

Sonra:

```
Snew = merge(S, include(S))
```

ve infeasible/dominated pairs çıkarılır.

15. Feasibility pruning

Şu pair capacity'yi aşıyorsa atılır:

```
weight > W
```

Örneğin üçüncü item eklendiğinde oluşan:

```
(8, 9)
```

capacity 8'i aştığı için infeasible'dır.

16. Dominance rule

Bir state $A=(p_A, w_A)$, başka bir state $B=(p_B, w_B)$ tarafından dominate edilir eğer:

$$p_B \geq p_A$$

$$w_B \leq w_A$$

ve inequalities'den en az biri strict ise.

Bu durumda A hiçbir future item combination için Bden daha iyi olamaz ve silinebilir.

Örnek:

$$(3,5) \text{ is dominated by } (5,4)$$

Çünkü ikinci pair daha yüksek profit'i daha düşük weight ile verir.

Videoda "profit artarken weight azalması mümkün değildir" biçiminde duyulan açıklama yanıltıcıdır. Tam tersine bu mümkündür ve daha iyi state'i gösterir; kaldırılması gereken düşük-profit/yüksek-weight pair'dir.

17. Örneğin frontier sets'i

Item 1 sonrası:

$$S_1 = \{(0,0), (1,2)\}$$

Item 2 sonrası:

$$S_2 = \{(0,0), (1,2), (2,3), (3,5)\}$$

Item 3 sonrası infeasible ve dominated states temizlenince:

$$S_3 = \{(0,0), (1,2), (2,3), (5,4), (6,6), (7,7)\}$$

Item 4 sonrası:

$$S_4 = \{(0,0), (1,2), (2,3), (5,4), (6,5), (7,7), (8,8)\}$$

Maximum feasible profit:

$$(8,8)$$

18. Set Method reconstruction Final pair (8,8) önceki frontier S3te yoktur; dolayısıyla item 4 eklenmiştir:

$$(8,8) - (6,5) = (2,3)$$

(2,3) hem S3 hem S2de bulunduğu için item 3 alınmamıştır.

(2,3) S2de var fakat S1de yoktur; item 2 alınmıştır:

$$(2,3) - (2,3) = (0,0)$$

Item 1 alınmamıştır. Yine:

$$x = [0, 1, 0, 1]$$

elde edilir.

19. Set Method complexity Dominance pruning practice'te frontier'ı küçültebilir. Fakat worst case'de çok sayıda pair birbirini dominate etmez:

$$\text{Time/space can be exponential in } n$$

Bu yöntem Pareto Frontier yaklaşımıdır. Tabulation'ın $O(nW)$ bound'u ile aynı garantiye sahip değildir; avantajı reachable/nondominated states az olduğunda ortaya çıkar.

20. Sık yapılan hatalar Fractional seçim yapmak Decision yalnız 0 veya 1dir.

Include case'te current row'a bakmak Item'ın tekrar kullanılmasına yol açar.

Negative capacity index'ini kullanmak $w[i] > c$ ise include case hesaplanmaz; previous-row value kopyalanır.

$O(nW)$ yi her anlamda polynomial sanmak Numeric wye bağlı olduğu için pseudo-polynomial'dır.

Equality görünce item kesin seçilmedi demek Equality multiple optimal solutions gösterebilir.

Dominance yönünü ters kurmak Higher/equal profit ve lower/equal weight sunan state korunur.

21. Kavram sözlüğü

English term	Türkçe açıklama
0/1 Knapsack	Her item'ın tamamen alındığı veya hiç alınmadığı optimization problemi
Capacity	Bag'ın taşıyabileceği maximum total weight
Include/Exclude	Bir item için iki possible decision
Pseudo-Polynomial	Numeric value'ya polynomial, bit-length'e polynomial olmayan runtime
Feasible State	Capacity constraint'i sağlayan state
Dominance Rule	Her açıdan daha kötü state'i eleme kuralı
Pareto Frontier	Birbirini dominate etmeyen states kümesi
Backtracking	DP table'dan selected items'ı geriye doğru çıkarma

22. Kendini sinama

1. $V[i][c]$ neyi temsil eder?
2. Item capacity'ye sığıyorsa hangi iki value karşılaştırılır?
3. Örnekte maximum profit ve selected items nedir?
4. Table method complexity nedir?
5. Neden bu complexity pseudo-polynomial'dır?
6. One-dimensional DP'de capacity neden descending dolaşılır?
7. (3,5) neden (5,4) tarafından dominate edilir?
8. Set Method'ın worst-case complexity'si nasıldır?

Kısa cevaplar

1. İlk i item ve capacity c ile elde edilebilen maximum profit'i.
2. $V[i-1][c]$ ve $p[i]+V[i-1][c-w[i]]$.
3. 8; items {2,4}.
4. $O(nW)$ time ve $O(nW)$ space.
5. w , binary input length'i $\log w$ olan numeric value olduğu için.
6. Current item'ın aynı iteration'da yeniden kullanılmasını önlemek için.
7. Daha düşük weight ile daha yüksek profit verdiği için.
8. Exponential olabilir.

Sonraki ders: **55 – 0/1 Knapsack Program.**

55 – 0/1 Knapsack Program

Video: 4.5.1 0/1 Knapsack Program (16:59)

Bu videonun ana fikri Program, $(n+1) \times (\text{capacity}+1)$ DP table doldurur:

```
dp[i][c] = ilk i items ve capacity c ile maximum profit
```

Transition:

```
if weight[i] > c
    dp[i][c] = dp[i-1][c]
else
    dp[i][c] = max(
        dp[i-1][c],
        profit[i] + dp[i-1][c-weight[i]]
    )
```

Sonra bottom-right cell'den geriye gidilerek selected item indices bulunur.

1. Input ve indexing Videodaki arrays 1-based indexing kolaylığı için index 0'ı boş bırakır:

```
profit = [0,1,2,5,6]
weight = [0,2,3,4,5]
n = 4
capacity = 8
```

Bu bir convenience choice'tur. Modern languages'da 0-based arrays ile de aynı recurrence yazılabilir; önemli olan indexing convention'ı tutarlı kullanmaktır.

2. Table dimensions

```
rows = n+1
columns = capacity+1
```

Örnekte:

```
5 rows:    0..4
9 columns: 0..8
```

Zero row "hiç item yok", zero column "zero capacity" anlamındadır.

3. Program loop'larının anlamı Outer loop:

```
i = 1..n
```

hangi item'a kadar izin verildiğini belirler.

Inner loop:

```
c = 0..capacity
```

o subproblem'in available capacity'sini belirler.

Her cell yalnız previous row'a bağlı olduğundan row-by-row fill order geçerlidir.

4. C++ implementation

```

#include <algorithm>
#include <cstdint>
#include <stdexcept>
#include <utility>
#include <vector>

struct KnapsackResult {
    std::int64_t maximumProfit;
    std::vector<int> selectedItems;
    std::vector<std::vector<std::int64_t>> dp;
};

KnapsackResult zeroOneKnapsack(
    const std::vector<int>& weight,
    const std::vector<std::int64_t>& profit,
    int capacity) {

    if (weight.size() != profit.size()) {
        throw std::invalid_argument("weight/profit size mismatch");
    }
    if (capacity < 0) {
        throw std::invalid_argument("capacity cannot be negative");
    }

    const int n = static_cast<int>(weight.size());
    for (int w : weight) {
        if (w < 0) {
            throw std::invalid_argument("weight cannot be negative");
        }
    }

    std::vector<std::vector<std::int64_t>> dp(
        n + 1,
        std::vector<std::int64_t>(capacity + 1, 0));

    for (int i = 1; i <= n; ++i) {
        const int itemWeight = weight[i - 1];
        const std::int64_t itemProfit = profit[i - 1];

        for (int c = 0; c <= capacity; ++c) {
            dp[i][c] = dp[i - 1][c];

            if (itemWeight <= c) {
                dp[i][c] = std::max(
                    dp[i][c],
                    itemProfit + dp[i - 1][c - itemWeight]);
            }
        }
    }

    std::vector<int> selected;
    int c = capacity;

    for (int i = n; i >= 1; --i) {
        if (dp[i][c] != dp[i - 1][c]) {
            selected.push_back(i); // user-facing item number
            c -= weight[i - 1];
        }
    }

    std::reverse(selected.begin(), selected.end());
}

```

```
    return {dp[n][capacity], std::move(selected), std::move(dp)};
}
```

std::move, large result vectors'ın gereksiz kopyalanmasını önler.

5. Örnek kullanım

```
std::vector<int> weight{2, 3, 4, 5};
std::vector<std::int64_t> profit{1, 2, 5, 6};

KnapsackResult result = zeroOneKnapsack(weight, profit, 8);

// result.maximumProfit == 8
// result.selectedItems == {2,4}
```

Programdaki vectors 0-based olsa da output item numbers videodaki gibi 1-based döndürülür.

6. Cell tracing i=1, item (weight=2,profit=1):

```
c=0,1 -> item sığmaz, profit 0
c=2..8 -> max(0,1+0)=1
```

i=2, item (3,2) ve c=5:

```
exclude = dp[1][5] = 1
include = 2 + dp[1][2] = 3
dp[2][5] = 3
```

i=3, item (4,5) ve c=7:

```
exclude = dp[2][7] = 3
include = 5 + dp[2][3] = 7
dp[3][7] = 7
```

i=4, item (5,6) ve c=8:

```
exclude = dp[3][8] = 7
include = 6 + dp[3][3] = 8
dp[4][8] = 8
```

7. Reconstruction tracing Başlangıç:

```
i=4, c=8
```

```
dp[4][8]=8 != dp[3][8]=7
-> item 4 selected
-> c=8-5=3
```

```
dp[3][3]=2 == dp[2][3]=2
-> item 3 not selected
```

```
dp[2][3]=2 != dp[1][3]=1
-> item 2 selected
-> c=3-3=0
```

```
dp[1][0]=dp[0][0]
-> item 1 not selected
```

Result:

```
item 2 and item 4
```

8. Equality ve multiple optima

Program equality durumunda item'ı seçmemeyi tercih eder:

```
if dp[i][c] == dp[i-1][c]
    skip item i
```

Bu valid bir optimal solution üretir. Ancak include seçeneği de aynı value'yu veriyorsa başka bir optimum item'ı içeriyor olabilir.

Bütün optimal selections için her tie'da iki legal predecessor state de izlenmelidir.

9. Negative profits

Base table zero olduğundan negative-profit item'lar doğal olarak seçilmez. Standard problem çoğunlukla nonnegative profits varsayar.

Eğer "en az bir item seçilmeli" gibi ek constraint varsa zero initialization tek başına yeterli değildir; unreachable states için $-\infty$ gibi ayrı representation gerekir.

10. Zero-weight items

Two-dimensional recurrence, zero-weight item'ı previous row'dan aldığı için onu en fazla bir kez seçer:

```
profit[i] + dp[i-1][c]
```

Reconstruction'da capacity değişmese bile i her iteration'da azalır; infinite loop oluşmaz.

11. Overflow

Profit sums 32-bit sınırını aşabilir. Örnekte küçük values olsa da generic program daha geniş type veya checked addition kullanılmalıdır.

Yukarıdaki code `int64_t` kullanır; untrusted/extreme input'ta `itemProfit + dp[...]` için ayrıca overflow kontrolü gerekir.

12. Complexity

Full table:

```
Time =  $\Theta(nW)$ 
Space =  $\Theta(nW)$ 
```

Reconstruction:

```
Time =  $O(n)$ 
```

Yalnız maximum value gerekiyorsa space $\Theta(W)$ ye düşürülebilir.

13. One-dimensional implementation

```
std::vector<std::int64_t> dp(capacity + 1, 0);

for (int i = 0; i < n; ++i) {
    for (int c = capacity; c >= weight[i]; --c) {
        dp[c] = std::max(dp[c], profit[i] + dp[c - weight[i]]);
    }
}
```

Descending capacity order, item'ın iteration içinde tekrar kullanılmasını engeller.

Bu form maximum profit'i verir; selected items reconstruction'ı için ek bookkeeping gerekir.

14. Input-driven program tasarımı

Videodaki program fixed example'a odaklanır. Reusable implementation:

- n ve capacityyi input'tan almalı,

- array lengths'i doğrulamalı,
- negative weights/capacity'yi reddetmeli,
- memory allocation'ın $(n+1)(W+1)$ için güvenli olduğunu kontrol etmeli,
- result ile selected item indices'i birlikte döndürmelidir.

Çok büyük W için table memory'si pratik olmayabilir; 1D DP, sparse frontier veya farklı algorithmic yaklaşım gerekebilir.

15. Sık yapılan program hataları Table'ı $n \times W$ boyutlandırmak Base row ve base column için $(n+1) \times (W+1)$ gerekir.

Weight ve profit arrays'ini karıştırmak Include index $c - \text{weight}[i]$, eklenen value $\text{profit}[i]$ dir.

Previous row yerine current row kullanmak 0/1 constraint bozulur.

Reconstruction'da capacity'yi her durumda azaltmak Yalnız item selected ise weight çıkarılır.

1D DP'yi ascending capacity ile doldurmak Aynı item tekrar kullanılabilir.

Tie'ları unique solution sanmak Equality birden çok optimum saklıyor olabilir.

16. Kavram sözlüğü

English term	Türkçe açıklama
DP Table	Subproblem optimum values'ını saklayan table
Previous Row	Current item kullanılmadan önceki item set'inin states'i
Backtracking	Bottom-right cell'den decisions'ı geriye doğru izleme
Selected Item	Optimal solution'a dahil edilen item
Space Optimization	İki-dimensional table'ı one-dimensional array'e düşürme
Input Validation	Program assumptions'ın input tarafından sağlandığını kontrol etme
Tie	Include ve exclude decisions'ın aynı profit'i vermesi

17. Kendini sinama

1. Table neden $(n+1) \times (W+1)$ boyutundadır?
2. Item sığmadığında hangi value kopyalanır?
3. Include candidate nasıl hesaplanır?
4. Örnek programın sonucu nedir?
5. Selected items nasıl bulunur?
6. Equality ne anlama gelebilir?
7. 1D DP'de capacity hangi yönde dolaşılır?
8. Time ve table space complexity nedir?

Kısa cevaplar

1. Zero-item ve zero-capacity base states'i de saklamak için.
2. $dp[i-1][c]$.
3. $\text{profit}[i] + dp[i-1][c - \text{weight}[i]]$.
4. Maximum profit 8, selected items {2,4}.
5. Bottom-right'tan previous-row values karşılaştırılarak.
6. Multiple optimal selections bulunabileceğini.
7. Descending.
8. $\theta(nW)$ time ve $\theta(nW)$ space.

Sonraki ders: **56 – Optimal Binary Search Tree: Successful Search Only.**

56 – Optimal Binary Search Tree: Successful Search Only

Video: 4.6 Optimal Binary Search Tree – Successful Search Only (30:19)

Bu videonun ana fikri Optimal Binary Search Tree (OBST), sorted keys ve bunların search frequencies'i verildiğinde weighted successful-search cost'u minimum yapan Binary Search Tree'yi bulur.

Amaç yalnız tree height'ını minimize etmek değildir. Sık aranan keys root'a yakın, daha seyrek aranan keys daha derinde tutulabilir.

Videonun interval notation'ıyla:

```
C[a][b] = K(a+1)..K(b) keys'i için minimum weighted search cost
W[a][b] = f(a+1)+...+f(b)
```

Recurrence:

```
C[a][b] = W[a][b]
        + min over a < r <= b of (
            C[a][r-1] + C[r][b]
        )
```

1. Binary Search Tree özelliği Her node için:

```
left subtree keys < node key
right subtree keys > node key
```

olur.

Bu ordering sayesinde search her comparison'da left veya right subtree'ye yönelir.

2. Search cost ve tree height Root'taki key için bir comparison, depth 1'deki child için iki comparisons gerekir. Bu derste root depth'i 0 sayılsa bile comparison count:

```
comparisons(key) = depth(key)+1
```

Balanced BST'de height $\Theta(\log n)$ olabilir. Fakat sıradan bir BST her zaman balanced değildir:

```
best/balanced height =  $\Theta(\log n)$ 
worst/skewed height =  $\Theta(n)$ 
```

Dolayısıyla "BST search her zaman $\Theta(\log n)$ dir" doğru değildir; worst case $\Theta(n)$ dir.

3. Aynı keys ile kaç BST mümkündür? Sorted n distinct keys için possible BST shapes sayısı ninci Catalan number'dır:

```
Catalan(n) = binomial(2n,n)/(n+1)
```

Örneğin $n=3$ için:

```
Catalan(3)=5
```

Brute force bütün trees'i üretirse search space exponential büyür.

4. Balanced tree ile optimal tree aynı değildir Frequencies equal ise shallow/balanced tree çoğu zaman iyi sonuç verir. Frequencies farklıysa high-frequency key'i root'a yaklaştırmak, tree'yi geometrik olarak daha az balanced yapsa bile expected cost'u düşürebilir.

Optimal burada şu objective'e göredir:

$$\text{sum over keys } i \text{ of } f[i] * (\text{depth}[i] + 1)$$

Tree shape tek başına objective değildir.

5. Frequency ve probability kullanımı

$f[i]$ raw search count olabilir. Toplam frequency:

$$F = \text{sum } f[i]$$

ise normalized probability:

$$p[i] = f[i] / F$$

Cost frequencies ile hesaplanırsa total weighted comparisons elde edilir. Expected comparisons:

$$\text{weightedCost} / F$$

Normalization optimal tree'yi değiştirmez; bütün costs aynı positive constant ile ölçeklenir.

6. Bu videonun kapsamı: successful search only

Bu model yalnız mevcut keys'in successful-search frequencies'ini kullanır:

$$f_1, f_2, \dots, f_n$$

Aranan value hiçbir key'e eşit değilse oluşan unsuccessful-search probabilities bu derste hesaba katılmaz. Onları içeren general OBST modeli bir sonraki videonun konusudur.

7. Videodaki interval indexing

Keys:

$$K_1, K_2, \dots, K_n$$

Table indices boundaries olarak $0..n$ kullanılır:

$C[a][b]$ represents keys $K(a+1), \dots, K(b)$

Örnekler:

$C[0][1] \rightarrow$ yalnız K_1
 $C[1][3] \rightarrow K_2, K_3$
 $C[0][4] \rightarrow K_1, K_2, K_3, K_4$
 $C[i][i] \rightarrow$ empty interval

Bu convention, standard $\text{cost}[i][j] = K_i \dots K_j$ notation'ından farklı görünür fakat aynı problemi ifade eder.

8. Base case

Empty key interval için:

$$C[i][i] = 0$$

Tek-key interval:

$$\begin{aligned} C[i-1][i] &= f[i] \\ \text{root}[i-1][i] &= i \end{aligned}$$

Çünkü key root olur ve bir comparison yapılır.

9. Recurrence nasıl türetilir?

$K(a+1) \dots K(b)$ interval'ında root olarak K_r seçilsin:

$$a < r \leq b$$

BST ordering nedeniyle:

```
left subtree = K(a+1)..K(r-1) -> C[a][r-1]
right subtree = K(r+1)..K(b) -> C[r][b]
```

Bu iki subtree kendi içinde optimal olmalıdır.

Subtree costs, kendi local roots'ını comparison level 1 kabul eder. Bütün subtree nodes final tree'de bir level aşağı indiği için interval'daki her frequency bir kez daha eklenir:

```
W[a][b]=sum frequencies in interval
```

Bu nedenle:

```
C[a][b] = min_r(C[a][r-1]+C[r][b]) + W[a][b]
```

10. Principle of Optimality Optimal tree'nin root'u K_r ise left subtree de kendi keys'i için optimal olmak zorundadır. Daha ucuz bir left subtree bulunsaydı onu mevcut left subtree ile değiştirerek whole tree cost'u azaltılabilirdi.

Aynı argument right subtree için geçerlidir. Bu optimal substructure DP recurrence'ını doğrular.

11. Videodaki ana örnek

Key index	Key	Frequency
1	10	4
2	20	2
3	30	6
4	40	3

```
Total frequency = 15
```

12. Length-1 intervals

```
C[0][1]=4, root=1
C[1][2]=2, root=2
C[2][3]=6, root=3
C[3][4]=3, root=4
```

13. Length-2 intervals $C[0][2]$: keys 10 ve 20

```
W[0][2]=4+2=6
```

Root 1:

```
C[0][0]+C[1][2]+6 = 0+2+6 = 8
```

Root 2:

```
C[0][1]+C[2][2]+6 = 4+0+6 = 10
```

```
C[0][2]=8, root=1
```

Diğer length-2 sonuçları

$C[1][3]=10$, root=3
 $C[2][4]=12$, root=3

14. Length-3 intervals $C[0][3]$: keys 10, 20, 30

$W[0][3]=4+2+6=12$

Candidates:

root 1: $C[0][0]+C[1][3]+12 = 0+10+12 = 22$
root 2: $C[0][1]+C[2][3]+12 = 4+6+12 = 22$
root 3: $C[0][2]+C[3][3]+12 = 8+0+12 = 20$

$C[0][3]=20$, root=3

$C[1][4]$: keys 20, 30, 40

$W[1][4]=2+6+3=11$

Candidates:

root 2: $0+12+11 = 23$
root 3: $2+3+11 = 16$
root 4: $10+0+11 = 21$

$C[1][4]=16$, root=3

15. Full interval $C[0][4]$

$W[0][4]=4+2+6+3=15$

Root candidate	Left cost	Right cost	Weight sum	Total
1	0	16	15	31
2	4	12	15	31
3	8	3	15	26
4	20	0	15	35

Minimum:

$C[0][4]=26$
root[0][4]=3

16. Tamamlanmış cost table

$C[a][b]$	0	1	2	3	4
0	0	4	8	20	26
1	–	0	2	10	16
2	–	–	0	6	12
3	–	–	–	0	3
4	–	–	–	–	0

Root table'ın ilgili entries'i:

root[a][b]	1	2	3	4
0	1	1	3	3
1	–	2	3	3
2	–	–	3	3
3	–	–	–	4

17. Optimal tree reconstruction Full interval:

root[0][4]=3 → K3=30

Left interval:

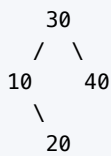
root[0][2]=1 → K1=10

root[1][2]=2 → K2=20

Right interval:

root[3][4]=4 → K4=40

Optimal tree:



18. Cost'u tree üzerinden doğrulama

Key	Frequency	Comparisons	Contribution
30	6	1	6
10	4	2	8
20	2	3	6
40	3	2	6

Total weighted cost = 6+8+6+6 = 26

Expected successful comparisons = 26/15 ≈ 1.733

19. Prefix sums ile weight hesabı Her interval frequency sum'ını tekrar tekrar loop ile hesaplamak gereksizdir.

```

prefix[0]=0
prefix[i]=prefix[i-1]+f[i]

```

```

W[a][b]=prefix[b]-prefix[a]

```

Böylece interval weight 0(1) zamanda bulunur.

20. Bottom-up pseudocode

```

OptimalBSTSuccessful(f[1..n])
  prefix[0] ← 0
  for i = 1 to n

```

```

prefix[i] <- prefix[i-1]+f[i]

for i = 0 to n
  C[i][i] <- 0

for length = 1 to n
  for a = 0 to n-length
    b <- a+length
    W <- prefix[b]-prefix[a]
    C[a][b] <- INF

    for r = a+1 to b
      candidate <- C[a][r-1]+C[r][b]+W

      if candidate < C[a][b]
        C[a][b] <- candidate
        root[a][b] <- r

return C[0][n], root

```

21. Complexity Intervals:

$\theta(n^2)$

Her interval için $\theta(n)$ root candidates denenebilir:

Time = $\theta(n^3)$
Space = $\theta(n^2)$

Prefix sums sayesinde $W[a][b]$ hesabı $O(1)$ dir.

Uygun assumptions altında Knuth Optimization root search ranges'ı daraltıp OBST'yi $O(n^2)$ time'a indirebilir; bu videonun standard cubic DP kapsamının dışındadır.

22. Successful-only modelin sınırı Gerçek search workload unsuccessful queries de içeriyorsa yalnız successful frequencies ile optimize edilen tree global expected cost için doğru model olmayabilir.

General OBST:

- successful probabilities $p[i]$
- keys arasındaki gaps için unsuccessful probabilities $q[i]$

kullanır. Base cases ve recurrence buna göre genişler.

23. Sık yapılan hatalar Optimal ile balanced'ı eş anlamlı sanmak Optimality frequency-weighted objective'e bağlıdır.

Her BST search'ünü $O(\log n)$ kabul etmek Skewed tree'de worst case $O(n)$ dir.

Interval indexing'i karıştırmak Videoda $C[a][b]$, $K(a+1) \dots K(b)$ keys'ini kapsar.

$W[a][b]$ term'ini unutmak Subtree nodes bir level aşağı indiği için interval frequencies toplamı eklenmelidir.

Root table tutmamak Minimum cost bulunur ancak actual tree reconstruct edilemez.

Unsuccessful searches'i bu modele dahil sanmak Bu ders yalnız successful-search frequencies'i kullanır.

24. Kavram sözlüğü

English term	Türkçe açıklama
Optimal Binary Search Tree	Weighted search cost'u minimum BST
Successful Search	Aranan key'in tree'de bulunduğu search
Search Frequency	Bir key'in kaç kez arandığını gösteren weight
Weighted Path Length	Frequency ile comparison depth çarpımlarının toplamı
Root Table	Her interval için optimal root index'ini saklayan tablo
Interval Weight	Bir interval'daki frequencies toplamı
Catalan Number	Possible BST shapes sayısını veren sequence
Prefix Sum	Interval sums'ı constant time bulmayı sağlayan cumulative array

25. Kendini sına

1. OBST hangi objective'i minimize eder?
2. Balanced BST her zaman optimal midir?
3. $C[a][b]$ videonun notation'ında hangi keys'i kapsar?
4. Recurrence'a $W[a][b]$ neden eklenir?
5. Örnekte optimal root hangi key'dir?
6. Minimum weighted cost nedir?
7. Optimal tree'de key 20'nin comparison count'u kaçtır?
8. Standard DP time ve space complexity nedir?
9. Bu ders unsuccessful searches'i hesaba katar mı?

Kısa cevaplar

1. $\sum f[i] * (\text{depth}[i] + 1)$ weighted successful-search cost'unu.
2. Hayır; frequencies farklıysa unbalanced bir shape daha ucuz olabilir.
3. $K(a+1) \dots K(b)$.
4. Left/right subtree keys final tree'de bir level daha derine indiği için.
5. $K3=30$.
6. 26.
7. 3.
8. $\theta(n^3)$ time ve $\theta(n^2)$ space.
9. Hayır; yalnız successful-search frequencies'i kullanır.

Sonraki ders: **57 – Optimal BST: Successful and Unsuccessful Probability.**

57 – Optimal BST: Successful and Unsuccessful Probability

Video: 4.6.1 Optimal BST – Successful and Unsuccessful Probability (57:00)

Bu videonun ana fikri General Optimal Binary Search Tree (OBST) modeli iki tür search olasılığını birlikte kullanır:

- $p[i]$: mevcut key $K[i]$ için successful-search probability
- $q[i]$: keys arasındaki gap için unsuccessful-search probability

Amaç successful ve unsuccessful searches'in toplam expected comparison cost'unu minimum yapan BST'yi kurmaktır.

Videonun boundary notation'ıyla:

```
C[i][j] = K(i+1)..K(j) interval'ı için minimum cost
W[i][j] = q[i] + p[i+1] + q[i+1] + ... + p[j] + q[j]
```

Recurrence:

```
C[i][j] = W[i][j]
        + min over i < k <= j of (
            C[i][k-1] + C[k][j]
        )
```

1. Successful ve unsuccessful search Successful Search, aranan value'nun tree'deki bir key'e eşit olmasıdır.

Unsuccessful Search, comparison path'in bir null child'da bitmesi ve key'in bulunmamasıdır.

Örneğin sorted keys:

```
10, 20, 30, 40
```

ise unsuccessful gaps:

```
q0: (-infinity, 10)
q1: (10, 20)
q2: (20, 30)
q3: (30, 40)
q4: (40, +infinity)
```

n keys için $n+1$ unsuccessful gaps vardır.

2. Dummy/external nodes neyi temsil eder? OBST çizimlerinde null children bazen Dummy Nodes veya External Nodes olarak gösterilir. Bunlar actual keys değildir; aynı null position'da sonuçlanan unsuccessful queries kümesini temsil eder.

Bir unsuccessful search'te dummy node ile key comparison yapılmaz. Cost, null'a ulaşana kadar karşılaştırılan internal key sayısıdır.

3. Probability dağılımı Gerçek probabilities kullanılıyorsa:

```
sum(p[1..n]) + sum(q[0..n]) = 1
```

olmalıdır.

Frequencies veya ortak bir constant ile ölçeklenmiş integer weights de kullanılabilir. Bütün weights aynı positive constant ile çarpılırsa optimal tree değişmez; yalnız total cost aynı factor ile çarpılır.

4. Bir tree'nin expected cost'u Successful key $K[i]$ için comparisons:

```
depth(K[i])+1
```

Unsuccessful gap $q[i]$ için comparisons, o null position'a ulaşmadan önce görülen internal nodes sayısıdır.
Expected cost:

```
sum_i p[i] * successfulComparisons(i)
+ sum_i q[i] * unsuccessfulComparisons(i)
```

Video external node'un drawn level'ından 1 çıkararak aynı quantity'yi hesaplar.

5. Neden yalnız balanced tree yetmez? Height önemli olsa da bütün queries eşit olasılıklı değildir. High-probability successful key veya gap'ı daha shallow yapmak, başka nodes'u biraz derinleştirmekten daha değerli olabilir.

Bu nedenle:

```
balanced != necessarily optimal
```

Optimality verilen p ve q distribution'ına göredir.

6. Brute force search space n sorted keys ile possible BST sayısı:

```
Catalan(n) = binomial(2n,n)/(n+1)
```

Bu sayı exponential büyür. DP bütün tree shapes'i açıkça üretmeden possible root decisions'ı interval subproblems üzerinden değerlendirir.

7. State ve interval convention Videoda table boundaries $0..n$ dir:

```
C[i][j] covers keys K(i+1),...,K(j)
```

Örnek:

```
C[0][1] -> K1
C[1][3] -> K2,K3
C[0][n] -> bütün keys
C[i][i] -> hiç key içermeyen gap qi
```

8. Base cases Empty key interval için key comparison yoktur:

```
C[i][i]=0
```

Fakat empty interval'ın probability weight'i vardır:

```
W[i][i]=q[i]
```

Bu ayrım önemlidir: gap probability recurrence'ın weight term'ine katılır fakat tek başına key comparison cost'u üretmez.

9. Weight recurrence

```
W[i][j] = W[i][j-1] + p[j] + q[j]
```

Base:

```
W[i][i]=q[i]
```

Equivalently prefix sums ile interval weight $O(1)$ zamanda hesaplanabilir.

10. Cost recurrence'ın türetilmesi

$K(i+1) \dots K(j)$ interval'ında root olarak $K[k]$ seçilsin:

$$i < k \leq j$$

Subtrees:

$$\begin{aligned} \text{left} &= K(i+1) \dots K(k-1) \rightarrow C[i][k-1] \\ \text{right} &= K(k+1) \dots K(j) \rightarrow C[k][j] \end{aligned}$$

Her left/right event final tree'de bir comparison daha derine taşındığı için interval'daki bütün successful ve unsuccessful weights bir kez eklenir:

$$C[i][j] = C[i][k-1] + C[k][j] + W[i][j]$$

Bütün possible k values denenerek minimum seçilir.

11. Principle of Optimality

Whole optimal tree root $K[k]$ yi kullanıyorsa left ve right subtrees kendi probability intervals'ı için optimal olmalıdır.

Daha ucuz bir subtree ile değiştirme whole cost'u azaltabiliyorsa original tree optimal değildir. Bu, recurrence'ın optimal substructure temelidir.

12. Videodaki scaled example

Keys:

$$K_1=10, K_2=20, K_3=30, K_4=40$$

Scaled successful weights:

$$p_1=3, p_2=3, p_3=1, p_4=1$$

Scaled unsuccessful weights:

$$q_0=2, q_1=3, q_2=1, q_3=1, q_4=1$$

Total:

$$3+3+1+1 + 2+3+1+1+1 = 16$$

Actual probabilities bu values'ın 16ya bölünmüş hâlidir.

13. Weight table

Diagonal:

$$\begin{aligned} W[0][0] &= 2 \\ W[1][1] &= 3 \\ W[2][2] &= 1 \\ W[3][3] &= 1 \\ W[4][4] &= 1 \end{aligned}$$

Tam upper-triangular table:

w[i][j]	0	1	2	3	4
0	2	8	12	14	16
1	–	3	7	9	11
2	–	–	1	3	5
3	–	–	–	1	3
4	–	–	–	–	1

Örneğin:

$$\begin{aligned} W[0][2] &= q_0 + p_1 + q_1 + p_2 + q_2 \\ &= 2 + 3 + 3 + 3 + 1 \\ &= 12 \end{aligned}$$

14. Length-1 costs Tek possible root vardır:

$$\begin{aligned} C[0][1] &= 8, \text{ root}=1 \\ C[1][2] &= 7, \text{ root}=2 \\ C[2][3] &= 3, \text{ root}=3 \\ C[3][4] &= 3, \text{ root}=4 \end{aligned}$$

Örneğin $C[0][1]$:

$$\begin{aligned} C[0][0] + C[1][1] + W[0][1] \\ &= 0 + 0 + 8 \\ &= 8 \end{aligned}$$

15. Length-2 costs $C[0][2]$

$$W[0][2] = 12$$

$$\begin{aligned} \text{root 1: } C[0][0] + C[1][2] + 12 &= 0 + 7 + 12 = 19 \\ \text{root 2: } C[0][1] + C[2][2] + 12 &= 8 + 0 + 12 = 20 \end{aligned}$$

$$C[0][2] = 19, \text{ root}=1$$

Diğer results:

$$\begin{aligned} C[1][3] &= 12, \text{ root}=2 \\ C[2][4] &= 8, \text{ root}=3 \end{aligned}$$

$C[2][4]$ te roots 3 ve 4 equal cost verebilir; video first minimum olan 3ü saklar.

16. Length-3 costs $C[0][3]$

$$W[0][3] = 14$$

$$\begin{aligned} \text{root 1: } 0 + C[1][3](12) + 14 &= 26 \\ \text{root 2: } 8 + C[2]3 + 14 &= 25 \\ \text{root 3: } 19 + 0 + 14 &= 33 \end{aligned}$$

$$C[0][3] = 25, \text{ root}=2$$

$C[1][4]$

$$W[1][4] = 11$$

root 2: $0 + 8 + 11 = 19$
 root 3: $7 + 3 + 11 = 21$
 root 4: $12 + 0 + 11 = 23$

$C[1][4]=19$, root=2

17. Full interval

$W[0][4]=16$

Root	Left + right	Weight	Total
1	$0+19=19$	16	35
2	$8+8=16$	16	32
3	$19+3=22$	16	38
4	$25+0=25$	16	41

$C[0][4]=32$
 $root[0][4]=2$

Normalize edilmiş expected cost:

$32/16 = 2$ comparisons

18. Cost ve root tables Cost:

$C[i][j]$	0	1	2	3	4
0	0	8	19	25	32
1	–	0	7	12	19
2	–	–	0	3	8
3	–	–	–	0	3
4	–	–	–	–	0

Root:

$root[i][j]$	1	2	3	4
0	1	1	2	2
1	–	2	2	2
2	–	–	3	3
3	–	–	–	4

19. Tree reconstruction

$root[0][4]=2 \rightarrow K2=20$

Left interval:

$root[0][1]=1 \rightarrow K1=10$

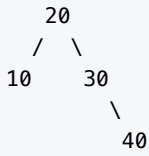
Right interval:

```

root[2][4]=3 -> K3=30
root[3][4]=4 -> K4=40

```

Optimal tree:



Dummy gap nodes tree'deki remaining null children'a bağlanır.

20. Bottom-up algorithm

```

OptimalBST(p[1..n],q[0..n])
  for i = 0 to n
    C[i][i] <- 0
    W[i][i] <- q[i]

  for length = 1 to n
    for i = 0 to n-length
      j <- i+length
      W[i][j] <- W[i][j-1]+p[j]+q[j]
      C[i][j] <- INF

      for k = i+1 to j
        candidate <- C[i][k-1]+C[k][j]+W[i][j]

        if candidate < C[i][j]
          C[i][j] <- candidate
          root[i][j] <- k

  return C[0][n], root

```

21. Complexity

```

States =  $\theta(n^2)$ 
root candidates per state =  $O(n)$ 
Time =  $\theta(n^3)$ 
Space =  $\theta(n^2)$ 

```

Weight table ayrı saklanmadan prefix sums ile de $O(1)$ interval weight hesaplanabilir.

Appropriate quadrangle/monotonicity properties sayesinde Knuth Optimization time'ı $O(n^2)$ ye indirebilir; standard video algorithm'ı cubic'tir.

22. Bir önceki successful-only modelden farkı

Özellik	Successful-only	General OBST
Existing keys	$p[i]$ /frequency	$p[i]$
Missing keys	Hesaba katılmaz	$q[0..n]$ ile modellenir
Empty interval weight	0	$q[i]$
Objective	Successful searches	Successful + unsuccessful searches

23. Sık yapılan hatalar n keys için yalnız n q values kullanmak

Gaps sayısı $n+1$ dir.

$C[i][i]$ ile $W[i][i]$ yi aynı sanmak

$C[i][i]=0$
 $W[i][i]=q[i]$

Weight term'den q values'ı çıkarmak Her interval hem successful hem unsuccessful probability mass içerir.

Scaled cost 32yi doğrudan expected comparisons sanmak Weights toplamı 16 olduğundan normalize cost $32/16=2$ dir.

Root indices'i actual key values ile karıştırmak root=2, actual key $K_2=20$ demektir.

External node'da ayrıca key comparison yapıldığını sanmak Unsuccessful comparison count null'a ulaşmadan önce görülen internal keys sayısıdır.

24. Kavram sözlüğü

English term	Türkçe açıklama
Successful Probability	Mevcut bir key'in arama olasılığı $p[i]$
Unsuccessful Probability	Bir gap'te bulunmayan value arama olasılığı $q[i]$
Dummy Node	Unsuccessful search gap'ini temsil eden external/null node
Probability Mass	Bir interval'a ait toplam $p+q$ weight'i
Expected Cost	Olasılıklarla ağırlıklandırılmış comparison sayısı
Root Table	Her interval'ın optimal root index'ini saklayan table
Scaled Weight	Ortak factor ile büyütülmüş probability/frequency
Normalization	Weights'i toplamlarına bölerek probabilities'e çevirme

25. Kendini sinama

1. n keys için kaç unsuccessful gap vardır?
2. q_0 ve q_n hangi ranges'i temsil eder?
3. $C[i][i]$ ve $W[i][i]$ values'ı nedir?
4. Recurrence'ta root k hangi aralıktadır?
5. Videodaki scaled example'ın minimum cost'u nedir?
6. Normalize expected cost nedir?
7. Optimal root ve actual key nedir?
8. Standard DP complexity nedir?
9. Successful-only modelden temel fark nedir?

Kısa cevaplar

1. $n+1$.
2. Sırasıyla K_1 den küçük ve K_n den büyük values'ı.
3. $C[i][i]=0$, $W[i][i]=q[i]$.
4. $i < k \leq j$.
5. 32 scaled units.
6. $32/16=2$ comparisons.
7. root=2, yani $K_2=20$.
8. $\theta(n^3)$ time ve $\theta(n^2)$ space.
9. Missing-key searches'in q probabilities ile objective'e eklenmesi.

Sonraki ders: **58 – Traveling Salesman Problem Using DP Formula.**

58 – Traveling Salesman Problem Using DP Formula

Video: 4.7 Traveling Salesman Problem Using DP Formula (17:18)

Bu videonun ana fikri Traveling Salesman Problem (TSP), bir start vertex'ten başlayıp diğer bütün vertices'i exactly once ziyaret ederek start'a dönen minimum-cost tour'u arar.

Dynamic Programming state'i:

$g(i, S)$ = şu anda i vertex'indeyken,
S set'indeki vertices'i ziyaret edip start'a dönmenin minimum cost'u

Recurrence:

$g(i, S) = \min \text{ over } k \text{ in } S \text{ of } (\text{cost}[i][k] + g(k, S - \{k\}))$

Base case:

$g(i, \text{empty}) = \text{cost}[i][\text{start}]$

Bu yöntem Held-Karp Dynamic Programming olarak bilinir.

1. TSP problem tanımı Bir valid tour:

- Start'tan çıkar.
- Her vertex'i exactly once ziyaret eder.
- Start vertex'e geri döner.
- Total edge cost'u minimize eder.

Bu, minimum Hamiltonian Cycle problemidir.

2. Videodaki directed cost matrix

	1	2	3	4
1	0	10	15	20
2	5	0	9	10
3	6	13	0	12
4	8	8	9	0

Matrix symmetric değildir:

$\text{cost}[1][2] = 10$
 $\text{cost}[2][1] = 5$

Dolayısıyla example Asymmetric/Directed TSPdir.

3. Start vertex neden sabitlenebilir? Bir cycle farklı vertices'ten yazılabilir fakat aynı cyclic edge sequence'i temsil eder. Rotational duplicates'i kaldırmak için start 1 olarak sabitlenir.

Initial problem:

$g(1, \{2, 3, 4\})$

4. State'in anlamı State iki bilgiye ihtiyaç duyar:

current vertex i
unvisited set S

Yalnız current vertex yeterli değildir; daha önce hangi vertices'in ziyaret edildiği future legal decisions'ı değiştirir.

5. Principle of Optimality Optimal $g(i, S)$ solution'ı ilk olarak kye gidiyorsa kalan route:

$$g(k, S - \{k\})$$

subproblem'i için optimal olmalıdır. Daha ucuz bir suffix bulunsaydı whole tour'daki suffix değiştirilerek total cost azaltılabilirdi.

6. Recurrence expansion Start state:

$$g(1, \{2, 3, 4\}) = \min(\begin{aligned} &c_{12} + g(2, \{3, 4\}), \\ &c_{13} + g(3, \{2, 4\}), \\ &c_{14} + g(4, \{2, 3\}) \end{aligned})$$

Bir alt state:

$$g(2, \{3, 4\}) = \min(\begin{aligned} &c_{23} + g(3, \{4\}), \\ &c_{24} + g(4, \{3\}) \end{aligned})$$

7. Base states Ziyaret edilecek vertex kalmadığında start'a dönülür:

$$\begin{aligned} g(2, \text{empty}) &= c_{21} = 5 \\ g(3, \text{empty}) &= c_{31} = 6 \\ g(4, \text{empty}) &= c_{41} = 8 \end{aligned}$$

Base value zero değildir; tour'un closing edge cost'u eklenmelidir.

8. Tek elemanlı sets

$$\begin{aligned} g(2, \{3\}) &= c_{23} + g(3, \text{empty}) = 9 + 6 = 15 \\ g(2, \{4\}) &= c_{24} + g(4, \text{empty}) = 10 + 8 = 18 \\ g(3, \{2\}) &= c_{32} + g(2, \text{empty}) = 13 + 5 = 18 \\ g(3, \{4\}) &= c_{34} + g(4, \text{empty}) = 12 + 8 = 20 \\ g(4, \{2\}) &= c_{42} + g(2, \text{empty}) = 8 + 5 = 13 \\ g(4, \{3\}) &= c_{43} + g(3, \text{empty}) = 9 + 6 = 15 \end{aligned}$$

9. İki elemanlı sets $g(2, \{3, 4\})$

$$\begin{aligned} \text{via 3: } &9 + g(3, \{4\}) = 20 \rightarrow 29 \\ \text{via 4: } &10 + g(4, \{3\}) = 15 \rightarrow 25 \end{aligned}$$

$$g(2, \{3, 4\}) = 25, \text{ next} = 4$$

$g(3, \{2, 4\})$

$$\begin{aligned} \text{via 2: } &13 + g(2, \{4\}) = 18 \rightarrow 31 \\ \text{via 4: } &12 + g(4, \{2\}) = 13 \rightarrow 25 \end{aligned}$$

$$g(3, \{2, 4\}) = 25, \text{ next} = 4$$

$g(4, \{2, 3\})$

```
via 2: 8 + g(2,{3})=15 -> 23
via 3: 9 + g(3,{2})=18 -> 27
```

```
g(4,{2,3})=23, next=2
```

10. Final state

```
via 2: c12 + g(2,{3,4}) = 10+25 = 35
via 3: c13 + g(3,{2,4}) = 15+25 = 40
via 4: c14 + g(4,{2,3}) = 20+23 = 43
```

```
g(1,{2,3,4})=35
next=2
```

11. Optimal tour reconstruction

Stored decisions:

```
(1,{2,3,4}) -> 2
(2,{3,4})    -> 4
(4,{3})      -> 3
(3,empty)    -> start 1
```

Tour:

```
1 -> 2 -> 4 -> 3 -> 1
```

Cost:

```
10+10+9+6=35
```

12. Top-down memoization

```
TSP(i,S)
  if S is empty
    return cost[i][start]

  if memo contains (i,S)
    return memo[i,S]

  answer <- INF
  for each k in S
    candidate <- cost[i][k]+TSP(k,S-{k})
    if candidate < answer
      answer <- candidate
      next[i,S] <- k

  memo[i,S] <- answer
  return answer
```

Without memoization aynı (i,S) states farklı prefixes üzerinden tekrar hesaplanır.

13. Bitmask representation

Subset S, integer bitmask ile saklanabilir:

```
bit k is 1 -> vertex k hâlâ set'te
```

Operations:

```
membership: mask & (1<<k)
remove k:    mask ^ (1<<k)
empty set:   mask == 0
```

Actual indexing implementation'a göre bit positions ayarlanmalıdır.

14. Bottom-up table order Base mask=0 states önce doldurulur. Sonra increasing subset size ile:

```
size 1 -> size 2 -> ... -> size n-1
```

Her state daha küçük set'e bağlı olduğu için bu order geçerlidir.

15. Complexity Kabaca:

```
states = 0(n*2^n)
transitions per state = 0(n)
Time = 0(n^2*2^n)
Space = 0(n*2^n)
```

Bu brute-force $\theta(n!)$ enumeration'dan önemli ölçüde iyidir fakat hâlâ exponential'dır. DP, TSP'yi polynomial-time probleme dönüştürmez; exact TSP NP-harddır.

16. Missing edges ve INF Graph complete değilse olmayan edge:

```
cost[i][j]=INF
```

olarak ele alınabilir. Candidate hesaplanmadan önce edge ve suffix'in finite olduğu kontrol edilmelidir.

Final state INF kalırsa Hamiltonian tour yoktur.

17. Negative edges TSP tour her vertex'i exactly once ziyaret ettiğinden arbitrary cycle tekrarları yapılmaz. Negative edges tek başına recurrence'ı bozmaz; fixed-length Hamiltonian tours arasında minimum aranabilir.

Bu, ordinary shortest-path problemindeki reachable negative-cycle sorunundan farklıdır.

18. Sık yapılan hatalar Base case'i zero yapmak Start'a dönüş edge'i unutulur.

Visited/unvisited set'i state'e koymamak Hangi next vertices'in legal olduğu bilinemez.

Seçilen kyi set'ten çıkarmamak Aynı vertex tekrar ziyaret edilebilir.

Directed matrix'i symmetric sanmak $c[i][j]$ ve $c[j][i]$ farklı olabilir.

DP complexity'sini polynomial sanmak Subset sayısı 2^n dir.

Yalnız cost saklayıp tour beklemek Reconstruction için next/parent decisions gerekir.

19. Kavram sözlüğü

English term	Türkçe açıklama
Traveling Salesman Problem	Bütün vertices'i bir kez ziyaret eden minimum cycle problemi
Hamiltonian Cycle	Her vertex'i exactly once içeren cycle
Held-Karp Algorithm	Subset DP kullanan exact TSP algorithm'ı
Unvisited Set	Henüz ziyaret edilmesi gereken vertices kümesi
Bitmask	Subset'i integer bits ile temsil etme
Closing Edge	Son vertex'ten start'a dönen edge
Asymmetric TSP	Direction'a göre costs'un farklı olabildiği TSP
Tour Reconstruction	Stored decisions'dan ordered cycle çıkarma

20. Kendini sına

1. $g(i, S)$ neyi temsil eder?
2. Base case neden $cost[i][start]$ tır?
3. Recurrence'ta k seçildikten sonra set nasıl değişir?
4. Videodaki minimum tour cost'u nedir?
5. Optimal tour hangisidir?
6. Cost matrix symmetric midir?
7. Held-Karp time ve space complexity nedir?
8. DP neden yine exponential'dır?

Kısa cevaplar

1. iden başlayıp S yi ziyaret ederek start'a dönmenin minimum cost'unu.
2. Tour'un start'a dönerek kapanması gerektiği için.
3. $S - \{k\}$ olur.
4. 35.
5. $1 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 1$.
6. Hayır.
7. $O(n^2 2^n)$ time ve $O(n 2^n)$ space.
8. 2^n subsets bulunduğu için.

Sonraki ders: **59 – Reliability Design.**

59 – Reliability Design

Video: 4.8 Reliability Design (26:32)

Bu videonun ana fikri Reliability Design Problem, series-connected stages'den oluşan bir system'in reliability'sini limited budget altında maximize eder. Her stage'de aynı device'ın parallel redundant copies'i kullanılabilir.

Bir device copy reliability'si $r[i]$, copy sayısı m ise stage reliability:

$$R_i(m) = 1 - (1 - r[i])^m$$

System stages series olduğunda:

$$R_{\text{system}} = \text{product over stages } i \text{ of } R_i(m_i)$$

Constraint:

$$\begin{aligned} \sum c[i] * m_i &\leq \text{budget} \\ m_i &\geq 1 \text{ integer} \end{aligned}$$

1. Series system neden product kullanır? System'in çalışması için bütün required stages'in çalışması gerekir. Stage failures independent varsayılsa:

$$P(\text{all stages work}) = \text{product}(\text{stage reliabilities})$$

Tek bir required stage tamamen fail olursa system fail olur.

2. Parallel redundancy neden reliability'yi artırır? Bir copy'nin failure probability'si:

$$1 - r$$

m independent copies'in hepsinin fail olma probability'si:

$$(1 - r)^m$$

En az birinin çalışma probability'si:

$$1 - (1 - r)^m$$

Örnek $r=0.9$, $m=3$:

$$1 - (0.1)^3 = 0.999$$

3. Model assumptions Formüller şu assumptions'a dayanır:

- Copies aynı reliability'ye sahiptir.
- Copy failures independent'dir.
- Bir stage'in çalışması için en az bir copy'nin çalışması yeterlidir.
- Bütün stages system için required'dir.
- Costs additive'dir.

Common-cause failures veya dependency varsa $1 - (1 - r)^m$ optimistic olabilir; farklı reliability model gerekir.

4. Videodaki example

Device stage	Unit cost	Single-copy reliability
1	30	0.9
2	15	0.8
3	20	0.5

Budget = 105

En az birer copy zorunlu:

minimum cost = 30+15+20 = 65
remaining budget = 105-65 = 40

5. Copy upper bounds Diğer stages'in minimum birer copy cost'u ayrıldıktan sonra, bütün remaining budget stage iye verilirse:

$u_i = \text{floor}((\text{budget} - \sum_j c[j]) / c[i]) + 1$

Example:

$u_1 = \text{floor}(40/30) + 1 = 2$
 $u_2 = \text{floor}(40/15) + 1 = 3$
 $u_3 = \text{floor}(40/20) + 1 = 3$

Search ranges:

$m_1 \in \{1, 2\}$
 $m_2 \in \{1, 2, 3\}$
 $m_3 \in \{1, 2, 3\}$

6. Frontier/Set Method state'i Her partial design ordered pair ile tutulur:

(reliability, cost)

Başlangıç:

$S_0 = \{(1, 0)\}$

Reliability identity 1, cost identity 0'dır:

$\text{newReliability} = \text{oldReliability} * \text{stageReliability}$
 $\text{newCost} = \text{oldCost} + \text{stageCost}$

Counts reconstruction için state içinde predecessor ve chosen copy count da saklanmalıdır.

7. Stage 1 states

$m_1=1$:
 $R_1=0.9$
 $\text{cost}=30$
 $\rightarrow (0.9, 30)$

$m_1=2$:
 $R_1=1-(0.1)^2=0.99$
 $\text{cost}=60$
 $\rightarrow (0.99, 60)$

$$S1 = \{(0.9, 30), (0.99, 60)\}$$

8. Stage 2 reliabilities

$$\begin{aligned} R2(1) &= 0.8 \\ R2(2) &= 1 - (0.2)^2 = 0.96 \\ R2(3) &= 1 - (0.2)^3 = 0.992 \end{aligned}$$

Feasible combinations, stage 3 için minimum 20 cost da reserve edilerek oluşturulur.

Raw useful candidates:

$$\begin{aligned} (0.9 \times 0.8, \quad 30+15) &= (0.72, 45) \\ (0.99 \times 0.8, \quad 60+15) &= (0.792, 75) \\ (0.9 \times 0.96, \quad 30+30) &= (0.864, 60) \\ (0.9 \times 0.992, \quad 30+45) &= (0.8928, 75) \end{aligned}$$

Cost 90 veya 105 olan partial choices stage 3'ün mandatory copy'sine yer bırakmadığı için erken elenebilir.

9. Dominance rule

State $A=(R_A, C_A)$, $B=(R_B, C_B)$ tarafından dominate edilir eğer:

$$\begin{aligned} R_B &\geq R_A \\ C_B &\leq C_A \end{aligned}$$

ve en az bir inequality strict ise.

Örnek:

$$(0.792, 75) \text{ is dominated by } (0.864, 60)$$

İkinci design hem daha reliable hem daha ucuzdur.

Pruning sonrası:

$$S2 = \{(0.72, 45), (0.864, 60), (0.8928, 75)\}$$

10. Stage 3 options

$$\begin{aligned} R3(1) &= 0.5 \\ R3(2) &= 1 - (0.5)^2 = 0.75 \\ R3(3) &= 1 - (0.5)^3 = 0.875 \end{aligned}$$

Bir-copy candidates:

$$\begin{aligned} (0.36, 65) \\ (0.432, 80) \\ (0.4464, 95) \end{aligned}$$

İki-copy feasible candidates:

$$\begin{aligned} (0.54, 85) \\ (0.648, 100) \end{aligned}$$

Üç-copy feasible candidate:

$$(0.63, 105)$$

Dominance:


```
(0.4464,95) dominated by (0.54,85)
(0.63,105) dominated by (0.648,100)
```

Final Pareto frontier:

```
S3={
  (0.36,65),
  (0.432,80),
  (0.54,85),
  (0.648,100)
}
```

11. Optimal design

Maximum reliability:

```
R_system=0.648
total cost=100
```

Choices:

```
m1=1
m2=2
m3=2
```

Verification:

```
R1(1)=0.9
R2(2)=0.96
R3(2)=0.75

0.9*0.96*0.75=0.648
30+2*15+2*20=100
```

Budget'in tamamını kullanmak zorunlu değildir; 5 units unused kalır.

12. Reconstruction

Her generated state için şu bilgiler saklanabilir:

```
previousState
stageIndex
chosenCopies
```

Final (0.648,100) state'inden predecessors izlenerek:

```
stage 3 -> 2 copies
stage 2 -> 2 copies
stage 1 -> 1 copy
```

bulunur.

13. Budget-indexed DP formulation

Integer budget için alternatif state:

```
dp[i][b] = ilk i stages, cost en fazla b iken maximum reliability
```

Recurrence:

```
dp[i][b] = max over 1 <= m <= u_i and m*c[i] <= b of (
  dp[i-1][b-m*c[i]] * (1-(1-r[i])^m)
)
```

Base:

```
dp[0][0]=1
other unreachable states marked invalid
```

Bu formulation bounded integer resource allocation problemidir.

14. Log-domain computation Çok sayıda stages olduğunda reliability products floating-point underflow'a yaklaşabilir. Product maximize etmek equivalent olarak log sum maximize etmektir:

```
maximize sum log(R_i(m_i))
```

$R_i(m) > 0$ için logarithm monotonic olduğundan optimum değişmez.

15. Complexity Set/frontier method'ın performance'ı nondominated state sayısına bağlıdır. Dominance practice'te güçlü pruning yapılabilir fakat worst case'de frontier exponential büyüyebilir.

Budget-indexed DP için, maximum copy-choice count U ile kaba bound:

```
Time = O(n*budget*U)
Space = O(n*budget)
```

Rolling rows ile value-only space $O(\text{budget})$ olabilir. Numeric budget'a bağlı olduğu için bu yaklaşım pseudo-polynomial'dır.

16. Floating-point karşılaştırma Computed reliabilities exact decimal olmayabilir. Dominance/equality checks için blindly == kullanmak yerine tolerance veya log-domain values düşünülmelidir.

Ancak tolerance çok büyük seçilirse gerçekten farklı designs yanlışlıkla merge edilebilir.

17. 0/1 Knapsack ile ilişki Set merging ve dominance pruning fikri Knapsack frontier'ına benzer. Fakat bu problem tam olarak 0/1 değildir:

- Her stage zorunludur.
- Her stage için bounded multiple copy count seçilir.
- Objective additive profit değil multiplicative reliability'dir.

Bu nedenle Bounded Resource Allocation olarak düşünmek daha doğrudur.

18. Sık yapılan hatalar Parallel reliability'yi $m \cdot r$ sanmak Probability 1'i aşamaz; doğru formula $1 - (1-r)^m$ dir.

Independence assumption'ı söylememek Parallel formula bağımsız failures varsayar.

Stage reliabilities'i toplamak Series system için multiply edilir.

Her stage'den en az bir copy constraint'ini unutmak Başlangıç minimum budget ve upper bounds yanlış çıkar.

Dominance yönünü ters uygulamak Higher reliability ve lower cost sunan state korunur.

Budget'in mutlaka tamamen harcanacağını sanmak Optimum cost budget'dan küçük olabilir.

19. Kavram sözlüğü

English term	Türkçe açıklama
Reliability	Component/system'in doğru çalışma probability'si
Series System	Çalışmak için bütün stages'in çalışması gereken system
Parallel Redundancy	En az bir copy çalışırsa stage'in çalıştığı yedeklilik
Failure Probability	Component'in fail olma olasılığı $1-r$
Copy Upper Bound	Budget altında bir stage için alınabilecek maximum copy count
Dominance Rule	Daha pahalı ve daha az reliable state'i eleme
Pareto Frontier	Cost/reliability bakımından nondominated designs
Common-Cause Failure	Multiple copies'i birlikte etkileyen ortak failure

20. Kendini sinama

1. m parallel copies'in stage reliability formula'sı nedir?
2. Series stages'in system reliability'si nasıl bulunur?
3. Örnekte minimum one-copy cost nedir?
4. Copy upper bounds nelerdir?
5. $(0.792, 75)$ neden elenir?
6. Optimal copy counts nelerdir?
7. Optimal reliability ve cost nedir?
8. Frontier method'ın worst-case davranışı nasıldır?
9. Independence neden önemlidir?

Kısa cevaplar

1. $1-(1-r)^m$.
2. Stage reliabilities multiply edilerek.
3. 65.
4. $[2, 3, 3]$.
5. $(0.864, 60)$ daha reliable ve daha ucuz olduğu için.
6. $[1, 2, 2]$.
7. 0.648 ve 100.
8. Exponential sayıda nondominated state oluşabilir.
9. Parallel failure formula'sının geçerliliği buna bağlıdır.

Sonraki ders: **60 – Longest Common Subsequence: Recursion and DP.**

60 – Longest Common Subsequence: Recursion and DP

Video: 4.9 Longest Common Subsequence – Recursion and DP (23:35)

Kaynak notu: Videonun English caption track'ı mevcuttur ancak uzun zaman aralıklarında caption text boş bırakılmıştır. Aşağıdaki ders, captionlarda görülen recursion/memoization/table kapsamı temel alınarak standard LCS recurrence, doğrulanmış örnek ve complexity ayrıntılarıyla tamamlanmıştır.

Bu videonun ana fikri Longest Common Subsequence (LCS), iki sequence içinde relative order'ı bozmadan bulunan en uzun ortak subsequence'ı arar.

Prefix-state recurrence:

$L[i][j]$ = X'in ilk i karakteri ile Y'nin ilk j karakterinin LCS length'i

```
if i==0 or j==0:
    L[i][j]=0
else if X[i-1]==Y[j-1]:
    L[i][j]=1+L[i-1][j-1]
else:
    L[i][j]=max(L[i-1][j],L[i][j-1])
```

1. Subsequence nedir? Bir sequence'den bazı elements silinip remaining order korunarak subsequence elde edilir.

Sequence: A B C D E
Subsequence: A C E

Elements contiguous olmak zorunda değildir.

2. Substring ile farkı Substring veya Subarray contiguous olmalıdır; Subsequence için yalnız order korunur.

ABCDE içinde:

BCD → substring ve subsequence
ACE → subsequence, substring değil

Longest Common Substring farklı recurrence gerektiren başka problemidir.

3. Common subsequence Bir sequence hem X'in hem Y'nin subsequence'ı ise Common Subsequencedir. Birden fazla common subsequence aynı maximum length'e sahip olabilir. LCS value'su length olabilir; actual LCS string unique olmak zorunda değildir.

4. Recursive state – suffix notation Top-down yaklaşımda:

$LCS(i,j)$ = X[i..] ve Y[j..] suffix'lerinin LCS length'i

Base:

```
if i==len(X) or j==len(Y): return 0
```

Match:

```
if X[i]==Y[j]:
    return 1+LCS(i+1,j+1)
```

Mismatch:

```
return max(
    LCS(i+1,j),
    LCS(i,j+1)
)
```

Prefix ve suffix formulations aynı problem için farklı indexing conventions'dır; karıştırılmamalıdır.

5. Match case neden diagonal ilerler? $X[i] == Y[j]$ olduğunda bu character ortak subsequence'e eklenebilir. İki sequence'de de kullanıldığı için ikisinin index'i ilerler:

```
1+LCS(i+1,j+1)
```

Equivalent bottom-up prefix transition:

```
1+L[i-1][j-1]
```

6. Mismatch case neden iki branch üretir? $X[i] != Y[j]$ olduğunda ikisi aynı LCS position'ında birlikte kullanılamaz. Optimal solution en az birini dışarıda bırakır:

```
skip X[i]
skip Y[j]
```

İki possibility'nin maximum'u alınır.

7. Naive recursion ve overlapping subproblems Mismatch'ler recursion tree'yi ikiye böler. Farklı paths aynı (i,j) state'ine tekrar ulaşır.

Örneğin:

```
LCS(i+1,j+1)
```

hem $LCS(i+1,j)$ hem $LCS(i,j+1)$ branches'inden yeniden doğabilir.

Naive runtime worst case'de exponential'dır; kaba upper bound:

```
 $O(2^{(m+n)})$ 
```

Recursion stack depth'i $O(m+n)$ olabilir.

8. Memoization Her (i,j) result bir kez hesaplanıp cache'lenir:

```
MemoLCS(i,j)
    if i==m or j==n
        return 0

    if memo[i][j] known
        return memo[i][j]

    if X[i]==Y[j]
        memo[i][j] <- 1+MemoLCS(i+1,j+1)
    else
        memo[i][j] <- max(
            MemoLCS(i+1,j),
            MemoLCS(i,j+1)
        )

    return memo[i][j]
```

Possible states yalnız $m \times n$ tane dir:

Time = $\theta(mn)$
Space = $\theta(mn)$ + recursion stack

9. Tabulation Bottom-up table dimensions:

$(m+1) \times (n+1)$

Zero row ve zero column empty prefixes'i temsil eder.

```
for i = 1 to m
  for j = 1 to n
    if X[i-1]==Y[j-1]
      L[i][j] <- 1+L[i-1][j-1]
    else
      L[i][j] <- max(L[i-1][j], L[i][j-1])
```

Bu Bottom-Up Dynamic Programmingdir. Memoization ise top-down recursion + cache yaklaşımıdır; iki terim aynı şey değildir.

10. Açıklayıcı örnek

X = ABCBDAB
Y = BDCABA

DP table:

L	∅	B	D	C	A	B	A
∅	0	0	0	0	0	0	0
A	0	0	0	0	1	1	1
B	0	1	1	1	1	2	2
C	0	1	1	2	2	2	2
B	0	1	1	2	2	3	3
D	0	1	2	2	2	3	3
A	0	1	2	2	3	3	4
B	0	1	2	2	3	4	4

LCS length = $L[7][6] = 4$

Possible LCS strings:

BCBA
BDAB
BCAB

Bu example multiple optimal answers olduğunu gösterir.

11. Table cell örnekleri $X[2]=C, Y[3]=C$ match olduğunda:

$L[3][3]=1+L[2][2]=2$

Son characters mismatch olduğunda:

```
L[7][6]=max(L[6][6],L[7][5])
          =max(4,4)
          =4
```

Bu equality reconstruction sırasında branch seçimi doğurur.

12. Bir LCS'i reconstruct etme

Bottom-right'tan başlanır:

```
i=m, j=n
```

```
if X[i-1]==Y[j-1]
    character'ı answer'a ekle
    i--, j--
else if L[i-1][j] > L[i][j-1]
    i--
else
    j--
```

Characters reverse order'da toplanacağı için sonunda reverse edilir.

Tie'da seçilen yön hangi valid LCS'in üretileceğini belirler.

13. Bütün LCS strings'i üretme

Mismatch ve:

```
L[i-1][j] == L[i][j-1]
```

ise her iki predecessor optimum olabilir. Bütün distinct LCS strings için iki branch izlenip duplicates deduplicate edilmelidir.

Output sayısı exponential olabileceğinden bütün answers'ı üretmenin runtime'ı da exponential olabilir; DP length hesabı $\theta(mn)$ olsa bile output maliyeti kaybolmaz.

14. Space optimization

Length-only computation'da current row yalnız previous row'a bağlıdır:

```
Space =  $\theta(\min(m,n))$ 
```

İki rows veya dikkatli one-dimensional update kullanılabilir.

Straightforward reconstruction full $\theta(mn)$ table ister. Hirschberg's Algorithm, divide-and-conquer ile $\theta(mn)$ time ve $\theta(\min(m,n))$ space kullanarak bir LCS reconstruct edebilir.

15. Complexity

Yaklaşım	Time	Space
Naive recursion	Exponential	$\theta(m+n)$ stack
Memoization	$\theta(mn)$	$\theta(mn)$ + stack
Full tabulation	$\theta(mn)$	$\theta(mn)$
Length-only rolling rows	$\theta(mn)$	$\theta(\min(m,n))$

16. Empty strings ve edge cases

```
LCS("",Y)=0
LCS(X,"")=0
```

Repeated characters multiple alignments oluşturabilir. Character comparison case-sensitive ise A ve a farklıdır; normalization problem contract'ına bağlıdır.

Unicode text'te "character"ın code point mi grapheme cluster mı olduğu ayrıca belirlenmelidir.

17. LCS uygulamaları

- Version-control diff mantığı
- File comparison
- DNA/protein sequence similarity
- Edit alignment
- Similar ordered event sequences bulma

Shortest Common Supersequence length'i de LCS length ile ilişkilidir:

$$\text{SCS length} = m+n-\text{LCS length}$$

18. Sık yapılan hatalar Subsequence'i substring sanmak LCS characters contiguous olmak zorunda değildir.

Match'te yalnız bir index ilerletmek Matched character iki sequence'de de tüketilmelidir.

Mismatch'te diagonal'a gitmek Doğru choices up ve left states'tir.

Memoization ile tabulation'ı eş anlamlı kullanmak Biri top-down cache, diğeri bottom-up table'dır.

Table dimensions'i m x n almak Empty-prefix base row/column için (m+1)x(n+1) gerekir.

Length'i bulunca unique LCS olduğunu sanmak Multiple strings aynı maximum length'e sahip olabilir.

19. Kavram sözlüğü

English term	Türkçe açıklama
Subsequence	Relative order korunarak bazı elements'in silindiği sequence
Substring	Contiguous character block'u
Longest Common Subsequence	İki sequence'in en uzun ortak subsequence'i
Overlapping Subproblem	Farklı recursion branches'de tekrar oluşan aynı state
Memoization	Top-down results cache'i
Tabulation	Bottom-up DP table doldurma
Reconstruction	Length table'dan actual LCS çıkarma
Tie	Birden çok predecessor'ın aynı optimal length'i vermesi

20. Kendini sinama

1. Subsequence ile substring arasındaki fark nedir?
2. $L[i][j]$ neyi tutar?
3. Characters match olduğunda transition nedir?
4. Mismatch olduğunda hangi states karşılaştırılır?
5. Naive recursion neden yavaştır?
6. Memoized/tabulated complexity nedir?
7. Örneğin LCS length'i kaçtır?
8. Multiple LCS nasıl ortaya çıkar?
9. Length-only space ne kadar düşürülebilir?

Kısa cevaplar

1. Subsequence contiguous olmak zorunda değildir; substring zorundadır.
2. İlk i ve j characters'ın LCS length'ini.
3. $1+L[i-1][j-1]$.

4. $L[i-1][j]$ ve $L[i][j-1]$.
5. Aynı (i, j) subproblems'i tekrar hesapladığı için.
6. $\theta(mn)$ time ve full table için $\theta(mn)$ space.
7. 4.
8. Equal optimal predecessor choices olduğunda.
9. $\theta(\min(m, n))$.

Sonraki ders: **61 – Graph Traversals: BFS and DFS.**

61 – Graph Traversals: BFS and DFS

Video: 5.1 Graph Traversals – BFS and DFS (18:31)

Bu videonun ana fikri Breadth-First Search (BFS) ve Depth-First Search (DFS), graph vertices'ini sistematik biçimde ziyaret eden iki temel traversal yöntemidir.

- BFS, keşfedilen vertices'i Queue ile level-by-level işler.
- DFS, bir path boyunca mümkün olduğunca derine gider; Recursion veya Stack kullanır.

Adjacency list ile ikisinin de complexity'si:

```
Time =  $\Theta(V+E)$   
Space =  $\Theta(V)$ 
```

1. Visit ve explore ayrımı Videonun kullandığı iki terim:

- Visit/Discover: Vertex'e ilk kez ulaşmak ve visited işaretlemek
- Explore/Process: Vertex'in adjacency list'indeki edges'i incelemek

Bir vertex yalnız bir kez discover edilmeli, adjacency list'i bir kez explore edilmelidir.

2. Traversal order neden unique değildir? Order şu choices'a bağlıdır:

- Hangi start vertex seçildiği
- Adjacency lists'in hangi sırada tutulduğu
- Equal-level/equal-depth neighbors'ın hangi sırada işlendiği

Aynı graph için birden fazla valid BFS ve DFS order bulunabilir. Algorithm rule'u sabittir; tie order değişebilir.

3. BFS intuition BFS start'tan distance layers biçiminde ilerler:

```
distance 0 -> start  
distance 1 -> start'ın unvisited neighbors'ı  
distance 2 -> onların yeni neighbors'ı  
...
```

Tree üzerinde BFS, Level-Order Traversal ile aynıdır.

4. Queue neden gerekir? First discovered vertex first explored edilmelidir:

```
FIFO = First In, First Out
```

Bu order, önce yakın layers'ın tamamen işlenmesini sağlar.

5. BFS pseudocode

```
BFS(G, start)  
  for each vertex v  
    visited[v] <- false  
    distance[v] <- INF  
    parent[v] <- NIL  
  
  visited[start] <- true  
  distance[start] <- 0  
  enqueue(Q, start)  
  
  while Q is not empty  
    u <- dequeue(Q)  
  
    for each v in adj[u]
```

```
if not visited[v]
  visited[v] <- true
  distance[v] <- distance[u]+1
  parent[v] <- u
  enqueue(Q,v)
```

6. Neden enqueue ederken visited işaretlenir? Visited işareti dequeue zamanına bırakılırsa aynı vertex farklı neighbors tarafından queue'ya birden çok kez eklenebilir.

Doğru an:

```
discover -> immediately mark visited -> enqueue
```

Bu duplicate work'ü önler ve parent'ın ilk shortest discovery olarak kalmasını sağlar.

7. Küçük BFS örneği Start 1; adjacency order:

```
adj[1]=[2,4,5]
adj[2]=[3,6,7]
```

Tree biçimindeki graph için possible BFS:

```
1,2,4,5,3,6,7
```

Önce start, sonra bütün level-1 vertices, sonra level-2 vertices ziyaret edilir.

8. BFS shortest paths Unweighted graph'ta veya bütün edges aynı cost'a sahipse BFS:

```
distance[v] = start'tan v'ye minimum edge count
```

bulur.

parent pointers shortest-path tree oluşturur. Weighted graph'ta arbitrary weights için normal BFS yeterli değildir; Dijkstra, Bellman-Ford veya başka uygun algorithm gerekir.

9. DFS intuition DFS yeni bir vertex discover ettiğinde current vertex'in exploration'ını geçici olarak suspend eder ve yeni vertex'i hemen explore eder.

Dead end'e ulaştığında Backtrack ederek önceki vertex'e döner.

Tree üzerinde uygun child order ile DFS discovery order, Preorder Traversal'a benzer.

10. Recursive DFS

```
DFSVisit(G,u)
  visited[u] <- true
  process u

  for each v in adj[u]
    if not visited[v]
      parent[v] <- u
      DFSVisit(G,v)
```

Call stack suspended vertices'i ve geri dönüş noktalarını doğal olarak saklar.

11. Iterative DFS

```
DFSIterative(G,start)
  push(stack,start)
```

```

while stack is not empty
    u <- pop(stack)

    if visited[u]
        continue

    visited[u] <- true
    process u

    for neighbors v in reverse desired order
        if not visited[v]
            push(stack,v)

```

Stack LIFO'dur. Recursive DFS ile aynı neighbor order isteniyorsa iterative version'da neighbors çoğu zaman reverse sırada push edilir.

Alternatif iterative design, stack frame içinde "next neighbor index" saklayarak recursive behavior'ı exactly taklit eder.

12. Küçük DFS örneği Aynı tree ve neighbor choices ile possible DFS order:

```
1,2,3,6,7,4,5
```

Algorithm önce 1→2→3 path'inde derine gider, sonra backtrack eder.

13. Traversal tree Bir unvisited vertex'i ilk keşfeden edge:

```
Tree Edge
```

olur. parent relationships bir BFS Tree veya DFS Tree oluşturur.

Start component connected ise bütün discovered vertices'i kapsayan spanning tree elde edilir. Graph disconnected ise bütün graph için tek tree değil Spanning Forest gerekir.

14. Non-tree edges hakkında dikkat Videoda BFS'teki dotted edges Cross Edges, DFS'tekiler Back Edges olarak adlandırılır. Bu, gösterilen undirected example için sezgisel bir anlatımdır; genel edge classification daha ayrıntılıdır.

Directed DFS'te edges:

- Tree Edge
- Back Edge
- Forward Edge
- Cross Edge

olarak ayrılabilir.

Undirected DFS'te her non-tree edge genellikle bir vertex'i ancestor'a bağlayan back edge olarak görülür. BFS non-tree edges aynı veya adjacent levels arasında olabilir.

15. Disconnected graph Tek BFS(start) veya DFS(start) yalnız start'ın connected/reachable component'ini dolaşır.

Bütün vertices için:

```

for each vertex v
    if not visited[v]
        start BFS/DFS from v

```

Bu outer loop traversal forest üretir.

Directed graph'ta "component" yerine start'tan directed paths ile reachable region düşünülmelidir.

16. Cycle handling Visited structure olmadan cyclic graph'ta traversal aynı vertices arasında sonsuza kadar dolaşabilir.

Visited:

- Infinite repetition'ı önler.
- Her vertex'in bir kez process edilmesini sağlar.
- Tree/non-tree edge ayırmasına yardım eder.

Directed DFS'te recursion-stack/color bilgisi cycle detection için kullanılabilir:

```
WHITE -> undiscovered
GRAY  -> active recursion path
BLACK -> finished
```

GRAY vertex'e edge bir directed back edge ve cycle işaretidir.

17. Complexity – adjacency list Her vertex bir kez discover/explore edilir:

$\theta(V)$

Adjacency lists toplamında directed graph için E, undirected graph için 2E entries taranır:

$\theta(E)$

Toplam:

```
Time =  $\theta(V+E)$ 
Auxiliary space =  $\theta(V)$ 
```

Video sonunda söylenen yalnız $\theta(n)$ ifadesi edges'i ayrı saymaz. Standard ve representation-aware ifade $\theta(V+E)$ dir.

18. Complexity – adjacency matrix Her explored vertex için bütün possible neighbors row'u taranır:

```
Time =  $\theta(V^2)$ 
Space =  $\theta(V^2)$  for matrix input
```

Graph sparse ise adjacency list genellikle daha verimlidir.

19. BFS ve DFS karşılaştırması

Özellik	BFS	DFS
Main structure	Queue	Stack / recursion
Pattern	Layer-by-layer	Deep then backtrack
Unweighted shortest path	Evet	Genel olarak hayır
Tree analogy	Level order	Preorder-like
Memory pressure	Wide frontier	Deep recursion/path
Typical uses	Distances, layers, bipartite check	Cycle, topological sort, components

20. DFS recursion depth riski Skewed/deep graph'ta recursion depth $\theta(V)$ olabilir ve language call-stack limit'ini aşabilir. Büyük untrusted graphs için explicit stack daha güvenli olabilir.

BFS queue da worst case'de $\theta(V)$ vertices tutabilir.

21. Sık yapılan hatalar BFS'te Stack kullanmak Traversal DFS behavior'ına dönüşür.

DFS'te Queue kullanmak Traversal BFS behavior'ına dönüşür.

BFS visited işaretini dequeue'ya bırakmak Duplicate queue entries oluşabilir.

Cyclic graph'ta visited kullanmamak Infinite traversal oluşabilir.

Tek start ile disconnected graph'ın tamamını beklemek Yalnız reachable component dolaşılır.

Traversal order'ı unique sanmak Adjacency order sonucu değiştirir.

Complexity'yi yalnız $O(V)$ yazmak Adjacency list'te edges de taranır: $O(V+E)$.

22. Kavram sözlüğü

English term	Türkçe açıklama
Breadth-First Search	Graph'ı distance layers halinde dolaşan traversal
Depth-First Search	Bir path boyunca derine gidip backtrack eden traversal
Queue	FIFO data structure
Stack	LIFO data structure
Discovery	Vertex'e ilk kez ulaşma
Exploration	Vertex'in adjacency edges'ini inceleme
Tree Edge	Yeni vertex'i ilk keşfeden edge
Spanning Forest	Disconnected graph components'inin traversal trees bütünü
Backtracking	Dead end sonrası suspended vertex'e geri dönme

23. Kendini sinama

1. Visit ile explore arasındaki fark nedir?
2. BFS hangi data structure'ı kullanır?
3. DFS hangi data structure ile gerçekleştirilir?
4. BFS'te vertex ne zaman visited işaretlenmelidir?
5. BFS ne zaman shortest path verir?
6. Tek start disconnected graph'ın tamamını dolaşır mı?
7. Adjacency-list complexity nedir?
8. Adjacency-matrix complexity nedir?
9. Traversal order neden değişebilir?

Kısa cevaplar

1. Visit ilk discovery; explore adjacency list'i işlemektir.
2. Queue.
3. Recursion/call stack veya explicit Stack.
4. İlk discovery/enqueue anında.
5. Unweighted veya equal-edge-cost graph'ta.
6. Hayır; yalnız reachable component'i.
7. $O(V+E)$ time ve $O(V)$ auxiliary space.
8. $O(V^2)$ time.
9. Start vertex ve adjacency ordering choices nedeniyle.

Sonraki ders: **62 – Articulation Point and Biconnected Components.**

62 – Articulation Point and Biconnected Components

Video: 5.2 Articulation Point and Biconnected Components (8:37)

Kaynak notu: Bu videoda YouTube English captions bulunmuyor. Aşağıdaki ders resmî konu kapsamı temel alınarak standard DFS discovery/low-link yöntemi, root istisnası, bridges ve biconnected-component extraction ayrıntıları doğrulanarak hazırlanmıştır.

Bu videonun ana fikri Undirected graph'ta bir vertex kaldırıldığında connected-component sayısı artıyorsa bu vertex bir Articulation Point veya Cut Vertex'tir.

DFS sırasında iki value tutulur:

```
disc[u] = u'nun discovery time'ı
low[u]  = u'nun subtree'sinden, en fazla bir back edge kullanarak
         ulaşılabilen en erken discovered vertex'in discovery time'ı
```

Non-root vertex u , bir DFS child v için:

```
low[v] >= disc[u]
```

ise articulation point'tir. DFS root'u için ayrı condition gerekir:

```
root has at least 2 DFS-tree children
```

1. Articulation Point tanımı Connected undirected graph G 'de vertex u ve ona incident edges çıkarıldığında graph daha fazla connected component'e ayrılıyorsa u articulation point'tir.

Disconnected input'ta da tanım component count'un artması üzerinden yapılır; algorithm her DFS forest tree'sini ayrı işler.

2. Neden önemlidir? Articulation points single points of failure gösterebilir:

- Communication network routers
- Road intersections
- Electrical/network infrastructure
- Dependency graphs
- Social-network connectors

Bu vertex'in kaybı graph'ın bazı kısımlarını birbirinden ayırır.

3. Bridge ile farkı Bridge veya Cut Edge, kaldırıldığında connected-component sayısını artıran edge'dir.

- Articulation point: vertex failure
- Bridge: edge failure

Aynı low-link DFS ikisini de bulabilir fakat conditions farklıdır.

Tree edge (u, v) için:

```
low[v] > disc[u] -> (u,v) is a bridge
low[v] >= disc[u] -> u may be an articulation point
```

Equality durumunda child subtree u'ya geri dönebilir fakat onun strict ancestor'ına çıkamaz; vertex u kaldırılırsa ayrılır, edge (u, v) ise başka path ile cycle üzerinde olabilir.

4. DFS tree ve back edge Undirected DFS'te:

- Tree Edge: unvisited child'ı keşfeden edge
- Back Edge: current vertex/subtree'den bir ancestor'a giden non-tree edge

Parent edge back edge olarak sayılmamalıdır. Aksi hâlde low yanlış biçimde parent discovery time'a düşer. Parallel edges destekleniyorsa yalnız parent vertex'i karşılaştırmak yetmeyebilir; traversed parent edge'in unique edge id'si ayırt edilmelidir.

5. Discovery time Global counter her vertex ilk ziyaret edildiğinde artırılır:

```
time <- time+1
disc[u] <- time
low[u] <- time
```

Smaller discovery time, DFS'te daha önce keşfedilmiş vertex anlamına gelir.

6. low[u] neyi temsil eder? low[u], şunların minimum'udur:

```
disc[u]
disc[w] for a back edge from u/subtree to ancestor w
low[v] for each DFS child v
```

Update rules:

```
tree edge u->v tamamlandıktan sonra:
    low[u] = min(low[u], low[v])

visited neighbor v, parent edge değilse:
    low[u] = min(low[u], disc[v])
```

Back edge case'inde low[v] değil disc[v] kullanılır.

7. Non-root articulation condition u root değil ve child v için:

```
low[v] >= disc[u]
```

ise v subtree'sinden onun strict ancestor'larına ulaşan back edge yoktur.

u kaldırılırsa v subtree'si upper graph'tan kopar; dolayısıyla u articulation point'tir.

8. Root articulation condition DFS root'un parent'ı yoktur. Standard non-root condition root'a uygulanırsa tek child'lı root yanlışlıkla articulation sayılabilir.

Root yalnız şu durumda articulation point'tir:

```
DFS tree'de 2 veya daha fazla children
```

Farklı root children birbirine root dışından ulaşabilseydi DFS sırasında ayrı children olarak kalmazlardı.

9. Örnek graph İki triangles bir bridge ile bağlansın:

```
1---2
 \ /
 3 ---- 4
      / \
      5---6
```

Edges:

```
(1,2), (2,3), (3,1)
(3,4)
(4,5), (5,6), (6,4)
```

Results:


```
Articulation points = {3,4}
Bridge = {(3,4)}
```

Triangle içindeki tek bir edge kaldırıldığında connectivity korunur.

10. Biconnected graph Connected undirected graph, hiçbir articulation point içermiyorsa Biconnected veya 2-Vertex-Connected olarak adlandırılır.

Equivalent intuition: herhangi tek vertex failure remaining vertices arasındaki connectivity'yi bozmaz.

Bazı formal definitions en az üç vertices şartı koyar. Textbook convention'a göre single edge'in ayrı block sayılıp sayılmadığı açıkça belirtilmelidir.

11. Biconnected Component Biconnected Component (BCC) veya Block, daha büyük bir biconnected subgraph'a genişletilemeyen maximal edge set/subgraph'tır.

Articulation vertices birden fazla BCC'ye ait olabilir. Edges ise standard block decomposition'da tek BCC'ye aittir.

Örnek graph'taki blocks:

```
triangle {1,2,3}
bridge edge {3,4}
triangle {4,5,6}
```

Convention bridge'i iki-vertex block olarak kabul eder.

12. Edge stack ile BCC çıkarma DFS traversed edges'i stack'e koyar:

- Tree edge keşfedildiğinde push
- Ancestor'a back edge görüldüğünde push

Child v tamamlandığında:

```
low[v] >= disc[u]
```

ise stack'ten (u,v) edge'i çıkana kadar pop edilen edges bir BCC oluşturur.

Disconnected component DFS'i bittiğinde stack'te remaining edges varsa onlar da bir BCC olarak çıkarılır.

13. Articulation-point pseudocode

```
DFS(u,parentEdge)
  visited[u] <- true
  disc[u] <- low[u] <- ++time
  childCount <- 0

  for each edge e=(u,v)
    if e == parentEdge
      continue

    if not visited[v]
      parent[v] <- u
      childCount++
      DFS(v,e)
      low[u] <- min(low[u],low[v])

    if parent[u] == NIL and childCount >= 2
      articulation[u] <- true

    if parent[u] != NIL and low[v] >= disc[u]
      articulation[u] <- true
```

```
else
    low[u] <- min(low[u], disc[v])
```

Outer loop:

```
for each vertex u
    if not visited[u]
        DFS(u, NIL)
```

14. Complexity Adjacency list ile her vertex bir kez, her undirected edge iki adjacency entry üzerinden constant sayıda işlenir:

```
Time =  $\Theta(V+E)$ 
Space =  $\Theta(V+E)$  including graph and edge stack
Auxiliary arrays/recursion =  $\Theta(V)$ 
```

Adjacency matrix kullanılırsa neighbor scans $\Theta(V^2)$ time alır.

15. Recursion depth DFS recursion depth worst case $\Theta(V)$ dir. Büyük/deep graph'ta call-stack overflow oluşabilir; iterative DFS frame stack'i kullanılabilir.

Iterative low-link implementation sıradan iterative DFS'ten daha dikkat ister çünkü child tamamlandığında postorder low update yapılmalıdır.

16. Sık yapılan hatalar Root'a non-root condition uygulamak Root için child-count rule ayrıdır.

Bridge ve articulation inequalities'ini karıştırmak Bridge $>$, articulation non-root $\geq$ kullanır.

Parent edge'i back edge saymak low values yanlış düşer.

Back edge update'te low[v] kullanmak Visited ancestor için disc[v] kullanılmalıdır.

Yalnız vertex 0 dan DFS başlatmak Disconnected graph components kaçırılır.

BCC ile ordinary connected component'i aynı sanmak BCC single-vertex failures'a karşı connectivity structure'ını gösterir.

17. Kavram sözlüğü

English term	Türkçe açıklama
Articulation Point	Kaldırıldığında component sayısını artıran vertex
Cut Vertex	Articulation point'in diğer adı
Bridge	Kaldırıldığında component sayısını artıran edge
Discovery Time	DFS'te vertex'in ilk ziyaret sırası
Low-Link Value	Subtree'nin ulaşabildiği en erken discovery time
Back Edge	DFS subtree'den ancestor'a giden non-tree edge
Biconnected Component	Maximal articulation-free block
DFS Forest	Disconnected graph için birden çok DFS tree

18. Kendini sına

1. Articulation point nedir?
2. disc[u] ve low[u] ne tutar?
3. Non-root articulation condition nedir?
4. Root articulation condition nedir?
5. Bridge condition nedir?

6. Equality neden articulation için yeterli fakat bridge için değil?
7. BCC edges nasıl stack'ten çıkarılır?
8. Adjacency-list complexity nedir?

Kısa cevaplar

1. Kaldırıldığında connected-component sayısını artıran vertex.
2. Discovery time'ı ve subtree'nin erişebildiği en erken discovery time'ı.
3. Bir child v için $low[v] \geq disc[u]$.
4. DFS root'un en az iki tree child'ı olması.
5. $low[v] > disc[u]$.
6. uya dönüş edge'i tree edge'i kurtarabilir ama u kaldırılınca subtree yine kopar.
7. $low[v] \geq disc[u]$ olduğunda (u, v) dahil olana kadar pop edilerek.
8. $\Theta(V+E)$.

Sonraki ders: **63 – Backtracking: Brute Force Approach.**

63 – Backtracking: Brute Force Approach

Video: 6.1 Backtracking: Brute Force Approach (8:15)

Bu videonun ana fikri Backtracking, solution'ı karar karar kuran, bir partial assignment constraint'i ihlal ettiğinde o branch'i tamamlamadan geri dönen depth-first search strategy'sidir.

choose → check/promising → recurse → undo

Tüm candidates'i körlemesine üretmek yerine impossible partial states erken Prune edilir.

1. Brute Force ile ilişkisi Brute force bütün possible complete candidates'i dener. Backtracking aynı state space'i systematic olarak araştırır fakat constraint'ler sayesinde bazı complete candidates'i hiç üretmez.

Worst case'de hiçbir pruning işe yaramazsa backtracking brute-force complexity'sine geri döner.

2. State-Space Tree State-Space Tree içinde:

- Root: henüz decision verilmemiş empty state
- Level k: ilk k decisions verilmiş partial solution
- Edge: bir candidate choice
- Leaf: complete solution veya dead end

Root-to-leaf path bir complete assignment temsil eder.

3. Videodaki permutation örneği Üç farklı öğrenci üç sandalyeye yerleştirilsin:

B1, B2, G1

Possible arrangements:

$3! = 6$

State-space tree'de:

Level 1 → first chair
Level 2 → second chair
Level 3 → third chair

Her level'da daha önce kullanılmamış öğrenci seçilir.

4. Constraint ile pruning Constraint:

G1 middle chair'da oturamaz

İkinci level'da G1 seçildiği anda partial assignment invalid olur. Third chair için descendants üretmeye gerek yoktur.

Valid arrangements sayısı:

4

Bu, erken constraint check'in search work'ü nasıl azalttığını gösterir.

5. Promising/Bounding function Videoda pruning condition Bounding Function olarak adlandırılır. Backtracking literatüründe sık kullanılan adlar:

- Promising Function
- Feasibility Test
- Constraint Check

Function partial state'in en az bir valid completion'a sahip olma ihtimalini test eder.

```
if not promising(state)
    prune branch
```

Test sound olmalıdır: valid solution içerebilecek branch yanlışlıkla kesilmemelidir.

6. Generic backtracking template

```
Backtrack(state, level)
    if state is complete
        report/process solution
        return

    for each candidate choice at level
        if choice is consistent with state
            apply(choice)
            Backtrack(state, level+1)
            undo(choice)
```

undo, sibling branch'in temiz state'ten başlamasını sağlar.

7. Apply ve undo invariant'ı Recursive call öncesi state current path'i exactly temsil etmelidir. Call bittikten sonra:

```
state = recursive call öncesindeki hâl
```

olmalıdır.

Mutation geri alınmazsa önceki branch'in choices'ı next branch'e sızar ve eksik/yanlış solutions oluşur.

8. DFS traversal Backtracking tipik olarak DFS kullanır:

1. Bir choice seç.
2. Mümkünse daha derine git.
3. Solution/dead end'de geri dön.
4. Next sibling choice'u dene.

Recursion call stack current path'i saklar. Explicit stack ile de uygulanabilir.

9. Backtracking yalnız "bütün çözümler" için değildir Videoda backtracking'in optimization için olmadığı ve bütün solutions istendiğinde kullanıldığı söylenir; bu fazla dar bir ifadedir.

Backtracking şunlar için kullanılabilir:

- İlk feasible solution'ı bulma
- Bütün solutions'ı enumerate etme
- Solution sayısını count etme
- Best solution'ı objective karşılaştırarak bulma
- Decision problem'e yes/no cevap verme

Optimization'da objective bounds eklenirse yaklaşım Branch and Bound teknikleriyle örtüşebilir.

10. Backtracking ve Dynamic Programming

Özellik	Backtracking	Dynamic Programming
Ana fikir	Decision tree search + pruning	Repeated subproblems'i cache/table ile çözme
Typical state	Partial assignment	Subproblem value
Repeated states	Tekrar oluşabilir	Bir kez hesaplanır
Output	Feasible assignments	Optimal/count/value/structure
Traversal	Genellikle DFS	Top-down veya bottom-up

Problem hem overlapping subproblems hem constraints içeriyorsa memoized backtracking de mümkündür.

11. Backtracking ve Branch and Bound Video farkı "Backtracking DFS, Branch and Bound BFS kullanır" şeklinde sadeleştirir. Bu evrensel değildir.

Branch and Bound live nodes'u:

- FIFO queue ile breadth-first,
- LIFO stack ile depth-first,
- Priority queue ile least-cost/best-first

seçebilir.

Temel conceptual fark:

- Backtracking çoğunlukla feasibility/constraint nedeniyle prune eder.
- Branch and Bound objective'in optimistic bound'u mevcut best'ten iyi olamıyorsa prune eder.

Traversal order implementation choice'udur.

12. Complexity Branching factor b , decision depth d ise kaba worst-case:

$$\text{Time} = O(b^d)$$

Permutation probleminde:

$$\text{complete leaves} = n!$$

Recursion path space:

$$O(d)$$

State copy ediliyorsa veya candidate structures tutuluyorsa additional memory değişebilir.

Pruning average/practical runtime'ı dramatik azaltabilir fakat worst-case exponential bound'u otomatik olarak kaldırmaz.

13. Solution enumeration ve output cost Problem gerçekten exponential sayıda valid solutions ürettiyorsa hepsini yazdırmak en az output size kadar zaman alır.

$$Z \text{ solutions, each length } d \rightarrow \Omega(Z \cdot d) \text{ output work}$$

Hiçbir algorithm bütün output'u bundan daha hızlı materialize edemez.

14. Candidate ordering Correctness'i değiştirmeden candidates farklı sırada denenebilir. İyi heuristic:

- Failure'a hızlı götüren choice'u önce denemek
- Most constrained variable'ı önce seçmek
- Promising solution'ı erken bulmak

özellikle first-solution veya optimization searches'i hızlandırabilir.

15. Backtracking problem örnekleri

- N-Queens
- Sum of Subsets
- Graph Coloring
- Hamiltonian Cycle
- Sudoku
- Constraint satisfaction
- Permutation/combination generation

16. Sık yapılan hatalar Constraint'i yalnız complete leaf'te kontrol etmek Early pruning avantajı kaybolur.

Choice'u geri almamak Sibling branches corrupt state görür.

Pruning function'ı aşırı güçlü yazmak Valid solutions yanlışlıkla elenebilir.

Backtracking'in her zaman hızlı olduğunu sanmak Worst case exponential olabilir.

Branch and Bound'ı zorunlu BFS sanmak FIFO, LIFO veya best-first node selection kullanılabilir.

İlk solution'da durup "hepsini buldum" demek Goal first/all/count olduğuna göre termination değişir.

17. Kavram sözlüğü

English term	Türkçe açıklama
Backtracking	Choice yapıp invalid branch'ten geri dönen search strategy
State-Space Tree	Partial ve complete assignments'ın search tree'si
Promising Function	Branch'in valid completion ihtimalini sınavan test
Pruning	Sonuç veremeyecek subtree'yi üretmeden kesme
Candidate	Current level'da denenebilecek choice
Apply/Undo	Choice'u state'e ekleme ve recursion sonrası geri alma
Feasible Solution	Bütün constraints'i sağlayan complete assignment
Branch and Bound	Objective bound ile nodes'u eleyen search yaklaşımı

18. Kendini sınama

1. State-space tree'de bir level neyi temsil eder?
2. Backtracking'in temel dört adımı nelerdir?
3. Promising function ne yapar?
4. undo neden zorunludur?
5. Üç öğrencinin kaç permutation'ı vardır?
6. Middle-chair constraint sonrası kaç valid arrangement kalır?
7. Backtracking yalnız all-solutions problemlerinde mi kullanılır?
8. Branch and Bound zorunlu olarak BFS mi kullanır?
9. Worst-case neden exponential kalabilir?

Kısa cevaplar

1. Bir decision position'ını/partial-solution uzunluğunu.
2. Choose, check, recurse, undo.
3. Branch'in valid completion içerebilme ihtimalini test edip impossible branch'i keser.
4. Sibling branch'in önceki choice'tan etkilenmemesi için.

5. $3! = 6$.
6. 4.
7. Hayır; first, all, count, decision ve optimization için kullanılabilir.
8. Hayır; farklı live-node selection policies mümkündür.
9. Hiçbir branch prune edilemeyebilir ve bütün state space aranabilir.

Sonraki ders: **64 – N-Queens Using Backtracking**.

64 – N-Queens Using Backtracking

Video: 6.2 N-Queens Using Backtracking (13:41)

Kaynak notu: Videonun English caption track'i mevcuttur ancak yalnız giriş bölümünü kapsar; geri kalan zaman aralıklarında caption text yoktur. Aşağıdaki ders videonun problem tanımı temel alınarak standard row-by-row backtracking, safety condition, iki 4-Queens solution'ı ve complexity ile tamamlanmıştır.

Bu videonun ana fikri N-Queens Problem, $N \times N$ chessboard'a N queens'i hiçbir ikisi birbirine saldırmayacak biçimde yerleştirmeyi ister.

Queen aynı:

- Row
- Column
- Diagonal

üzerinde hareket edebilir.

Backtracking her row'a bir queen yerleştirir. Safe olmayan column choices anında prune edilir.

1. Constraint'ler İki queen positions $(r1, c1)$ ve $(r2, c2)$ olsun.

Attack conditions:

```
same row:      r1 == r2
same column:   c1 == c2
same diagonal: abs(r1-r2) == abs(c1-c2)
```

Valid solution'da bu conditions'ın hiçbirisi farklı queen pair'i için sağlanmaz.

2. Neden her row'a tam bir queen koyuyoruz? N queens ve N rows vardır. Aynı row'da iki queen bulunamayacağına göre valid solution'da her row exactly one queen içerir.

Bu observation search representation'ı küçültür:

```
position[row] = o row'daki queen'in column'u
```

Row conflict construction gereği otomatik olarak ortadan kalkar; yalnız column ve diagonal kontrol edilir.

3. State-Space Tree

```
Level 0 -> empty board
Level 1 -> row 1 queen column choice
Level 2 -> row 2 queen column choice
...
Level N -> complete arrangement
```

Her node safe partial placement'tır. Unsafe candidate'ın descendants'i generate edilmez.

4. Safety check Yeni queen row r , column c ye konacaksa önceki rows $i=0..r-1$ için:

```
position[i] == c
```

ise same column conflict vardır.

```
abs(position[i]-c) == r-i
```

ise same diagonal conflict vardır.

Hiç conflict yoksa candidate Safe/Promisingdir.

5. Basic pseudocode

```
Solve(row)
  if row == N
    report position[0..N-1]
    return

  for col = 0 to N-1
    if IsSafe(row,col)
      position[row] <- col
      Solve(row+1)
      position[row] <- UNASSIGNED
```

```
IsSafe(row,col)
  for previousRow = 0 to row-1
    previousCol <- position[previousRow]

    if previousCol == col
      return false

    if abs(previousCol-col) == row-previousRow
      return false

  return true
```

6. 4-Queens problem 4 x 4 board'a four queens yerleştirilir. Column vector 1-based yazıldığında iki solution vardır:

```
[2,4,1,3]
[3,1,4,2]
```

Vector index row'u, value column'u gösterir.

7. Birinci 4-Queens solution'ı

```
position=[2,4,1,3]
```

Board:

```
. Q . .
. . . Q
Q . . .
. . Q .
```

Queen coordinates:

```
(1,2),(2,4),(3,1),(4,3)
```

Columns unique'dir ve hiçbir pair için row difference column difference'a eşit değildir.

8. İkinci 4-Queens solution'ı

```
position=[3,1,4,2]
```

```

. . Q .
Q . . .
. . . Q
. Q . .

```

Bu, first solution'ın mirror symmetry'sidir.

9. Backtracking trace intuition Örneğin first row column 1 seçilsin. Next row:

- Column 1: same column, reject
- Column 2: diagonal, reject
- Column 3: safe olabilir, recurse

Deeper row'da hiçbir safe column kalmazsa current row'un queen'i kaldırılır ve previous row'daki next column denenir.

Bu "place, recurse, remove" döngüsü bütün valid arrangements'ı systematic olarak bulur.

10. O(1) safety structures Her candidate için önceki queens'i O(N) taramak yerine üç sets/boolean arrays tutulabilir:

```

usedColumn[c]
usedMainDiagonal[r-c+(N-1)]
usedAntiDiagonal[r+c]

```

Ranges:

```

r-c+(N-1) in [0, 2N-2]
r+c       in [0, 2N-2]

```

Candidate check ve apply/undo O(1) olur.

11. Optimized pseudocode

```

Solve(row)
  if row == N
    report solution
    return

  for col = 0 to N-1
    d1 <- row-col+(N-1)
    d2 <- row+col

    if usedColumn[col] or usedD1[d1] or usedD2[d2]
      continue

    position[row] <- col
    usedColumn[col] <- usedD1[d1] <- usedD2[d2] <- true

    Solve(row+1)

    usedColumn[col] <- usedD1[d1] <- usedD2[d2] <- false

```

Undo sırasında üç structures'ın da temizlenmesi zorunludur.

12. Bitmask implementation fikri N machine word'e sığyorsa columns ve diagonals bitmasks ile tutulabilir:

```

available = allColumns & ~(columns | diagLeft | diagRight)

```

Her available bit sırayla seçilir. Recursive call'da diagonal masks shift edilir.

Bitmask constant factors'ı ciddi azaltır; search'in exponential nature'ını değiştirmez.

13. First solution, all solutions ve count

Goal'a göre base-case behavior değişir:

- İlk solution: bulununca bütün recursion durdurulur.
- Bütün solutions: her complete placement output edilir.
- Count: yalnız counter artırılır.

Known small counts:

```
N=1 -> 1
N=2 -> 0
N=3 -> 0
N=4 -> 2
N=8 -> 92
```

Counts board rotations/reflections ayrı arrangements olarak sayıldığında verilmiştir.

14. Complexity

Bir row'da used column tekrar seçilmediği için kaba search bound:

```
O(N!) nodes/leaves scale
```

Naive IsSafe her candidate için $O(N)$ tararsa kaba time bound:

```
O(N*N!)
```

$O(1)$ sets/bitmasks ile state work azalır; enumeration worst case yine exponential/factorial karakterdedir.

Auxiliary state ve recursion:

```
Space = O(N)
```

Bütün boards aynı anda saklanırsa output storage ayrıca solution sayısı kadar büyür.

15. Symmetry breaking

Count/search hızlandırmak için first row'da board symmetry kullanılabilir. Örneğin yalnız columns'ın ilk yarısı denenip mirrored solutions hesaplanabilir.

Odd N'de middle column ayrı ele alınır. Symmetry optimization doğru count reconstruction gerektirir; all labeled boards tek tek output edilecekse mirrored solutions yine üretilmelidir.

16. Sık yapılan hatalar

Yalnız column conflict kontrol etmek Diagonals da kontrol edilmelidir.

Diagonal condition'ı $r1+c1==r2+c2$ ile sınırlamak Bu yalnız bir diagonal direction'ı yakalar; iki direction gerekir veya absolute-difference testi kullanılmalıdır.

Recursive call sonrası marks'ı temizlememek Valid sibling choices yanlışlıkla blocked kalır.

Bir row'da birden fazla queen yerleştirmek Row-by-row representation bunu construction ile engeller.

N=2 veya N=3 için solution beklemek Bu sizes için solution yoktur.

Complexity'yi polynomial sanmak Pruning'e rağmen worst-case search factorial/exponential büyür.

17. Kavram sözlüğü

English term	Türkçe açıklama
N-Queens Problem	N queens'i birbirine saldırmadan board'a yerleştirme problemi
Column Vector	Her row'daki queen column'unu saklayan array
Diagonal Conflict	Row ve column differences'ın absolute olarak eşit olması
Safe Position	Önceki queens ile conflict üretmeyen cell
Symmetry Breaking	Mirror/rotation equivalent searches'i azaltma
Bitmask	Used columns/diagonals'ı bits ile saklama
Place/Remove	Backtracking apply/undo operation'ı

18. Kendini sına

1. İki queen hangi üç durumda birbirine saldırır?
2. `position[row]` ne tutar?
3. Diagonal conflict formula'sı nedir?
4. 4-Queens için kaç solution vardır?
5. İki column vectors nelerdir?
6. $O(1)$ safety için hangi structures tutulur?
7. Undo neden gerekir?
8. Kaba worst-case search bound nedir?

Kısa cevaplar

1. Aynı row, column veya diagonal'da olduklarında.
2. O row'daki queen'in column'unu.
3. $\text{abs}(r1-r2) == \text{abs}(c1-c2)$.
4. 2.
5. `[2,4,1,3]` ve `[3,1,4,2]`.
6. Used columns ve iki diagonal direction set/arrays'i.
7. Sibling branch'in temiz state görmesi için.
8. Yaklaşık $O(N!)$; naive safety ile $O(N*N!)$ kaba bound.

Sonraki ders: **65 – Sum of Subsets Using Backtracking.**

65 – Sum of Subsets Using Backtracking

Video: 6.3 Sum of Subsets Using Backtracking (12:19)

Bu videonun ana fikri Sum of Subsets probleminde verilen weights arasından total'ı target Mye exactly eşit olan subsets aranır.

Her weight için binary decision vardır:

```
x[i]=1 -> weight i dahil  
x[i]=0 -> weight i dahil değil
```

Backtracking include/exclude state-space tree'sini DFS ile arar. Positive weights için iki güçlü bound kullanılır:

```
currentSum > target          -> fazla, prune  
currentSum + remainingSum < target -> yetişemez, prune
```

1. Videodaki problem

```
weights = [5,10,12,13,15,18]  
target M = 30
```

Total weight:

```
5+10+12+13+15+18 = 73
```

Solution binary vector, hangi indices'in seçildiğini gösterir.

2. State representation

Bir recursive state:

```
index      = sıradaki karar verilecek weight  
currentSum = seçilen weights toplamı  
remainingSum = henüz karar verilmemiş weights toplamı  
chosen     = current subset/bit vector
```

Root:

```
index=0, currentSum=0, remainingSum=73
```

3. State-Space Tree

Her level bir weight'e karşılık gelir:

```
left/include branch -> x[i]=1  
right/exclude branch -> x[i]=0
```

n weights için pruning olmadan:

```
2^n complete subsets  
1+2+...+2^n = 2^(n+1)-1 total tree nodes
```

4. Include branch update

Weight w[i] dahil edilirse:

```
currentSum <- currentSum+w[i]  
remainingSum <- remainingSum-w[i]  
chosen[i] <- 1
```

Exclude branch'te:

```
currentSum unchanged
remainingSum <- remainingSum-w[i]
chosen[i] <- 0
```

5. Bound 1 – target’ı aşmak

All weights positive ise:

```
currentSum > target
```

olduğunda future additions sum’ı yalnız artırır; exact target’a geri dönülemez.

Video include branch’i oluşturmada önce eşdeğer test kullanır:

```
currentSum + nextWeight <= target
```

6. Bound 2 – target’a yetişememek

Henüz seçilebilecek bütün weights eklense bile:

```
currentSum+remainingSum < target
```

ise hiçbir descendant solution olamaz.

Örneğin:

```
currentSum=27
remainingSum=0
27+0<30
```

branch prune edilir.

7. Video trace’inden bir branch

İlk üç weights dahil:

```
5+10+12=27
```

Next 13 dahil edilirse:

```
27+13=40>30
```

include branch prune edilir.

13 ve 15 dışarıda bırakıldıktan sonra yalnız 18 kalır:

```
27+18=45>30
```

include edilemez; exclude edilirse 27de kalınır ve remaining zero olduğundan target’a ulaşamaz.

8. Videoda bulunan solution

Branch:

```
5 included
10 included
12 excluded
13 excluded
15 included
```

gives:

```
5+10+15=30
```

Binary vector:

```
[1,1,0,0,1,0]
```

9. Örneğin bütün solutions'ı

Index-distinct subsets:

```
{5,10,15} -> 30
{5,12,13} -> 30
{12,18}   -> 30
```

Binary vectors:

```
[1,1,0,0,1,0]
[1,0,1,1,0,0]
[0,0,1,0,0,1]
```

10. Robust pseudocode

Strictly positive weights varsayımıyla:

```
Search(i,currentSum,remainingSum)
  if currentSum == target
    report chosen
    return

  if i == n
    return

  if currentSum > target
    return

  if currentSum+remainingSum < target
    return

  chosen[i] <- 1
  Search(
    i+1,
    currentSum+weight[i],
    remainingSum-weight[i]
  )

  chosen[i] <- 0
  Search(
    i+1,
    currentSum,
    remainingSum-weight[i]
  )
```

11. Promising function

Equivalent compact test:

```
promising =
  currentSum <= target
  and currentSum+remainingSum >= target
```

Bu necessary condition'dır; promising olması solution kesin var demek değildir. Yalnız branch'in henüz impossible olmadığını söyler.

12. Positive-weight assumption

Bounds monotonicity'ye dayanır. Negative weights varsa:

- $currentSum > target$ sonrası negative weight ile geri dönülebilir.
- Remaining weights'in yalnız sum'ı maximum achievable additional value'yu temsil etmez.

Dolayısıyla bu pruning rules negative weights için sound değildir. Problem variant'ı ve bounds yeniden tasarlanmalıdır.

13. Zero weights

Strictly positive weights'te $currentSum == target$ olduğunda return etmek güvenlidir;

future additions sum'ı bozar.

Zero weights varsa aynı numeric subset sum'ı veren farklı index subsets'i kaçırmamak için target bulunduğu remaining zero-valued choices enumerate edilmek istenebilir.

Goal'un value-distinct mi index-distinct subsets mi olduğu açık olmalıdır.

14. Duplicate weights Equal values farklı indices olarak kabul edilirse farklı binary vectors aynı multiset value solution'ını verebilir.

Value-distinct solutions isteniyorsa:

- Weights sort edilir.
- Aynı recursion level'da duplicate choices skip edilir.

Ancak include/exclude indexing ve original indices mapping dikkatle korunmalıdır.

15. Sorting ve stronger bounds Video weights'i increasing order'da kullanır. Basit aggregate bounds için sorting zorunlu değildir; fakat candidate ordering ve additional bounds açısından faydalı olabilir.

Large weights'i önce denemek overshoot'u erken gösterebilir; small weights'i önce denemek solutions'ı farklı sırada bulur. Correctness sound bounds ile korunur.

16. Complexity Worst case'de hiçbir branch prune edilmez:

```
Time =  $O(2^n)$ 
Recursion/chosen space =  $O(n)$ 
```

All solutions output ediliyorsa output cost ayrıca vardır.

Pruning practical node count'u azaltır fakat worst-case exponential bound kalır.

17. Backtracking ve 0/1 Knapsack farkı İki problem de include/exclude decisions kullanır fakat objectives farklıdır:

- Sum of Subsets: exact target equality ve feasible subsets
- 0/1 Knapsack: capacity altında maximum profit

Sum of Subsets DP ile de çözülebilir; nonnegative integer target için reachability table $O(nM)$ pseudo-polynomial time kullanır. Backtracking ise actual solutions'ı doğal biçimde enumerate eder.

18. Sık yapılan hatalar **remainingSumdan current weight'i çıkarmamak** Potential bound yanlış kalır.

currentSum+remainingSum == target olduğunda yalnız tek solution varsaymak Remaining bütün weights'i almak bir solution olabilir; zero/duplicates ve enumeration goal dikkat ister.

Negative weights ile positive-only bounds kullanmak Valid solutions prune edilebilir.

Solution bulunca chosen vector'ü kopyalamadan saklamak Later undo operations stored solution'ı değiştirebilir.

Worst-case'i pruning sonrası polynomial sanmak Hâlâ $O(2^n)$ olabilir.

19. Kavram sözlüğü

English term	Türkçe açıklama
Sum of Subsets	Total'ı target'a eşit subsets bulma problemi
Include/Exclude	Bir weight'i alma veya almama kararları
Current Sum	Şu ana kadar seçilmiş weights toplamı
Remaining Sum	Henüz karar verilmemiş weights toplamı
Target	Ulaşılmak istenen exact sum
Promising Node	Henüz target'a ulaşma ihtimali bounds ile elenmemiş state
Binary Vector	Selected indices'i 0/1 values ile gösteren solution
Monotonicity	Positive additions ile sum'ın yalnız artması özelliği

20. Kendini sına

1. Her tree level neyi temsil eder?
2. Include branch'te state nasıl güncellenir?
3. İki temel pruning condition nedir?
4. Videodaki weights ve target nedir?
5. Kaç solution vardır ve nelerdir?
6. Bounds neden negative weights ile geçersizdir?
7. Worst-case time ve auxiliary space nedir?
8. `currentSum==target` sonrası return hangi assumption'a dayanır?

Kısa cevaplar

1. Bir weight için include/exclude kararını.
2. Weight current sum'a eklenir, remaining sum'dan çıkarılır.
3. `currentSum>target` ve `currentSum+remainingSum<target`.
4. `[5,10,12,13,15,18]`, target 30.
5. Üç: `{5,10,15}`, `{5,12,13}`, `{12,18}`.
6. Future negative additions overshoot'u geri düşürebileceği için.
7. $O(2^n)$ time ve $O(n)$ space.
8. Bütün remaining weights'in strictly positive olmasına.

Sonraki ders: **66 – Graph Coloring Using Backtracking**.

66 – Graph Coloring Using Backtracking

Video: 6.4 Graph Coloring Using Backtracking (15:52)

Bu videonun ana fikri Vertex Coloring, graph vertices'ine colors atayıp her edge'in endpoints'inin farklı color almasını ister:

```
for every edge (u,v): color[u] != color[v]
```

m-Coloring backtracking algorithm'ı vertices'i sırayla renklendirir. Current vertex için her color denenir; colored neighbor ile conflict oluştursa branch anında prune edilir.

Worst-case search:

```
 $O(m^V)$ 
```

assignments ölçeğindedir; safety-check maliyeti representation'a göre ayrıca eklenir.

1. Proper Vertex Coloring Proper Coloring, adjacent vertices'in aynı color'a sahip olmadığı assignment'tır.

Nonadjacent vertices aynı color'ı paylaşabilir. Amaç her vertex'e unique color vermek değil, constraints'i sağlayacak kadar colors kullanmaktır.

2. Videodaki giriş örneği Üç labeled colors:

```
Red, Green, Blue
```

ile example graph renklendirilebilir. Video bir assignment verir:

Vertex:	1	2	3	4	5
Color:	Red	Green	Red	Green	Blue

Aynı graph farklı valid assignments'a sahip olabilir. Color labels globally permute edildiğinde de yeni labeled assignment oluşur.

Video example'ı two colors ile renklendirilemez, three colors ile renklendirilebilir; dolayısıyla o graph'ın chromatic number'ı 3tür.

3. Üç farklı problem türü Search/Enumeration

```
Given G and m, one/all valid color assignments'ı bul.
```

Decision

```
Is G m-colorable?
```

Yalnız yes/no istenir.

Optimization

```
G'yi color etmek için gereken minimum color sayısını bul.
```

Bu minimum sayıya Chromatic Number, notation olarak $\chi(G)$, denir.

Videoda decision ve chromatic-number terimleri birbirine yaklaştırılır; kesin ayrım şudur: m-colorability decision problemidir, chromatic number minimum myi bulan optimization value'dur.

4. Complexity bağlamı

- $m=1$: Graph edge içermiyorsa mümkün.
- $m=2$: Bipartite Graph testi BFS/DFS ile $\Theta(V+E)$ zamanda yapılır.
- Fixed $m \geq 3$: General graph için decision problem NP-complettir.
- Chromatic number optimization NP-harddır.

Backtracking exact çözüm verir fakat worst case exponential'dır.

5. State representation

```
color[v] = 0      -> uncolored
color[v] in 1..m  -> assigned color
```

State-space tree:

```
Level 0 -> no vertex colored
Level 1 -> vertex 1 color choice
...
Level V -> complete coloring
```

Branching factor en fazla m dir.

6. Naive state-space size

Constraint check yapılmadan every vertex any of m colors alır:

```
 $m^V$  complete assignments
```

Total tree nodes:

```
1+m+m^2+...+m^V
=(m^(V+1)-1)/(m-1), m>1
= $\Theta(m^V)$ 
```

Video three colors/four vertices için $1+3+3^2+3^3+3^4$ expansion'ını gösterir.

7. Safe color test

Vertex v ye color c vermek güvenlidir eğer hiçbir already-colored neighbor aynı color'a sahip değilse:

```
for each neighbor u of v
  if color[u] == c
    return false
return true
```

Uncolored neighbor $color[u]=0$ conflict oluşturmaz; onun constraint'i kendi turn'ünde veya forward checking ile ele alınır.

8. Basic backtracking pseudocode

```
ColorVertex(v)
  if v == V
    report solution
    return true/continue

  for c = 1 to m
    if IsSafe(v,c)
      color[v] <- c

      if ColorVertex(v+1) and onlyFirstSolutionWanted
        return true

  color[v] <- 0
```

```
return false
```

All solutions isteniyorsa successful child sonrası search devam eder. Decision probleminde first solution bulunduğunda early return yapılabilir.

9. Backtracking trace mantığı Vertex 1 Red olsun. Vertex 2, vertex 1'e adjacent ise Red denenir ve reject edilir; Green safe ise recursion ilerler.

Daha ileride bir vertex için bütün colors adjacent conflicts yüzünden reject edilirse:

1. Current vertex uncolored yapılır.
2. Previous vertex'e dönülür.
3. Previous vertex'in next color'ı denenir.

Bu, feasibility-based pruning'dir.

10. Adjacency representation ve safety cost Adjacency list ile:

```
IsSafe(v,c) =  $\theta(\deg(v))$ 
```

Adjacency matrix ile:

```
IsSafe(v,c) =  $\theta(V)$ 
```

Kaba worst-case runtime:

```
 $O(m^V * V)$ 
```

olarak yazılabilir. Search nodes sayısını vurgulayan yaygın shorthand $O(m^V)$ dir.

Auxiliary assignment/recursion space:

```
 $O(V)$ 
```

11. Vertex ordering neden önemlidir? Fixed order correctness'i bozmaz fakat pruning gücünü etkiler. Daha constrained vertex'i erken seçmek conflicts'i daha hızlı gösterir.

Heuristics:

- Highest degree first
- MRV: En az legal color'ı kalan vertex
- DSATUR: Colored neighbors arasında en çok distinct color gören vertex

Dynamic ordering kullanılıyorsa next vertex index yerine uncolored vertex seçme function'ı gerekir.

12. Color ordering Colors'ın deneme sırası first solution'ın hangisi olacağını değiştirir. Decision probleminde promising color'ı önce denemek runtime'ı azaltabilir.

All labeled solutions enumerate ediliyorsa bütün colors denenmelidir.

13. Symmetry breaking Color names arbitrary ise first vertex'in color'ını sabitlemek mümkündür:

```
color[firstVertex]=1
```

Her unlabeled color partition'ın global color permutations'ından gelen equivalent branches azaltılır.

Ancak amaç labeled color assignments'ın tamamını saymaksa bu symmetry breaking count'u değiştirir; sonuç uygun permutation factor'ıyla yorumlanmalıdır.

14. Forward checking Her uncolored vertex için available colors set'i tutulabilir. Assignment sonrası neighbors'ın domain'inden chosen color çıkarılır.

Bir neighbor'ın domain'i empty olursa branch daha o vertex seçilmeden prune edilir.

Undo sırasında removed domain values geri yüklenmelidir. Bu, Constraint Satisfaction Problem tekniklerine geçiştir.

15. Chromatic number bulma Basit exact yöntem:

```
lowerBound <= m <= upperBound
```

aralığında m-colorability testleri çalıştırmaktır.

Bounds:

- Clique size bir lower bound'dur.
- Greedy coloring'in kullandığı color sayısı upper bound'dur.

Smallest feasible m , $\chi(G)$ dir. Binary search her zaman doğrudan yararlı değildir çünkü her decision testinin maliyeti ve solution reuse stratejisi önemlidir; fakat feasibility monotonic'tir: m ile colorable graph daha fazla colors ile de colorable'dır.

16. Map coloring Map regions graph'a dönüştürülür:

- Her region bir vertex.
- Common boundary segment paylaşan regions arasında edge.

Yalnız bir point'te temas eden regions standard map-coloring modelinde adjacent sayılmaz.

Graph coloring sonucu region colors'a geri aktarılır. Planar maps için Four Color Theorem, four colors'ın her zaman yeterli olduğunu söyler; bazı maps daha azıyla renklendirilebilir.

17. Diğer uygulamalar

- Exam/course scheduling
- Register allocation in compilers
- Frequency/channel assignment
- Timetable construction
- Resource conflict allocation
- Sudoku ve general constraint satisfaction transformations

Edge, aynı resource/time/color'ı paylaşamayan iki entity arasındaki conflict'i temsil eder.

18. Edge cases Self-loop (v, v) edge'i varsa condition:

```
color[v] != color[v]
```

imkânsızdır; hiçbir proper coloring yoktur.

Disconnected graph Components bağımsız color edilebilir fakat aynı global color palette paylaşılır. Chromatic number:

```
 $\chi(G) = \max \text{ of component chromatic numbers}$ 
```

Empty graph Edge olmayan nonempty graph one color ile colorable'dır.

19. Bütün solutions ve output maliyeti Graph sparse/constraint-free olduğunda çok sayıda valid colorings olabilir. Hepsini output etmek solution sayısı kadar kaçınılmaz zaman gerektirir.

Decision version first solution'da durabildiği için enumeration'dan çok daha hızlı tamamlanabilir.

20. Sık yapılan hatalar Only previous vertex ile color karşılaştırmak Current vertex'in bütün colored neighbors'ı kontrol edilmelidir.

Recursive return sonrası color'ı sıfırlamamak Sibling branches yanlış conflict görür.

m-colorable? ile chromatic number'ı aynı problem sanmak Biri fixed-m decision, diğeri minimum-m optimization'dır.

İki-coloring için exponential backtracking kullanmak Bipartite test linear zamanda yapılabilir.

Self-loop'u görmezden gelmek Proper coloring immediately impossible olur.

$O(m^V)$ yi her input'ta gerçekleştiren süre sanmak Worst-case search scale'dir; pruning/ordering practice'i değiştirebilir.

21. Kavram sözlüğü

English term	Türkçe açıklama
Proper Vertex Coloring	Adjacent vertices'e farklı colors veren assignment
m-Colorability	Graph'ın en fazla m colors ile colorable olup olmadığı
Chromatic Number	Gerekli minimum color sayısı $\chi(G)$
Color Conflict	Adjacent vertices'in aynı color'ı alması
Bipartite Graph	Two colors ile proper colorable graph
DSATUR	Saturation degree temelli vertex-ordering heuristic'i
Forward Checking	Future vertices'in domains'ini erken güncelleyen pruning
Symmetry Breaking	Equivalent color-label permutations'ını azaltma

22. Kendini sinama

1. Proper coloring condition nedir?
2. m-colorability ile chromatic number farkı nedir?
3. State-space tree branching factor nedir?
4. IsSafe(v,c) neyi kontrol eder?
5. Worst-case search neden exponential'dır?
6. m=2 için hangi linear-time test kullanılabilir?
7. Vertex ordering runtime'ı nasıl etkiler?
8. Self-loop neden coloring'i imkânsız yapar?
9. Map hangi kuralla graph'a dönüştürülür?

Kısa cevaplar

1. Her edge (u,v) için $color[u] \neq color[v]$.
2. İlki fixed m için yes/no; ikincisi feasible minimum myi bulur.
3. En fazla m.
4. Bütün colored neighbors'ın cden farklı olduğunu.
5. V vertices için m^V assignments bulunabildiği için.
6. BFS/DFS bipartite test.
7. Constrained vertices erken seçilirse branches daha erken prune olabilir.
8. $color[v] \neq color[v]$ sağlanamaz.
9. Regions vertices, common boundary paylaşılan pairs edges yapılır.

Sonraki ders: **67 – Hamiltonian Cycle Using Backtracking.**

67 – Hamiltonian Cycle Using Backtracking

Video: 6.5 Hamiltonian Cycle Using Backtracking (18:35)

Bu videonun ana fikri Hamiltonian Cycle, graph'taki her vertex'i exactly once ziyaret edip başlangıç vertex'ine dönen cycle'dır.

Backtracking, path'i bir vertex at a time kurar. Yeni vertex şu üç condition'ı sağlamalıdır:

1. Previous vertex ile edge/arc bulunmalı.
2. Path'te daha önce kullanılmamış olmalı.
3. Last position'daysa start'a dönüş edge'i bulunmalı.

Invalid partial paths erkenden prune edilir.

1. Problem tanımı Graph $G=(V,E)$ için Hamiltonian cycle:

$v_0, v_1, \dots, v_{(n-1)}, v_0$

sequence'idir ve:

- $v_0 \dots v_{(n-1)}$ bütün vertices'i birer kez içerir.
- Her consecutive pair arasında edge vardır.
- Last vertex ile first vertex arasında closing edge vardır.

Directed graph'ta edge directions da sequence ile uyuşmalıdır.

2. Hamiltonian Path ile farkı Hamiltonian Path, her vertex'i exactly once ziyaret eder fakat start'a dönmek zorunda değildir.

Hamiltonian Path → return edge gerekmez
Hamiltonian Cycle → return edge zorunlu

3. TSP ile farkı Traveling Salesman Problem da Hamiltonian cycle arar fakat weighted graph'ta minimum-cost olanı seçer.

- Hamiltonian-cycle decision: Böyle bir cycle var mı?
- Hamiltonian-cycle enumeration: Cycles nelerdir?
- TSP optimization: Minimum-cost cycle hangisi?

Backtracking yalnız feasibility'yi, TSP ise objective'i de değerlendirir.

4. Complexity sınıfı Hamiltonian Cycle decision problemi general directed ve undirected graphs için NP-complete'tir. Bir cycle verildiğinde doğrulamak polynomial'dır; cycle'ın varlığını exact biçimde bulmak için bilinen general polynomial-time algorithm yoktur.

Search/enumeration ve optimization variants en az bu decision problemi kadar zordur (NP-hard).

5. Necessary fakat sufficient olmayan conditions Undirected graph'ta Hamiltonian cycle varsa:

- Graph connected olmalıdır.
- Her vertex degree en az 2 olmalıdır.
- $n \geq 3$ için articulation point bulunmamalıdır.
- Bridge bulunmamalıdır.

Bu conditions necessary'dir fakat tek başına cycle garantisi vermez.

Directed graph için strong connectivity ve her vertex'te en az bir incoming/outgoing arc gerekli conditions'dır; yine sufficient değildir.

6. Pendant vertex ve articulation point Pendant Vertex degree 1 olan vertex'tir. Cycle vertex'e bir edge ile girip farklı edge ile çıkmak zorunda olduğundan pendant vertex Hamiltonian cycle'da bulunamaz. Articulation point kaldırıldığında graph parçalanır. Hamiltonian cycle içeren undirected graph, tek vertex kaldırıldığında remaining cycle path üzerinden connected kalacağından articulation point içeremez.

7. Videodaki cycle örnekleri Beş-vertex example graph'ta gösterilen iki cycle:

```
1 -> 2 -> 3 -> 4 -> 5 -> 1
1 -> 2 -> 5 -> 4 -> 3 -> 1
```

Her sequence bütün five vertices'i birer kez içerir ve start'a döner.

8. Duplicate cycles Aynı cycle farklı start'tan yazılabilir:

```
1-2-3-4-5-1
2-3-4-5-1-2
```

Bunlar rotation bakımından aynı cycle'dır. Start vertex'i fixed yapmak rotational duplicates'i kaldırır.

Undirected graph'ta reverse order da aynı edge cycle'dır:

```
1-2-3-4-5-1
1-5-4-3-2-1
```

Unique undirected cycles için ek symmetry rule kullanılabilir:

```
path[1] < path[n-1]
```

Directed graph'ta reversed sequence valid veya aynı cycle olmak zorunda değildir.

9. State representation

```
path[position] = cycle'daki position'a yerleştirilen vertex
used[v]         = v daha önce path'e eklendi mi?
```

Start fixed:

```
path[0]=start
used[start]=true
```

Level position, partial path'teki vertex sayısını gösterir.

10. Promising test Candidate v, positiona konabilir eğer:

```
adj[path[position-1]][v] is true
used[v] is false
```

Complete position sonrası:

```
adj[path[n-1]][path[0]] is true
```

olmalıdır.

11. Backtracking pseudocode

```
Hamiltonian(position)
    if position == n
        if edge(path[n-1],path[0]) exists
```

```

        report path + path[0]
    return

    for v = 0 to n-1
        if not used[v]
            and edge(path[position-1],v) exists

            path[position] <- v
            used[v] <- true

            Hamiltonian(position+1)

            used[v] <- false
            path[position] <- UNASSIGNED

```

Initial call:

```

path[0]=start
Hamiltonian(1)

```

12. Closing-edge check neden leaf'te yapılır? Bir path bütün vertices'i ziyaret etse bile last vertex start'a adjacent değilse Hamiltonian path olabilir fakat Hamiltonian cycle değildir.

Bu yüzden complete state output edilmeden önce closing edge doğrulanır.

Bazı implementations final vertex candidate'ı eklerken aynı check'i erken yapar; iki yöntem eşdeğerdir.

13. Video algorithm'ındaki next-value yaklaşımı Video $x[k]$ position'ı için vertex labels'ı modular biçimde sırayla dener. 0 değeri "bütün candidates denendi, backtrack et" sentinel'idir.

Conceptual checks:

```

edge(x[k-1],x[k]) exists
x[k] not in x[0..k-1]
if k==n-1: edge(x[k],x[0]) exists

```

Modern pseudocode'da explicit for each candidate v ve used array aynı mantığı daha açık gösterir.

14. First, all ve count variants

- Decision: First cycle bulununca true döndür.
- Search: Bir cycle döndür.
- Enumeration: Her complete valid path'i output et.
- Count: Counter artır; duplicate convention'ı açıkça belirle.

All unique cycles output ediliyorsa output sayısı kendisi factorial/exponential olabilir.

15. Complexity Start fixed ve repeated vertices yasak olduğundan complete permutations en fazla:

```

(n-1)!

```

Adjacency-matrix edge check ve used array membership $O(1)$ ise kaba search time:

```

O(n!)

```

Video $O(n^n)$ verir; $n! = O(n^n)$ doğru fakat daha gevşek upper bound'dur. Bu representation için factorial bound daha açıklayıcıdır.

Auxiliary path, used ve recursion:

```

Space = O(n)

```

Adjacency matrix input $\Theta(n^2)$ yer kaplar.

16. Stronger pruning fikirleri

Sound necessary checks partial search'i hızlandırabilir:

- Remaining vertex'in possible degree'si $2n$ in altına düştüyse prune.
- Partial path premature smaller cycle kapatıyorsa prune.
- Remaining induced connectivity imkânsızsa prune.
- Symmetry breaking kullan.

Bu checks worst-case NP-complete nature'ı kaldırmaz.

17. Eulerian Cycle ile farkı

Hamiltonian → her vertex exactly once
Eulerian → her edge exactly once

Eulerian-cycle existence degree/connectivity conditions ile polynomial zamanda test edilebilir. Hamiltonian cycle için aynı kadar basit characterization bilinmez.

18. Sık yapılan hatalar Start'a dönüş edge'ini kontrol etmemek

Hamiltonian path yanlışlıkla cycle sayılır.

Vertex tekrarına izin vermek Exactly-once constraint bozulur.

Start'ı her branch'te yeniden seçmek Rotational duplicates çoğalır.

Connected graph'ın mutlaka Hamiltonian olduğunu sanmak Connectivity yalnız necessary condition'dır.

Undirected reversal duplicates'ını saymak Counting convention normalize edilmelidir.

Hamiltonian ile Eulerian'ı karıştırmak Biri vertices, diğeri edges üzerindedir.

19. Kavram sözlüğü

English term	Türkçe açıklama
Hamiltonian Cycle	Her vertex'i exactly once ziyaret edip start'a dönen cycle
Hamiltonian Path	Her vertex'i exactly once içeren, kapanması gerek-meyen path
Closing Edge	Last vertex'ten start'a dönen edge
Pendant Vertex	Degree'si one olan vertex
Rotation Duplicate	Aynı cycle'ın farklı start vertex'ten yazımı
Reversal Duplicate	Undirected cycle'ın ters yönde yazımı
NP-complete	Hem NP'de olan hem bütün NP problemleri kadar zor decision sınıfı
Symmetry Breaking	Equivalent paths'i search'ten kaldırma

20. Kendini sına

1. Hamiltonian cycle'ın üç temel özelliği nedir?
2. Hamiltonian path'ten farkı nedir?
3. TSP ile ilişkisi nedir?
4. Candidate vertex için hangi checks yapılır?
5. Start neden fixed tutulur?
6. Undirected reversal duplicate nasıl azaltılabilir?
7. Pendant vertex neden cycle'ı engeller?

8. Tighter worst-case permutation bound nedir?

Kısa cevaplar

1. Her vertex once, consecutive edges mevcut, start'a dönüş mevcut.
2. Path'in kapanması gerekmez.
3. TSP minimum-cost Hamiltonian cycle arar.
4. Previous edge mevcut, unused, leaf'te closing edge mevcut.
5. Rotational duplicates'i kaldırmak için.
6. Örneğin $path[1] < path[n-1]$ symmetry rule'uyla.
7. Cycle'a girip çıkmak için iki incident edge gerektiğinden.
8. Start fixed iken $(n-1)!$, asymptotically $O(n!)$.

Sonraki ders: **68 – Branch and Bound: Introduction.**

68 – Branch and Bound: Introduction

Video: 7.1 Branch and Bound: Introduction (9:40)

Bu videonun ana fikri Branch and Bound (B&B), discrete/combinatorial optimization problems'ını exact çözen state-space search strategy'sidir.

- Branch: Problem'i decision children'a ayır.
- Bound: Partial node'un ulaşabileceği en iyi objective için optimistic limit hesapla.
- Prune: Bu limit mevcut best feasible solution'ı yenemiyorsa node'u kapat.

Traversal FIFO, LIFO veya priority-based best-first olabilir.

1. Optimization context B&B hem:

- Minimization
- Maximization

problemlerini doğrudan çözebilir.

Videoda yalnız minimization için olduğu ve maximization'ın sign değiştirerek çevrilmesi gerektiği söylenir. Sign conversion mümkündür fakat zorunlu değildir; maximization için optimistic upper bound kullanmak standard yaklaşımdır.

2. State-Space Tree Her node bir partial decision/state'tir. Children next decision possibilities'i temsil eder.

Örnek binary representation:

```
x[k]=1 -> item/job included  
x[k]=0 -> excluded
```

Fixed-size vector her candidate için n decisions tutar. Subset representation yalnız included elements'i saklayabilir ve variable length olabilir.

3. Feasible solution ve incumbent Şimdiye kadar bulunan en iyi complete feasible solution'a Incumbent denir.

Minimization:

```
incumbentValue = current best upper bound on optimum
```

Maximization:

```
incumbentValue = current best lower bound on optimum
```

Yeni daha iyi feasible solution bulununca incumbent güncellenir.

4. Minimization pruning rule Node N için optimistic Lower Bound $LB(N)$ hesaplanır:

```
 $LB(N) \leq N\text{'den tamamlanabilecek her feasible solution cost'u}$ 
```

Eğer:

```
 $LB(N) \geq incumbentCost$ 
```

ise node daha iyi solution veremez ve prune edilir.

All optimal solutions gerekiyorsa equality branches'ini korumak için $>$ kullanmak gerekebilir.

5. Maximization pruning rule Node için optimistic Upper Bound $UB(N)$:

```
UB(N) >= N'den tamamlanabilecek her feasible solution profit'i
```

Eğer:

```
UB(N) <= incumbentProfit
```

ise node prune edilir.

Bound optimism yönü yanlış kurulursa optimal solution içeren branch kesilebilir.

6. Live, E-node ve dead node

- Live Node: Üretilmiş fakat henüz expand edilmemiş node
- E-node/Expansion Node: Şu anda children'ı üretilen node
- Dead Node: Prune edilmiş veya artık expand edilecek child'ı olmayan node

Live-node container traversal policy'yi belirler.

7. FIFO Branch and Bound

 Live nodes Queueda tutulur:

```
first generated -> first expanded
```

Bu breadth-first/level-order expansion verir.

Avantajı basitliktir; dezavantajı geniş level'da çok fazla memory kullanabilmesidir.

8. LIFO Branch and Bound

 Live nodes Stackte tutulur:

```
last generated -> first expanded
```

Video otomatik captions'ta LIFO gibi duyulan terim LIFOdur.

Bu policy depth-first-benzeri expansion verir ve hızlı incumbent bulabilir; fakat promising başka branch'i geç ziyaret edebilir.

9. Least-Cost / Best-First Branch and Bound

 Minimization'da smallest lower bound'a sahip live node priority queue'dan seçilir:

```
expand argmin LB(node)
```

Maximization'da largest upper bound seçilir:

```
expand argmax UB(node)
```

Bu Best-First Branch and Bounddur.

Videoda least-cost'un mutlaka daha hızlı olduğu söylenir; çoğu problemde iyi bounds ile avantajlı olabilir fakat her input'ta garanti edilmez. Priority-queue overhead ve bound quality önemlidir.

10. "B&B her zaman BFS" düzeltmesi

 Bir node expand edildiğinde immediate children'ın oluşturulması, entire traversal'ın BFS olduğu anlamına gelmez.

- FIFO -> BFS/level-order
- LIFO -> DFS-like
- Least-cost -> best-first

Branch and Bound'ı tanımlayan şey tek bir traversal order değil, objective bounds ile exact pruning'dir.

11. Cost ve bound ayırımı

 Cost terimi problem/video arasında farklı kullanılabilir:

- Accumulated actual partial cost
- Node lower bound

- Feasible completion cost

Derslerde belirsizliği önlemek için explicit isimler kullanılmalıdır:

```
partialCost
lowerBound
upperBound
incumbent
```

Bound complete feasible solution olmak zorunda değildir; optimistic relaxation'dan gelebilir. Incumbent ise mutlaka feasible olmalıdır.

12. Bound nasıl elde edilir? Problem'e özel relaxation kullanılır:

- 0/1 Knapsack: Fractional Knapsack upper bound
- TSP: Row/column matrix reduction lower bound
- Scheduling: Remaining penalties/profits üzerinden bound
- Assignment Problem: Row minima lower bound

Tight bound daha çok prune eder fakat hesaplaması daha pahalı olabilir. İyi design bound cost ile pruning gücü arasında denge kurar.

13. Generic minimization pseudocode

```
BranchAndBound(root)
  incumbentCost <- INF
  incumbentSolution <- NIL
  push root with LB(root)

  while live container not empty
    node <- select according to policy

    if LB(node) >= incumbentCost
      continue

    if node is complete and feasible
      if cost(node) < incumbentCost
        incumbentCost <- cost(node)
        incumbentSolution <- node
      continue

    for each child in Branch(node)
      if child infeasible
        continue

      compute LB(child)

      if LB(child) < incumbentCost
        push child

  return incumbentSolution
```

14. Correctness intuition Pruned node ancak optimistic best-case'i bile incumbent'tan daha iyi değilse silinir. Bu nedenle node'un hiçbir descendant'ı optimal incumbent'ı yenemez.

Live nodes tükendiğinde veya en promising live bound incumbent'ı yenemediğinde incumbent global optimum'dur.

Correctness bound'ın validity'sine ve branching'in bütün feasible solutions'ı kapsamasına dayanır.

15. Backtracking ile farkı

Özellik	Backtracking	Branch and Bound
Typical goal	Feasibility/enumeration	Optimization
Main pruning	Constraint violation	Objective bound + feasibility
Node order	Genellikle DFS	FIFO, LIFO, best-first
Incumbent	Gerekli olmayabilir	Temel yapı
Bound	Genellikle feasibility	Objective'in optimistic limiti

Sınırlar keskin değildir; optimization backtracking ve depth-first B&B pratikte benzer görünebilir.

16. Complexity B&B worst case'de state-space tree'nin tamamını explore edebilir:

Time = exponential/factorial depending on problem

Memory:

- LIFO/DFS-like çoğu zaman current depth ile sınırlı olabilir.
- FIFO/best-first çok sayıda live node tutup exponential memory kullanabilir.

B&B worst-case hardness'i kaldırmaz; good bounds real instances'i çözülebilir hâle getirir.

17. Initial incumbent nasıl iyileştirilir? Fast heuristic ile erken feasible solution bulunursa incumbent güçlü olur ve daha fazla node prune edilir.

Örneğin:

- Knapsack greedy feasible selection
- TSP nearest-neighbor tour
- Scheduling greedy schedule

Heuristic solution optimal olmak zorunda değildir; yalnız feasible olması gerekir.

18. Sık yapılan hatalar Maximization'ı B&B ile doğrudan çözülemez sanmak Optimistic upper bound ve best feasible lower bound kullanılabilir.

Bound'ı feasible solution sanmak Relaxation bound infeasible/fractional olabilir.

Invalid bound ile prune etmek Optimal branch kaybolabilir; correctness bozulur.

Her policy'yi BFS sanmak LIFO DFS-like, priority policy best-first'tür.

Least-cost'un her zaman daha hızlı olduğunu varsaymak Input, bound quality ve overhead sonucu etkiler.

Incumbent'ı infeasible state ile güncellemek Pruning yanlış ve tehlikeli olur.

19. Kavram sözlüğü

English term	Türkçe açıklama
Branch and Bound	Objective bounds ile exact state-space search
Branch	Node'u child decisions'a ayırma
Bound	Descendants'ın ulaşabileceği objective için optimistic limit
Incumbent	Şimdiye kadarki en iyi feasible solution
Live Node	Üretilmiş fakat expand edilmemiş node
Dead Node	Prune edilmiş veya tamamlanmış node
FIFO B&B	Queue tabanlı level-order policy
LIFO B&B	Stack tabanlı depth-first-benzeri policy
Least-Cost B&B	En promising bound'ı priority ile seçen policy

20. Kendini sınama

1. Branch, bound ve prune ne demektir?
2. Incumbent nedir?
3. Minimization pruning condition nedir?
4. Maximization pruning condition nedir?
5. FIFO, LIFO ve least-cost hangi structures'ı kullanır?
6. Bound feasible olmak zorunda mıdır?
7. B&B yalnız minimization için midir?
8. Worst-case neden exponential kalabilir?

Kısa cevaplar

1. Decisions'a ayırma, optimistic limit hesaplama, yenemeyen branch'i kesme.
2. Şimdiye kadarki en iyi complete feasible solution.
3. $LB(node) \geq incumbentCost$.
4. $UB(node) \leq incumbentProfit$.
5. Queue, stack ve priority queue.
6. Hayır; optimistic relaxation olabilir.
7. Hayır, maximization'ı da doğrudan çözer.
8. Bounds hiçbir node'u prune edemeyebilir.

Sonraki ders: **69 – Job Sequencing with Deadline: Branch and Bound.**

69 – Job Sequencing with Deadline: Branch and Bound

Video: 7.2 Job Sequencing with Deadline – Branch and Bound (10:56)

Bu videonun ana fikri Jobs'ın processing times, deadlines ve missed-job penalties'i vardır. Tek machine'da deadline'larına kadar tamamlanabilecek bir subset seçilir; yapılmayan jobs'ın total penalty'si minimize edilir.

Bir partial subset node'u için video iki değer kullanır:

```
lowerBound = şimdiye kadar kesin dışarıda bırakılmış jobs'ın penalties'i  
feasibleCompletionCost = node'daki jobs dışında bütün jobs'ı atmanın penalty'si
```

İkinci value feasible incumbent'ı günceller. Eğer node lower bound mevcut best penalty'yi yenemiyorsa prune edilir.

1. Problem modeli Her job j:

```
processing time t[j]  
deadline d[j]  
penalty p[j]
```

değerlerine sahiptir.

Selected jobs deadline'larına kadar tamamlanır. Unselected job için penalty ödenir:

```
minimize sum(p[j] for unselected jobs)
```

Equivalent maximization:

```
maximize saved penalty of selected jobs
```

2. B&B maximization'ı da doğrudan çözebilir Video problemi minimization convention'ına çevirmek için profit yerine penalty kullanır. Bu geçerli ve öğretici bir transformation'dır.

Ancak Branch and Bound yalnız minimization'a bağlı değildir. Saved penalty maximization da optimistic upper bound ile doğrudan çözülebilir.

3. Videodaki data Penalties:

```
J1=5, J2=10, J3=6, J4=3
```

Processing times:

```
t1=1, t2=2, t3=1, t4=1
```

Caption anlatımındaki önemli deadlines:

```
d1=1, d2=3, d3=2
```

Video table'ındaki fourth job da horizon 3 içinde bir deadline'a sahiptir. Gösterilen feasibility checks ve optimum için kritik sonuç: J2, J3 schedule edilebilir, bunlara J4 eklenirse total processing time 4 olduğundan deadline constraints sağlanamaz.

4. Bir selected set feasible mi? Single-machine, release-time olmayan bu feasibility testinde selected jobs nondecreasing deadline sırasıyla (Earliest Deadline First) dizilir.

```
time=0
for job in sorted(selected, by deadline)
    time += processing[job]
    if time > deadline[job]
        infeasible
```

Bu schedule bütün selected jobs'ı on time tamamlayabiliyorsa set feasible'dır.

5. Örnek feasible set

$S=\{J2, J3\}$

Deadline order'da:

```
J3: finish at 1 <= d3=2
J2: finish at 3 <= d2=3
```

Feasible'dır.

Penalty:

```
unselected={J1, J4}
5+3=8
```

6. Subset state-space representation Video selected jobs'ı increasing index order'da ekler. Node:

```
S = selected feasible jobs
k = S'deki last considered/included job index
```

Children yalnız $j > k$ jobs'ından oluşturulur. Bu, aynı subset'in farklı permutations ile tekrar üretilmesini önler.

7. Node lower bound Node'un last index'i k olsun. $1..k$ arasında S de bulunmayan jobs artık future child'da eklenemez; onların penalties'i kesin ödenecektir:

```
LB(S,k)=sum(p[j] for j<=k and j not in S)
```

Future unconsidered jobs'ın penalty contribution'ı optimistic olarak zero kabul edilir. Bu nedenle valid lower bound'dur.

Video buna cost der.

8. Feasible completion upper cost Node'daki selected set S feasible ise, remaining bütün jobs'ı skip etmek de feasible complete solution verir:

```
completionPenalty(S)=sum(p[j] for j not in S)
```

Bu minimization optimum'u için bir upper bound/incumbent candidate'dır.

Video buna node upper bound der ve global upper value'yu küçültür.

9. Root children Initial best:

```
incumbent=INF
```

$S=\{J1\}$

```
LB=0
completion penalty=10+6+3=19
incumbent=19
```

S={J2}

```
LB=penalty(J1)=5
completion penalty=5+6+3=14
incumbent=14
```

S={J3}

```
LB=5+10=15 >= 14
prune
```

S={J4}

```
LB=5+10+6=21 >= 14
prune
```

10. Expanding {J1} {J1, J2}

```
LB=0
completion penalty=6+3=9
incumbent=9
```

{J1, J3} J2 skipped:

```
LB=10 >= 9
prune
```

{J1, J4} J2, J3 skipped:

```
LB=10+6=16 >= 9
prune
```

11. Expanding {J2} {J2, J3}

```
LB=penalty(J1)=5
completion penalty=penalty(J1)+penalty(J4)=5+3=8
incumbent=8
```

{J2, J4}

```
LB=penalty(J1)+penalty(J3)=5+6=11 >= 8
prune
```

12. Feasibility pruning Subset {J1, J2} total processing time 3tür ve kendi deadlines'ında feasible olabilir; ek J3 veya J4 total time'ı 4e çıkarıp video instance'ındaki deadline horizon'ı aşar.

{J2, J3} zaten time 3 kullanır; J4 eklendiğinde feasible schedule yoktur.

Objective bound'dan bağımsız olarak infeasible children üretilmez.

13. Final solution

```
Selected jobs = {J2, J3}
Unselected jobs = {J1, J4}
```

Minimum penalty = $5+3 = 8$
Saved penalty = $10+6 = 16$

Bir feasible execution order:

J3 → J2

14. Direct maximization formulation Total penalty:

$5+10+6+3=24$

Min penalty 8 ile max saved penalty:

$24-8=16$

Maximization B&B'de:

- currentSaved feasible lower bound/incumbent candidate olur.
- Remaining jobs'ın penalties'i optimistic eklenerek upper bound bulunur.
- $UB \leq \text{bestSaved}$ ise prune edilir.

Aynı optimum elde edilir.

15. Pseudocode skeleton

```
Search(selected, lastIndex)
  if not Feasible(selected)
    return

  completion ← penalty of all jobs not in selected
  if completion < incumbent
    incumbent ← completion
    bestSet ← selected

  for j = lastIndex+1 to n
    child ← selected union {j}

    if not Feasible(child)
      continue

    lower ← penalties of skipped jobs through j
    if lower < incumbent
      push/expand child
```

Node selection FIFO, LIFO veya best-bound olabilir.

16. Bounds'in correctness'i Lower bound yalnız artık geri alınamayacak skipped penalties'i sayar. Future penalties'i zero varsaymak optimistic'tir; gerçek completion bundan daha ucuz olamaz.

Completion penalty ise selected feasible set'i aynen tutup bütün remaining jobs'ı skip eden gerçek feasible solution'dır. Bu nedenle incumbent olarak güvenle kullanılabilir.

17. Complexity Worst case'de bütün subsets feasible ve bounds zayıf olabilir:

Time = $O(2^n * \text{feasibilityCheck})$

Naive EDD feasibility her node'da sort edilirse extra $O(n \log n)$ maliyet doğabilir. Incremental/önceden deadline-order structures ile azaltılabilir.

Live-node policy'ye göre memory exponential olabilir.

18. Sık yapılan hatalar Selected set'i feasibility check etmeden incumbent yapmak Infeasible schedule pruning'i yanlış yönlendirir.

Node cost ile actual final penalty'yi karıştırmak Video cost'u optimistic lower bound'dur.

Deadline'ları yalnız total time ile kontrol etmek Her prefix completion kendi job deadline'ını sağlamalıdır.

Jobs'ı index order'da execute etmek zorunda sanmak State generation index order'ı duplicate önler; actual schedule EDD order olabilir.

B&B'yi yalnız minimization'a özgü sanmak Saved-penalty maximization doğrudan çözülebilir.

19. Kavram sözlüğü

English term	Türkçe açıklama
Penalty	Job tamamlanmazsa ödenen objective cost
Processing Time	Job'ın machine üzerinde kullandığı süre
Deadline	Job'ın tamamlanması gereken latest time
Earliest Deadline First	Jobs'ı artan deadline sırasına koyma
Lower Bound	Partial node'dan kaçınılamayan minimum penalty
Incumbent	Şimdiye kadarki en düşük feasible total penalty
Feasible Completion	Partial selected set'ten oluşturulan complete valid schedule
Saved Penalty	Selected jobs sayesinde ödenmeyen penalty toplamı

20. Kendini sınamaya

1. Objective nedir?
2. Selected set feasibility nasıl test edilir?
3. Node lower bound neyi toplar?
4. Feasible completion penalty nasıl bulunur?
5. Root'ta {J2} incumbent'ı kaçta indirir?
6. Optimal selected jobs nelerdir?
7. Minimum penalty ve saved penalty nedir?
8. State generation order ile execution order neden farklı olabilir?

Kısa cevaplar

1. Unselected jobs'ın total penalty'sini minimize etmek.
2. Selected jobs EDD sırasına konup her completion deadline ile karşılaştırılarak.
3. Artık seçilemeyecek skipped jobs'ın penalties'ini.
4. Selected set dışındaki bütün jobs'ın penalties'i toplanarak.
5. 14.
6. {J2, J3}.
7. 8 ve 16.
8. Index order duplicate subsets'i önler; feasible machine order deadlines'a göre seçilir.

Sonraki ders: **70 – 0/1 Knapsack Using Branch and Bound.**

70 – 0/1 Knapsack Using Branch and Bound

Video: 7.3 0/1 Knapsack Using Branch and Bound (10:48)

Bu videonun ana fikri 0/1 Knapsack Branch and Bound çözümünde her node first level items için include/exclude decisions'ı temsil eder.

Node'un gerçek feasible profit'i incumbent'ı günceller. Remaining capacity'nin Fractional Knapsack ile doldurulması, 0/1 optimum için optimistic upper bound verir:

```
if fractionalBound(node) <= bestFeasibleProfit
    prune node
```

Video profit signs'ını negatife çevirerek least-cost minimization uygular; derste aynı hesap daha doğal direct maximization notation'ıyla açıklanır.

1. Videodaki instance

Item	Profit	Weight	profit/weight
1	10	2	5.0
2	10	4	2.5
3	12	6	2.0
4	18	9	2.0

Capacity = 15

Items nonincreasing ratio order'dadır. Fractional bound'ın greedy hesaplanması için bu sorting önemlidir.

2. Node state

```
level   = son karar verilen item index'i
weight  = included items total weight'i
profit  = included items total profit'i
bound   = node descendants'ının ulaşabileceği optimistic max profit
choices = partial 0/1 vector
```

Overweight node infeasible'dır ve hemen elenir.

3. Fractional upper bound Node'dan sonra undecided items ratio order'da ele alınır:

1. Sığan whole items eklenir.
2. Next item tam sığmıyorsa remaining capacity kadar fraction eklenir.

```
bound = currentProfit

for undecided item j
    if currentWeight+w[j] <= capacity
        take whole item
    else
        bound += remainingCapacity * p[j]/w[j]
        break
```

0/1 problem fraction'a izin vermediği için relaxation original feasible set'i genişletir. Bu yüzden fractional optimum gerçek 0/1 optimum'dan küçük olamaz; valid upper bound'dur.

4. Root bound Whole first three items:

```
weight=2+4+6=12
profit=10+10+12=32
```

Remaining capacity:

```
15-12=3
```

Item 4 fraction:

```
18/9 * 3 = 6
```

Root fractional bound:

```
32+6=38
```

Whole-item greedy feasible solution:

```
items {1,2,3}, profit=32
```

initial incumbent 32 olabilir.

5. Video sign convention Video maximization'ı minimization'a dönüştürür:

```
least-cost value = -fractionalBound
feasible upper value in minimization = -wholeProfit
```

Root:

```
cost=-38
incumbent=-32
```

More negative, original problemde daha yüksek profit demektir.

Bu transformation geçerlidir fakat doğrudan max-priority upper-bound formulation daha okunaklıdır.

6. First decision Include item 1

```
weight=2
profit=10
fractional bound=38
```

Exclude item 1 Items 2 ve 3 whole:

```
weight=10
profit=22
```

Remaining 5 capacity item 4 fraction'ı:

```
5*(18/9)=10
```

```
bound=22+10=32
```

bound=32 mevcut incumbent 32yi strict olarak iyileştiremez; yalnız one optimum isteniyorsa prune edilebilir.

7. Include item 1 branch'inde item 2 Include item 2


```
weight=6
profit=20
bound=38
```

Exclude item 2 Item 3 whole:

```
weight=2+6=8
profit=10+12=22
```

Remaining 7 capacity item 4 fraction:

```
7*(18/9)=14
```

```
bound=36
```

Both nodes initially incumbent 32'yi aşma potansiyeline sahiptir.

8. Include items 1 ve 2 branch'inde item 3 Include item 3

```
weight=12
profit=32
bound=32 + 3*(18/9) = 38
```

Item 4 whole include edilirse weight 21 olur ve infeasible'dır.

Exclude item 3 Item 4 whole sağlar:

```
weight=2+4+9=15
profit=10+10+18=38
```

Bu complete feasible selection fractional bound'a eşittir:

```
bestProfit=38
```

9. Pruning sonrası optimum

Incumbent 38 olduğunda remaining nodes için:

```
bound <= 38
```

ise daha yüksek profit mümkün değildir. One optimal solution yeterliyse hepsi prune edilir.

Optimal vector:

```
x=[1,1,0,1]
```

```
selected items={1,2,4}
total weight=2+4+9=15
maximum profit=10+10+18=38
```

10. Best-first maximization pseudocode

```
KnapsackBnB(items,capacity)
  sort items by nonincreasing profit/weight

  bestProfit <- profit of a fast feasible solution
  bestChoices <- its choices

  root.bound <- FractionalBound(root)
```

```

push root into max-priority queue by bound

while queue not empty
    node <- pop maximum bound

    if node.bound <= bestProfit
        continue

    create include child
    if include.weight <= capacity
        if include.profit > bestProfit
            update incumbent
            include.bound <- FractionalBound(include)
            if include.bound > bestProfit
                push include

    create exclude child
    exclude.bound <- FractionalBound(exclude)
    if exclude.bound > bestProfit
        push exclude

return bestProfit, bestChoices

```

11. Bound calculation'da forced decisions Node level'ından önce:

- Included items kesin alınır.
- Excluded items kesinlikle bound'a eklenmez.
- Yalnız undecided suffix fractional relaxation ile doldurulur.

Excluded item'ı bound hesabına yanlışlıkla geri eklemek invalid/çok gevşek state semantics üretir.

12. Ratio sorting neden gerekir? Fractional Knapsack greedy optimum'u descending p/w order ile bulunur. Items arbitrary order'da fractionally doldurulursa hesap gerçek fractional optimum'dan küçük çıkabilir.

Maximization upper bound'ın gerçek attainable 0/1 profit'ten küçük olması unsafe pruning'e yol açabilir. Bu nedenle sorting veya exact fractional relaxation zorunludur.

13. Tie ve equality pruning Yalnız maximum value ve bir solution isteniyorsa:

```
bound <= bestProfit
```

prune edilebilir.

Bütün optimal subsets isteniyorsa $\text{bound} == \text{bestProfit}$ branches başka optimum içerebilir; pruning condition $\text{strict} <$ olmalı ve equal complete solutions saklanmalıdır.

14. Complexity Sorting:

```
0(n log n)
```

Worst-case state-space:

```
0(2^n) nodes
```

Bound naive olarak her node'da suffix tararsa $0(n)$ maliyetle kaba time $0(n^2n)$ olabilir. Prefix/optimized calculations constant factors'ı azaltır.

Priority queue çok sayıda live node tutarsa worst-case space exponential olabilir.

15. B&B ve DP karşılaştırması

Özellik	Branch and Bound	Capacity DP
Worst time	Exponential	$\Theta(nW)$ pseudo-polynomial
Capacity büyüklüğü	State count'a doğrudan dimension değil	Table width W
Bound	Fractional relaxation	Exact subproblem recurrence
İyi olduğu durum	Tight bounds, large numeric W	Moderate integer W
Solution	Incumbent choices	Backtrack table/decisions

16. Sık yapılan hatalar Fractional item'ı final solution'a koymak Fraction yalnız bound içindir; output decisions 0/1 dir.

Ratio'ya göre sort etmemek Greedy fractional bound unsafe veya yanlış olabilir.

Overweight child'ın bound'unu kullanmak Node infeasible olduğundan prune edilmelidir.

Bound ile actual profit'i karıştırmak Bound optimistic ve infeasible fraction içerebilir.

Maximization için sign conversion'ı zorunlu sanmak Max-priority upper-bound B&B doğrudan uygulanabilir.

Equality branch'lerini kesip bütün optima beklemek bound==best başka equal solutions içerebilir.

17. Kavram sözlüğü

English term	Türkçe açıklama
Fractional Upper Bound	Fractions'a izin veren relaxation'ın optimistic profit'i
Incumbent Profit	Şimdiye kadarki en yüksek feasible 0/1 profit
Profit-to-Weight Ratio	Fractional greedy order için p/w
Include Child	Current item'ı alan decision node
Exclude Child	Current item'ı almayan decision node
Best-First Search	En yüksek bound'lı node'u önce expand etme
Relaxation	Constraints'in bir kısmını kaldırıp kolay optimistic problem oluşturma

18. Kendini sına

1. Videodaki items ve capacity nedir?
2. Root fractional bound nasıl 38 olur?
3. Bound neden valid upper bound'dır?
4. Items neden ratio order'da olmalıdır?
5. Optimal selected items hangileridir?
6. Maximum profit ve weight nedir?
7. Direct maximization pruning condition nedir?
8. Fraction final solution'da kullanılabilir mi?

Kısa cevaplar

1. $(p,w)=\{(10,2), (10,4), (12,6), (18,9)\}$, capacity 15.
2. İlk üç whole profit 32 ve item 4'ün three-weight fraction profit'i 6.
3. Fractions original feasible set'i genişlettiği için.
4. Fractional relaxation optimum'unu greedy doğru hesaplamak için.
5. $\{1,2,4\}$.
6. Profit 38, weight 15.
7. $bound \leq bestProfit$ ise prune; all optima için equality dikkatle ele alınır.

8. Hayır, yalnız bound hesabındadır.

Sonraki ders: **71 – Traveling Salesman Problem: Branch and Bound.**

71 – Traveling Salesman Problem: Branch and Bound

Video: 7.4 Traveling Salesman Problem – Branch and Bound (24:42)

Bu videonun ana fikri TSP için matrix-reduction Branch and Bound, her partial tour node'una optimistic Lower Bound atar.

Cost matrix'te her row ve column'dan minimum çıkarılır. Çünkü complete tour her vertex için exactly one outgoing ve one incoming edge seçmek zorundadır; zorunlu minima toplamı tour cost'u için lower bound verir.

Child bound:

```
LB(child) = LB(parent)
            + reducedCost[i][j]
            + additionalReductionCost
```

Best-first search en küçük lower-bound node'u expand eder.

1. Problem Weighted directed graph'ta:

- Start 1'den çık.
- Her vertex'i exactly once ziyaret et.
- Start'a dön.
- Total cost'u minimize et.

Bu minimum-cost Hamiltonian cycle problemidir.

2. Videodaki original cost matrix

	1	2	3	4	5
1	INF	20	30	10	11
2	15	INF	16	4	2
3	3	5	INF	2	4
4	19	6	18	INF	3
5	16	4	7	16	INF

Diagonal INF'dir çünkü self-loop tour step'i olarak seçilemez.

Matrix asymmetric olduğundan directed TSP instance'ıdır.

3. Neden row minima lower bound verir? Her complete tour her row'dan exactly one outgoing edge seçer. Row 1'de seçilecek edge cost'u en az row minimum kadar olmalıdır.

Bu minimum'u bütün finite entries'den çıkarmak:

- Relative feasible tour costs'u azaltır.
- Çıkarılan total'ı lower bound'a ekler.
- Her active row'da en az bir zero bırakır.

4. Row reduction Row minima:

```
[10, 2, 2, 3, 4]
```

Sum:

```
10+2+2+3+4=21
```

Row-reduced matrix:

	1	2	3	4	5
1	INF	10	20	0	1
2	13	INF	14	2	0
3	1	3	INF	0	2
4	16	3	15	INF	0
5	12	0	3	12	INF

5. Column reduction

Row reduction sonrası column minima:

[1,0,3,0,0]

Additional sum:

1+3=4

Root lower bound:

21+4=25

Fully reduced root matrix:

	1	2	3	4	5
1	INF	10	17	0	1
2	12	INF	11	2	0
3	0	3	INF	0	2
4	15	3	12	INF	0
5	11	0	0	12	INF

Her active row ve column en az bir zero içerir.

6. Bound 25 ne demektir?

optimal tour cost ≥ 25

demektir. 25'in kendisi feasible tour olmak zorunda değildir. Matrix reduction, one-in/one-out assignment relaxation'ı üzerinden optimistic cost üretir; subtours veya tek connected cycle constraint'i henüz tam uygulanmamış olabilir.

7. Branching from start

Start 1 fixed:

1→2
1→3
1→4
1→5

children oluşturulur.

Bir edge $i \rightarrow j$ seçildiğinde:

1. Row i INF yapılır: i den başka outgoing edge seçilemez.
2. Column j INF yapılır: j ye başka incoming edge seçilemez.
3. Premature return/subtour edge'i yasaklanır.
4. Remaining active matrix yeniden reduce edilir.

8. Premature subtour prevention

Path $1 \rightarrow \dots \rightarrow j$ bütün vertices'i içermeden:

$j \rightarrow 1$

seçilirse küçük cycle erken kapanır ve remaining vertices dışarıda kalır. Bu yüzden leaf'e kadar return edge INF yapılır.

General Little-style TSP B&B arbitrary partial arcs seçiyorsa yalnız reverse edge'i kapatmak yetmez; oluşabilecek her premature subtour'u tespit edip closing arc'ını yasaklamak gerekir.

9. Child bound formula Root reduced matrix'te selected edge'in reduced entry'si $R[i][j]$ olsun:

$$LB(child) = LB(parent) + R[i][j] + newReduction$$

Original edge cost yeniden doğrudan eklenmez; original costs'un zorunlu parçaları önceki reductions içinde zaten bound'a katılmıştır.

10. First-level child costs 1 → 2

```
R[1][2]=10
new reduction=0
LB=25+10=35
```

1 → 3

```
R[1][3]=17
new reduction=11
LB=25+17+11=53
```

1 → 4

```
R[1][4]=0
new reduction=0
LB=25
```

1 → 5

```
R[1][5]=1
new reduction=5
LB=25+1+5=31
```

Least-bound node:

1→4 with LB=25

11. Matrix after selecting 1 → 4 Row 1, column 4 ve premature edge 4→1 kapatılır:

	1	2	3	4	5
1	INF	INF	INF	INF	INF
2	12	INF	11	INF	0
3	0	3	INF	INF	2
4	INF	3	12	INF	0
5	11	0	0	INF	INF

Active rows/columns zaten reduced'dır; extra reduction zero.

12. Expanding vertex 4 Remaining possible next vertices:

4→2, 4→3, 4→5

Video calculations:

```
path 1->4->2 : LB=28
path 1->4->3 : LB=50
path 1->4->5 : LB=36
```

Best live node:

```
1->4->2, LB=28
```

For 4->2, current reduced entry 3, extra reduction zero:

```
25+3=28
```

13. Expanding path 1 -> 4 -> 2 Remaining vertices 3 ve 5:

```
1->4->2->3 : LB=52
1->4->2->5 : LB=28
```

Least-bound choice:

```
2->5
```

Then yalnız vertex 3 kalır:

```
1->4->2->5->3
```

Closing edge:

```
3->1
```

14. Feasible tour ve actual cost Tour:

```
1 -> 4 -> 2 -> 5 -> 3 -> 1
```

Original matrix costs:

```
1->4 = 10
4->2 = 6
2->5 = 2
5->3 = 7
3->1 = 3
```

```
total=10+6+2+7+3=28
```

Leaf lower bound da 28dir; feasible actual tour cost ile eşleşir.

15. Incumbent update ve pruning Leaf tour bulununca:

```
incumbentCost=28
```

Remaining live nodes:

```
LB >= 28
```

ise one optimum isteniyorsa prune edilebilir. Video'daki diğer live lower bounds 31,35,36,50,52,53 gibi values olduğundan hiçbiri 28i yenemez.

Sonuç global optimum'dur.

16. Best-first pseudocode

```
TSPBranchAndBound(cost)
  rootMatrix, rootLB <- Reduce(cost)
  incumbent <- INF
  bestTour <- NIL
  push root into min-priority queue by LB

  while queue not empty
    node <- pop minimum LB

    if node.LB >= incumbent
      continue

    if node path contains all vertices
      if closing edge exists
        actual <- actual tour cost
        if actual < incumbent
          incumbent <- actual
          bestTour <- node path + start
      continue

    for each legal next vertex j
      childMatrix <- copy node.matrix
      enforce chosen edge and subtour constraints
      reduction <- Reduce(childMatrix)
      child.LB <- node.LB
                    + node.matrix[current][j]
                    + reduction

      if child.LB < incumbent
        push child

  return incumbent, bestTour
```

17. Early feasible upper bound Video incumbent'ı first leaf'e kadar **INF** tutar. Correct olsa da fast heuristic tour ile başlangıçta finite incumbent elde etmek pruning'i güçlendirebilir:

- Nearest Neighbor
- Greedy insertion
- Local search tour

Heuristic tour mutlaka feasible olmalıdır; optimal olması gerekmez.

18. Matrix copy ve incremental state Her child için full $n \times n$ matrix copy/reduction basit fakat pahalıdır. Implementations:

- Persistent/copy-on-write matrices
- Row/column active flags
- Incremental reductions
- Compact forbidden-edge structures

kullanabilir.

Correctness için sibling nodes birbirinin matrix mutations'ını görmemelidir.

19. Complexity Worst case'de TSP tours'ın çoğu explore edilir:

```
tour permutations = (n-1)!
```

Her node'da naive matrix reduction $O(n^2)$ ise kaba worst time:

$$O(n^2 * n!)$$

Bound pruning practice'te çok node kesebilir fakat factorial worst-case nature kalır.

Best-first priority queue ve stored matrices worst case'de çok yüksek, hatta exponential/factorial memory kullanabilir.

20. Held-Karp DP ile karşılaştırma

Özellik	Matrix B&B	Held-Karp DP
Worst time	Factorial-scale	$O(n^2 2^n)$
Bound/pruning	Instance-dependent	Bütün subset states systematic
Memory	Live nodes/matrices	$O(n^2 n)$
Güçlü olduğu durum	Tight bounds, good incumbent	Moderate n, predictable exact DP

Her ikisi exact'tir; practical performance instance'a bağlıdır.

21. Sık yapılan hatalar Reduced entry yerine original edge'i child formula'ya eklemek

Previous reductions double-count edilebilir.

Row/column'u kapatmamak Vertex birden fazla outgoing/incoming edge alabilir.

Premature subtour'u engellemek Remaining vertices dışarıda kalan smaller cycles oluşabilir.

Lower bound'ı actual partial-path cost sanmak Matrix relaxation'dan gelen optimistic değerdir.

Infeasible/nonclosing leaf'i incumbent yapmak Complete Hamiltonian tour doğrulanmalıdır.

Equality pruning yapıp bütün optimal tours'ı beklemek LB==incumbent başka optimum içerebilir.

22. Kavram sözlüğü

English term	Türkçe açıklama
Cost Matrix Reduction	Row/column minima çıkararak lower bound üretme
Reduced Matrix	Her active row/column'da en az bir zero bulunan matrix
Reduction Cost	Çıkarılan minima toplamı
Lower Bound	Node descendants'ının attainable cost'u için optimistic alt limit
Subtour	Bütün vertices'i içermeyen kapanan smaller cycle
Premature Return	Tour tamamlanmadan start'a dönme
Incumbent Tour	Şimdiye kadarki en ucuz feasible complete tour
Little's Algorithm	Matrix reduction kullanan klasik TSP B&B yaklaşımı

23. Kendini sinama

1. Row/column reduction neden lower bound verir?
2. Root row ve column reduction sums nedir?
3. Root lower bound kaçtır?
4. Edge $i \rightarrow j$ seçilince matrix'te hangi entries kapatılır?
5. Child bound formula nedir?
6. First level'da en promising edge hangisidir?
7. Optimal tour ve actual cost nedir?
8. Premature subtour neden engellenmelidir?
9. Worst-case neden factorial kalır?

Kısa cevaplar

1. Her tour her row/column'dan bir edge seçmek zorunda olduğu için.
2. 21 ve 4.
3. 25.
4. Row i , column j ve premature subtour'u kapatan return edge.
5. $\text{parentLB} + \text{reducedEntry}[i][j] + \text{newReduction}$.
6. $1 \rightarrow 4$, lower bound 25.
7. $1 \rightarrow 4 \rightarrow 2 \rightarrow 5 \rightarrow 3 \rightarrow 1$, cost 28.
8. Remaining vertices dışarıda kalan küçük cycle oluşmasını önlemek için.
9. Bounds zayıfsa $(n-1)!$ tours'ın çoğu explore edilebildiği için.

Sonraki ders: **72 – FIFO Branch and Bound: 0/1 Knapsack.**

72 – NP-Hard and NP-Complete Problems

Video: 8.1 NP-Hard and NP-Complete Problems (31:53)

Bu videonun ana fikri Bazı problemlerin bilinen exact algoritmaları exponential time alır. Complexity Theory, “Henüz hızlı algoritma bulamadık” demekle yetinmez; problemleri polynomial-time çözüm, doğrulama ve reduction ilişkilerine göre sınıflandırır.

Bu dersin dört temel cümlesi şunlardır:

- P: polynomial time içinde çözülebilen decision problems.
- NP: verilen bir yes cevabının certificate’ı polynomial time içinde doğrulanabilen decision problems.
- NP-Hard: NP içindeki her problemten en az o kadar zor olan problemler.
- NP-Complete: hem NP içinde hem NP-Hard olan problemler.

1. Önce problem biçimini netleştirelim Formal sınıflar öncelikle decision problem için tanımlanır; cevap yalnız yes/nodur.

Problem	Optimization/Search biçimi	Decision biçimi
0/1 Knapsack	Maximum profit’i bul	Profit en az K olacak feasible subset var mı?
TSP	Minimum tour cost’u bul	Cost’u en fazla B olan tour var mı?
Clique	Maximum clique’i bul	Size’ı en az k olan clique var mı?
SAT	Satisfying assignment bul	Formula satisfiable mı?

Optimization sürümü çoğunlukla decision sürümünden en az o kadar zordur. Ancak “NP-Complete” ifadesini kullanırken formal olarak decision biçimi kastedilir.

2. Polynomial ve exponential büyüme Polynomial örnekleri:

$O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(n^{10})$

Exponential örnekleri:

$O(2^n)$, $O(3^n)$, $O(n!)$

Sabit bir k için n^k , asymptotically 2’nden yavaş büyür. Küçük input’ta constants önemli olabilir; complexity-class ayrımı large-input davranışını anlatır.

3. P nedir? P, deterministic bir modelde input size’in polynomial’i kadar zamanda çözülebilen decision problems sınıfıdır.

$L \in P$

demek, her input için doğru yes/no sonucunu $O(n^k)$ zamanda veren bir deterministic algorithm vardır demektir.

Örnekler:

- Bir graph connected mı?
- İki vertex arasında path var mı?
- Shortest path length en fazla B mi? (non-negative veya uygun shortest-path modelinde)
- Bir listede key var mı?

P, “pratikte her zaman hızlı” anlamına gelmez. n^{100} polynomial’dır ama pratik değildir. Buna rağmen polynomial time, tractability için temel teorik sınırdır.

4. NP nedir? En kullanışlı tanım certificate ve verifier üzerindendir.

Bir problem NP içindeyse, yes instance için polynomial uzunlukta bir certificate vardır ve deterministic verifier bunu polynomial time içinde kontrol eder.

$L \in NP \Leftrightarrow$
yes-instance x için bir certificate c vardır ve
 $V(x,c)$ polynomial time içinde accept eder.

TSP Örneği Soru:

Cost $\leq B$ olan Hamiltonian tour var mı?

Certificate, vertex sırasıdır. Verifier:

1. Her vertex'in exactly once yer aldığını kontrol eder.
2. Ardışık edge'lerin varlığını kontrol eder.
3. Cost'ları toplar.
4. Total'in B yi aşmadığını kontrol eder.

Tour'u bulmak zor olabilir; verilmiş tour'u doğrulamak polynomial'dir.

5. Nondeterministic ne demektir? Formal nondeterministic machine, bir noktada birçok olası choice'a branch edebilen soyut computation modelidir. Input yes ise branch'lerden en az biri accept eder.

Sezgisel şema:

guess certificate
verify certificate deterministically

Buradaki guess:

- Random seçim değildir.
- Kodda boş bırakılan satır değildir.
- "Nasıl çalıştığını bilmediğimiz magic function" değildir.
- Gelecekte keşfedilecek deterministic teknik anlamına gelmez.

NP, Nondeterministic Polynomial time kısaltmasıdır; "Non-Polynomial" değildir.

6. Neden $P \subseteq NP$? Polynomial time içinde çözebildiğimiz bir problemi polynomial time içinde doğrulayabiliriz. Nondeterministic model ayrıca choice yapmak zorunda değildir; deterministic computation onun özel durumudur.

$P \subseteq NP$

Bilmediğimiz şey:

$P = NP$?

Eğer $P = NP$ ise bütün NP-Complete decision problems için polynomial-time algorithms vardır. Eğer $P \neq NP$ ise hiçbir P içinde değildir. Bugün bu soru çözülmüş değildir.

7. Polynomial-Time Reduction A problemini B problemine polynomial time içinde dönüştürebiliyorsak:

$A \leq_p B$

yazarız. Dönüşüm f , cevabı korumalıdır:

$x \in A \Leftrightarrow f(x) \in B$

Bu ifade şunu söyler:

B'yi hızlı çözebilirim A'yı da hızlı çözerim.

Dolayısıyla B, Adan en az o kadar zordur.

Reduction yönünü karıştırma Bnin zor olduğunu göstermek istiyorsak bilinen zor problemden Bye gideriz:

$\text{KnownHard} \leq_p B$

Ters yönü göstermek, Bnin zor olduğunu kanıtlamaz.

8. NP-Hard Bir problem H, NP içindeki her problem ona polynomial time içinde reduce edilebiliyorsa NP-Harddır:

$\forall L \in \text{NP}, L \leq_p H$

NP-Hard problem:

- Decision problem olmak zorunda değildir.
- NP içinde olmak zorunda değildir.
- Hatta bazı NP-Hard problemler undecidable olabilir.

Bu nedenle NP-Hard ile “bilinen algorithm exponential” aynı cümle değildir. Exponential algorithm bilmek, tek başına hardness proof oluşturmaz.

9. NP-Complete Bir decision problem C şu iki koşulu sağlıyorsa NP-Complete'tir:

1. $C \in \text{NP}$
2. C is NP-Hard

Proof template:

1. Candidate solution'ın polynomial-time verifier'ını göster.
2. Bilinen NP-Complete A probleminden C'ye $A \leq_p C$ reduction kur.
3. Reduction'ın polynomial time aldığını ve yes/no'yu koruduğunu kanıtla.

10. SAT ve Cook-Levin Theorem SAT, bir Boolean formula'yı true yapan assignment bulunup bulunmadığını sorar.

Örnek CNF:

$(x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee \neg x_3)$

n variable için brute force 2^n assignment deneyebilir. Fakat SATın önemi yalnız brute-force tree'sinden gelmez.

Cook-Levin Theorem şunu kanıtlar:

SAT is NP-Complete.

Yani SAT $\in \text{NP}$ dir ve NP içindeki her problem polynomial time içinde SATa reduce edilebilir. “SAT $\in \text{P}$ iff P = NP” bunun sonucudur; theorem yalnız bu koşullu cümleden ibaret değildir.

11. SAT'tan yeni bir NP-Complete probleme geçiş Yeni decision problem X için:

$\text{SAT} \leq_p X$

kanıtlanırsa X NP-Hard olur. Ayrıca X $\in \text{NP}$ gösterilirse X NP-Complete olur.

Reduction transitive'dir:

$$A \leq_p B \text{ ve } B \leq_p C \Rightarrow A \leq_p C$$

Bu yüzden her yeni proof doğrudan SAT'tan başlamak zorunda değildir; daha uygun bir bilinen NP-Complete problem seçilebilir.

12. State-space tree benzerliği neden proof değildir? Videoda SAT ve 0/1 Knapsack için binary choice tree gösteriliyor. Bu iyi bir sezgidir: ikisinde de n tane binary decision 2^n aday oluşturabilir.

Fakat aynı şekilli state-space tree görmek, polynomial reduction kanıtlamaz. Formal proof için:

- Her SAT instance'ından belirli bir Knapsack instance üretilmeli.
- Üretim polynomial time almalı.
- SAT instance satisfiable ise ve yalnız ise Knapsack instance yes olmalı.

Bu üç madde olmadan yalnızca "search spaces benziyor" denmiş olur.

13. Sık kullanılan sınıflandırmalar

Problem biçimi	Sınıf
SAT decision	NP-Complete
3-SAT decision	NP-Complete
Clique decision	NP-Complete
Vertex Cover decision	NP-Complete
Hamiltonian Cycle decision	NP-Complete
TSP decision (cost $\leq B$?)	NP-Complete
TSP optimization	NP-Hard
0/1 Knapsack decision	NP-Complete
0/1 Knapsack optimization	NP-Hard

0/1 Knapsack ayrıca pseudo-polynomial $O(nW)$ DP'ye sahiptir. Bu, input length'e göre polynomial olmak zorunda değildir; W binary yazıldığında input'ta yalnız $\log W$ bit kaplar.

14. Videodaki anlatımı kesinleştiren düzeltmeler "Nondeterministic = bilmediğimiz satır" değil Formal nondeterminism, computation branches ve existential acceptance ile tanımlanır. Araştırma notundaki boşluk değildir.

Dün bilinmeyen algorithm NP, bugün bilinen algorithm P değildir Problem sınıfı insan bilgisinin tarihine göre tanımlanmaz. Bir polynomial algorithm matematiksel olarak varsa problem zaten Pdedir; bizim henüz onu bilmememiz yalnız epistemik durumumuzdur.

P $\subseteq$ NP sebebi tarihsel keşif değildir Sebep, deterministic modelin nondeterministic modelin özel durumu olması ve çözmenin doğrulamayı sağlamasıdır.

NP-Hard demek "exponential algorithm kullanıyor" değildir NP-Hardlık polynomial reductions ile kanıtlanır. Bir problemin bugün bilinen en iyi algorithm'inin exponential olması tek başına proof değildir.

SAT için doğru theorem Cook-Levin, SAT'ın NP-Complete olduğunu kanıtlar. $P=NP$ 'i kanıtlamaz ve SAT'ın P içinde olduğunu söylemez.

15. Sık yapılan hatalar

- NP'yi "Not Polynomial" diye açmak.
- Optimization problem ile decision version'ı ayırmamak.
- Verification kolay diye solution bulmanın da kolay olduğunu sanmak.
- Reduction arrow'unu ters kurmak.
- Bir exponential brute-force algorithm gösterip NP-Hardlık ilan etmek.

- NP-Hard ile NP-Complete eş anlamlı kullanmak.
- $P \neq NP$ nin kanıtlanmış olduğunu söylemek.
- Shared state-space tree'yi formal reduction sanmak.

16. Kavram sözlüğü

English term	Türkçe açıklama
Decision Problem	Cevabı yes/no olan problem
Certificate	yes cevabını kanıtlayan kısa aday bilgi
Verifier	Certificate'ı kontrol eden deterministic algorithm
Deterministic	Her state'te sonraki adımı belirli computation
Nondeterministic	Olası choice branches'den en az birinin accept etmesinin yeterli olduğu formal model
Polynomial-Time Reduction	Bir problem instance'ını cevabı koruyarak polynomial time'da diğerine çevirme
NP-Hard	NP'deki bütün problemler kadar zor
NP-Complete	Hem NP içinde hem NP-Hard
CNF	Clauses'ın conjunction'ı biçimindeki Boolean formula
Satisfiable	En az bir assignment altında true olan formula

17. Kendini sinama

1. P ile NP arasındaki temel fark nedir?
2. NP neden "Non-Polynomial" değildir?
3. $P \subseteq NP$ neden doğrudur?
4. $A \leq_p B$ ne söyler?
5. B nin NP -Hard olduğunu göstermek için reduction hangi yönde olmalıdır?
6. NP -Complete olmanın iki koşulu nedir?
7. Optimization TSP ile decision TSP nasıl ayrılır?
8. Cook-Levin Theorem neyi kanıtlar?
9. İki problemin state-space tree'lerinin benzemesi neden yeterli değildir?
10. $P = NP$ bugün çözülmüş müdür?

Kısa cevaplar

1. P polynomial time çözüm, NP polynomial time verification koşuludur.
2. Açılımı Nondeterministic Polynomial timedır.
3. Deterministic computation nondeterministic modelin özel durumudur.
4. B için solver kullanılarak A polynomial overhead ile çözülebilir; B , A dan en az o kadar zordur.
5. Bilinen NP -Complete problemden B ye.
6. $B \in NP$ ve B NP -Hard.
7. Biri minimum tour'u ister; diğeri "cost $\leq$ bound olan tour var mı?" diye sorar.
8. SAT'ın NP -Complete olduğunu.
9. Cevabı koruyan polynomial-time instance mapping kanıtlanmamıştır.
10. Hayır.

73 – Clique Decision Problem

Video: 8.2 Clique Decision Problem (17:14)

Bu videonun ana fikri Undirected graph içinde birbirleriyle pairwise adjacent olan vertex kümesine Clique denir.

Clique Decision Problem:

Input: Undirected graph $G=(V,E)$ ve integer k
Soru: G içinde size'ı en az k olan bir clique var mı?

Bu problem NP-Complettir. Verilen clique kolay doğrulanır; fakat genel k için clique'ı bulmaya yarayan polynomial-time algorithm bilinmemektedir.

1. Clique nedir? $C \subseteq V$ kümesinin clique olması için farklı her $u, v \in C$ çifti arasında edge bulunmalıdır:

$\forall u, v \in C, u \neq v \Rightarrow \{u, v\} \in E$

Başka bir ifadeyle $G[C]$ induced subgraph'ı complete graph olmalıdır.

Size k clique tam olarak $k(k-1)/2$ pairwise edge gerektirir.

$\text{requiredEdges} = k(k-1)/2$

2. Küçük örnek Şu undirected graph'ı ele alalım:

$V = \{A, B, C, D, E\}$
 $E = \{AB, AC, BC, AD, BD, CE\}$

$\{A, B, C\}$ bir 3-clique'tır:

$AB \in E$
 $AC \in E$
 $BC \in E$

$\{A, B, D\}$ de bir 3-clique'tır. Fakat $\{A, B, C, D\}$ clique değildir; çünkü örneğin CD edge'i yoktur.

Sonuç:

$k=3 \rightarrow \text{yes}$
 $k=4 \rightarrow \text{no}$

3. Complete Graph ile Clique farkı Complete Graph, graph'ın bütün vertex'leri arasında edge olmasını ister. Clique, graph içindeki bir vertex subset'inin complete olmasını ister.

$G \text{ complete} \Leftrightarrow V$ 'nin tamamı clique
 $C \text{ clique} \Leftrightarrow \text{yalnız } G[C] \text{ complete}$

Bir graph complete değilken içinde birçok clique bulunabilir.

4. Maximal Clique ve Maximum Clique Bu iki terim aynı değildir.

Maximal Clique Başka vertex eklenerek büyütülemeyen clique'tır. Local olarak genişletilemez.

Maximum Clique Graph'taki en büyük size'a sahip clique'tır. Global optimumdur.

Her maximum clique maximal'dır; her maximal clique maximum değildir.

Maximum clique size:

$\omega(G)$

Decision sorusu şu biçimde de yazılabilir:

$\omega(G) \geq k$?

5. Neden problem NP içindedir? Certificate olarak k vertex'ten oluşan set verilsin.

Verifier bütün vertex pair'lerini kontrol eder:

```
for each pair (u,v) in certificate:
    if edge(u,v) does not exist:
        reject
accept
```

Adjacency matrix kullanıldığında:

Time = $O(k^2)$
Space = $O(k)$ certificate dışında $O(1)$

Certificate length polynomial olduğu ve verifier polynomial zamanda çalıştığı için:

CLIQUE $\in$ NP

6. Naive çözüm Size k olan bütün vertex subsets denenebilir:

```
CliqueDecision(G,k):
    for each  $C \subseteq V$  with  $|C| = k$ :
        if every pair in  $C$  is adjacent:
            return true
    return false
```

Candidate sayısı:

$C(n,k)$

Her candidate verification $O(k^2)$:

$O(C(n,k) \cdot k^2)$

k input'un parçası olduğunda bu worst case exponential'dır. Örneğin $k \approx n/2$ iken binomial coefficient exponential büyür.

7. Önemli nuance: fixed k k önceden sabit bir constant ise $O(n^k)$ polynomial'dir. Örneğin triangle, $k=3$ clique'tır ve polynomial-time methods ile bulunabilir.

Clique Decision Problemın NP-Complete olması, k nin input ile birlikte büyüyebildiği genel problem içindir.

8. Backtracking ile arama State şu bilgilerden oluşabilir:

- candidates: clique'a eklenebilecek vertex'ler.
- chosen: şu anki clique.
- need: hedefe ulaşmak için gereken vertex sayısı.

Yeni vertex v seçilince candidates yalnız v nin neighbors'ıyla intersect edilir. Böylece chosen kümesinin clique invariant'ı korunur.

```
HasClique(candidates, chosenSize, k):
    if chosenSize  $\geq$  k:
```

```

    return true

    if chosenSize + |candidates| < k:
        return false

    choose a vertex v from candidates

    includeCandidates = candidates_after_v  $\cap$  N(v)
    if HasClique(includeCandidates, chosenSize + 1, k):
        return true

    excludeCandidates = candidates_after_v
    return HasClique(excludeCandidates, chosenSize, k)

```

Pruning condition:

```
chosenSize + remainingCandidates < k
```

ise hedef artık ulaşamaz.

Worst-case yine exponential'dır; pruning yalnız birçok practical instance'ı hızlandırır.

9. Independent Set ilişkisi Independent Set, kendi içinde hiçbir edge bulunmayan vertex subset'idir.

Complement graph $\bar{G}$ şu şekilde tanımlanır:

```
u ve v,  $\bar{G}$ 'de adjacent  $\Leftrightarrow$  u ve v, G'de adjacent değil
```

Temel eşdeğerlik:

```
C, G'de clique  $\Leftrightarrow$  C,  $\bar{G}$ 'de independent set
```

Dolayısıyla:

```
(G, k)  $\in$  CLIQUE  $\Leftrightarrow$  ( $\bar{G}$ , k)  $\in$  INDEPENDENT-SET
```

Complement graph $O(n^2)$ zamanda kurulabilir.

10. Clique neden NP-Harddır? Bilinen NP-Complete problem INDEPENDENT-SETten reduction kurulur:

```
INDEPENDENT-SET  $\leq_p$  CLIQUE
```

Mapping:

```
(G, k)  $\mapsto$  ( $\bar{G}$ , k)
```

Çünkü G'de independent set olan aynı vertex kümesi $\bar{G}$ 'de clique olur ve bunun tersi de doğrudur.

- Transformation polynomial time alır.
- yes/no cevabını korur.
- INDEPENDENT-SET NP-Complete olduğundan CLIQUE NP-Hard olur.
- Önceki verifier sayesinde CLIQUE ayrıca NP içindedir.

Sonuç:

```
CLIQUE is NP-Complete.
```

11. Vertex Cover ilişkisi Bir vertex set S independent ise complement vertex set $V-S$ bütün edge'leri cover eder:

S independent in $G \Leftrightarrow V-S$ vertex cover in G

Clique ile birleştirince:

G 'de size k clique
 $\Leftrightarrow \bar{G}$ 'de size k independent set
 $\Leftrightarrow \bar{G}$ 'de size $n-k$ vertex cover

Bu ilişkiler, farklı graph problems arasında polynomial reductions kurmayı kolaylaştırır.

12. Decision, Search ve Optimization

Biçim	Soru
Decision	Size en az k clique var mı?
Search	Böyle bir clique'ı ver
Optimization	Maximum clique'ı bul

Decision solver kullanarak maximum size binary search veya artan/azalan k sorgularıyla bulunabilir. Search solution da self-reduction ile çıkarılabilir. Bu yüzden biçimler polynomial overhead altında yakından ilişkilidir; formal sınıf etiketi decision sürümüne verilir.

13. Clique ile Graph Coloring'i karıştırma

Clique size k , proper coloring için lower bound verir:

$$\chi(G) \geq \omega(G)$$

Çünkü clique içindeki her vertex farklı color almak zorundadır. Fakat equality her graph için doğru değildir; chromatic number clique number'dan daha büyük olabilir.

14. Complexity özeti

İşlem	Time
Verilen k vertex'i doğrulama	$O(k^2)$
Bütün k -subsets'i deneme	$O(C(n,k) \cdot k^2)$
Bütün subsets'i kaba tarama	$O(2^n \cdot n^2)$
Complement graph üretme	$O(n^2)$

Backtracking space, representation hariç recursion depth ve candidate sets'e göre $O(n)$ ile $O(n^2)$ arasında uygulanabilir.

15. Videodaki sınıflandırmayı kesinleştirme

Videonun caption açılışında problem "NP-Hard" olarak adlandırılıyor. Bu eksik ama tamamen yanlış olmayan bir ifadedir:

Clique optimization $\rightarrow$ NP-Hard
Clique decision $\rightarrow$ NP-Complete (dolayısıyla ayrıca NP-Hard)

Video başlığı Clique Decision Problem olduğu için en kesin sınıflandırma NP-Complete'tir.

16. Sık yapılan hatalar

- Clique için yalnız connected subset kontrol etmek; pairwise adjacency gerekir.
- k vertex arasında yalnız $k-1$ edge aramak; gereken $k(k-1)/2$ dir.
- Maximal ile Maximumu karıştırmak.
- Clique'ı original graph yerine yanlışlıkla complement graph'ta aramak.
- $k=3$ özel durumu polynomial diye genel CLIQUE'in polynomial olduğunu sanmak.
- Certificate verification ile clique discovery complexity'sini karıştırmak.

- Decision sürümünü yalnız NP-Hard deyip NP-Complete üyeliğini atlamak.

17. Kavram sözlüğü

English term	Türkçe açıklama
Clique	Pairwise adjacent vertex subset
k-Clique	Size'ı k olan clique
Maximum Clique	En büyük clique
Maximal Clique	Daha fazla vertex eklenemeyen clique
Clique Number	Maximum clique size, $\omega(G)$
Independent Set	Kendi içinde edge bulunmayan vertex subset
Complement Graph	Edge ve non-edge'leri tersleyen graph
Vertex Cover	Her edge'in en az bir endpoint'ini içeren vertex set
Pairwise Adjacent	Kümedeki her farklı vertex çiftinin adjacent olması

18. Kendini sinama

1. Size k clique içinde kaç edge bulunmalıdır?
2. Connected subset neden clique olmak zorunda değildir?
3. Maximal Clique ile Maximum Clique farkı nedir?
4. Verilen candidate clique nasıl doğrulanır?
5. Naive k-subset algoritmasının complexity'si nedir?
6. Fixed k ile input'un parçası olan k neden farklıdır?
7. Clique ile Independent Set arasında nasıl ilişki vardır?
8. CLIQUE'in NP-Hard olduğunu gösteren reduction nedir?
9. Clique decision'in kesin sınıfı nedir?
10. $\omega(G)$ ile $\chi(G)$ arasında hangi inequality vardır?

Kısa cevaplar

1. $k(k-1)/2$.
2. Connected olmak yalnız yolları garanti eder; clique her vertex çiftinde doğrudan edge ister.
3. Maximal büyütülemez; maximum bütün clique'lar arasında en büyüktür.
4. Bütün $O(k^2)$ vertex pair'leri için edge kontrol edilir.
5. $O(C(n, k) \cdot k^2)$.
6. Constant k için n^k polynomial; variable k için exponent input ile büyür.
7. Gde clique olan set, $\bar{G}$ de independent set'tir.
8. $(G, k) \mapsto (\bar{G}, k)$ ile INDEPENDENT-SET $\leq_p$ CLIQUE.
9. NP-Complete.
10. $\chi(G) \geq \omega(G)$.

74 – Knuth-Morris-Pratt (KMP) String Matching

Video: 9.1 Knuth-Morris-Pratt KMP String Matching Algorithm (18:56)

Bu videonun ana fikri Naive String Matching, mismatch sonrası pattern'ı bir position kaydırıp text'teki bazı characters'ı yeniden karşılaştırır. KMP, pattern'ın kendi iç yapısını önceden analiz ederek bu tekrarları önler.

KMP'nin tuttuğu bilgi:

```
lps[j] = pattern[0..j] için  
longest proper prefix'in aynı zamanda suffix olan length'i
```

Mismatch olduğunda text index i geri gitmez; yalnız pattern index j , lps üzerinden fallback yapar.

1. Problem Input:

```
text    T, length n  
pattern P, length m
```

Amaç, Pnin T içinde başladığı bütün indices'i bulmaktır.

Örnek:

```
T = "ABABDABACDABABCABAB"  
P = "ABABCABAB"
```

Pattern, zero-based index 10da başlar.

2. Naive algorithm Her possible alignment için pattern baştan karşılaştırılır:

```
for s = 0 to n-m:  
    j = 0  
    while j < m and T[s+j] == P[j]:  
        j++  
    if j == m:  
        report s
```

Worst case:

```
Time  =  $O((n-m+1)m) = O(nm)$   
Space =  $O(1)$ 
```

Örneğin repeated A characters ve sonunda B içeren pattern, birçok alignment'ta uzun partial match oluşturabilir.

3. KMP'nin temel gözlemi Şimdiye kadar j character match olduysa:

```
P[0..j-1] = T[i-j..i-1]
```

Mismatch'te bu match bilgisinin tamamını atmak zorunda değiliz. Matched prefix'in sonundaki bir suffix, pattern'ın başındaki prefix ile aynıysa o bölüm zaten doğrulanmıştır.

Bu nedenle:

```
j = lps[j-1]
```

yapılır. i değişmez; aynı text character daha kısa candidate prefix ile denenir.

4. Prefix, Suffix ve Proper Prefix String:

S = "ABABC"

Prefixes:

"", "A", "AB", "ABA", "ABAB", "ABABC"

Suffixes:

"", "C", "BC", "ABC", "BABC", "ABABC"

Proper Prefix, string'in kendisi olmayan prefix'tir. LPS hesabında full string kullanılmaz; aksi halde her position kendisini match edip yararsız bir value üretti.

5. LPS ne saklar? Pattern:

P = "ABABCABAB"

LPS table:

j	P[0..j]	Longest proper prefix = suffix	lps[j]
0	A	empty	0
1	AB	empty	0
2	ABA	A	1
3	ABAB	AB	2
4	ABABC	empty	0
5	ABABCA	A	1
6	ABABCAB	AB	2
7	ABABCABA	ABA	3
8	ABABCABAB	ABAB	4

lps = [0,0,1,2,0,1,2,3,4]

LPS value bir index değil, length'tir. Fallback sonrası bu length aynı zamanda next pattern index olur.

6. LPS table nasıl oluşturulur? iki pointer kullanılır:

- i: yeni prefix-suffix value hesaplanan position.
- len: mevcut candidate border length.

```
BuildLPS(P):
    m = length(P)
    lps = array(m, 0)
    len = 0
    i = 1

    while i < m:
        if P[i] == P[len]:
            len++
            lps[i] = len
            i++
        else if len > 0:
            len = lps[len-1]
        else:
            lps[i] = 0
            i++
```

```
return lps
```

Neden mismatch'te i artmıyor? $len > 0$ iken aynı $P[i]$, daha kısa bir known border ile hâlâ match edebilir. Önce:

```
len = lps[len-1]
```

ile bu possibility denenir. Bütün borders tükenirse $len=0$ olur ve i ilerler.

7. LPS oluşturma örneği

$P = \text{"ABABCABAB"}$ için:

```
i=1: B != A      → lps[1]=0
i=2: A == A      → lps[2]=1
i=3: B == B      → lps[3]=2
i=4: C != A, len=2 → len=lps[1]=0
      C != A, len=0 → lps[4]=0
i=5: A == A      → lps[5]=1
i=6: B == B      → lps[6]=2
i=7: A == A      → lps[7]=3
i=8: B == B      → lps[8]=4
```

Özellikle $i=4$ te fallback sırasında i sabit kalır. Bu, LPS preprocessing'in de kendi içinde KMP mantığı kullanmasıdır.

8. Search algorithm

Zero-based standart sürüm:

```
KMP(T, P):
    n = length(T)
    m = length(P)
    if m == 0:
        return [0]          // seçilen API convention

    lps = BuildLPS(P)
    matches = []
    i = 0                    // text index
    j = 0                    // pattern index

    while i < n:
        if T[i] == P[j]:
            i++
            j++

            if j == m:
                matches.append(i-m)
                j = lps[m-1]

            else if j > 0:
                j = lps[j-1]
            else:
                i++

    return matches
```

9. Search trace

```
T = ABABDABACDABABCABAB
P = ABABCABAB
```


İlk dört character match eder:

```
T[0..3] = ABAB  
P[0..3] = ABAB
```

Sonra:

```
T[4] = D  
P[4] = C mismatch
```

j=4 iken:

```
j = lps[3] = 2
```

Text index i=4 değişmez. Aynı D, P[2]='A' ile denenir; yine mismatch:

```
j = lps[1] = 0
```

D != P[0] olduğundan bu kez i artar. Daha sonra i=10dan başlayan alignment bütün pattern'ı match eder:

```
match index = i-m = 19-9 = 10
```

10. Text index neden geri gitmez? KMP invariant'ı, j matched characters'ın text'teki son j characters ile eşit olduğunu bilir. LPS, bu matched block içindeki hangi suffix'in yeni prefix olarak korunabileceğini söyler.

Bu bilgi sayesinde daha önce doğrulanmış text characters'a fiziksel olarak dönülmez:

```
i monoton artar  
j match'te artar, mismatch'te fallback yapar
```

11. Overlapping matches Match bulununca search'ü bitirmek zorunda değiliz. Şu assignment:

```
j = lps[m-1]
```

overlapping matches'i korur.

Örnek:

```
T = "AAAAA"  
P = "AAA"
```

Matches:

```
[0,1,2]
```

Match sonrası j=0 yapmak bazı overlap bilgisini gereksiz yere kaybeder.

12. Correctness sezgisi Mismatch'te j=lps[j-1] yapmak hiçbir possible match'i atlamaz:

1. İlk j pattern characters zaten match olmuştur.
2. Yeni valid alignment, bu matched text suffix'ile başlamalıdır.
3. Böyle bir alignment'ın pattern prefix'i, matched pattern prefix'inin suffix'i olmalıdır.
4. LPS bunun en uzun olanını verir.
5. O da olmazsa LPS chain daha kısa bütün borders'ı sırayla dener.

Dolayısıyla atlanan lengths'in hiçbirisi valid border değildir.

13. Complexity neden linear? LPS preprocessing:

$O(m)$

Search:

$O(n)$

Toplam:

Time = $O(n+m)$
Space = $O(m)$

Search loop'ta j bazen geri gider; fakat i geri gitmez ve fallback'lerin toplam sayısı advances ile amortized olarak sınırlıdır. Bu nedenle nested görünüm $O(nm)$ üretmez.

14. Video gösterimi ile standart gösterim Videoda one-based pattern indices, $j+1$ karşılaştırması ve başlangıç için ek 0 cell kullanılan bir π table anlatılıyor. Bu geçerli bir convention'dır.

Bu notta yaygın zero-based form kullanıldı:

```
compare T[i] with P[j]
fallback j = lps[j-1]
```

İki gösterimi satır satır karıştırmamak gerekir; off-by-one errors'ın ana kaynağı budur.

15. KMP ne zaman tercih edilir?

- Worst-case linear guarantee gerektiğinde.
- Aynı pattern ile streaming text taranırken.
- Text üzerinde backtracking istenmediğinde.
- Exact single-pattern matching yapılırken.

Çok sayıda pattern için Aho-Corasick; approximate matching için başka algorithms gerekir.

16. Sık yapılan hatalar

- LPS'yi longest common prefix-suffix sanıp proper koşulunu unutmak.
- $lps[j]$ yi length yerine text index gibi kullanmak.
- Search mismatch'inde $j=lps[j]$ yazmak; doğru fallback $lps[j-1]$ dir.
- $j=0$ mismatch'inde yine j fallback yapıp infinite loop oluşturmak.
- LPS preprocessing mismatch'inde her zaman $i++$ yapmak.
- Match sonrası overlap aranırken $j=0$ a dönmek.
- One-based video convention ile zero-based pseudocode'u karıştırmak.
- Complexity'yi videodaki son sözlü sürümle $n+n$ yazmak; genel ifade $n+m$ dir.

17. Kavram sözlüğü

English term	Türkçe açıklama
Pattern Matching	Pattern'ın text içindeki occurrences'larını bulma
Prefix	String'in başından başlayan parça
Suffix	String'in sonunda biten parça
Proper Prefix	String'in tamamından kısa prefix
Border	Hem proper prefix hem suffix olan string
LPS	Longest Proper Prefix which is also Suffix length table
Failure Function	Mismatch sonrası next pattern state'i veren bilgi
Fallback	Daha kısa valid border'a geçiş
Alignment	Pattern'ın text üzerinde denendiği başlangıç position'ı

18. Kendini sınama

1. Naive matching'in worst-case time complexity'si nedir?
2. $\text{lps}[j]$ tam olarak neyi saklar?
3. Proper prefix koşulu neden gerekir?
4. Mismatch ve $j > 0$ iken hangi assignment yapılır?
5. Bu durumda text index neden artmaz?
6. ABABCABAB pattern'ının LPS table'ı nedir?
7. KMP'nin toplam time ve space complexity'si nedir?
8. Match sonrası overlapping occurrences için j ne olur?
9. Video ile bu not arasındaki indexing farkı nedir?

Kısa cevaplar

1. $O(nm)$.
2. $P[0..j]$ nin en uzun proper prefix=suffix length'ini.
3. Full string seçilirse yararlı daha kısa fallback bilgisi elde edilemez.
4. $j = \text{lps}[j-1]$.
5. Aynı text character daha kısa valid prefix ile denenebilir.
6. $[0,0,1,2,0,1,2,3,4]$.
7. Time $O(n+m)$, extra space $O(m)$.
8. $j = \text{lps}[m-1]$.
9. Video one-based/extra zero cell; not zero-based standard lps kullanıyor.

75 – Rabin-Karp String Matching

Video: 9.2 Rabin-Karp String Matching Algorithm (23:50)

Bu videonun ana fikri Rabin-Karp, pattern ile her text window'unu character-by-character karşılaştırmak yerine önce hash values'ı karşılaştırır.

```
hash(window) != hash(pattern) → kesinlikle match değil  
hash(window) == hash(pattern) → possible match; characters doğrulanmalı
```

Bir window'dan sonraki window'un hash'ini $O(1)$ zamanda üretmeye Rolling Hash denir.

1. Problem

```
Text T, length n  
Pattern P, length m
```

P nin T içindeki bütün occurrences'larını bulmak istiyoruz. Toplam possible window sayısı:

```
n-m+1
```

eğer $n \geq m$; aksi halde hiç window yoktur.

2. Neden hash kullanılır? Naive algorithm her alignment'ta en fazla m character karşılaştırır:

```
 $O((n-m+1)m) = O(nm)$ 
```

Rabin-Karp her length- m window'u tek numeric fingerprint ile temsil eder. Hash farklıysa m comparisons yapmadan sonraki window'a geçer.

Ancak hash, string'in kendisi değildir. Aynı hash'e sahip farklı strings olabilir.

3. Basit hash fikri ve sorunu Videoda sezgi için character codes toplanıyor:

```
A=1, B=2, C=3, ...  
hash("AAB") = 1+1+2 = 4
```

İlk window'un toplamı biliniyorsa rolling update:

```
nextHash = currentHash - outgoingCode + incomingCode
```

$O(1)$ dir. Fakat toplam, character order'ını kaybeder:

```
hash("ABC") = 1+2+3 = 6  
hash("CAB") = 3+1+2 = 6
```

Bu yüzden çok fazla collision ve Spurious Hit üretebilir.

4. Position-sensitive polynomial hash Order'ı korumak için characters positional number gibi ele alınır:

```
hash(s[0..m-1]) =  
s[0] · d(m-1) + s[1] · d(m-2) + ... + s[m-1]
```

Burada:

- d : alphabet/base size'a uygun base.
- $code(c)$: character'ın numeric code'u.

Videodaki küçük alfabet örneğinde $d=10$:

```
D=4, B=2, A=1
hash("DBA") =  $4 \cdot 10^2 + 2 \cdot 10 + 1 = 421$ 
```

Bu, "ABD"=124ten farklıdır; order artık hash'e yansır.

5. Rolling update nasıl türetilir? Current window:

```
 $x_0 \ x_1 \ \dots \ x_{(m-1)}$ 
```

Hash:

```
 $H = x_0 \cdot d^{(m-1)} + x_1 \cdot d^{(m-2)} + \dots + x_{(m-1)}$ 
```

Sonraki window için:

1. Leading term çıkarılır.
2. Kalan digits bir position sola kaydırılır: d ile çarpılır.
3. Incoming character eklenir.

```
 $H_{next} = d \cdot (H - x_0 \cdot d^{(m-1)}) + incoming$ 
```

Videodaki örnek:

```
current = "CCA" =  $3 \cdot 10^2 + 3 \cdot 10 + 1 = 331$ 
next    = "CAC"

 $H_{next} = 10 \cdot (331 - 3 \cdot 10^2) + 3$ 
        =  $10 \cdot 31 + 3$ 
        = 313
```

Window yeniden $0(m)$ hesaplanmadı; üç arithmetic operation ile güncellendi.

6. Büyük sayılar ve modulus $d^{(m-1)}$ hızla büyür. Fixed-width integer overflow'u önlemek ve values'ı sınırlamak için büyük bir prime q ile modulus alınır:

```
 $hash(s) = polynomialValue(s) \bmod q$ 
```

Rolling form:

```
 $H_{next} = (d \cdot (H - outgoing \cdot h) + incoming) \bmod q$ 
 $h = d^{(m-1)} \bmod q$ 
```

Negative result normalize edilir:

```
 $H_{next} = (H_{next} + q) \bmod q$ 
```

Practical implementation'da multiplication overflow'u için language'in integer semantics'i de dikkate alınmalıdır.

7. Modulus seçimi Yaygın yaklaşım:

- d , character domain'e uygun base seçilir; byte alfabet için sıklıkla 256.
- q , yeterince büyük prime seçilir.
- Daha düşük collision probability için two independent moduli kullanılabilir.

$q=2^{31}$ seçmek genel bir zorunluluk veya ideal kural değildir. Prime modulus, polynomial hashing'in dağılım özellikleri için daha standarttır.

Modulus ne olursa olsun collision tamamen imkânsız hale gelmez.

8. Spurious Hit Şu durum spurious hit'tir:

```
hash(window) == hash(pattern)
window != pattern
```

Hash equality yalnız candidate üretir. Correctness için actual characters doğrulanır:

```
if patternHash == windowHash:
    if T[s..s+m-1] == P:
        report s
```

Character verification kaldırılırsa algorithm false positive verebilir.

9. Tam pseudocode

```
RabinKarp(T, P, d, q):
    n = length(T)
    m = length(P)
    matches = []

    if m == 0:
        return [0]                // API convention
    if m > n:
        return []

    h = 1
    for i = 1 to m-1:
        h = (h*d) mod q           // d^(m-1) mod q

    patternHash = 0
    windowHash = 0

    for i = 0 to m-1:
        patternHash = (d*patternHash + code(P[i])) mod q
        windowHash = (d*windowHash + code(T[i])) mod q

    for s = 0 to n-m:
        if patternHash == windowHash:
            if T[s..s+m-1] equals P:
                matches.append(s)

        if s < n-m:
            outgoing = code(T[s])
            incoming = code(T[s+m])
            windowHash = (d*(windowHash - outgoing*h) + incoming) mod q
            if windowHash < 0:
                windowHash += q

    return matches
```

10. Trace mantığı Her iteration'da invariant:

```
windowHash = hash(T[s..s+m-1]) mod q
```

Loop sırası önemlidir:

1. Current window hash karşılaştırılır.
2. Hash eşitse exact comparison yapılır.
3. Son window değilse outgoing çıkarılıp incoming eklenir.

$s < n-m$ guard'ı olmazsa son iteration'da $T[s+m]$ out-of-bounds olur.

11. Correctness

Algorithm'ın correctness'i iki parçadır:

Hiçbir gerçek match kaçmaz Gerçek match varsa strings identical olduğundan aynı deterministic hash'i üretir. Hash equality branch'i exact comparison'a gider ve match raporlanır.

False positive raporlanmaz Collision durumunda exact character comparison başarısız olur; position raporlanmaz.

Bu yüzden hash yalnız filter'dır, proof değildir.

12. Complexity

Preprocessing:

$$O(m)$$

Window hashes'ın rolling scan'i:

$$O(n-m+1) = O(n)$$

Expected/average durumda az sayıda hash hit varsa:

$$\text{Time} = O(n+m)$$

Worst case'te her window hash'i pattern hash'ine eşit olabilir ve her seferinde $O(m)$ verification yapılır:

$$\text{Time} = O((n-m+1)m) = O(nm)$$

Extra space, output hariç:

$$O(1)$$

Hash arithmetic'in machine-word constant time olduğu varsayılır.

13. KMP ile karşılaştırma

Özellik	Rabin-Karp	KMP
Preprocessing	Pattern/window hash	LPS table
Expected time	$O(n+m)$	$O(n+m)$
Worst time	$O(nm)$	$O(n+m)$
Extra space	$O(1)$	$O(m)$
False hash hits	Var, exact check gerekir	Yok
Güçlü kullanım	Multiple patterns / fingerprints	Tek exact pattern, worst-case guarantee

Rabin-Karp aynı length'teki birçok pattern hash'ini bir set'te tutarak multi-pattern filtering için kullanışlı olabilir.

14. Video anlatımını kesinleştiren düzeltmeler Güçlü hash spurious hits'i yok etmez

Videoda positional hash sonrası "artık spurious hit yok" sezgisi veriliyor; hemen ardından possibility'nin kalabileceği de söyleniyor. Kesin ifade:

Daha iyi hash collision ihtimalini azaltır, sıfırlamaz.

Hash function call otomatik olarak $O(1)$ değildir İlk window hash'ini sıfırdan hesaplamak $O(m)$ dir. Yalnız rolling update $O(1)$ dir.

Modulus yalnız overflow sınırı değildir Prime modulus hem bounded arithmetic sağlar hem hash space tanımlar. Collision hâlâ mümkündür; exact comparison zorunludur.

Average ifade $n-m+1$ yalnız window sayısıdır. Input'ı okumak ve pattern hash'ini hazırlamakla genel expected time $O(n+m)$ diye yazılır.

15. Sık yapılan hatalar

- Character codes'u yalnız toplamak ve order bilgisini kaybetmek.
- Hash eşitliğini direct match kabul etmek.
- Her window hash'ini baştan $O(m)$ hesaplayıp rolling avantajını yok etmek.
- Leading factor $h=d^{(m-1)}$ i yanlış exponent ile hesaplamak.
- Subtraction sonrası negative modulo'yu normalize etmemek.
- Son window'dan sonra incoming character okumak.
- $m>n$ durumunu ele almamak.
- Large powers'ı önce hesaplayıp sonra mod almak; her adımda mod uygulanmalıdır.
- "Double hash collision imkânsızdır" demek; yalnız ihtimal çok küçüldür.

16. Kavram sözlüğü

English term	Türkçe açıklama
Hash Function	String'i bounded numeric fingerprint'e dönüştüren function
Rolling Hash	Sliding window hash'ini previous value'dan $O(1)$ güncelleme
Collision	Farklı strings'in aynı hash'i üretmesi
Spurious Hit	Hash match olup actual string match olmaması
Fingerprint	String'i temsil eden kısa hash value
Base	Positional polynomial'deki d değeri
Modulus	Hash'i bounded tutan q değeri
Outgoing Character	Window kayınca dışarı çıkan character
Incoming Character	Yeni window'a giren character

17. Kendini sına

1. Rabin-Karp neden önce hash karşılaştırır?
2. Basit sum hash'in temel kusuru nedir?
3. "DBA" için $D=4, B=2, A=1, d=10$ hash'i kaçtır?
4. Rolling update'in üç adımı nedir?
5. $h=d^{(m-1)}$ ne işe yarar?
6. Hash values eşit olduğunda neden characters karşılaştırılır?
7. Expected ve worst-case time complexity nedir?
8. Prime modulus collision'ı tamamen kaldırır mı?
9. $m>n$ durumunda sonuç nedir?
10. KMP'nin Rabin-Karp'a göre worst-case avantajı nedir?

Kısa cevaplar

1. Farklı hash, window'u $O(1)$ test ile elemek için.
2. Character order'ını kaybeder ve çok collision üretir.
3. 421.
4. Leading term'i çıkar, kalan değeri base ile çarp, incoming code'u ekle.
5. Outgoing character'ın highest-place contribution'ını çıkarmak için.
6. Collision false positive üretmesin diye.

7. Expected $O(n+m)$, worst $O(nm)$.
8. Hayır, yalnız probability'yi azaltır.
9. Hiç match yoktur; empty result.
10. KMP worst case'te de $O(n+m)$ dir.

76 – AVL Tree: Insertion and Rotations

Video: 10.1 AVL Tree – Insertion and Rotations (43:08)

Bu videonun ana fikri Normal Binary Search Tree (BST), insertion order'a göre linked list gibi skewed hale gelebilir. Bu durumda search, insert ve delete $O(n)$ ye düşer.

AVL Tree, her node'da left ve right subtree heights arasındaki farkı en fazla 1 tutan self-balancing BST'dir:

```
BalanceFactor(node) = height(left) - height(right)
BalanceFactor ∈ {-1, 0, +1}
```

Insertion bu koşulu bozarsa Rotation ile BST ordering korunarak denge geri getirilir.

1. BST özelliği

Her node x için:

```
left subtree keys < x.key
right subtree keys > x.key
```

Bu notta duplicate keys eklenmiyor. Başka implementations duplicate için count tutabilir veya tutarlı bir side policy seçebilir; duplicate yasağı BST'nin tek mümkün tanımı değildir.

Search path root'tan leaf'e iner. Dolayısıyla:

```
Search time =  $O(h)$ 
```

Burada h tree height'tır.

2. Normal BST'nin sorunu

Aynı keys farklı order'da insert edilince farklı shapes oluşur.

```
30, 20, 10
```

şu skewed tree'yi üretir:

```
  30
 /
20
/
10
```

Sorted veya reverse-sorted input normal BST'yi chain'e dönüştürebilir:

```
h =  $O(n)$ 
Search/Insert/Delete =  $O(n)$ 
```

Dengeli durumda:

```
h =  $O(\log n)$ 
Search/Insert/Delete =  $O(\log n)$ 
```

AVL'nin amacı her zaman mathematically minimum-height tree üretmek değil, height'ı garantili $O(\log n)$ sınırında tutmaktır.

3. Height convention

Bu notta:

```
height(empty) = -1
height(leaf) = 0
height(node) = 1 + max(height(left), height(right))
```

Videoda edges sayılarak aynı convention kullanılıyor; yalnız bazı gösterimlerde single node height 1 diye de tanımlanabilir. Balance factors değişmez, çünkü iki taraf aynı offset'i alır.

4. Balance Factor

$$BF(x) = \text{height}(x.\text{left}) - \text{height}(x.\text{right})$$

Yorum:

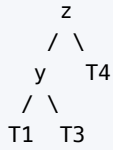
BF	Durum
+1	Left subtree bir level daha yüksek
0	Heights eşit
-1	Right subtree bir level daha yüksek
+2	Left-heavy, rebalance gerekir
-2	Right-heavy, rebalance gerekir

AVL invariant:

$$|BF(x)| \leq 1, \text{ every node } x \text{ için}$$

Yalnız root'u kontrol etmek yetmez; invariant bütün nodes için geçerlidir.

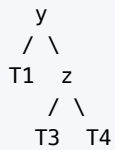
5. Rotation neden BST order'ını bozmaz? Right rotation öncesi:



Key order:

$$T1 < y < T3 < z < T4$$

Right rotation sonrası:



Aynı in-order sequence korunur. T3, yden büyük fakat zden küçük olduğu için z. left olur.

Rotation pointer değişiklikleri $O(1)$ zaman alır.

6. Right Rotation

```
RotateRight(z):
  y = z.left
  T3 = y.right

  y.right = z
  z.left = T3

  updateHeight(z)
```

```
updateHeight(y)
return y
```

Önce lower node z, sonra new root y height'ı güncellenir.

7. Left Rotation



```
RotateLeft(z):
    y = z.right
    T2 = y.left

    y.left = z
    z.right = T2

    updateHeight(z)
    updateHeight(y)
    return y
```

8. Dört imbalance case Case adı, unbalanced node'dan inserted key'e giden ilk iki direction'ı anlatır.

Case	Shape	Düzeltilme
LL	left child's left side	Right Rotation(z)
RR	right child's right side	Left Rotation(z)
LR	left child's right side	Left Rotation(z.left), sonra Right Rotation(z)
RL	right child's left side	Right Rotation(z.right), sonra Left Rotation(z)

Önemli adlandırma ayrımı:

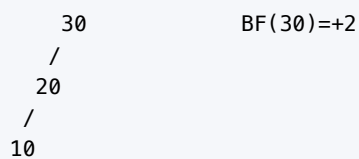
```
LL case'i düzelten actual operation = Right Rotation
RR case'i düzelten actual operation = Left Rotation
```

Bazı dersler "LL rotation" diyerek case'in adını kullanır. Pointer operation adıyla karıştırmamak gerekir.

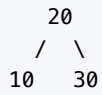
9. LL Case Insertion order:

30,20,10

Insertion sonrası:

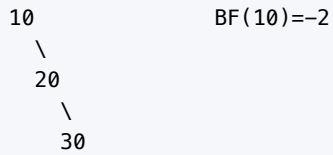


30 üzerinde right rotation:

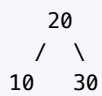


10. RR Case Insertion order:

10,20,30

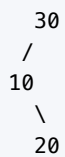


10 üzerinde left rotation:



11. LR Case Insertion order:

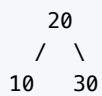
30,10,20



İki operation gerekir:

1. RotateLeft(10)
2. RotateRight(30)

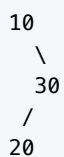
Sonuç:



Bu bir Double Rotationdır.

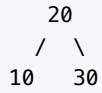
12. RL Case Insertion order:

10,30,20



1. RotateRight(30)
2. RotateLeft(10)

Sonuç yine:

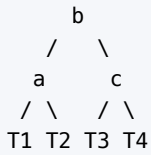


13. Üç node'dan büyük subtree'de rotation Rotation yalnız üç key'i sıralamak değildir; attached subtrees doğru yerlere aktarılmalıdır.

$a < b < c$ olsun. LR/RL double rotation sonrası median key b subtree root olur:



Fakat a, b, c'nin altındaki T1, T2, T3, T4 subtrees de key ranges'e göre bağlanır:



$T1 < a < T2 < b < T3 < c < T4$

Bu transfer atlanırsa nodes kaybolur veya BST property bozulur.

14. AVL insertion algorithm Insertion üç aşamalıdır:

1. Normal BST insertion yap.
2. Recursion geri açılırken heights'ı update et.
3. BF üzerinden gerekirse rotation yap.

```

Insert(node, key):
    if node == null:
        return new Node(key, height=0)

    if key < node.key:
        node.left = Insert(node.left, key)
    else if key > node.key:
        node.right = Insert(node.right, key)
    else:
        return node                // duplicate policy

    node.height = 1 + max(height(node.left), height(node.right))
    bf = height(node.left) - height(node.right)

    // LL
    if bf > 1 and key < node.left.key:
        return RotateRight(node)

    // RR
    if bf < -1 and key > node.right.key:
        return RotateLeft(node)

    // LR
    if bf > 1 and key > node.left.key:
        node.left = RotateLeft(node.left)
        return RotateRight(node)

    // RL
    if bf < -1 and key < node.right.key:

```

```

node.right = RotateRight(node.right)
return RotateLeft(node)

return node

```

Caller, returned subtree root'u child pointer'ına bağlamalıdır. Rotation sonrası root değişebileceği için return value'yu yok saymak ciddi bug'dır.

15. Hangi ancestor'da rotation yapılır? Inserted leaf'ten root'a doğru çıkarken ilk unbalanced ancestor bulunur. Insertion için bu node'da yapılan single/double rotation ilgili subtree'nin insertion öncesi height'ını geri getirir; daha yukarıda ikinci rotation gerekmez.

Deletion farklıdır: height azalışı ancestors boyunca yayılabilir ve birden fazla rebalance gerekebilir.

16. Videodaki insertion dizisi Keys:

40,20,10,25,30,22,50

40,20,10 40'ta LL case oluşur. RotateRight(40):

```

  20
 /  \
10   40

```

25 Normal BST insertion:

```

  20
 /  \
10   40
     /
    25

```

Tree hâlâ balanced'dır.

30 30, 25. right olur. 40 node'unda LR case:

```

RotateLeft(25)
RotateRight(40)

```

Sonuç:

```

  20
 /  \
10   30
     / \
    25 40

```

22 22, 25. left olur. Root 20 ilk unbalanced ancestor'dır ve RL case oluşur:

```

RotateRight(30)
RotateLeft(20)

```

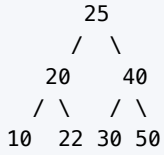
Sonuç:

```

    25
   /  \
  20   30
 /  \   \
10 22  40

```

50 50, 40. right olur. 30da RR case; RotateLeft(30):



Bu final tree perfect balanced görünür; fakat her AVL tree perfect veya complete olmak zorunda değildir.

17. Height neden $O(\log n)$ kalır? Height h olan en seyrek AVL tree'nin minimum node sayısı:

$$N(h) = 1 + N(h-1) + N(h-2)$$

Çünkü en kötü balanced durumda subtree heights $h-1$ ve $h-2$ olabilir. Bu recurrence Fibonacci gibi büyür:

$$N(h) = \Omega(\phi^h)$$

Ters çevirince:

$$h = O(\log n)$$

Daha kesin bilinen bound, convention'a bağlı küçük constants ile yaklaşık:

$$h < 1.4405 \cdot \log_2(n+2)$$

Videodaki "yaklaşık $1.44 \log n$ " ifadesinin temeli budur.

18. Complexity

Operation	Worst Time
Search	$O(\log n)$
Insert	$O(\log n)$
Single/Double rotation	$O(1)$
Delete	$O(\log n)$

Stored height field varsa BF ve height update $O(1)$ dir. Height her seferinde subtree traversal ile yeniden hesaplanırsa implementation gereksiz yere yavaşlar.

Space:

$O(n)$ tree storage
 $O(\log n)$ recursive call stack

Iterative implementation parent pointers/explicit path ile stack kullanımını değiştirebilir.

19. AVL ve Red-Black Tree

Özellik	AVL	Red-Black Tree
Balance	Daha strict	Daha gevşek
Height guarantee	Daha kısa/tight	$O(\log n)$, biraz daha uzun olabilir
Search	Sıklıkla çok iyi	$O(\log n)$
Update rotations	Daha sık olabilir	Genellikle update-friendly
Metadata	Height/BF	Color bit

İkisi de self-balancing BST'dir. "Red-Black her durumda AVL kadar aynı height'a sahiptir" doğru değildir; asymptotic class aynıdır, exact balance farklıdır.

20. Video anlatımını kesinleştiren noktalar AVL her zaman minimum-height BST değildir AVL local balance invariant'ını korur ve height $O(\log n)$ olur. Aynı keys için mümkün olan mutlak minimum height'ı garanti etmez.

Rotation yalnız "üç nodes'u taşıma" değildir Case üç critical nodes ile belirlenir; attached subtrees de pointer transferiyle korunur.

Case adı ile operation direction farklıdır LL case $\rightarrow$ Right Rotation, RR case $\rightarrow$ Left Rotation.

Rebalance geciktirilmez Normal online AVL insertion, her insertion'ın dönüş yolunda invariant'ı hemen geri kurar. Bütün keys'i normal BST'ye ekleyip en sonda birkaç local rotation yapmak aynı algorithm değildir.

21. Sık yapılan hatalar

- BF formülünü right-left alıp case conditions'ı değiştirmemek.
- Height convention'larını aynı calculation içinde karıştırmak.
- Rotation sonrası transferred subtree T2/T3ü kaybetmek.
- Height'ları yanlış sırada update etmek.
- Returned subtree root'u parent'a bağlamamak.
- Unbalanced node'dan inserted key'e bütün path'i case adı sanmak; ilk iki direction yeterlidir.
- Yalnız inserted node'un parent'ını kontrol etmek; ancestors da update edilmelidir.
- LL casete left rotation yapmak.
- AVL'nin complete/perfect tree olduğunu sanmak.
- Duplicate policy'yi tanımlamamak.

22. Kavram sözlüğü

English term	Türkçe açıklama
AVL Tree	Height-balanced self-balancing BST
Balance Factor	$height(left) - height(right)$
Rotation	BST order'ını koruyan local pointer transformation
Single Rotation	Bir left veya right rotation
Double Rotation	LR ya da RL için iki bağlı rotation
Skewed Tree	Bir side'a uzayıp chain'e yaklaşan tree
Ancestor	Bir node'un root yönündeki üst node'u
Subtree Transfer	Rotation sırasında orta key-range subtree'nin yeni parent'a bağlanması
Self-Balancing	Updates sonrası height invariant'ını otomatik geri kurma

23. Kendini sına

1. Normal BST operation time neden $O(h)$ dir?
2. AVL invariant'ı nedir?
3. Bu notta leaf ve empty tree height'ları kaçtır?
4. LL case hangi actual rotation ile düzelir?
5. LR casete iki operation sırası nedir?
6. Rotation BST order'ını neden korur?
7. Videodaki insertion dizisinin final root'u nedir?
8. Final tree'nin left ve right subtrees'i nasıldır?
9. AVL height neden $O(\log n)$ dir?

10. Insert ile delete rebalancing arasında hangi fark vardır?
11. Stored height field neden önemlidir?

Kısa cevaplar

1. Search path en fazla root-to-leaf height kadar ilerler.
2. Her node için $|\text{height}(\text{left}) - \text{height}(\text{right})| \leq 1$.
3. $\text{height}(\text{empty}) = -1$, $\text{height}(\text{leaf}) = 0$.
4. Right Rotation.
5. Önce left child üzerinde left rotation, sonra node üzerinde right rotation.
6. Subtrees key ranges'e uygun aktarılır ve in-order sequence değişmez.
7. 25.
8. Left 20(10,22), right 40(30,50).
9. Minimum-node recurrence Fibonacci hızında büyüdüğü için inverse height logarithmic olur.
10. Insert'te ilk rebalance yeterlidir; delete'te birden fazla ancestor rebalance gerekebilir.
11. BF ve height update'in $O(1)$ kalmasını sağlar.

77 – B-Trees and B+ Trees in Databases

Video: 10.2 B-Trees and B+ Trees in Databases (39:41)

Bu videonun ana fikri Database index'i RAM'den çok storage pages üzerinde yaşar. Burada asıl maliyet CPU comparison değil, page I/O'dur. B-Tree ve B+ Tree, her node'a çok sayıda key/pointer koyarak yüksek fan-out, küçük height ve az page access sağlar.

1. Storage ve page mantığı Disk/SSD'den veri genellikle byte byte değil fixed-size page/block halinde okunur. Videoda öğretim amacıyla 512 byte block kullanılıyor; modern sistemlerde page size sıklıkla daha büyüktür ve sistem tarafından belirlenir.

Bir record 128 byte, page 512 byte ise basitleştirilmiş unspanned modelde:

```
blockingFactor = floor(512/128) = 4 record/page  
100 record → ceil(100/4) = 25 page
```

Gerçekte page header, slot directory, variable-length fields ve fragmentation da yer kaplar.

2. Index neden gerekir? Index yoksa aranan record için worst case bütün data pages taranabilir. Dense Indexte her search key/record için entry bulunabilir:

```
(key, recordPointer)
```

Videodaki değerlerle:

```
key = 10 byte  
record pointer = 6 byte  
entry = 16 byte  
512/16 = 32 entry/page  
100 entry → ceil(100/32) = 4 index page
```

Linear index taraması worst case 4 index page ve ardından 1 data page ister. Sorted index içinde binary search yapılabilir; daha büyük amaç index'in kendisini de çok seviyeli hale getirmektir.

3. Multi-Level Index Üst index level, alttaki her page için separator ve child pointer tutar. Index büyüdükçe yeni root level otomatik oluşur:

```
root index page  
↓  
internal index pages  
↓  
leaf/index pages  
↓  
records
```

Her level'da bir page okunursa I/O yaklaşık tree height kadardır.

4. m-Way Search Tree Order/degree m tanımıyla bir node en fazla:

```
m children  
m-1 sorted keys
```

tutar. Keys search ranges'i böler:

```
 $P_0 < K_1 < P_1 < K_2 < \dots < K_{(m-1)} < P_{(m-1)}$ 
```

Binary Search Tree, m=2 özel durumudur. Ancak sıradan m-way tree insertion order'a göre skewed olabilir; doluluk ve balance garantisi yoktur.

5. B-Tree kuralları Terminoloji kitaplar arasında değişebilir. Bu notta order m = maximum children:

- Her node en fazla m child ve $m-1$ key tutar.
- Root dışındaki internal nodes en az $\lceil m/2 \rceil$ child tutar.
- Dolayısıyla root dışı nodes en az $\lceil m/2 \rceil - 1$ key tutar.
- Non-leaf root en az 2 child tutar.
- Bütün leaves aynı depth'tedir.
- Keys node içinde sorted'dır ve child ranges'i ayırır.

Bu kurallar height'ın düşük kalmasını sağlar.

6. B-Tree node'u Database index olarak conceptual node:

$[P_0 \mid K_1, R_1 \mid P_1 \mid K_2, R_2 \mid \dots \mid K_r, R_r \mid P_r]$

- P_i : child/page pointer.
- K_i : separator/search key.
- R_i : B-Tree sürümünde record veya record pointer.

Node size genellikle bir storage page'e sığacak şekilde seçilir.

7. Search Her node'da key aranır:

1. Key node içindeyse record pointer bulunur.
2. Küçük/büyük aralığa göre bir child seçilir.
3. Leaf'e kadar tekrarlanır.

Node içi search linear veya binary olabilir. Asıl kazanç, büyük fan-out nedeniyle az levels olmasıdır.

$I/O = O(\log_m N)$ yaklaşık

8. Insertion ve split Insertion uygun leaf'e yapılır. Node capacity aşılsa:

1. Keys sıralanır.
2. Median separator parent'a promote edilir.
3. Kalan keys iki node'a split edilir.
4. Parent overflow ederse split yukarı yayılır.
5. Root split olursa yeni root oluşur ve height 1 artar.

Bu nedenle video insertion'ı bottom-up growth olarak anlatır.

Order 4 örneği Maximum 3 key/node olsun:

10, 20, 40

50 eklenince temporary dört key oluşur. Seçilen split convention'a göre middle separator parent'a çıkar; iki child minimum occupancy'yi sağlamalıdır. Even number of temporary keys'te lower/upper median seçimi implementation policy'ye bağlı olabilir.

9. Deletion özeti Video deletion'ı işlemiyor; bütünlük için temel mantık:

- Leaf'ten key sil.
- Minimum occupancy bozulursa sibling'dan borrow/redistribute et.
- Borrow mümkün değilse sibling ile merge et ve parent separator'ı aşağı indir.
- Underflow parent'a yayılabilir.
- Root boşalırsa only child yeni root olur; height azalır.

10. B+ Tree farkı B+ Tree database index'lerinde çok yaygındır:

- Actual records/record pointers yalnız leaves'dedir.
- Internal nodes yalnız separator keys ve child pointers tutar.

- Her search key leaf level’da bulunur.
- Separator keys internal levels’da duplicate görünebilir.
- Leaves ordered linked list şeklinde bağlanır.

```
internal: [separator | child pointers]
leaf:     [key, recordPointer] ↔ [key, recordPointer] ↔ ...
```

11. B-Tree ve B+ Tree karşılaştırması

Özellik	B-Tree	B+ Tree
Record pointer	Internal veya leaf node’da olabilir	Yalnız leaf’te
Search finish	Internal node’da bitebilir	Leaf’e iner
Internal fan-out	Record pointers yüzünden daha düşük olabilir	Genellikle daha yüksek
Range scan	Ek traversal gerekebilir	Linked leaves çok uygundur
Sequential access	Daha zor	Doğrudan leaf chain
Key duplication	Genellikle tek stored key	Separators leaf key’lerin copies’i olabilir

12. Dense ve sparse nuance B+ Tree leaf level, indexed entries’in dense ordered sequence’ını taşır; upper levels sparse separators gibidir. Ancak gerçek DBMS index’in dense/clustering davranışı index type ve duplicate-key representation’a bağlıdır. “Her B+ Tree mutlaka table’ın her physical record’u için ayrı leaf entry tutar” genellemesi her sistem için aynı değildir.

13. Range query Örnek:

```
WHERE key BETWEEN 30 AND 70
```

1. Tree search ile 30’un bulunduğu leaf’e git.
2. Leaf entries’i sırayla oku.
3. Linked next leaves üzerinden 70 aşılanı kadar devam et.

Cost yaklaşık:

```
tree height + result'ın kapladığı leaf pages
```

Bu, B+ Tree’nin database için temel avantajıdır.

14. Height ve fan-out Bir internal page yüzlerce child pointer tutabiliyorsa:

```
fan-out ≈ 100
height 3 → yaklaşık 100³ = 1,000,000 leaf range
```

Bu yalnız intuition’dır; occupancy ve root farkları exact capacity’yi değiştirir. Yine de küçük height az I/O demektir.

15. Videodaki anlatımı kesinleştiren noktalar

- 512 byte evrensel block size değildir; yalnız video örneğidir.
- Database işlemleri yalnız magnetic disk geometry ile açıklanamaz; SSD ve buffer pool’da da page abstraction korunur.
- DBMS yalnız “diskte data structure” değildir; transactions, recovery, concurrency, query processing gibi alanları da kapsar.
- B-Tree order tanımı kaynaklar arasında değişir; formülü kullanmadan önce convention belirtilmelidir.

- AVL gibi binary trees yerine B/B+ Tree seçilmesinin ana nedeni CPU comparison değil page I/O ve high fan-out'tur.

16. Complexity Balanced height:

Search = $O(\log_m N)$ node/page visits
 Insert = $O(\log_m N)$, split başına $O(m)$
 Delete = $O(\log_m N)$, borrow/merge dahil

In-memory comparison cost representation'a göre ayrıca değerlendirilir. Database bağlamında en değerli metrik çoğu zaman page reads/writes'tır.

17. Sık yapılan hatalar

- myi bazen maximum keys, bazen maximum children diye kullanmak.
- Leaves'in aynı level'da olması kuralını unutmak.
- Overflow'da median'ı parent'a bağlamadan yalnız node'u bölmek.
- Split sonrası child pointers'ı kaybetmek.
- B+ Tree internal separator'ını actual record sanmak.
- Range scan için her key'de root'tan yeniden başlamak.
- B-Tree'nin binary tree olduğunu sanmak.
- Page header/unused space'i yok sayan kapasite hesabını production değeri sanmak.

18. Kavram sözlüğü

English term	Türkçe açıklama
Page/Block	Storage ile memory arasında taşınan temel unit
Fan-Out	Bir internal node'un child sayısı
Multi-Level Index	Index pages'in üst index levels ile aranması
Separator Key	Child key ranges'ini ayıran internal key
Split	Overflow node'u iki node'a bölme
Promote	Median separator'ı parent'a çıkarma
Underflow	Node'un minimum occupancy altına düşmesi
Redistribution	Sibling'dan key/child ödünç alma
Leaf Link	B+ Tree leaves arasındaki ordered next/previous bağlantısı
Range Scan	Bir key interval'ındaki ordered records'ı dolaşma

19. Kendini sinama

1. B+ Tree neden high fan-out ister?
2. Order m convention'ında maximum keys kaçtır?
3. Root dışı minimum children kaçtır?
4. B-Tree insertion overflow'unda ne yapılır?
5. Root split olursa ne değişir?
6. B+ Tree'de record pointers nerede tutulur?
7. Leaves neden linked'dir?
8. B-Tree ile B+ Tree'nin search finish farkı nedir?
9. Page size neden sabit 512 kabul edilmemelidir?

Kısa cevaplar

1. Height ve page I/O'yu azaltmak için.
2. $m-1$.
3. $\text{ceil}(m/2)$.

4. Median promote edilir ve node split edilir; gerekirse işlem parent'a yayılır.
5. Yeni root oluşur, height bir artar.
6. Leaves'de.
7. Efficient ordered/range scan için.
8. B-Tree internal node'da bitebilir; B+ Tree leaf'e iner.
9. Page/block size storage engine ve sistem configuration'ına bağlıdır.

78 – Asymptotic Notations: Simplified

Video: Asymptotic Notations – Simplified (22:44)

Ana fikir Big O, Big Omega ve Big Theta, input büyürken functions'ın growth rate'ini constant factors ve lower-order terms'den bağımsız karşılaştırır.

$O(g)$: asymptotic upper bound
 $\Omega(g)$: asymptotic lower bound
 $\Theta(g)$: asymptotically tight bound

1. Frequency count'tan growth'a Videodaki örnekler:

single statement	→ constant	
single loop	→ $2n+3$	→ $\Theta(n)$
nested two loops	→ $2n^2+2n+1$	→ $\Theta(n^2)$
three nested loops	→ $\Theta(n^3)$	
binary search	→ $\Theta(\log n)$	
$T(n)=2T(n-1)+1$	→ $\Theta(2^n)$	

Exact coefficients machine/model'e bağlı olabilir; dominant growth class daha karardır.

2. Formal tanımlar Pozitif functions için:

$f(n) \in O(g(n))$
 $\Leftrightarrow \exists c > 0, n_0: 0 \leq f(n) \leq c \cdot g(n), \text{ every } n \geq n_0$

$f(n) \in \Omega(g(n))$
 $\Leftrightarrow \exists c > 0, n_0: 0 \leq c \cdot g(n) \leq f(n), \text{ every } n \geq n_0$

$f(n) \in \Theta(g(n))$
 $\Leftrightarrow \exists c_1, c_2 > 0, n_0: 0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n), \text{ every } n \geq n_0$

3. Polynomial örneği

$f(n) = 2n^2 + 3n + 1$

$n \geq 1$ için:

$2n^2 \leq f(n) \leq 6n^2$

Dolayısıyla:

$f(n) \in \Theta(n^2)$

Ayrıca gevşek ama doğru bounds vardır:

$f(n) \in O(n^3)$
 $f(n) \in O(2^n)$
 $f(n) \in \Omega(n)$
 $f(n) \in \Omega(1)$

En bilgilendirici standart ifade tight bound olan $\Theta(n^2)$ dir.

4. Dominant term kuralı Fixed-degree polynomial:

$$a_k n^k + \dots + a_1 n + a_0, a_k > 0$$

için:

$$\theta(n^k)$$

Bu kural her expression'a körlemesine uygulanmaz; exponential, factorial, logarithmic ve oscillating functions ayrıca analiz edilir.

5. Yaygın growth sırası

$$1 < \log n < n < n \log n < n^2 < n^3 < 2^n < n! < n^n$$

Bu sıra yeterince büyük n içindir. Log base constant factor farkı oluşturduğundan asymptotic class'ı değiştirmez.

6. $n!$ hakkındaki önemli düzeltme Videoda $n!$ için farklı kaba upper/lower bounds bulunup "tight bound yok" deniyor. Bu yanlıştır. Her function kendi başına tight bound verir:

$$n! \in \theta(n!)$$

Daha açıklayıcı Stirling bound:

$$n! = \theta(\sqrt{n} \cdot (n/e)^n)$$

Ayrıca:

$$n! \in O(n^n)$$

doğru ama gevşektir. $\Omega(1)$ de doğru ama aşırı gevşektir. Gevşek bounds'ın eşleşmemesi, tight bound'ın bulunmadığını göstermez.

7. Notation ile case analysis farklıdır Big O = worst case, Omega = best case değildir.

Önce hangi function/case'in analiz edildiği belirtilir:

$$\begin{aligned} T_{\text{worst}}(n) &\in \theta(n^2) \\ T_{\text{best}}(n) &\in \theta(n) \end{aligned}$$

Her bir function için ayrıca O , Ω , θ kullanılabilir.

8. O ile θ kullanım farkı

$$\begin{aligned} 3n+5 &\in O(n^2) && // \text{doğru, gevşek} \\ 3n+5 &\in \theta(n) && // \text{doğru ve tight} \end{aligned}$$

"Algorithm $O(n^2)$ dir" cümlesi yalnız upper guarantee verir; exact asymptotic growth iddia edilmek isteniyorsa θ tercih edilir.

9. Limit testi Uygun pozitif functions için:

$$\begin{aligned} \lim f(n)/g(n) = c, 0 < c < \infty &\Rightarrow f \in \theta(g) \\ \lim f(n)/g(n) = 0 &\Rightarrow f \text{ grows slower than } g \\ \lim f(n)/g(n) = \infty &\Rightarrow f \text{ grows faster than } g \end{aligned}$$

Limit yoksa formal inequalities kullanılabilir.

10. Sık yapılan hatalar

- Yalnız "known complexities list" içinden bound seçilebileceğini sanmak.
- = işaretini set membership gibi yorumlamak; matematiksel olarak $f \in O(g)$ daha nettir.
- Constant ve lower-order terms'i küçük n için de önemsiz sanmak.
- Big O'yu otomatik worst case kabul etmek.
- Her function'ın θ bound'ı olmadığını söylemek.
- $n!$ i yalnız $O(n^n)$ ve $\Omega(1)$ ile bırakmak.

11. Kavram sözlüğü

English term	Türkçe açıklama
Upper Bound	Yeterince büyük input'ta growth'u yukarıdan sınırlama
Lower Bound	Growth'u aşağıdan sınırlama
Tight Bound	Aynı class ile hem upper hem lower bound
Dominant Term	Asymptotic growth'u belirleyen en hızlı büyüyen term
Threshold n_0	Inequality'nin bundan sonra hep geçerli olduğu input
Constant Factor	Asymptotic class'ı değiştirmeyen sabit multiplier

12. Kendini sinama

1. O , Ω , θ neyi ifade eder?
2. $2n^2+3n+1$ neden $\theta(n^2)$ dir?
3. Aynı function $\theta(n^3)$ olabilir mi?
4. Big O neden doğrudan worst case değildir?
5. $n!$ için bir tight bound yaz.
6. $3n+5$ için $\theta(n^2)$ mi $\theta(n)$ mi daha bilgilendiricidir?

Kısa cevaplar

1. Sırasıyla upper, lower ve tight asymptotic bound.
2. Pozitif constants ile hem alttan hem üstten n^2 nin katları arasında kalır.
3. Evet; bu gevşek upper bound'dır.
4. Notation, önce seçilmiş herhangi bir case function'ına uygulanabilir.
5. $\theta(n!)$; Stirling ile $\theta(\sqrt{n}(n/e)^n)$.
6. $\theta(n)$.

79 – Hashing Technique: Simplified

Video: Hashing Technique – Simplified (17:04)

Ana fikir Hash Table, key'i bir Hash Function ile array index'e dönüştürür. İyi dağılım ve kontrollü Load Factor altında search/insert/delete expected $O(1)$ olabilir; collision resolution zorunludur.

1. Search yöntemleri

Linear Search $\rightarrow O(n)$, sorting gerekmez
Binary Search $\rightarrow O(\log n)$, sorted random-access data gerekir
Hashing $\rightarrow$ expected $O(1)$, order bilgisi sağlamaz

Hashing worst-case $O(1)$ garantisi değildir.

2. Direct Addressing

Key doğrudan index yapılır:

`table[key] = value`

Bu gerçek $O(1)$ erişim sağlar fakat key universe çok büyük ve sparse ise büyük memory israfı yaratır.

3. Hash Function

Table size m için basit integer hash:

$h(k) = k \bmod m$

Örnek $m=10$:

$h(8)=8$
 $h(3)=3$
 $h(13)=3$

3 ve 13 aynı slot'a gider; bu bir Collision'dır. Finite table'a daha büyük key space map edildiğinde collisions genel olarak kaçınılmazdır.

İyi hash:

- Deterministic ve hızlıdır.
- Eşit keys için eşit hash üretir.
- Expected input üzerinde slots'a dengeli dağıtır.

4. Load Factor

$\alpha = n/m$

- n : stored entries.
- m : bucket/slot count.

Chaining'de α 1'i aşabilir. Open addressing'de $n < m$, yani $\alpha < 1$ olmalıdır. α büyüdükçe collisions ve probes artar; threshold aşılmıca resize/rehash yapılır.

5. Separate Chaining

Her table slot bir bucket/list taşır:

`table[3]  $\rightarrow$  3  $\rightarrow$  13  $\rightarrow$  23`

Search:

1. $i = h(\text{key})$ hesapla.
2. Yalnız `table[i]` chain'ini tara.

Uniform hashing altında expected chain length α :

Expected search = $O(1+\alpha)$
Worst search = $O(n)$

Bucket linked list olmak zorunda değildir; dynamic array veya tree de olabilir.

6. Open Addressing Bütün entries table'ın kendi slots'larında tutulur. Collision'da deterministic probe sequence izlenir.

Search, insertion ile aynı sequence'i takip etmelidir; aksi halde displaced key bulunamaz.

7. Linear Probing

$h_i(k) = (h(k) + i) \bmod m$
 $i = 0, 1, 2, \dots$

13 mod 10 = 3 ve slot 3 doluysa 4, 5, ... denenir; array sonundan sonra wrap around yapılır.

Avantajı cache locality'dir. Sorunu Primary Clustering: consecutive occupied blocks giderek büyür.

8. Quadratic Probing Basit video biçimi:

$h_i(k) = (h(k) + i^2) \bmod m$

Örneğin home slot 3:

$i=0 \rightarrow 3$
 $i=1 \rightarrow 4$
 $i=2 \rightarrow 7$

Primary clustering azalır. Fakat arbitrary m ve coefficients ile bütün slots ziyaret edilmeyebilir. Table size/probe constants uygun seçilmeli ve load factor sınırlı tutulmalıdır.

Genel biçim:

$h_i(k) = (h(k) + c_1 \cdot i + c_2 \cdot i^2) \bmod m$

9. Deletion ve tombstone Open addressing'de key doğrudan empty yapılırsa probe chain erken kesilebilir ve arkadaki key kaybolmuş görünür. Bu yüzden:

EMPTY → hiç kullanılmamış
OCCUPIED → key var
DELETED → tombstone

Search DELETED üzerinde devam eder; insertion onu tekrar kullanabilir.

10. Resizing Load factor threshold aşılsa:

1. Daha büyük table allocate et.
2. Her live key'i yeni size'a göre yeniden hash et.
3. Eski table'ı bırak.

Resize tek seferde $O(n)$ dir; seyrek yapıldığında insertion amortized expected $O(1)$ kalabilir.

11. Complexity

Method	Expected Search	Worst Search
Chaining	$O(1+\alpha)$	$O(n)$
Open addressing	$O(1)$ düşük α	$O(n)$

Videodaki "hashing log n'den hızlıdır" cümlesi expected ve iyi dağılım koşuluyla doğrudur; adversarial collisions veya kötü hash'te linear olabilir.

12. Sık yapılan hatalar

- Collision olamayacağını varsaymak.
- $h(k)=k$ yi her key domain için ideal saymak.
- Linear probe'da table sonundan wrap etmemek.
- Search ve insertion'da farklı probe sequence kullanmak.
- Open-address deletion'da tombstone yerine empty yazmak.
- Quadratic probing'in her table size'da bütün slots'u gezeceğini sanmak.
- Load factor'ı izlememek.
- Hashing'in sorted traversal sağladığını sanmak.

13. Kavram sözlüğü

English term	Türkçe açıklama
Hash Table	Keys'i computed slots'ta tutan structure
Hash Function	Key'i table index'e map eden function
Collision	Farklı keys'in aynı slot'a map edilmesi
Separate Chaining	Slot başına bucket/chain tutma
Open Addressing	Entries'i table slots içinde probe ederek tutma
Load Factor	$\alpha=n/m$ doluluk oranı
Primary Clustering	Linear probing'de consecutive cluster oluşumu
Tombstone	Probe chain'i bozmayan deletion marker
Rehashing	Keys'i yeni table size ile yeniden yerleştirme

14. Kendini sınamaya

1. Direct addressing'in sorunu nedir?
2. $h(k)=k \bmod 10$ için 3 ve 13 ne üretir?
3. Chaining expected search complexity'si nedir?
4. Linear probing'in temel sorunu nedir?
5. Quadratic probing neden her zaman boş slot bulmayabilir?
6. Tombstone neden gerekir?
7. Hashing'in worst-case search complexity'si nedir?

Kısa cevaplar

1. Büyük sparse key universe için memory israfı.
2. İkisi de 3; collision.
3. $O(1+\alpha)$.
4. Primary clustering.
5. Probe sequence table arithmetic'ine bağlı olarak bütün slots'u kapsamayabilir.
6. Deletion sonrası probe chain erken kesilmesini diye.
7. $O(n)$.

80 – Shortest Path: Dijkstra and Bellman-Ford, Simplified

Video: Shortest Path – Dijkstra and Bellman-Ford, Simplified (26:13)

Ana fikir İki algorithm da Single-Source Shortest Paths çözer:

- Dijkstra: bütün reachable edge weights non-negative olmalı.
- Bellman-Ford: negative edges'e izin verir ve reachable negative cycle'ı tespit eder.

Ortak temel operation Relaxation'dır.

1. Relaxation

Edge (u,v,w) için:

```
if dist[u] != INF and dist[u] + w < dist[v]:
    dist[v] = dist[u] + w
    parent[v] = u
```

Bu, uya bilinen path üzerinden vye daha kısa path bulunup bulunmadığını sorar.

Initialization:

```
dist[source]=0
dist[others]=INF
parent[*]=NIL
```

parent array shortest paths'i yeniden kurar; yalnız distances tutmak route'u vermez.

2. Dijkstra mantığı Her adımda unsettled vertices arasından minimum tentative distance'a sahip u seçilir. Non-negative weights sayesinde bu distance artık final'dir. Sonra onun outgoing edges'i relax edilir.

```
Dijkstra(G,s):
    initialize dist, parent
    pq = min-priority-queue containing (0,s)

    while pq not empty:
        (du,u)=extractMin(pq)
        if du != dist[u]: continue // stale entry

        for each edge (u,v,w):
            if w < 0: reject input
            if dist[u]+w < dist[v]:
                dist[v]=dist[u]+w
                parent[v]=u
                push/decreaseKey(v)
```

3. Videodaki Dijkstra trace

Source P için successive selections ve relaxations sonunda:

```
dist(P)=0
dist(Q)=1
dist(R)=2      via P→Q→R
dist(U)=3      via P→Q→R→U
dist(S)=4      via P→Q→R→S
dist(T)=7      direct edge de optimum olabilir
```

Equal-cost alternative bulunması distance'ı değiştirmek zorunda değildir; parent tie policy implementation'a bağlıdır.

4. Dijkstra correctness'in kilidi Minimum tentative u seçildiğinde daha sonra bulunacak başka path, unsettled bir vertex üzerinden gelip negative olmayan edges ekleyecektir. Bu path $dist[u]$ dan daha ucuz olamaz. Bu nedenle u safe biçimde finalize edilir.

Negative edge bu argument'ı bozar.

5. Negative edge neden sorun? Video örneği:

```
1→4 cost 1
1→2 cost 2
2→3 cost 3
3→4 cost -6
```

Dijkstra 4ü cost 1 ile erken finalize eder. Oysa:

```
1→2→3→4 = 2+3-6 = -1
```

Bazı negative-edge graphs'ta Dijkstra tesadüfen doğru sonuç verebilir; bu algorithm'ın geçerli olduğu anlamına gelmez. Genel precondition:

```
all reachable edge weights  $\geq 0$ 
```

6. Dijkstra complexity

Implementation	Time
Array/matrix minimum scan	$O(V^2)$
Adjacency list + binary heap	$O((V+E) \log V)$

Space adjacency representation dışında $O(V)$ distance/parent/PQ state'tir. Red-Black Tree kullanılabilir ama gerekli değildir; binary heap standart seçimdir.

7. Bellman-Ford mantığı Her edge bütün olarak $V-1$ round relax edilir:

```
BellmanFord(G,s):
    initialize dist, parent

    repeat V-1 times:
        changed=false
        for each edge (u,v,w):
            if dist[u] != INF and dist[u]+w < dist[v]:
                dist[v]=dist[u]+w
                parent[v]=u
                changed=true
        if not changed: break

    for each edge (u,v,w):
        if dist[u] != INF and dist[u]+w < dist[v]:
            report reachable negative cycle
```

8. Neden $V-1$ round? Negative cycle olmayan graph'ta simple shortest path en fazla $V-1$ edge içerir. Her full relaxation round, edge order ne olursa olsun en az bir additional edge uzunluğundaki shortest-path bilgisinin yayılmasını garanti eder.

Favorable edge order'da bilgi aynı round içinde daha uzağa yayılabilir; $V-1$ worst-case guarantee'dir.

9. Videodaki Bellman-Ford örneği Directed edges şu order'da relax ediliyor:

```
D→B (-6)
C→D (3)
A→C (2)
A→B (1)
```

Source A. $V=4$, dolayısıyla en fazla üç round:

round 1: C=2, B=1, D=INF
round 2: D=5
round 3: B=-1 via A→C→D→B

Final:

dist(A)=0, dist(C)=2, dist(D)=5, dist(B)=-1

10. Negative cycle Örneğe B→C (1) eklenirse cycle:

B→C→D→B
cost = 1+3-6 = -2

Videoda bu toplam sözlü olarak -3 deniyor; doğru arithmetic -2dir. Cycle her tekrarlandığında path cost 2 daha azalır, bu nedenle affected vertices için finite shortest path yoktur.

V-1 round sonrası extra round'da hâlâ relaxation varsa source'tan reachable negative cycle vardır.

Unreachable negative cycle source'un shortest paths'ini etkilemez; dist[u] != INF guard'ı bu ayrımı sağlar.

11. Bellman-Ford complexity

Time = $O(VE)$
Space = $O(V)$

Dense directed graph'ta $E=O(V^2)$ olduğundan time $O(V^3)$ olabilir. Early stopping best/practical cases'i iyileştirir, worst bound değişmez.

12. Karşılaştırma

Özellik	Dijkstra	Bellman-Ford
Negative edge	Desteklemez	Destekler
Negative cycle detection	Hayır	Reachable cycle için evet
Heap time	$O((V+E) \log V)$	$O(VE)$
Strategy	Greedy finalization	Repeated relaxation

13. Sık yapılan hatalar

- Dijkstra'yı "negative cycle yoksa çalışır" sanmak; tek negative edge bile guarantee'yi bozar.
- Vertex'i visited yapınca daha sonra update etmeye çalışmak.
- Priority queue'daki stale entries'i ele almamak.
- Bellman-Ford'u V yerine V-1 main rounds mantığıyla kurmamak.
- Negative-cycle pass'inde unreachable edge'i kontrol etmek.
- Distance verirken path reconstruction için parent tutmamak.
- Undirected negative edge'in iki yönde geçilebiliyorsa negative closed walk oluşturacağını unutmak.

14. Kendini sinama

1. Relaxation condition nedir?
2. Dijkstra hangi edge precondition'ını ister?
3. Dijkstra heap complexity'si nedir?
4. Bellman-Ford neden V-1 round yapar?
5. Negative cycle nasıl tespit edilir?
6. Videodaki cycle cost'u doğru olarak kaçtır?
7. Unreachable negative cycle raporlanmalı mı?

Kısa cevaplar

1. $\text{dist}[u] + w < \text{dist}[v]$ ve u reachable ise update.
2. Reachable edges non-negative olmalıdır.
3. $O((V+E) \log V)$.
4. Simple shortest path en fazla $V-1$ edge içerir.
5. Main rounds sonrası bir reachable edge daha relax olabiliyorsa.
6. $1+3-6=-2$.
7. Source-specific sonuç için hayır.

81 – BFS and DFS: Simplified

Video: BFS and DFS – Simplified (19:13)

Ana fikir Breadth-First Search (BFS) graph'ı level by level ve Queue ile; Depth-First Search (DFS) mümkün olduğunca derine gidip backtrack ederek Stack/Recursion ile dolaşır.

İkisinde de bir vertex ilk keşfedildiği anda visited işaretlenmelidir.

1. Terminoloji

- Visit/Discover: Vertex ile ilk kez karşılaşmak ve visited yapmak.
- Explore: Vertex'in bütün outgoing/adjacent neighbors'ını incelemek.
- Finish: Bütün adjacency list işlendiğinde exploration'ın bitmesi.

Video "visit" ve "explore" ayrımını özellikle kullanır.

2. BFS BFS invariant'ı: Queue'nun önündeki vertex sıradaki explore edilecek vertex'tir.

```
BFS(G,s):
  for each v: visited[v]=false
  Q=empty queue

  visited[s]=true
  distance[s]=0
  parent[s]=NIL
  enqueue(Q,s)

  while Q not empty:
    u=dequeue(Q)
    for each v in Adj[u]:
      if not visited[v]:
        visited[v]=true
        distance[v]=distance[u]+1
        parent[v]=u
        enqueue(Q,v)
```

Visited işareti enqueue anında konur. Dequeue'a bırakılırsa aynı vertex farklı parents tarafından birden çok kez queue'ya eklenebilir.

3. Videodaki BFS örneği Start 1, komşular küçük sıra ile seçildiğinde örnek order:

1,2,4,3,5,8,9,6,7

Conceptual levels:

```
level 0: 1
level 1: 2,4
level 2: 3,5
level 3: 8,9,6,7
```

Adjacent vertices'in order'ı değiştirilirse aynı graph ve source için başka valid BFS order oluşabilir. Ancak level distances değişmez.

4. Valid BFS order nasıl kontrol edilir? Her dequeued vertex'in henüz görülmemiş neighbors'ı, daha sonra dequeued olacak vertices'ten önce queue'ya topluca girer.

Örneğin 1in neighbors'ı 2,4 ise, distance 2 olan bir vertex 2 veya 4ten önce gelemez. Yalnız level order'a bakmak bazen yetmez; aynı parent's adjacency grouping ve seçilen neighbor order da hesaba katılmalıdır.

5. BFS'nin shortest-path özelliği Unweighted graph'ta veya bütün edges eşit cost'taysa BFS:

```
distance[v] = source'tan minimum edge count
```

garanti eder. Weighted graph'ta genel shortest path için BFS kullanılmaz; weights'e göre Dijkstra/Bellman-Ford gibi algorithms gerekir.

6. BFS tree

Bir vertex ilk keşfedildiğinde:

```
parent[v]=u
```

edge'i traversal tree'ye girer. Connected graph'ta bu bir BFS Spanning Tree; disconnected graph'ta tek source yalnız kendi connected component'ını kapsar.

7. DFS

Recursive sürüm:

```
DFSVisit(G,u):
    visited[u]=true
    for each v in Adj[u]:
        if not visited[v]:
            parent[v]=u
            DFSVisit(G,v)
```

Yeni neighbor bulununca current vertex'in kalan exploration'ı suspend edilir; yeni vertex tamamen bitince geri dönülür. Recursion call stack bu suspended states'i tutar.

8. Iterative DFS

```
DFSIterative(G,s):
    stack=[s]
    visited[s]=true

    while stack not empty:
        u=pop(stack)
        process(u)
        for v in reverse(Adj[u]):
            if not visited[v]:
                visited[v]=true
                parent[v]=u
                push(stack,v)
```

Exact recursive order'ı taklit etmek için neighbors reverse push edilebilir. Fakat visited-on-push iterative sürümü, cross connections nedeniyle recursive order ile her graph'ta birebir aynı olmayabilir; exact simulation için (vertex,nextNeighborIndex) frame tutulur.

9. Videodaki DFS örneği

Bir valid choice order:

```
1→2→3→4→5→6
```

6 dead end olduğunda 5e backtrack edilip 7, ardından uygun ancestors'a dönülüp 8,9 ziyaret edilir:

```
1,2,3,4,5,6,7,8,9
```

Başka adjacency order başka valid DFS traversal verir. Fakat active path'te keşfedilmemiş neighbor varken alakasız branch'e atlamak valid DFS değildir.

10. DFS tree ve kullanım alanları

DFS parent edges DFS Tree oluşturur. Başlıca uygulamalar:

- Cycle detection.
- Topological sorting (DAG).
- Strongly connected components.

- Articulation points ve bridges.
- Maze/path exploration.

BFS ise unweighted shortest paths ve level structure için doğrudan uygundur.

11. Disconnected graph

Bütün graph'ı traverse etmek için outer loop gerekir:

```
for each vertex v:
    if not visited[v]:
        BFS/DFS from v
```

Sonuç tek spanning tree değil Spanning Foresttir. Component sayısı, başlatılan traversal-tree sayısına eşittir (undirected graph için).

12. Complexity

Adjacency list:

```
Time =  $O(V+E)$ 
Space =  $O(V)$  traversal metadata
```

Undirected graph'ta her edge adjacency lists'te iki kez görölse de $2E$, $O(E)$ dir.

Adjacency matrix:

```
Time =  $O(V^2)$ 
```

BFS queue ve DFS recursion/stack worst case $O(V)$ yer tutabilir.

13. BFS ve DFS karşılaştırması

Özellik	BFS	DFS
Structure	Queue	Stack/Recursion
Pattern	Level by level	Deep then backtrack
Unweighted shortest path	Evet	Hayır
Memory	Wide frontier'da yüksek	Deep path'te yüksek
Typical uses	Levels, shortest edge-count	Cycles, ordering, components

14. Sık yapılan hatalar

- Vertex'i enqueue/push ederken değil çok geç visited yapmak.
- BFS'de Queue yerine Stack kullanmak.
- DFS'de current vertex bitmeden sibling branch'e atlamak.
- Adjacency order farklı diye valid traversal'ı yanlış saymak.
- Single-source traversal'ın disconnected graph'ın tamamını gezeceğini sanmak.
- Weighted shortest path'i BFS ile çözmek.
- $O(V+E)$ bound'ını adjacency matrix için de aynen yazmak.
- Traversal output ile spanning-tree edges'i aynı şey sanmak.

15. Kavram sözlüğü

English term	Türkçe açıklama
Discover	Vertex'i ilk kez görmek
Explore	Bütün adjacent edges'i incelemek
Frontier	BFS'de keşfedilmiş ama henüz explore edilmemiş vertices
Backtracking	DFS'de dead end sonrası ancestor'a dönme
Traversal Tree	First-discovery parent edges'inden oluşan tree
Spanning Forest	Disconnected components için traversal trees kümesi
Dead End	Keşfedilmemiş neighbor kalmayan DFS state'i

16. Kendini sına

1. BFS ve DFS hangi structures'ı kullanır?
2. Vertex ne zaman visited yapılmalıdır?
3. BFS hangi shortest-path türünü garanti eder?
4. Aynı graph'ın birden fazla valid BFS/DFS order'ı neden olabilir?
5. DFS backtracking ne zaman olur?
6. Disconnected graph'ın tamamı nasıl gezilir?
7. Adjacency-list complexity nedir?
8. Adjacency-matrix complexity nedir?

Kısa cevaplar

1. BFS Queue, DFS Stack/Recursion.
2. İlk keşifte, queue/stack'e eklenirken.
3. Unweighted/equal-weight graph'ta minimum edge count.
4. Starting vertex ve adjacency iteration order değişebilir.
5. Current vertex'te keşfedilmemiş neighbor kalmadığında.
6. Unvisited her vertex için yeni traversal başlatılır.
7. $O(V+E)$.
8. $O(V^2)$.

82 – Tower of Hanoi: Made Easy

Video: Tower of Hanoi – Made Easy (9:32)

Ana fikir Tower of Hanoi, recursion'ın "önce küçük subproblem, sonra tek zorunlu move, sonra aynı subproblem" yapısını gösterir.

1. Kurallar Üç tower vardır:

Source A, Auxiliary B, Destination C

n disks başlangıçta A'da büyükten küçüğe dizilir. Kurallar:

1. Bir seferde yalnız bir disk taşınır.
2. Yalnız tower'ın en üstündeki disk taşınabilir.
3. Büyük disk küçük diskin üstüne konamaz.

Amaç bütün stack'i A'dan C'ye taşımaktır.

2. Base cases $n=0$: Move yoktur.

$n=1$:

A → C

3. Recursive decomposition En büyük disk'i A'dan C'ye taşıyabilmek için üstündeki $n-1$ disks önce B'ye gitmelidir:

1. Move $n-1$ disks A → B using C
2. Move largest disk A → C
3. Move $n-1$ disks B → C using A

İlk ve üçüncü adım aynı problemin daha küçük instance'larıdır.

4. Pseudocode

```
Hanoi(n, source, auxiliary, destination):
    if n == 0:
        return

    Hanoi(n-1, source, destination, auxiliary)
    print source, "→", destination
    Hanoi(n-1, auxiliary, source, destination)
```

Parameter order'ı dikkatle izle:

- İlk call'da destination ile auxiliary yer değiştirir.
- İkinci call'da old auxiliary new source olur.

5. $n=3$ trace

Hanoi(3,A,B,C)

Moves:

1. A → C
2. A → B
3. C → B
4. A → C

5. B → A
6. B → C
7. A → C

Her step'te smaller-on-larger invariant korunur.

6. Recursion tree ve move sayısı Bir n-disk call iki n-1 call ve bir direct move üretir:

$$T(0)=0$$

$$T(n)=2T(n-1)+1$$

Expansion:

$$T(n)=2^n-1$$

Örnek:

n	Minimum moves
1	1
2	3
3	7
4	15
10	1023

7. Neden optimal? En büyük disk taşınmadan önce üstündeki n-1 disks başka tower'a taşınmalıdır: en az $T(n-1)$ move. En büyük disk için bir move gerekir. Sonra n-1 disks onun üstüne taşınmalıdır: yeniden en az $T(n-1)$ move.

$$T(n) \geq 2T(n-1)+1$$

Recursive algorithm bu lower bound'a equality ile ulaşır; dolayısıyla optimaldir.

8. Complexity

$$\text{Time} = \Theta(2^n)$$

$$\text{Moves} = 2^n - 1$$

$$\text{Stack} = \Theta(n)$$

Output'un kendisi exponential uzunluktadır; bütün moves yazdıran hiçbir algorithm asymptotically bundan hızlı olamaz.

9. Overflow ve practical limit 2^n-1 hızla büyür. 64-bit unsigned integer bile yalnız sınırlı n için exact count tutar. Moves'i gerçekten üretmek, sayı representation'dan önce zaman nedeniyle imkânsızlaşır.

10. Sık yapılan hatalar

- $n=0/n=1$ base case koymamak.
- Auxiliary ve destination parameters'ı yanlış sırada geçirmek.
- En büyük disk'i üstteki disks taşınmadan move etmek.
- Time complexity'yi $\Theta(n)$ sanmak; recursion depth ile total calls farklıdır.
- Move count'u 2^n yazmak; exact değer 2^n-1 dir.

11. Kavram sözlüğü

English term	Türkçe açıklama
Source Tower	Başlangıç stack'inin olduğu tower
Auxiliary Tower	Geçici taşımalarda kullanılan tower
Destination Tower	Final stack'in hedefi
Base Case	Recursive call üretmeyen en küçük instance
Recursive Case	Problemi daha küçük aynı problemlere ayıran durum
Call Stack	Aktif recursive calls'ın state'lerini tutan stack

12. Kendini sına

1. Üç temel kural nedir?
2. Recursive çözümün üç adımı nedir?
3. $n=3$ için kaç move gerekir?
4. Recurrence nedir?
5. Exact solution nedir?
6. Space neden $\theta(n)$ dir?
7. Algorithm neden optimaldir?

Kısa cevaplar

1. Tek disk, yalnız üst disk, büyük küçük üstüne gelemmez.
2. $n-1$ source $\rightarrow$ auxiliary, büyük disk source $\rightarrow$ destination, $n-1$ auxiliary $\rightarrow$ destination.
3. 7.
4. $T(n)=2T(n-1)+1$, $T(0)=0$.
5. 2^n-1 .
6. Maximum recursion depth ndir.
7. En büyük disk öncesi ve sonrası zorunlu iki $n-1$ transfer lower bound'ına ulaşır.

83 – Row-Major and Column-Major Mapping

Video: Row-Major and Column-Major Mapping (19:16)

Ana fikir Multi-dimensional array, linear memory üzerinde tek sıra bytes olarak saklanır. Row-Major son index'i hızlı değiştirir; Column-Major ilk index'i hızlı değiştirir. Address formula, indices'i linear offset'e çevirir.

1. One-dimensional array Base address B, element width W, lower bound L:

$$\text{LOC}(A[i]) = B + (i-L) \cdot W$$

Zero-based array için $L=0$:

$$\text{LOC}(A[i]) = B + iW$$

Video örneği $B=100, W=2, i=2$:

$$\text{LOC}(A[2]) = 100 + 2 \cdot 2 = 104$$

$W=2$ yalnız örnektir; type size platform ve language'e bağlıdır.

2. 2D dimensions ve bounds

rows: $L_r \dots U_r$, count $M = U_r - L_r + 1$
cols: $L_c \dots U_c$, count $N = U_c - L_c + 1$

Element $A[i][j]$ için önce kaç element atlandığı bulunur, sonra W ile çarpılır.

3. Row-Major formula Her full previous row N element içerir:

$$\begin{aligned} \text{offset} &= (i - L_r) \cdot N + (j - L_c) \\ \text{LOC}(A[i][j]) &= B + W \cdot \text{offset} \end{aligned}$$

Zero-based:

$$B + W \cdot (iN + j)$$

4. Videodaki Row-Major örneği 3×4 , zero-based, $B=500, W=2$:

$$\begin{aligned} \text{LOC}(A[2][1]) &= 500 + 2 \cdot (2 \cdot 4 + 1) \\ &= 500 + 18 \\ &= 518 \end{aligned}$$

Memory order:

$A_{00}, A_{01}, A_{02}, A_{03}, A_{10}, A_{11}, \dots$

5. Column-Major formula Her full previous column M element içerir:

$$\begin{aligned} \text{offset} &= (j - L_c) \cdot M + (i - L_r) \\ \text{LOC}(A[i][j]) &= B + W \cdot \text{offset} \end{aligned}$$

Zero-based:

$$B + W \cdot (jM + i)$$

6. Videodaki Column-Major örneği 3×4 , $B=500$, $W=2$:

```
LOC(A[1][2])
=500+2*(2*3+1)
=514
```

Memory order:

```
A00,A10,A20,A01,A11,A21,...
```

7. One-based özel durum Bounds 1den başlıyorsa:

```
Row-major:    B+W*((i-1)N+(j-1))
Column-major: B+W*((j-1)M+(i-1))
```

Genel lower-bound formula ezberlenirse zero/one/arbitrary indexing aynı yapıdan çıkar.

8. 3D ve strides Row-major A[D1][D2][D3] zero-based:

```
offset = i*(D2D3)+j*D3+k
```

Burada strides:

```
stride0=D2D3, stride1=D3, stride2=1
```

Genel representation:

```
LOC = B + W * Σ((index_r-lower_r) * stride_r)
```

Views, padding ve non-contiguous slices'ta strides yalnız dimension products olmak zorunda değildir; runtime metadata'da saklanabilir.

9. Cache locality Physical order ile aynı yönde traversal daha iyi spatial locality sağlar.

Row-major:

```
for i:
  for j:
    use A[i][j]
```

Column-major'da çoğunlukla loop order ters çevrilir. Formulas aynı operation count'a sahip görünse de performance yalnız arithmetic değil cache line access pattern'ına bağlıdır.

10. Language örnekleri

- C/C++ built-in arrays: row-major.
- Fortran, MATLAB, Julia: column-major.
- NumPy default: row-major-like C order; Fortran order da desteklenir.

Videodaki "C'den sonra her language zero-based oldu" ifadesi doğru değildir. Birçok modern language one-based veya configurable bounds kullanır. Zero-based indexing'in gerekçesi de yalnız bir subtraction instruction kurtarmak değildir; pointer-offset modeli ve language design tercihidir.

11. Complexity Known dimensions/strides ile address calculation:

```
Time = O(1)
```

Array traversal:

Time = $\Theta(MN)$

Layout asymptotic element count'u deęiřtirmmez, fakat constants/cache behavior'ı ciddi etkiler.

12. Sık yapılan hatalar

- Row-major formula'da N yerine M kullanmak.
- Column-major formula'da M yerine N kullanmak.
- Lower bounds'i çıkarmamak.
- Byte offset ile element offset'i karıřtırıp w_i unutmak.
- Base address'i array object metadata adresi sanmak.
- Type width'ü evrensel sabit kabul etmek.
- Loop order'ın cache etkisini göz ardı etmek.

13. Kavram sözlüğü

English term	Türkçe açıklama
Base Address	İlk logical element'in adresi
Element Width	Bir element'in byte sayısı
Lower Bound	Bir dimension'ın ilk valid index'i
Linear Offset	Base'ten itibaren atlanan element sayısı
Stride	Bir index bir artınca linear offset'in artışı
Row-Major	Rows'un contiguous tutulduęu layout
Column-Major	Columns'un contiguous tutulduęu layout
Spatial Locality	Yakın addresses'a yakın zamanda erişme

14. Kendini sınama

1. Genel 1D address formula nedir?
2. Row-major 2D formula nedir?
3. Column-major 2D formula nedir?
4. 3×4 , $B=500$, $w=2$ için row-major $A[2][1]$ kaçtır?
5. Aynı bilgilerle column-major $A[1][2]$ kaçtır?
6. Row-major traversal'da hangi index inner loop'ta olmalıdır?
7. Zero-based olmayan arrays nasıl ele alınır?

Kısa cevaplar

1. $B+(i-L)W$.
2. $B+W((i-Lr)N+(j-Lc))$.
3. $B+W((j-Lc)M+(i-Lr))$.
4. 518.
5. 514.
6. Column/son index j .
7. Her dimension'da lower bound index'ten çıkarılır.

84 – Merge Sort Algorithm (Hindi)

Video: Merge Sort Algorithm – Hindi (16:38)

Bu videonun ana fikri Bu video Hindi anlatımlıdır; English translation caption kaynak olarak kullanılmıştır. Merge Sort, array'i single-element sorted lists'e kadar böler ve sorted halves'ı linear-time Merge işlemiyle birleştirir.

Divide → Recursively Sort → Merge

1. Merge işlemi İki sorted list:

A, length m
B, length n

üç pointer ile birleştirilir:

- i: A'nın current element'i.
- j: B'nin current element'i.
- k: output C'nin next free position'ı.

Küçük current element C'ye kopyalanır. Bir list bitince diğer list'in remainder'ı topluca kopyalanır.

2. Merge pseudocode

```
Merge(A,B):  
    i=j=0  
    C=[]  
  
    while i<length(A) and j<length(B):  
        if A[i] <= B[j]:  
            C.append(A[i])  
            i++  
        else:  
            C.append(B[j])  
            j++  
  
    while i<length(A):  
        C.append(A[i]); i++  
  
    while j<length(B):  
        C.append(B[j]); j++  
  
    return C
```

<= kullanımı equal keys'te left item'ı önce seçer ve stability'yi korur.

3. Merge correctness invariant'ı Her iteration başında:

C, A ve B'den tüketilen elements'ın sorted merge'idir.
C'nin bütün elements'ı kalan elements'tan küçük/eşittir.

Current heads içinden küçüğü seçmek safe'tir; her input list kendi içinde sorted olduğu için onun arkasında daha küçük bir element olamaz.

4. Birden fazla merge pattern Video dört/eight single-element lists'in farklı pairing patterns ile merge edilebileceğini gösterir. Balanced pairing:

8 lists → 4 → 2 → 1

Merge Sort'un balanced recursion'ı her level'da toplam n element işler ve $\log n$ levels oluşturur. Sequentially büyüyen bir list'e tek tek merge yapmak aynı iyi bound'ı sağlamayabilir.

5. Recursive Merge Sort

```
MergeSort(A, low, high):  
    if low >= high:  
        return  
  
    mid = low + floor((high-low)/2)  
    MergeSort(A, low, mid)  
    MergeSort(A, mid+1, high)  
    MergeRanges(A, low, mid, high)
```

$\text{low} + (\text{high} - \text{low}) / 2$, fixed-width integers'da $\text{low} + \text{high}$ overflow riskini azaltır.

6. Eight-element örneğinin yapısı

Video array'i önce sekiz sorted one-element list olarak görür:

```
[3] [8] [5] [7] [2] [9] [4] [6]
```

İlk merge level:

```
[3,8] [5,7] [2,9] [4,6]
```

İkinci level:

```
[3,5,7,8] [2,4,6,9]
```

Final:

```
[2,3,4,5,6,7,8,9]
```

8 singleton list'i bir tree biçiminde tek listeye dönüştürmek 7 merge calls gerektirir; genel olarak $n-1$ internal merge nodes vardır.

7. Recurrence ve analysis

Balanced halves için:

```
 $T(n) = 2T(n/2) + \theta(n)$ 
```

Recursion tree'de:

- Her level toplam $\theta(n)$ merge work yapar.
- Level sayısı $\theta(\log n)$ dir.

Sonuç:

```
Time =  $\theta(n \log n)$ 
```

Best, average ve worst case aynı asymptotic time'a sahiptir; input already sorted olsa da standard implementation divisions ve merges'i yapar.

8. Space

Array merge için auxiliary buffer:

```
Auxiliary array =  $\theta(n)$   
Recursion stack =  $\theta(\log n)$   
Total extra      =  $\theta(n)$ 
```

Video, merged result'ın temporary array'e yazılıp original range'e kopyalandığını vurgular. Tek reusable n -element buffer allocate etmek, her call'da yeni arrays üretmekten daha verimlidir.

Linked lists, pointer relinking ile merge edilebilir; yine recursion stack gerekir fakat array-style $\theta(n)$ buffer zorunlu olmayabilir.

9. Stability Equal keys original relative order'ını korursa sort Stable'dir.

```
(5,A), (3,X), (5,B)
→ (3,X), (5,A), (5,B)
```

Merge equality'de left element'ı önce seçerse Merge Sort stable olur.

10. Neden in-place değildir? Standard array Merge Sort, iki interleaved sorted range'i kolay ve linear biçimde birleştirmek için temporary buffer kullanır. Advanced in-place merge algorithms vardır, ancak daha karmaşık ve constants'ı farklıdır. Bu nedenle genel ifade:

```
Standard array Merge Sort is not in-place.
```

11. Iterative Bottom-Up sürüm Recursion zorunlu değildir:

```
width=1
while width<n:
    adjacent width-sized runs'ı merge et
    width*=2
```

Bu da $\theta(n \log n)$ time ve array için $\theta(n)$ buffer kullanır; recursion stack'i kaldırır.

12. Merge Sort ne zaman güçlüdür?

- Stable sorting gerektiğinde.
- Linked list sorting'de.
- Sequential external storage runs'larında (External Merge Sort).
- Predictable worst-case $\theta(n \log n)$ gerektiğinde.
- Data memory'e sığmadığında sorted runs üretilip disk üzerinde merge etmekte.

Random-access array ve düşük extra memory hedefinde Heap Sort; iyi average constants için uygun Quick Sort variants tercih edilebilir.

13. Sık yapılan hatalar

- Input halves sorted değilken Merge çağırmak.
- Bir list bitince diğer remainder'ı kopyalamamak.
- mid element'ını iki tarafa birden koymak veya iki taraftan da atmak.
- Equality'de right item'ı seçip stability'yi bozmak.
- Temporary result'ı original array'e geri kopyalamamak.
- Time'ı $\theta(n^2)$ sanmak; her level n, levels $\log n$ dir.
- Extra space'i yalnız recursion stack sayıp merge buffer'ı unutmak.
- "Fiziksel olarak her division'da array kopyalanır" sanmak; index ranges ile logical division yeterlidir.

14. Complexity özeti

Özellik	Değer
Best time	$\theta(n \log n)$
Average time	$\theta(n \log n)$
Worst time	$\theta(n \log n)$
Array extra space	$\theta(n)$
Recursive depth	$\theta(\log n)$
Stable	Evet, merge tie doğruysa
Standard array version in-place	Hayır

15. Kavram sözlüğü

English term	Türkçe açıklama
Merge	İki sorted sequence'ı tek sorted sequence yapma
Divide and Conquer	Böl, subproblems çöz, sonuçları birleştir
Sorted Run	Kendi içinde sorted contiguous sequence
Auxiliary Array	Merge output'u için temporary buffer
Stable Sort	Equal keys'in relative order'ını koruyan sort
Bottom-Up Merge Sort	Width 1 runs'dan iterative merge ile büyüyen sürüm
External Merge Sort	Diskteki büyük data için run üretme ve multi-way merge

16. Kendini sına

1. Merge input'larının hangi özelliği olmalıdır?
2. Bir input list bitince ne yapılır?
3. Recursive recurrence nedir?
4. Neden her level $\theta(n)$ work yapar?
5. Toplam time nedir?
6. Standard array extra space nedir?
7. Stability nasıl korunur?
8. Videodaki sekiz elementin sorted sonucu nedir?
9. Logical division neden physical copies gerektirmez?

Kısa cevaplar

1. İkisi de sorted olmalıdır.
2. Diğer list'in remainder'ı output'a kopyalanır.
3. $T(n)=2T(n/2)+\theta(n)$.
4. Disjoint merges toplamda bütün n elements'ı bir kez işler.
5. $\theta(n \log n)$.
6. $\theta(n)$ buffer; stack ayrıca $\theta(\log n)$.
7. Equal keys'te left half item önce seçilir.
8. [2,3,4,5,6,7,8,9].
9. Subarrays index ranges ile temsil edilebilir.